

Jigsaw Unintended Bias in Toxicity Classification

Détection de toxicité robuste et atténuation des biais d'identité

Enoncé du problème

Les plateformes en lignes modèrent automatiquement les commentaires pour limiter les messages haineux. Mais ces classifieurs de toxicité présentent souvent un biais : ils ont tendance à considérer certains mots liés à des identités ou communautés (par exemple « gay », « black », « muslim ») comme intrinsèquement toxiques. Cela entraîne une censure injuste qui touche en priorité des groupes marginalisés. Notre objectif est donc de développer un modèle de détection de la toxicité qui reste efficace pour modérer les propos haineux, tout en réduisant au maximum ces biais involontaires, afin de garantir une modération plus juste.

Pertinence pour le Big Data

Le dataset contient environ 2.0 millions de commentaires annotés avec des metadonnées sur les biais potentiels). L'approche Big Data est nécessaire car :

Volume : 2.38GB de données dont environ 2.0 millions de commentaires (texte libre) avec des colonnes de labels binaires et continus. Variété :

- Données non structurées : texte brut nécessitant un prétraitement lourd afin de pouvoir les exploiter
- Données structurées : variables numériques (binaires/continues) liées aux biais (genre, religion, orientation sexuelle)
- Nécessite des pipelines capables d'intégrer ces différents types de données

Vélocité : dans un scénario de déploiement, les plateformes en lignes génèrent des flux continus de nouveaux commentaires

- Nécessite une modération en temps réel ce qui impose des systèmes distribués de type stream processing pour l'ingestion, le prétraitement des données et la classification

Limitations des approches traditionnelles : L'approche traditionnelle Pandas/Scikit-learn qui fonctionne en mémoire sur un seul CPU est inadapté pour l'entraînement sur plusieurs millions de commentaires en texte brut. HDFS + Spark nous permettront de faire de la parallélisation pour entraîner les NLP.

Description de l'ensemble de données

Dataset Name: Jigsaw Unintended Bias in Toxicity Classification

Source: Compétition Kaggle <https://www.kaggle.com/competitions/jigsaw-unintended-bias-in-toxicity-classification/overview>

Type of Data: Semi-structuré (texte brut en CSV + colonnes numériques, binaires)

Volume: 2 millions de commentaires annotés ; 2.38GB de données dans tous les fichiers

Key Features:

- comment_text : commentaires (texte brut)
- target : score de toxicité (0-1)
- l'ensembles des colonnes du dataset identity_individual_annotations (double entre 0 et 1 pour la plupart mais aussi timestamp, integer et string)

Activation de findspark

imports spark

création de la session Spark :

- utilisation de tous les coeurs alloués à la vm
- on alloue 8g de RAM à spark

```
import findspark
findspark.init()

import pyspark
from pyspark.sql import SparkSession

# spark = SparkSession.builder \
#     .master("local[*]") \
#     .appName("Jigsaw Unintended Bias in Toxicity Classification") \
#     .config("spark.driver.memory", "12g") \
#     .config("spark.executor.memory", "12g") \
#     .config("spark.driver.maxResultSize", "4g") \
#     .config("spark.sql.shuffle.partitions", "16") \
#     .config("spark.memory.offHeap.enabled", "true") \
#     .config("spark.memory.offHeap.size", "2g") \
#     .config("spark.network.timeout", "600s") \
#     .getOrCreate()

spark = SparkSession.builder \
    .master("local[*]") \
    .appName("Jigsaw Unintended Bias in Toxicity Classification") \
    .config("spark.driver.memory", "8g") \
    .config("spark.driver.maxResultSize", "2g") \
    .config("spark.executor.memory", "8g") \
    .config("spark.sql.shuffle.partitions", "8") \
    .config("spark.network.timeout", "600s") \
    .config("spark.some.config.option", "some-value") \
    .getOrCreate()

WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-
defaults.properties
```

```
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use
setLogLevel(newLevel).
25/12/07 02:55:11 WARN NativeCodeLoader: Unable to load native-hadoop
library for your platform... using builtin-java classes where
applicable
```

Imports: matplotlib pour les graphiques ; des fonctions de pyspark.sql ; des fonctions de pyspark.ml pour la création de pipeline ; nltk et WordNet Lemmatizer pour la lemmatisation ; PipelineModel pour le chargement et la sauvegarde d'un pipeline spark

```
print(spark.version)
import matplotlib.pyplot as plt
from pyspark.sql.functions import isnan

from pyspark.sql.functions import udf, regexp_replace, lower, col,
when, isnan
from pyspark.sql.types import ArrayType, StringType
from pyspark.ml import Pipeline
from pyspark.ml.feature import Tokenizer, StopWordsRemover,
SQLTransformer
from pyspark.ml.evaluation import BinaryClassificationEvaluator

import nltk
nltk.download("wordnet")
nltk.download("omw-1.4")
from nltk.stem import WordNetLemmatizer

import os
import json
from pyspark.ml import PipelineModel

from pyspark.ml.feature import CountVectorizer, IDF
from pyspark.ml.classification import LogisticRegression,
RandomForestClassifier, LinearSVC, GBTClassifier
from pyspark.ml.tuning import CrossValidator, ParamGridBuilder

4.0.0

[nltk_data] Downloading package wordnet to /home/vboxuser/nltk_data...
[nltk_data] Package wordnet is already up-to-date!
[nltk_data] Downloading package omw-1.4 to /home/vboxuser/nltk_data...
[nltk_data] Package omw-1.4 is already up-to-date!
```

0. Chargement et inspection des données

```
# train = spark.read.csv('../train.csv', header=True,
inferSchema=True)
```

```
# train.select("id", "target", "comment_text").show(5, truncate=False)
# pas toutes les colonnes sinon illible
```

On print le schéma ainsi que la description de certaines colonnes pour vérifier si spark a correctement inféré les types des features

```
# train.printSchema()

# print(f"nombre de lignes : {train.count()}")
# print(f"nombre de colonnes : {len(train.columns)}")
# train.describe(['target', 'severe_toxicity', 'obscene',
'insult']).show()
```

Problème : on a des données non numériques dans ces colonnes --> données mal parsées ; de plus on a dans le schéma : type string au lieu d'un type numérique, ce qui confirme notre intuition. Nous réadaptions le script en conséquence :

```
train = (
    spark.read.format("csv")
    .option("header", True)
    .option("inferSchema", True)
    .option("multiLine", True) # commentaires multilignes
    .option("escape", "\\") # les " internes à un comment ne ferment
pas le champ
    .load("../train.csv")
)
```

```
train.printSchema()
```

```
root
|-- id: integer (nullable = true)
|-- target: double (nullable = true)
|-- comment_text: string (nullable = true)
|-- severe_toxicity: double (nullable = true)
|-- obscene: double (nullable = true)
|-- identity_attack: double (nullable = true)
|-- insult: double (nullable = true)
|-- threat: double (nullable = true)
|-- asian: double (nullable = true)
|-- atheist: double (nullable = true)
|-- bisexual: double (nullable = true)
|-- black: double (nullable = true)
|-- buddhist: double (nullable = true)
|-- christian: double (nullable = true)
|-- female: double (nullable = true)
|-- heterosexual: double (nullable = true)
```

```

|-- hindu: double (nullable = true)
|-- homosexual_gay_or_lesbian: double (nullable = true)
|-- intellectual_or_learning_disability: double (nullable = true)
|-- jewish: double (nullable = true)
|-- latino: double (nullable = true)
|-- male: double (nullable = true)
|-- muslim: double (nullable = true)
|-- other_disability: double (nullable = true)
|-- other_gender: double (nullable = true)
|-- other_race_or_ethnicity: double (nullable = true)
|-- other_religion: double (nullable = true)
|-- other_sexual_orientation: double (nullable = true)
|-- physical_disability: double (nullable = true)
|-- psychiatric_or_mental_illness: double (nullable = true)
|-- transgender: double (nullable = true)
|-- white: double (nullable = true)
|-- created_date: timestamp (nullable = true)
|-- publication_id: integer (nullable = true)
|-- parent_id: double (nullable = true)
|-- article_id: integer (nullable = true)
|-- rating: string (nullable = true)
|-- funny: integer (nullable = true)
|-- wow: integer (nullable = true)
|-- sad: integer (nullable = true)
|-- likes: integer (nullable = true)
|-- disagree: integer (nullable = true)
|-- sexual_explicit: double (nullable = true)
|-- identity_annotator_count: integer (nullable = true)
|-- toxicity_annotator_count: integer (nullable = true)

```

```

print(f"nombre de lignes : {train.count()}")
print(f"nombre de colonnes : {len(train.columns)}")
train.describe(['target', 'severe_toxicity', 'obscene',
'insult']).show()

```

```

nombre de lignes : 1804874
nombre de colonnes : 45

```

```

25/12/06 06:13:42 WARN SparkStringUtils: Truncated the string
representation of a plan since it was too large. This behavior can be
adjusted by setting 'spark.sql.debug.maxToStringFields'.

```

```

[Stage 5:> (0
+ 1) / 1]

```

```

+-----+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+
|summary|          target|      severe_toxicity|
obscene|          insult|

```

```

+-----+-----+-----+-----+
+-----+
| count|          1804874|          1804874|
1804874|          1804874|
| mean|0.10301729306264126|0.004582098790397937|
0.013877209195019816|0.08115273414475882|
| stddev|0.19707574479033887| 0.02286128244247058|
0.0646041914397229|0.17606571372957508|
| min|          0.0|          0.0|
0.0|          0.0|
| max|          1.0|          1.0|
1.0|          1.0|
+-----+-----+-----+-----+
+-----+

```

Les features ont été correctement inférées. La moyenne ainsi que l'écart type de ces features nous renseigne quand à un fort déséquilibre des classes, que nous approfondirons par la suite

```
train.limit(20).toPandas().head()
```

	id	target	comment_text
0	59848	0.000000	This is so cool. It's like, 'would you want yo...
1	59849	0.000000	Thank you!! This would make my life a lot less...
2	59852	0.000000	This is such an urgent design problem; kudos t...
3	59855	0.000000	Is this something I'll be able to install on m...
4	59856	0.893617	haha you guys are a bunch of losers.

	severe_toxicity	obscene	identity_attack	insult	threat	asian
0	0.000000	0.0	0.000000	0.000000	0.0	NaN
1	0.000000	0.0	0.000000	0.000000	0.0	NaN
2	0.000000	0.0	0.000000	0.000000	0.0	NaN
3	0.000000	0.0	0.000000	0.000000	0.0	NaN
4	0.021277	0.0	0.021277	0.87234	0.0	0.0

0	...	article_id	rating	funny	wow	sad	likes	disagree	\
1	...	2006	rejected	0	0	0	0	0	
2	...	2006	rejected	0	0	0	0	0	
3	...	2006	rejected	0	0	0	0	0	
4	...	2006	rejected	0	0	0	1	0	
		sexual_explicit	identity_annotator_count	toxicity_annotator_count					
0		0.0				0			4
1		0.0				0			4
2		0.0				0			4
3		0.0				0			4
4		0.0				4			47
[5 rows x 45 columns]									

A l'aide du schéma on identifie des groupes dans les features :

- La colonne commentaires --> **string**
- La colonne target --> **double entre [0,1]**
- Les sous-labels de toxicité --> **double entre [0,1]** : severe_toxicity, obscene, identity_attack, insult, threat, sexual_explicit
- Les colonnes d'identité --> **float entre [0,1]** : asian, atheist, bisexual, black, buddhist, christian, female, heterosexual, hindu, homosexual_gay_or_lesbian, intellectual_or_learning_disability, jewish, latino, male, muslim, other_disability, other_gender, other_race_or_ethnicity, other_religion, other_sexual_orientation, physical_disability, psychiatric_or_mental_illness, transgender, white
- Les colonnes temporelles --> **timestamp** : created_date
- Les features liées aux métadonnées --> **numériques mais catégorielles** : publication_id, parent_id, article_id
- Colonnes liées aux réactions --> compteurs d'interactions, **int** : funny, wow, sad, likes, disagree
- Nombre d'annotateurs sur un commentaire donné (mesure de la confiance des labels) --> **int** : identity_annotator_count, toxicity_annotator_count

Nous allons explorer les distributions de toutes ces colonnes. Sont affichées ci-dessous tous les distributions pour chaque feature représentative d'un groupe

Fonction pour afficher la distribution d'une col numerique + nb valeurs manquantes

```
def describe_col(spark_df, column_name, bins = 50):
    """
    Affiche la distribution d'une col numerique + nb valeurs
    manquantes
    """
    data = spark_df.select(column_name).toPandas()

    plt.figure(figsize=(8, 5))
    plt.hist(data[column_name], bins=bins, edgecolor='black')
    plt.title(f"Distribution de '{column_name}'")
    plt.xlabel(column_name)
    plt.ylabel("Fréquence")
    plt.grid(True, alpha=0.3)
    plt.show()

    tot = spark_df.count()

    dtype = dict(spark_df.dtypes)[column_name]

    if dtype in ("double", "float"):
        missing = spark_df.filter(
            col(column_name).isNull() | isnan(col(column_name))
        ).count()
    else:
        missing = spark_df.filter(
            col(column_name).isNull()
        ).count()

    missing_pct = (missing / tot) * 100
    print(f"Valeurs manquantes : {missing} / {tot} ({missing_pct:.2f}
    %)")

from pyspark.sql.functions import col, isnan, lit
import numpy as np

def preview_comments(df, col_name, comment_col="comment_text",
max_categories=5, bins=5, per_group=3):
    """
    Aperçu des commentaires :
    - Si nombre de valeurs distinctes inf ou egal à max_categories -->
    catégoriel
    - Sinon --> continu, on fait des bins
    --> Affiche un tableau récapitulatif des effectifs
    """

    # On compte le nb de valeurs distinctes
    distinct_vals = df.select(col_name).distinct().dropna().collect()
    distinct_count = len(distinct_vals)
```

```

print(f"Nombre de valeurs distinctes : {distinct_count}\n")

# VARIABLE CATÉGORIELLE
if distinct_count <= max_categories:
    print("--> Colonne détectée comme CATEGORIELLE\n")

    values = [row[col_name] for row in distinct_vals]

    # tableau des effectifs
    summary_rows = []
    for v in values:
        count_v = df.filter(col(col_name) == v).count()
        summary_rows.append((str(v), count_v))

    summary_df = spark.createDataFrame(summary_rows, ["Categorie",
"Effectif"])
    print("Tableau des effectifs par catégorie :")
    summary_df.show(truncate=False)

    print("\nAperçu des commentaires par catégorie :\n")

    for v in values:
        print(f">> Valeur : {v} ")
        subset = df.filter(col(col_name) == v)
        subset.select(col_name, comment_col).show(per_group,
truncate=100)
        print("\n")

    return

# VARIABLE CONTINUE
print("--> Colonne détectée comme CONTINUE (ou + de 5 catégories).
Création de bins.\n")

# on cherche min/max
values = df.select(col_name).dropna().rdd.map(lambda r:
r[0]).collect()
mn, mx = min(values), max(values)

print(f"Range = [{mn:.4f}, {mx:.4f}]\n")

edges = np.linspace(mn, mx, bins + 1)

# tableau des effectifs par bin
summary_rows = []

for i in range(bins):
    low, high = float(edges[i]), float(edges[i+1])

```

```

        # Tous les bins sauf le dernier : [low, high)
        if i < bins - 1:
            subset = df.filter((col(col_name) >= low) & (col(col_name)
< high))
        else:
            # Dernier bin : inclut la valeur max --> sinon on a pas
            les val avec 1.0 quand les valeurs sont entre [0,1]
            subset = df.filter((col(col_name) >= low) & (col(col_name)
<= high))

        count_bin = subset.count()
        summary_rows.append((f"[{low:.4f}, {high:.4f})", count_bin))

    summary_df = spark.createDataFrame(summary_rows, ["Intervalle",
"Effectif"])

    print("Tableau des effectifs par intervalle :")
    summary_df.show(truncate=False)
    print("\nAperçu des commentaires par bin :\n")

    # Affichage des commentaires par bin
    for i in range(bins):
        low, high = float(edges[i]), float(edges[i+1])
        print(f">> Intervalle {i+1}/{bins} : [{low:.4f}, {high:.4f}]
")

        if i < bins - 1:
            subset = df.filter((col(col_name) >= low) & (col(col_name)
< high))
        else:
            subset = df.filter((col(col_name) >= low) & (col(col_name)
<= high))

        subset.select(col_name, comment_col).show(per_group,
truncate=100)
        print("\n")

from pyspark.sql.functions import col

def explore_categorical(df, col_name="rating",
comment_col="comment_text", per_group=5):
    print(f">> Exploration de la colonne catégorielle : {col_name} \
n")

    # Valeurs distinctes + effectifs
    summary = (
        df.groupBy(col_name)
        .count()
        .orderBy(col("count").desc())

```

```

)

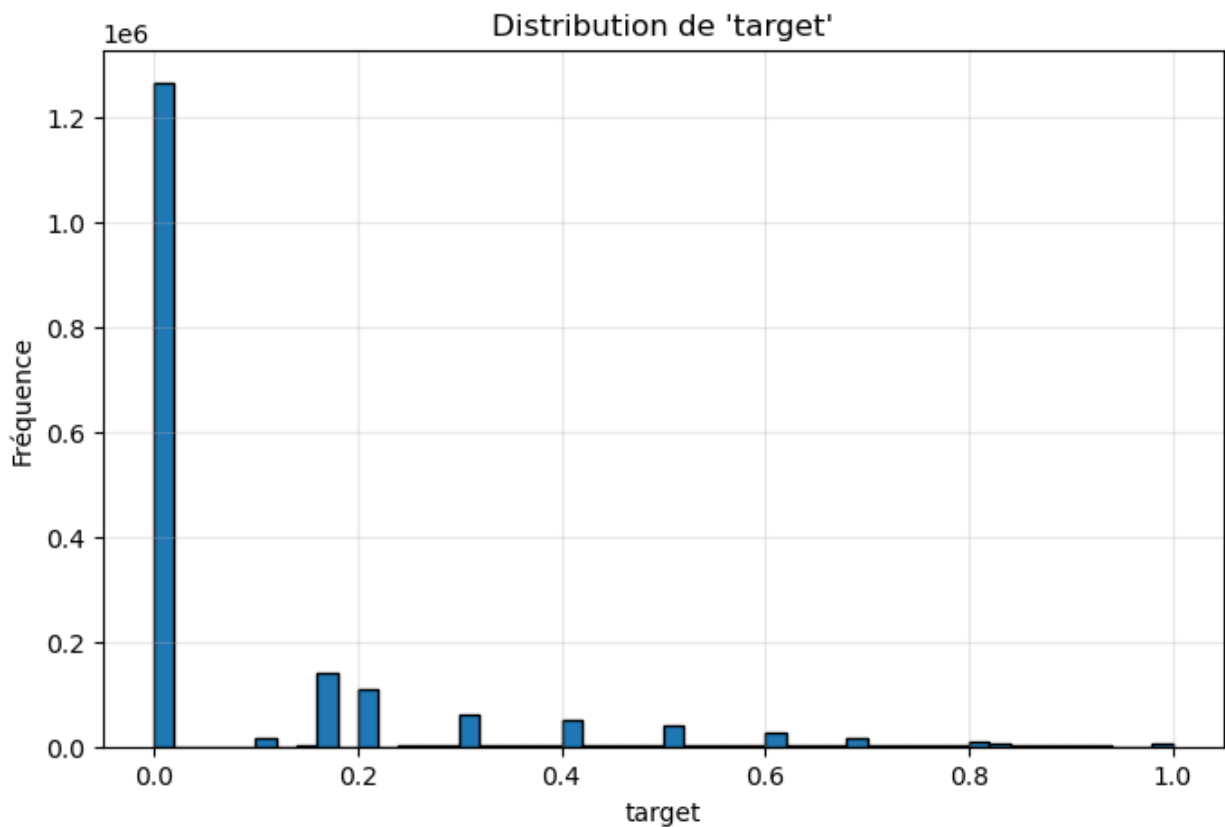
print("Valeurs et effectifs :")
summary.show(truncate=False)

# Aperçu des exemples par catégorie
categories = [row[col_name] for row in summary.collect()]

print("\n>> Aperçu des commentaires par catégorie \n")
for cat in categories:
    print(f">> {col_name} = '{cat}'")
    df.filter(col(col_name) == cat) \
        .select(col_name, comment_col) \
        .show(per_group, truncate=100)
    print("\n")

describe_col(train, column_name="target")
preview_comments(train, "target")

```



Valeurs manquantes : 0 / 1804874 (0.00%)

Nombre de valeurs distinctes : 2913

```
--> Colonne détectée comme CONTINUE (ou + de 5 catégories). Création de bins.
```

Range = [0.0000, 1.0000]

Tableau des effectifs par intervalle :

Intervalle	Effectif
[0.0000, 0.2000]	1427188
[0.2000, 0.4000]	176210
[0.4000, 0.6000]	129241
[0.6000, 0.8000]	41404
[0.8000, 1.0000]	30831

Aperçu des commentaires par bin :

```
>> Intervalle 1/5 : [0.0000, 0.2000]
```

```
+-----+
+-----+
+-----+
|target|
comment_text|
+-----+
+-----+
+-----+
| 0.0|This is so cool. It's like, 'would you want your mother to
read this??' Really great idea, well d...|
| 0.0|Thank you!! This would make my life a lot less anxiety-
inducing. Keep it up, and don't let anyone...|
| 0.0| This is such an urgent design problem; kudos to
you for taking it on. Very impressive!|
+-----+
+-----+
+-----+
only showing top 3 rows
```

```
>> Intervalle 2/5 : [0.2000, 0.4000]
```

```

+-----
+-----
+-----+
|target|
comment_text|
+-----
+-----
+-----+
| 0.2|I'm crazy about that illustration! I never would have thought
to pitch everything so yellow and o...|
| 0.2|Thank you, thank you, thank you! I love WW content but the
comment section has polluted for far t...|
| 0.3|                                Mormons have had a
complicated relationship with federal law.|
+-----
+-----
+-----+
only showing top 3 rows

```

```
>> Intervalle 3/5 : [0.4000, 0.6000]
```

```

+-----
+-----
+-----+
|          target|
comment_text|
+-----
+-----
+-----+
|0.4576271186440678|
hahahahahahahhha suck it.|
|          0.44|This is a great story. Man. I wonder if the person
who yelled "shut the fuck up!" at him ever hea...|
|          0.6|It's ridiculous that these guys are being called
"protesters". Being armed is a threat of violenc...|
+-----
+-----
+-----+
only showing top 3 rows

```

```
>> Intervalle 4/5 : [0.6000, 0.8000]
```

```

+-----
+-----
+-----+
|          target|
comment_text|
+-----
+-----
+-----+

```

```
|0.6666666666666666|
ur a sh*tty comment.|
|          0.7|Took this as an opportunity to check back in on
The Yard and the floorpans are finally up and the...|
|0.7818181818181817|THIS IS A TEST: \n\nAny fool knows Star Trek is
the best albeit lacking the technology available ...|
```

```
+-----+
+-----+
+-----+
only showing top 3 rows
```

```
>> Intervalle 5/5 : [0.8000, 1.0000]
```

```
+-----+
+-----+
+-----+
```

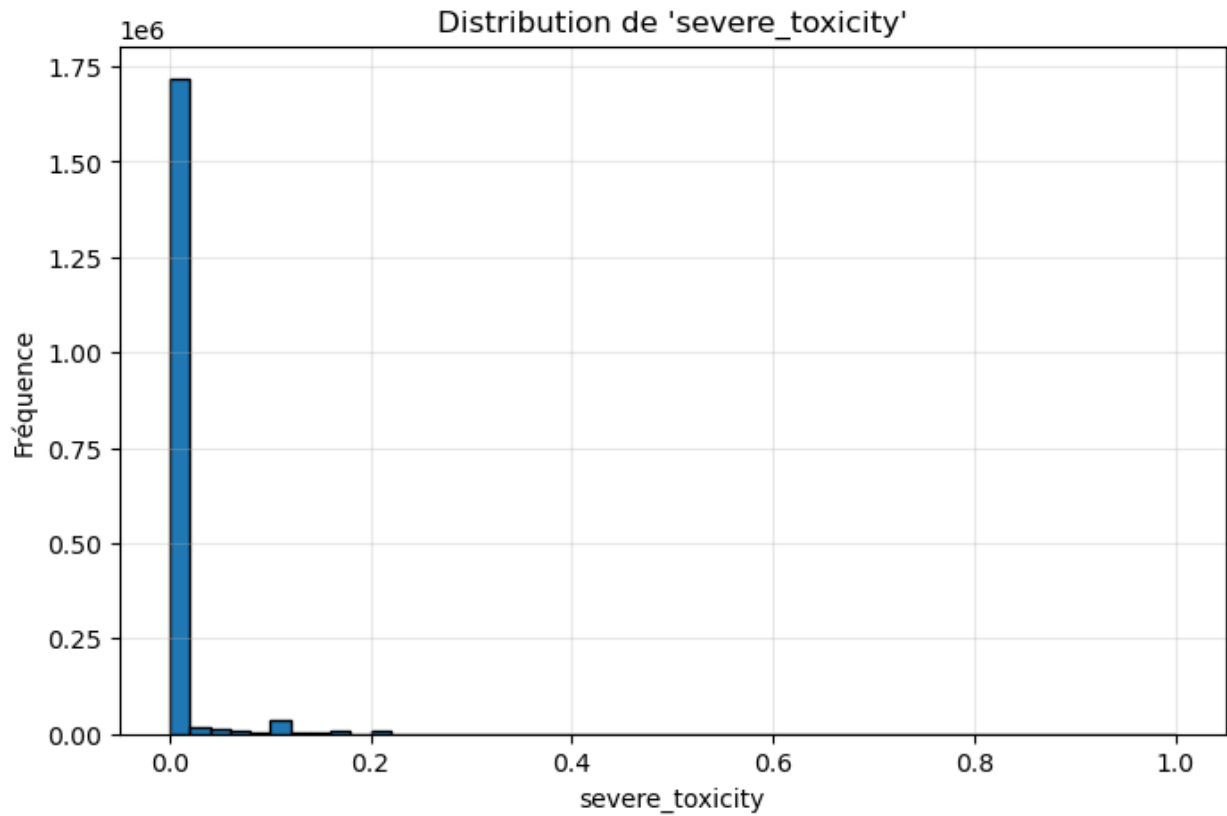
```
|          target|
comment_text|
```

```
+-----+
+-----+
+-----+
```

```
|0.8936170212765957|
haha you guys are a bunch of losers.|
|          0.9125|Yet call out all Muslims for the acts of a few
will get you pilloried. So why is it okay to sme...|
|0.8307692307692307|
This bitch is nuts. Who would read a book by a woman.|
```

```
+-----+
+-----+
+-----+
only showing top 3 rows
```

```
describe_col(train, "severe_toxicity")
preview_comments(train, "severe_toxicity")
```



Valeurs manquantes : 0 / 1804874 (0.00%)

Nombre de valeurs distinctes : 1460

--> Colonne détectée comme CONTINUE (ou + de 5 catégories). Création de bins.

Range = [0.0000, 1.0000]

Tableau des effectifs par intervalle :

Intervalle	Effectif
[0.0000, 0.2000]	1798436
[0.2000, 0.4000]	6363
[0.4000, 0.6000]	73

```
|[0.6000, 0.8000]|1|
|[0.8000, 1.0000]|1|
+-----+-----+
```

Aperçu des commentaires par bin :

>> Intervalle 1/5 : [0.0000, 0.2000]

```
+-----+
+-----+
+-----+
|severe_toxicity|
comment_text|
+-----+
+-----+
+-----+
|          0.0|This is so cool. It's like, 'would you want your
mother to read this??' Really great idea, well d...|
|          0.0|Thank you!! This would make my life a lot less
anxiety-inducing. Keep it up, and don't let anyone...|
|          0.0|          This is such an urgent design problem;
kudos to you for taking it on. Very impressive!|
+-----+
+-----+
+-----+
only showing top 3 rows
```

>> Intervalle 2/5 : [0.2000, 0.4000]

```
+-----+
+-----+
+-----+
|severe_toxicity|
comment_text|
+-----+
+-----+
+-----+
|          0.2|If you believe this bunch is standing up for the
Hammonds, you are sadly misinformed.\n\nThe Bund...|
|          0.2|
And ballots.|
|          0.2|The issue of the 19 - 16 year old is, who's the
aggressor and why? The 19 yo is college age, the ...|
+-----+
+-----+
+-----+
only showing top 3 rows
```

>> Intervalle 3/5 : [0.4000, 0.6000]

```

+-----+
+-----+
+-----+
|   severe_toxicity|
comment_text|
+-----+
+-----+
+-----+
|0.5348189415041783|
Kill all the niggers and pedophiles!|
|           0.4|Good. The NRA and tRump deserve each other.
Blam, blam, blam. Shoot 'em up, bang 'em up.|
|           0.4|                               WE NEED TO
PERSECUTE THESE CHRISTIANS,PLEASE JOIN US|
+-----+
+-----+
+-----+
only showing top 3 rows

```

>> Intervalle 4/5 : [0.6000, 0.8000]

```

+-----+
+-----+
+-----+
|   severe_toxicity|
comment_text|
+-----+
+-----+
+-----+
|0.6443629697525206|You are murdering feckless SCUM. Your mother
would slap you for being the piece of shit that you...|
+-----+
+-----+
+-----+

```

>> Intervalle 5/5 : [0.8000, 1.0000]

[Stage 76:> (0
+ 1) / 1]

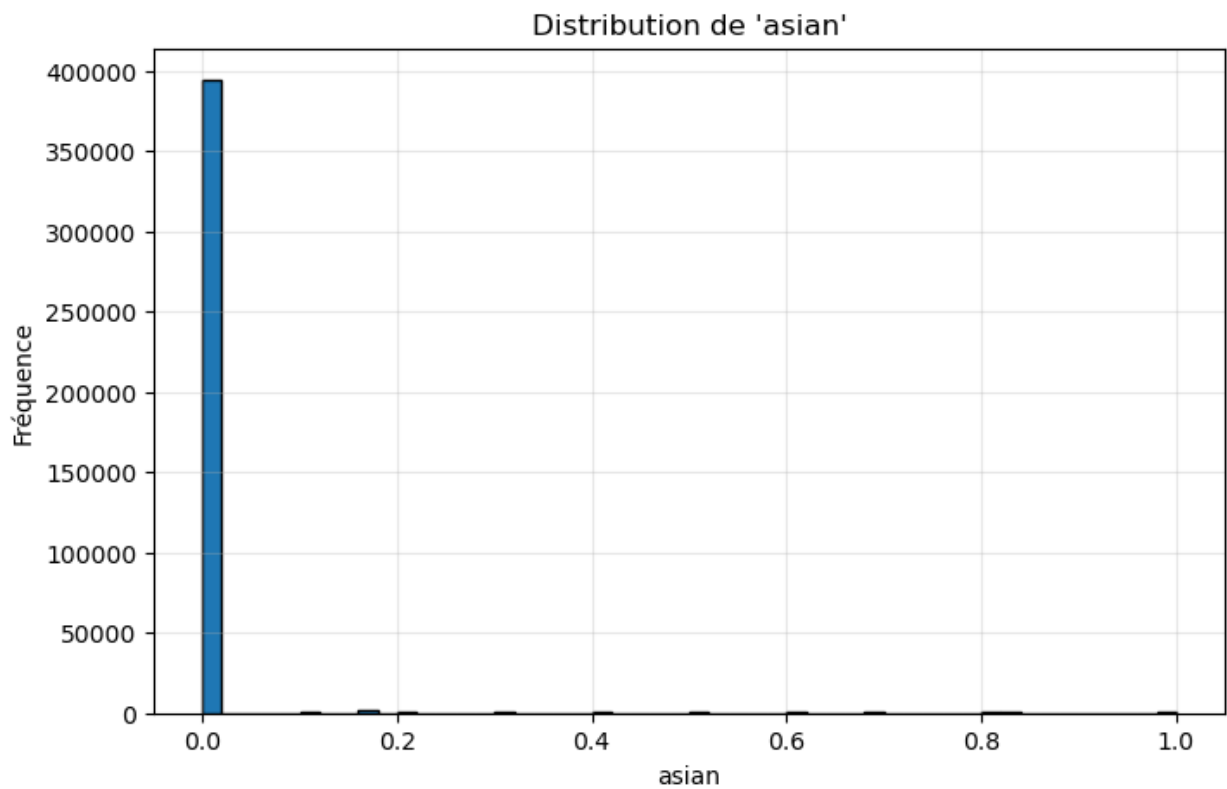
```

+-----+
+-----+
+-----+
|severe_toxicity|
comment_text|
+-----+

```

```
+-----+
+-----+
|          1.0|Awesome! Lets cut the head off hate! Lets stab it and
burn its house down! lets march together an...|
+-----+
+-----+
+-----+
```

```
describe_col(train, "asian")
preview_comments(train, "asian")
```



Valeurs manquantes : 1399744 / 1804874 (77.55%)

Nombre de valeurs distinctes : 125

--> Colonne détectée comme CONTINUE (ou + de 5 catégories). Création de bins.

Range = [0.0000, 1.0000]

Tableau des effectifs par intervalle :

Intervalle	Effectif
[0.0000, 0.2000]	397084
[0.2000, 0.4000]	2402
[0.4000, 0.6000]	2820
[0.6000, 0.8000]	553
[0.8000, 1.0000]	2271

Aperçu des commentaires par bin :

>> Intervalle 1/5 : [0.0000, 0.2000]

```
+-----+
+-----+
+-----+
|asian|
comment_text|
+-----+
+-----+
+-----+
| 0.0|
haha you guys are a bunch of losers.|
| 0.0|This is a great story. Man. I wonder if the person who yelled
"shut the fuck up!" at him ever hea...|
| 0.0|Angry trolls, misogynists and Racists", oh my. It doesn't take
all of my 150 IQ to see the slant ...|
+-----+
+-----+
+-----+
only showing top 3 rows
```

>> Intervalle 2/5 : [0.2000, 0.4000]

```
+-----+
+-----+
+-----+
```

```

-----+
|asian|
comment_text|
+-----
+-----+
-----+
| 0.3|True. But the idea is good--just send them to a sanctuary in SE
Asia, such as the Elephant Nature...|
| 0.2|Is Trump the Alpha Male, or is he the Great White Hope of those
abandoned by their country? \n\nI...|
| 0.3|Joshua, $15.00 wouldn't be enough to for a family of 4 to live
on. It would however, be the make...|
+-----
+-----+
-----+
only showing top 3 rows

```

```

>> Intervalle 3/5 : [0.4000, 0.6000]

```

```

+-----
+-----+
-----+
|asian|
comment_text|
+-----
+-----+
-----+
| 0.5|Relentless attempts by liberal pundits to prop-up the notion
that we live in a nation deranged an...|
| 0.5|I see CJ101's opinion as one of fear, saying it is "scary,
scary times". I also think this is wh...|
| 0.5|To blame Rubio is to find a scapegoat. Trump is a product of
many angry, mostly white Americans w...|
+-----
+-----+
-----+
only showing top 3 rows

```

```

>> Intervalle 4/5 : [0.6000, 0.8000]

```

```

+-----
+-----+
-----+
|asian|
comment_text|
+-----
+-----+
-----+
| 0.75|Seems like a mixed bag. Without knowing what people are being
arrested for, many of these numbers...|

```

```
| 0.75|I'd be more supportive of Miss Loretta if she'd said "why are  
there more Blacks in the system in ...|  
| 0.7|To address your question, I don't think Hillary is an honest  
person, and I would say that Obama i...|
```

```
+-----
```

```
+-----
```

```
-----+
```

```
only showing top 3 rows
```

```
>> Intervalle 5/5 : [0.8000, 1.0000]
```

```
+-----
```

```
+-----
```

```
-----+
```

```
|asian|  
comment_text|
```

```
+-----
```

```
+-----
```

```
-----+
```

```
| 1.0| Not a good idea, considering that the  
elephants at the zoo are Asian elephants.|
```

```
| 1.0|Or they could be moved to one of the two US sanctuaries (PAWS  
in California or The Elephant Sanct...|
```

```
| 1.0|FYI, Asian people have never been called Orientals. I mean  
they have but only in error and it i...|
```

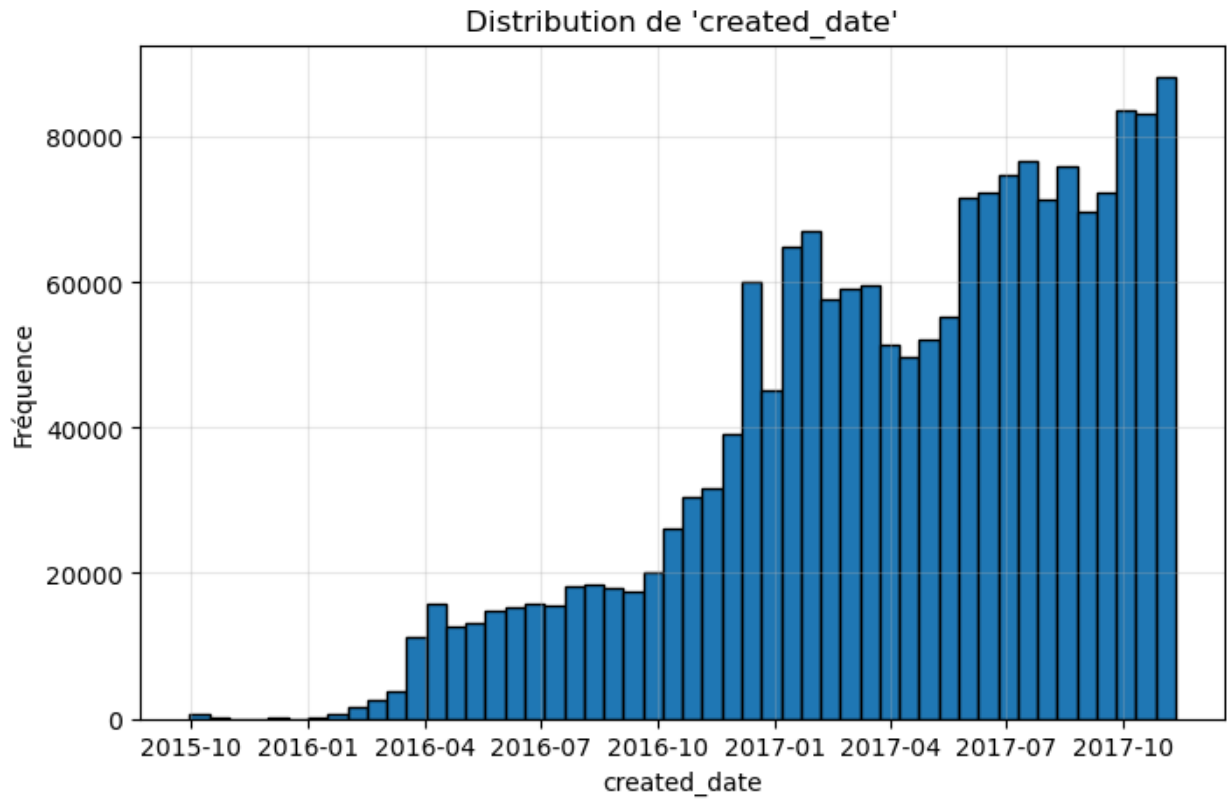
```
+-----
```

```
+-----
```

```
-----+
```

```
only showing top 3 rows
```

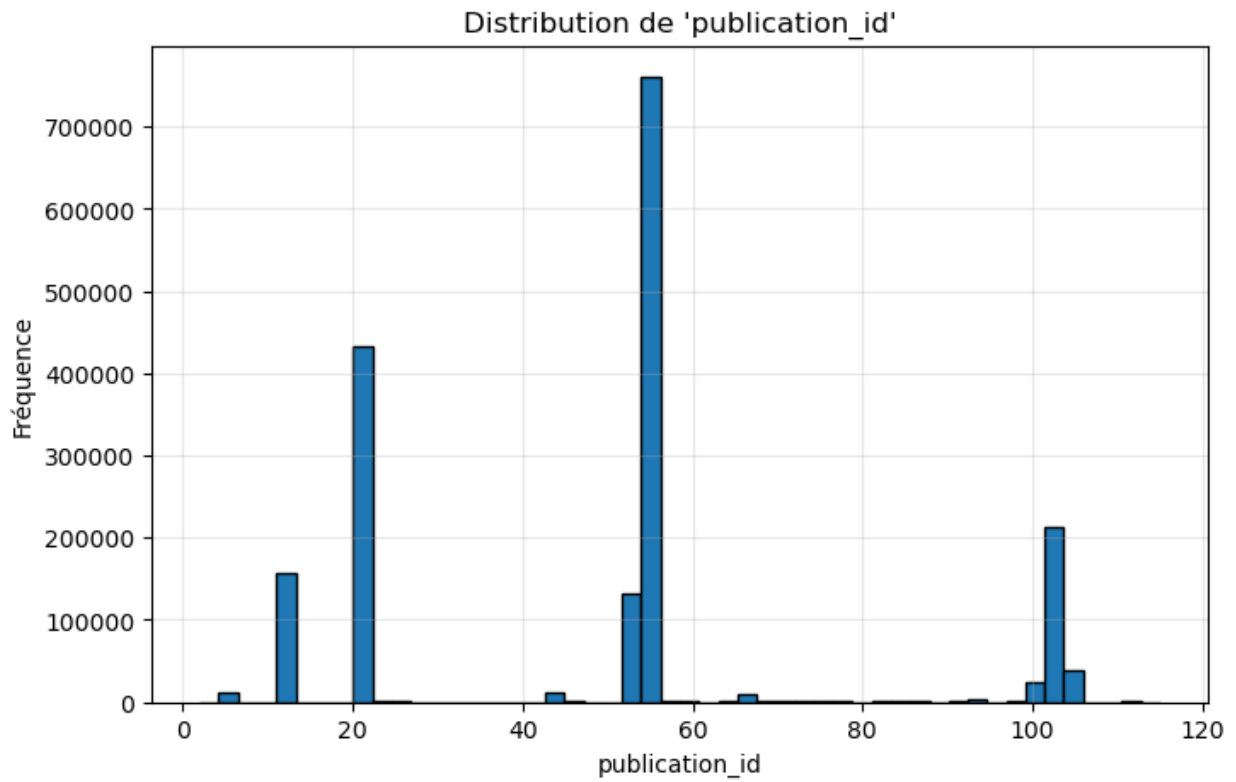
```
describe_col(train, "created_date")
```



```
[Stage 115:> (0  
+ 1) / 1]
```

Valeurs manquantes : 0 / 1804874 (0.00%)

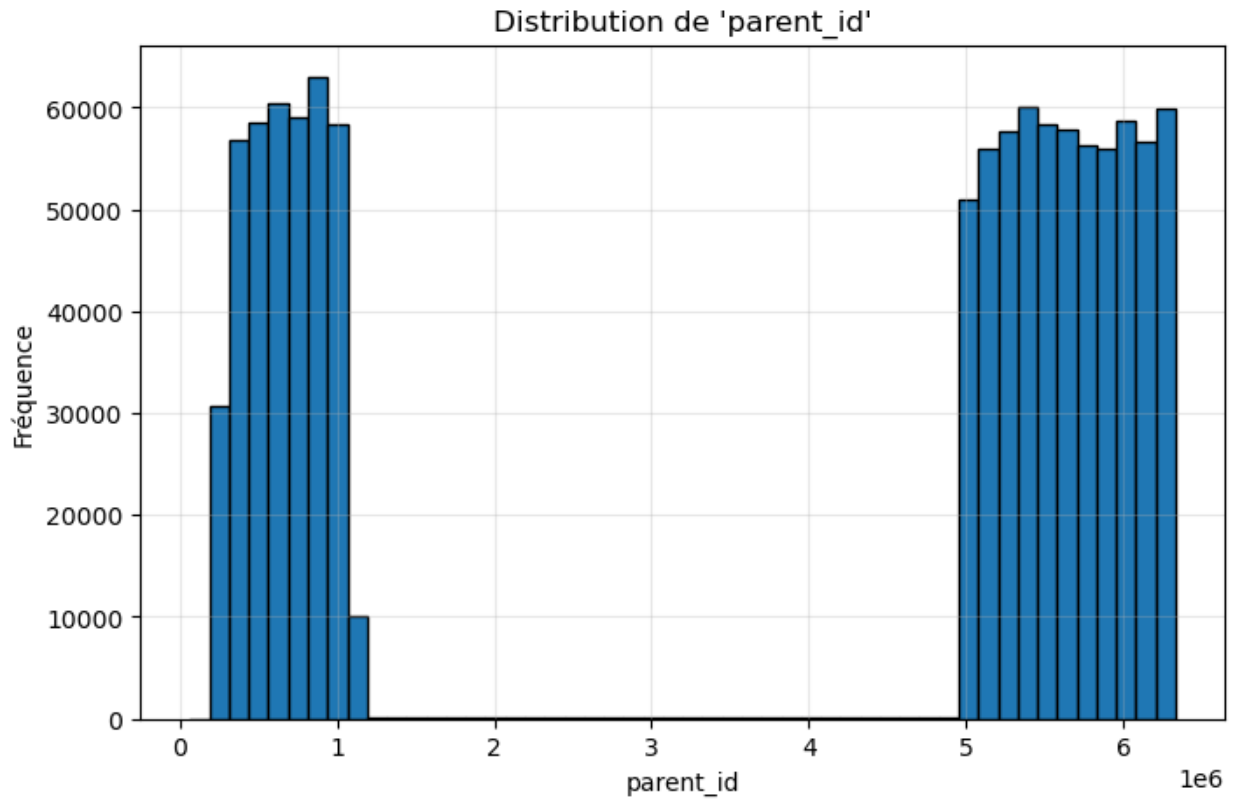
```
describe_col(train, "publication_id")
```



```
[Stage 122:> (0  
+ 1) / 1]
```

```
Valeurs manquantes : 0 / 1804874 (0.00%)
```

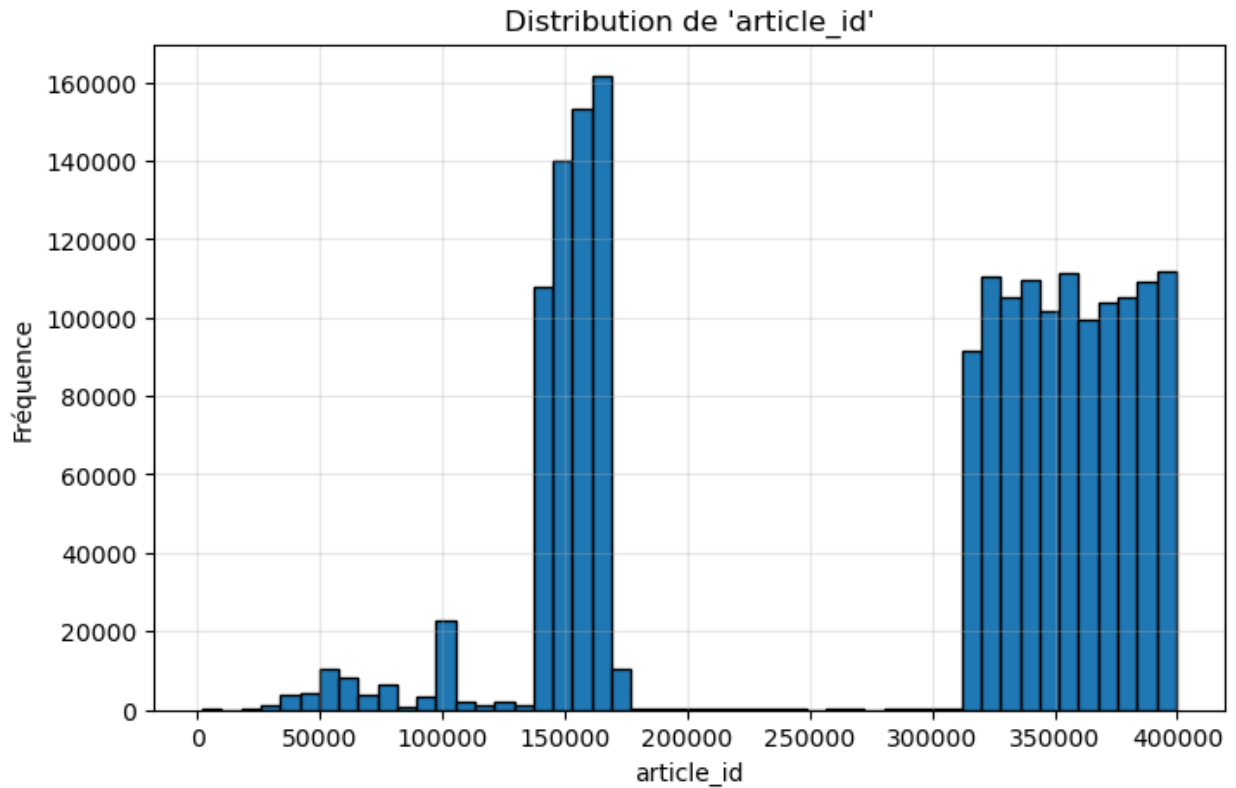
```
describe_col(train, "parent_id")
```



```
[Stage 129:> (0  
+ 1) / 1]
```

Valeurs manquantes : 778646 / 1804874 (43.14%)

```
describe_col(train, "article_id")
```

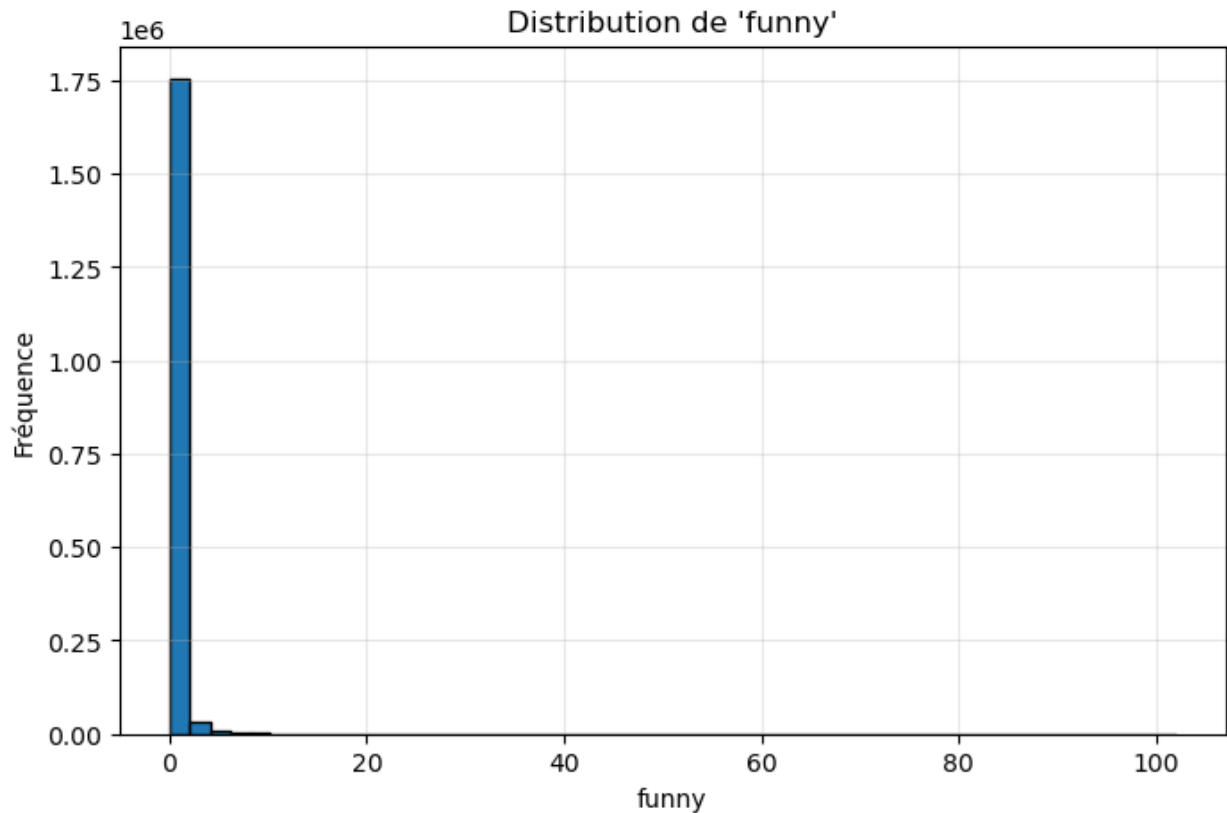


```
[Stage 136:>  
+ 1) / 1]
```

(0

Valeurs manquantes : 0 / 1804874 (0.00%)

```
describe_col(train, "funny")  
preview_comments(train, "funny")
```



Valeurs manquantes : 0 / 1804874 (0.00%)

Nombre de valeurs distinctes : 61

--> Colonne détectée comme CONTINUE (ou + de 5 catégories). Création de bins.

Range = [0.0000, 102.0000]

Tableau des effectifs par intervalle :

Intervalle	Effectif
[0.0000, 20.4000]	1804531
[20.4000, 40.8000]	309
[40.8000, 61.2000]	29

```
|[61.2000, 81.6000] |1      |
|[81.6000, 102.0000]|4      |
+-----+-----+
```

Aperçu des commentaires par bin :

```
>> Intervalle 1/5 : [0.0000, 20.4000]
```

```
+-----
+-----+
+-----+
|funny|
comment_text|
+-----
+-----+
+-----+
|    0|This is so cool. It's like, 'would you want your mother to read
this??' Really great idea, well d...|
|    0|Thank you!! This would make my life a lot less anxiety-
inducing. Keep it up, and don't let anyone...|
|    0|          This is such an urgent design problem; kudos to
you for taking it on. Very impressive!|
+-----
+-----+
+-----+
only showing top 3 rows
```

```
>> Intervalle 2/5 : [20.4000, 40.8000]
```

```
+-----
+-----+
+-----+
|funny|
comment_text|
+-----
+-----+
+-----+
|   23|Wait for it:  Trump will step down and let his children run
his business.      \n\nA future Trump...|
|   24|So, I hear there are a lot of people saying that Amy is a
bigot.  I'm not saying that I think tha...|
|   26|          It's because Trump knows more than the intelligence
community. He is an expert on everything.|
+-----
+-----+
+-----+
only showing top 3 rows
```

```
>> Intervalle 3/5 : [40.8000, 61.2000]
```

```

+-----
+-----+
+-----+
|funny|
comment_text|
+-----
+-----+
+-----+
| 52|
release Trumps Tax returns. Coming soon|
| 43|....the spokesperson went on to say that anyone who would like
to represent the PM in Davos is w...|
| 50|Trump is indeed cutting costs. Like how he's going to get
Mexico to pay for the wall. The U.S. ...|
+-----
+-----+
+-----+
only showing top 3 rows

```

>> Intervalle 4/5 : [61.2000, 81.6000]

```

+-----
+-----+
+-----+
|funny|
comment_text|
+-----
+-----+
+-----+
| 73|I'm sure prospective migrants will now think twice about trying
to enter Canada knowing that Libe...|
+-----
+-----+
+-----+

```

>> Intervalle 5/5 : [81.6000, 102.0000]

[Stage 172:> (0
+ 1) / 1]

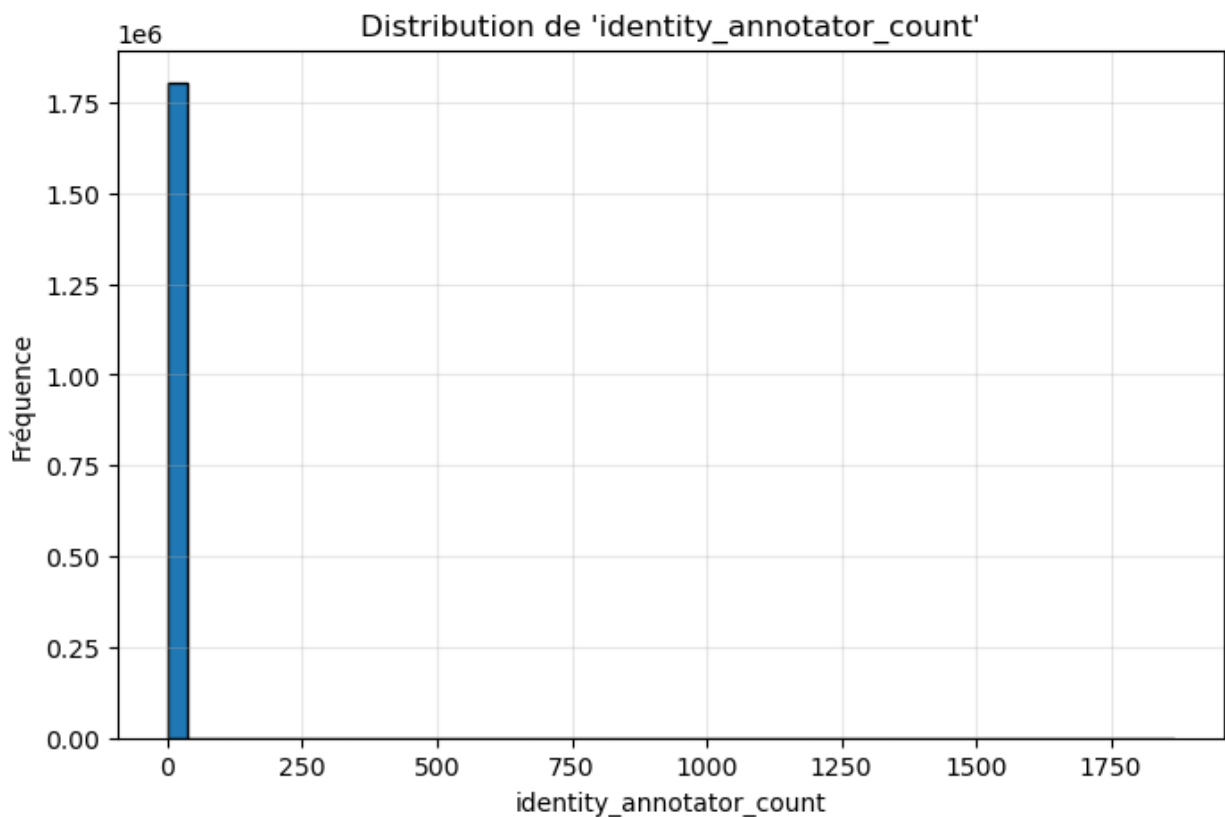
```

+-----+-----+
|funny|
comment_text|
+-----+-----+
| 102| It will run you $1500 for a double double though.|

```

```
| 94|Kevin considers himself quite the wit, and he's half right.|
| 93|Congrats to Harjit Sajjan - nice shot!|
+-----+
only showing top 3 rows
```

```
describe_col(train, "identity_annotator_count")
preview_comments(train, "identity_annotator_count")
```



Valeurs manquantes : 0 / 1804874 (0.00%)

Nombre de valeurs distinctes : 158

--> Colonne détectée comme CONTINUE (ou + de 5 catégories). Création de bins.

Range = [0.0000, 1866.0000]

Tableau des effectifs par intervalle :

Intervalle	Effectif
[0.0000, 373.2000]	1804659
[373.2000, 746.4000]	1
[746.4000, 1119.6000]	0
[1119.6000, 1492.8000]	40
[1492.8000, 1866.0000]	174

Aperçu des commentaires par bin :

>> Intervalle 1/5 : [0.0000, 373.2000]

```
+-----+
+-----+
+-----+
|identity_annotator_count|
comment_text|
+-----+
+-----+
+-----+
|                                0|This is so cool. It's like, 'would you want
your mother to read this??' Really great idea, well d...|
|                                0|Thank you!! This would make my life a lot
less anxiety-inducing. Keep it up, and don't let anyone...|
|                                0|This is such an urgent design
problem; kudos to you for taking it on. Very impressive!|
+-----+
+-----+
+-----+
only showing top 3 rows
```

>> Intervalle 2/5 : [373.2000, 746.4000]

```
+-----+
+-----+
+-----+
|identity_annotator_count|
comment_text|
```

```

+-----+
+-----+
+-----+
|           408|Taqiyya is the Islamic practice of
deception. It can take the form of concealing the full truth (...|
+-----+
+-----+
+-----+

```

>> Intervalle 3/5 : [746.4000, 1119.6000]

```

+-----+-----+
|identity_annotator_count|comment_text|
+-----+-----+
+-----+-----+

```

>> Intervalle 4/5 : [1119.6000, 1492.8000]

```

+-----+
+-----+
+-----+
|identity_annotator_count|
comment_text|
+-----+
+-----+
+-----+
|           1226|My understanding is that a PPS
employee/fundamentalist Christian complained because Frida is bise...|
|           1267|I really hope that this event will be able
to be back next year with the city trying to ban these...|
|           1257|With all due respect to Nick, how does he
know they are lesbians and not bisexual? And if they ha...|
+-----+
+-----+
+-----+
only showing top 3 rows

```

>> Intervalle 5/5 : [1492.8000, 1866.0000]

```

+-----+
+-----+
+-----+
|identity_annotator_count|
comment_text|
+-----+
+-----+

```

```

-----+
|           1840|beautiful story. Oregoncraig in an earlier
listing pondered that it was a "Christians and jews...|
|           1842|I will keep my Christian denotation to
myself. As far as it being up to me to decide who is Chr...|
|           1810|How about the fact that many in the Muslim
and Arab world hated us long before either of the Bush...|
+-----+
+-----+
-----+
only showing top 3 rows

```

```

explore_categorical(train, "rating", comment_col="comment_text")

```

```

>> Exploration de la colonne catégorielle : rating

```

```

Valeurs et effectifs :

```

```

+-----+-----+
|rating |count |
+-----+-----+
|approved|1684758|
|rejected|120116 |
+-----+-----+

```

```

[Stage 210:> (0
+ 1) / 1]

```

```

>> Aperçu des commentaires par catégorie

```

```

>> rating = 'approved'

```

```

+-----+
+-----+
-----+
| rating|
comment_text|
+-----+
+-----+
-----+
|approved|The ranchers seem motivated by mostly by greed; no one
should have the right to allow their anima...|
|approved|           It was a great show. Not a combo I'd of
expected to be good together but it was.|
|approved|
Wow, that sounds great.|

```

```
|approved|This is a great story. Man. I wonder if the person who
yelled "shut the fuck up!" at him ever hea...|
|approved| This
seems like a step in the right direction.|
+-----+
+-----+
+-----+
only showing top 5 rows

>> rating = 'rejected'
+-----+
+-----+
+-----+
| rating|
comment_text|
+-----+
+-----+
+-----+
|rejected|This is so cool. It's like, 'would you want your mother to
read this??' Really great idea, well d...|
|rejected|Thank you!! This would make my life a lot less anxiety-
inducing. Keep it up, and don't let anyone...|
|rejected| This is such an urgent design problem; kudos
to you for taking it on. Very impressive!|
|rejected| Is this something I'll be able to install on
my site? When will you be releasing it?|
|rejected|
haha you guys are a bunch of losers.|
+-----+
+-----+
+-----+
only showing top 5 rows
```

```
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.stat import Correlation
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

def heatmap_with_col(df, col_main, other_cols, method="pearson",
sample_fraction=0.1, seed=42):
    """
    Affiche une heatmap de corrélation entre col_main et d'autres
    colonnes.
```

```

"""
cols = [col_main] + other_cols

# Échantillonnage pour rapidité
sampled_df =
df.select(cols).dropna().sample(fraction=sample_fraction, seed=seed)

# Convertir en pandas
pdf = sampled_df.toPandas()

# Calculer la corrélation
corr_matrix = pdf.corr(method=method)

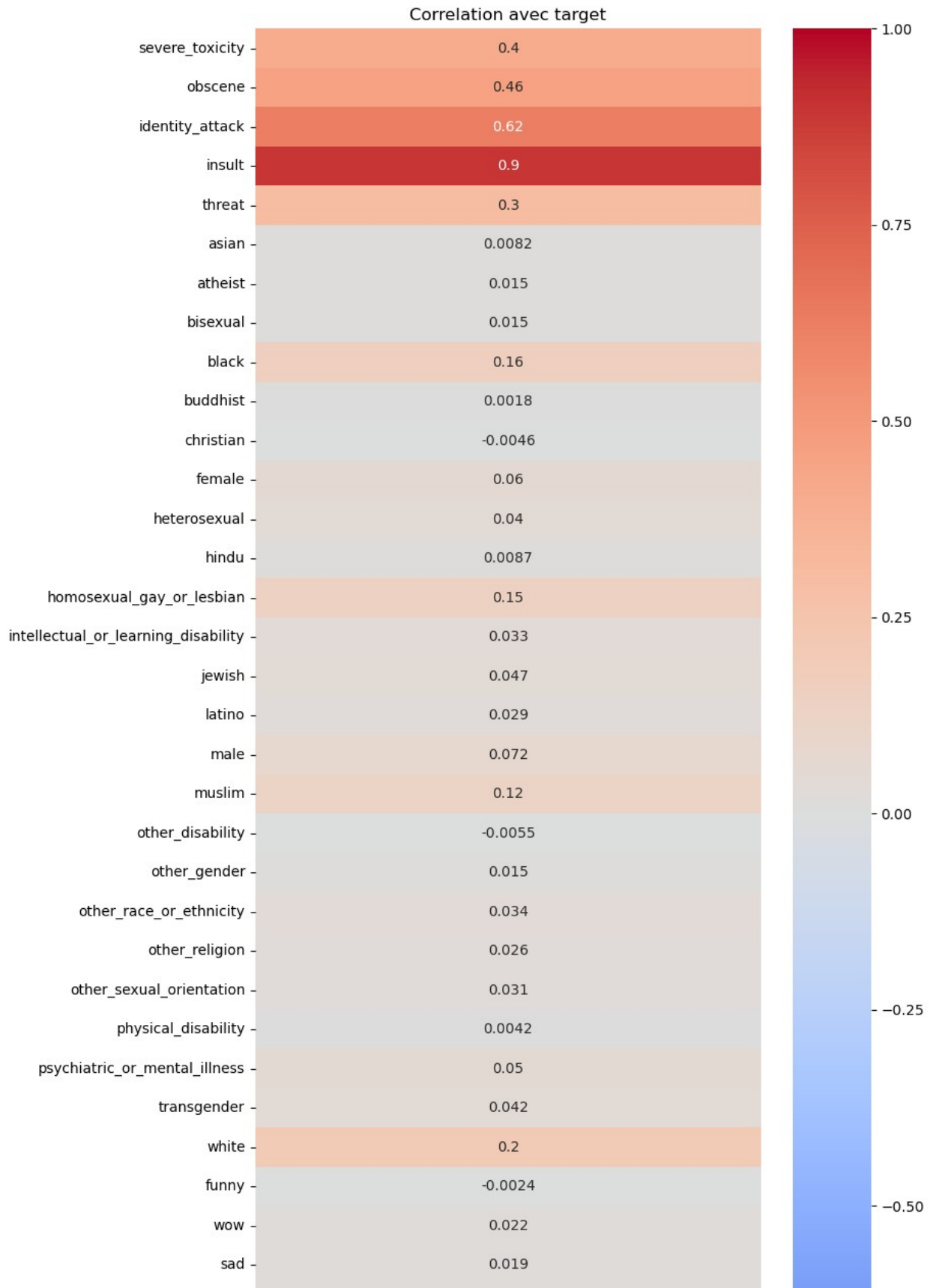
# Extraire uniquement la colonne principale vs les autres
corr_series = corr_matrix[col_main].drop(col_main)

# Affichage heatmap
plt.figure(figsize=(8, max(4, len(other_cols)*0.5)))
sns.heatmap(corr_series.to_frame(), annot=True, cmap="coolwarm",
vmin=-1, vmax=1)
plt.title(f"Correlation avec {col_main}")
plt.show()

numeric_cols = [
    "severe_toxicity", "obscene", "identity_attack", "insult",
    "threat",
    "asian", "atheist", "bisexual", "black", "buddhist", "christian",
    "female", "heterosexual", "hindu", "homosexual_gay_or_lesbian",
    "intellectual_or_learning_disability", "jewish", "latino", "male",
    "muslim", "other_disability", "other_gender",
    "other_race_or_ethnicity",
    "other_religion", "other_sexual_orientation",
    "physical_disability",
    "psychiatric_or_mental_illness", "transgender", "white",
    "funny", "wow", "sad", "likes", "disagree", "sexual_explicit",
    "identity_annotator_count", "toxicity_annotator_count",
    "publication_id", "parent_id", "article_id"
]

heatmap_with_col(train, "target", numeric_cols, sample_fraction=0.1)

```



```

import matplotlib.pyplot as plt
import numpy as np

def scatter_plot(df, col_x, col_y, sample_fraction=0.1, seed=42,
title=None, show_regression=True):
    """
    Affiche un scatter plot entre deux colonnes numériques et
    éventuellement la droite de régression.
    """
    # échantillonnage
    sampled_df = df.select(col_x,
col_y).dropna().sample(fraction=sample_fraction, seed=seed)

    # on passe en pandas
    pdf = sampled_df.toPandas()

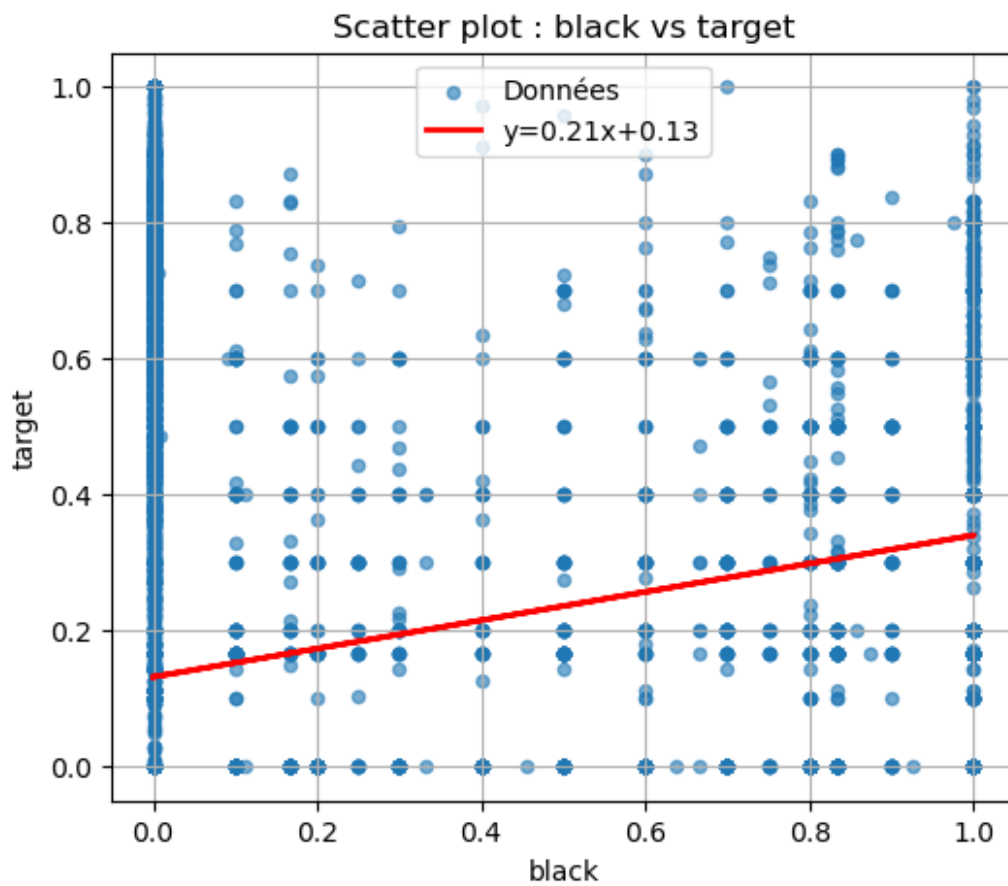
    plt.figure(figsize=(6,5))
    plt.scatter(pdf[col_x], pdf[col_y], alpha=0.6, s=20,
label="Données")

    # droite de regression
    if show_regression:
        x = pdf[col_x].values
        y = pdf[col_y].values
        if len(x) > 1:
            m, b = np.polyfit(x, y, 1) # régression linéaire  $y = m*x$ 
+ b
            plt.plot(x, m*x + b, color='red', lw=2,
label=f"y={m:.2f}x+{b:.2f}")

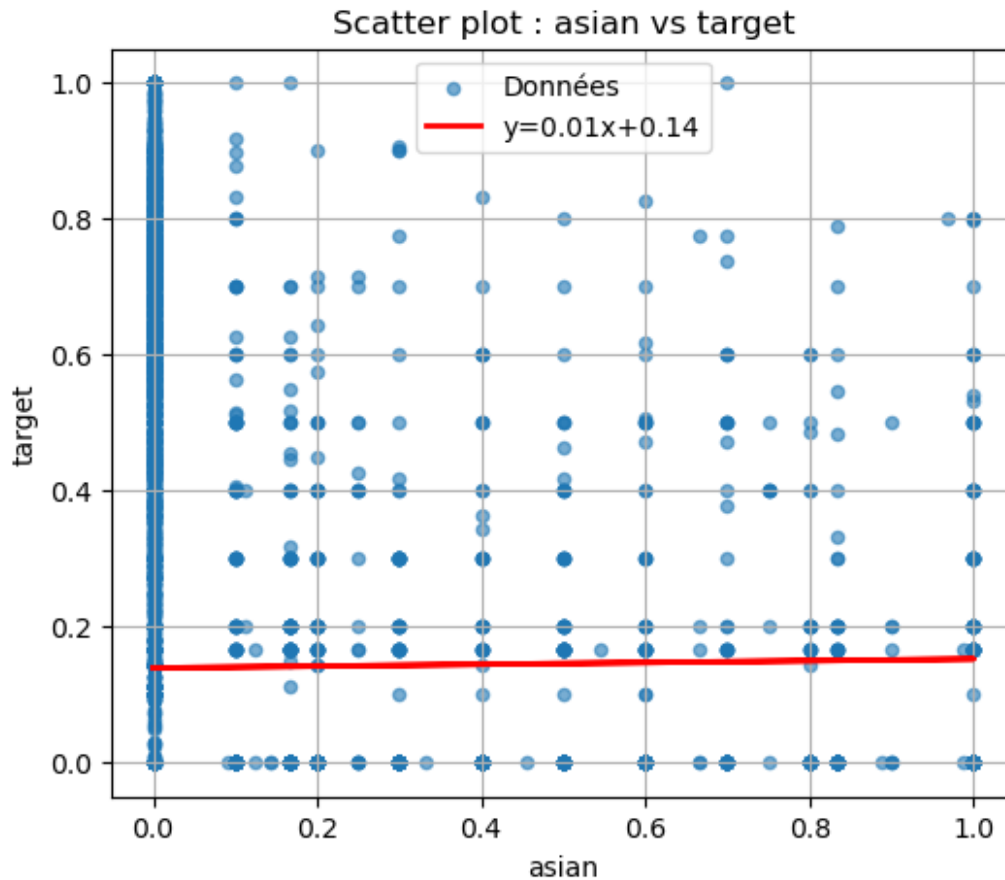
    plt.xlabel(col_x)
    plt.ylabel(col_y)
    if title:
        plt.title(title)
    else:
        plt.title(f"Scatter plot : {col_x} vs {col_y}")
    plt.grid(True)
    if show_regression:
        plt.legend()
    plt.show()

scatter_plot(train, "black", "target", sample_fraction=0.1)

```



```
scatter_plot(train, "asian", "target", sample_fraction=0.1)
```



L'analyse des distributions nous permet de mieux cerner nos variables.

Par exemple, la variable `severe_toxicity` est majoritairement proche de 0, avec seulement deux observations dépassant 0,6, bien que la variable soit bornée à 1. Cela révèle une distribution très déséquilibrée qui pourrait influencer l'apprentissage du modèle si elle n'est pas correctement prise en compte.

Un autre exemple est la feature `funny` (et toutes les autres features de réactions), bien qu'elle a un range de 0 à 102, la majorité de ces valeurs sont à 1. De plus, on peut questionner l'aspect subjectif d'une telle variable. Il serait judicieux de juxtaposer cette variable avec le nombre d'annotateurs sur un commentaire donné pour pouvoir juger de la confiance que l'on accorde à une telle feature.

Nous observons également que certaines features présentent des valeurs manquantes, notamment dans les colonnes d'identité ou `parent_id`. Pour éviter que ces absences n'introduisent du biais ou ne perturbent les algorithmes, il sera nécessaire d'imputer ou de remplacer ces valeurs lors du prétraitement.

Enfin, l'examen des corrélations révèle que certaines colonnes d'identité sont positivement corrélées avec la target. Bien que les annotations soient censées être correctement réalisées, cette observation peut indiquer soit un biais latent dans les données annotées.

1. Approche Classique Spark ML sur la colonne "comments"

Stratégie pour le prétraitement :

1. Prétraitement de la colonne "comments", cela passe par un nettoyage avec Spark SQL et regex, puis une tokenisation. On enlève ensuite les stop words puis on utilise un transformer SparkML personnalisé pour la lemmatisation
2. Pour la colonne "Target" : création d'une variable binaire label --> `when(col("target") >= 0.5, 1).otherwise(0)` --> pour le modèle de regression logistique
3. On s'occupe des features secondaires. Dans la partie précédente on avait identifié 3 types de features :
 - Les sous-labels de toxicité --> **float entre [0,1]**
 - Les colonnes d'identité --> **float entre [0,1]**
 - Les colonnes temporelles --> **timestamp**
 - Les features liées aux métadonnées --> **numériques mais catégorielles**
 - Colonnes liées aux réactions --> compteurs d'interactions, **int**
 - Nombre d'annotateurs sur un commentaire donné (mesure de la confiance des labels) --> **int**

1.1 Prétraitement de la colonne "comments"

--> utilisation d'une fonctions UDF afin que le traitement reste distribué sur les workers (pas d'utilisation de pandas sauf pour l'affichage d'un échantillon des données)

--> on fait tout le prétraitement dans spark

--> utilisation de Transformers spark ML pour l'encapsulation dans un pipeline --> structure robuste et modulaire

--> Encapsulation dans un pipeline --> structure robuste et modulaire

1.1.1 Définition d'un Transformer SPark ML pour la lemmatisation

Il n'y a pas d'objet Mlib natif dans Spark ML permettant de faire une lemmatisation. Pour pouvoir encapsuler cette étape dans le pipeline spark, nous ne pouvons pas utiliser une UDF python classique. Nous avons donc créé un Transformer Spark ML personnalisé "Lemmatizer", utilisant WordNetLemmatizer et compatible avec le pipeline Spark

```
from pyspark.ml import Transformer
from pyspark.ml.util import DefaultParamsReadable,
DefaultParamsWritable
from pyspark.sql.functions import udf
from pyspark.sql.types import ArrayType, StringType
```

```

lemm = WordNetLemmatizer()

class Lemmatizer(Transformer, DefaultParamsReadable,
DefaultParamsWritable):

    def __init__(self, inputCol="tokens", outputCol="lemmatized"):
        super(Lemmatizer, self).__init__()
        self.inputCol = inputCol
        self.outputCol = outputCol

    def _transform(self, df):
        lemm = WordNetLemmatizer()
        lemm_udf = udf(lambda tokens: [lemm.lemmatize(w) for w in
tokens], ArrayType(StringType()))
        return df.withColumn(self.outputCol,
lemm_udf(df[self.inputCol]))

```

1.1.2 Définition de la fonction Preprocess

Cette fonction permet de définir le pipeline de preprocessing. Les données de la colonnes commentaires passent par :

- regex_cleaner: met tout le texte en minuscules et supprime les caractères non alphabétiques
- tokenizer: découpe le texte nettoyé en mots individuels
- remover: retire les stopwords pour ne garder que les mots porteurs de sens
- lemmatizer: réduit les mots à leur forme de base afin d'obtenir des tokens normalisés

La colonne target est ensuite convertie en une étiquette binaire label pour la classification.

```

def preprocess(df, limit=None, debug=False):
    """
    Nettoyage (tout en minuscule + suppression de tous les caractères
    non alphabétiques) --> tokenisation --> stopwords --> lemmatisation →
    conversion de target en binaire
    Retourne un df contenant : tokens, target, label
    """
    # Pour choisir la taille de l'échantillon du dataset
    if limit:
        df = df.limit(limit)

    # Nettoyage du texte (Spark SQL + regex)
    regex_cleaner = SQLTransformer(
        statement="""
        SELECT *,
        regexp_replace(lower(comment_text), '^[^a-z ]', '') AS
clean_text
        FROM __THIS__
    """
    )

```

```

    """
)

tokenizer = Tokenizer(inputCol="clean_text", outputCol="words")
remover = StopWordsRemover(inputCol="words", outputCol="filtered")
lemmatizer = Lemmatizer(inputCol="filtered", outputCol="tokens")

# Pipeline de preprocessing
preprocess_pipeline = Pipeline(stages=[
    regex_cleaner,
    tokenizer,
    remover,
    lemmatizer
])

df_prep = preprocess_pipeline.fit(df).transform(df)

# Target --> binarisation
# Spark ML reglog = classif binaire --> on doit convertir la
# colonne target en binaire pour le modèle baseline, même si cela nous
# fait perdre de l'information
df_prep = df_prep.withColumn(
    "label",
    when(col("target") >= 0.5, 1).otherwise(0)
)

# Pour voir les différentes étapes du preprocessing
if (debug == True):

    print("\n>>> AFFICHAGE DES DIFFERENTES ETAPES DU
PREPROCESSING")

    print("\n# 1) regex_cleaner")
    df1_debug = regex_cleaner.transform(train)
    df1_debug.select("clean_text").show(5, truncate=False)

    print("\n# 2) tokenizer")
    df2_debug = tokenizer.transform(df1_debug)
    df2_debug.select("words").show(5, truncate=False)

    print("\n# 3) stopwords remover")
    df3_debug = remover.transform(df2_debug)
    df3_debug.select("filtered").show(5, truncate=False)

    print("\n# 4) lemmatizer (UDF)")
    df4_debug = lemmatizer.transform(df3_debug)
    df4_debug.select("tokens").show(5, truncate=False)

print("-----")

```

```
return df, df_prep
```

1.2 Prétraitement des colonnes secondaires

Les colonnes qui en ont encore besoin (year, réactions, annotateurs) seront standardisées par la suite lorsque nous ferons la vectorisation avec `vector assembler` et `standard scaler`, pour l'instant nous en sommes encore au modèle baseline et nous n'avons pas besoin de ces données. Nous entamerons la standardisation lorsque nous aurons une vision claire de la manière dont nous voudrions utiliser ces données

- Cette fonction commence par remplir les valeurs manquantes des colonnes d'identité avec des zéros pour normaliser les informations démographiques.
- Applique une transformation logarithmique aux colonnes de réactions (funny, wow, sad, likes, disagree) pour réduire l'influence des valeurs extrêmes.
- La colonne rating est convertie en binaire --> car elle ne contenait que 2 valeurs
- Les informations temporelles (created_date) sont extraites en année, mois, jour de la semaine et heure, puis codées en sin/cos --> pour l'aspect cyclique
- Les colonnes de comptage d'annotateurs (toxicity_annotator_count, identity_annotator_count) sont transformées avec log pour stabiliser la variance --> on garde aussi les colonnes d'origines pour voir par la suite ce qui marche le mieux
- Les colonnes ID (parent_id, article_id, publication_id) sont drop pour le moment--> risquent d'être mal interprétée par le modèle

```
from pyspark.sql.functions import (
    col, loglp, year, month, dayofweek, hour,
    sin, cos, when
)
from pyspark.ml import Pipeline
from pyspark.ml.feature import StringIndexer, OneHotEncoder

def preprocess_all(df, limit=None):
    """
    Prétraitement complet sans vectorisation,
    avec normalisation et suppression des colonnes ID.
    """

    if limit:
        df = df.limit(limit)

    # 1. Colonnes d'identité
    identity_cols = [
```

```

        "asian", "atheist", "bisexual", "black", "buddhist",
"christian",
        "female", "heterosexual", "hindu",
"homosexual_gay_or_lesbian",
        "intellectual_or_learning_disability", "jewish", "latino",
"male",
        "muslim", "other_disability", "other_gender",
"other_race_or_ethnicity",
        "other_religion", "other_sexual_orientation",
"physical_disability",
        "psychiatric_or_mental_illness", "transgender", "white"
    ]
    df = df.fillna({c: 0 for c in identity_cols})

    # 2. Colonnes de réactions (loglp)
    reaction_cols = ["funny", "wow", "sad", "likes", "disagree"]
    for c in reaction_cols:
        df = df.withColumn(f"{c}_log", loglp(col(c)))

    # 3. rating → decision
    df = df.withColumn("decision", when(col("rating") == "approved",
1).otherwise(0))
    df = df.drop("rating")

    # 4. Time features → sin/cos
    df = (
        df.withColumn("year", year("created_date"))
            .withColumn("month", month("created_date"))
            .withColumn("dow", dayofweek("created_date"))
            .withColumn("hour", hour("created_date"))
    )

    df = (
        df.withColumn("hour_sin", sin(2 * 3.14159 * col("hour")/24))
            .withColumn("hour_cos", cos(2 * 3.14159 * col("hour")/24))
            .withColumn("month_sin", sin(2 * 3.14159 * col("month")/12))
            .withColumn("month_cos", cos(2 * 3.14159 * col("month")/12))
            .withColumn("dow_sin", sin(2 * 3.14159 * col("dow")/7))
            .withColumn("dow_cos", cos(2 * 3.14159 * col("dow")/7))
    )

    # On drop les colonnes month/hour/dow/year pour éviter redondance
    df = df.drop("year", "month", "dow", "hour")

    # 5. Annotator counts --> on fait passer une des colonnes en log
    df = df.withColumn("toxicity_annotator_count_log",
loglp("toxicity_annotator_count"))
    df = df.withColumn("identity_annotator_count_log",
loglp("identity_annotator_count"))

```

```
# 6. Colonnes ID --> on supprime
df = df.drop("parent_id", "article_id", "publication_id")

return df
```

1.3 Vectorisation et Modélisation Baseline

1.3.1 Pipeline baseline : CountVectorizer + IDF + RegLog

On aurait aussi pu utiliser un hashing à la place de TF-IDF comme vu dans le cours

Dans cet fonction on construit un pipeline complet avec :

1. **CountVectorizer (CV)**

Les tokens issus du prétraitement sont transformés en sparse vectors

- vocabSize=100000 : on limite le vocabulaire aux 100 000 mots les plus fréquents.
- minDF=5 : seuls les mots présents dans au moins 5 commentaires sont conservés --> réduire le bruit.

2. **IDF (Inverse Document Frequency)**

Chaque vecteur brut est pondéré par l'importance des termes dans le corpus afin de mettre en avant les mots discriminants.

3. **Logistic Regression**

Les vecteurs TF-IDF sont utilisés pour entraîner une régression logistique:

- featuresCol="features" et labelCol="label" (version binaire de target)
- maxIter=50 pour limiter le nombre d'itérations de l'optimisation.
- regParam=0.0 : pas de régularisation appliquée pour le modèle baseline

```
from pyspark.ml.feature import CountVectorizer, IDF
from pyspark.ml.classification import LogisticRegression

def build_pipeline_baseline():
    """
    Pipeline ML complet : TF-IDF --> RegLog
    """
    # Ici on transforme les tokens en vecteurs numériques ; listes -->
    # sparse vector
    cv = CountVectorizer(
        inputCol="tokens",
        outputCol="raw_features",
        vocabSize=100000,
        minDF=5 # on garde que quand dans 5 commentaires ou +, pour
        # réduire le bruit
```

```

)

idf = IDF(inputCol="raw_features", outputCol="features")

lr = LogisticRegression(
    featuresCol="features",
    labelCol="label",
    maxIter=50,
    regParam=0.0
)

return Pipeline(stages=[cv, idf, lr])

```

1.5 Main Baseline

Les chemins PREPROCESSED_PATH, ALL_PREPROCESSED_PATH et MODEL_PATH désignent où sauvegarder ou charger les données prétraitées et le modèle. Les flags SAVE_PREPROCESSED et SAVE_MODEL permettent de sauvegarder le prétraitement et le modèle, tandis que LOAD_PREPROCESSED et LOAD_MODEL permettent de les charger depuis le disque au lieu de les recalculer ou réentraîner.

```

PREPROCESSED_PATH = "data/df_features_preprocessed.parquet"
BASE_MODEL_PATH = "models/"
ALL_PREPROCESSED_PATH = "data/df_features_df_all.parquet"

SAVE_PREPROCESSED = False # mettre True pour sauvegarder le
                           preprocessing dans un parquet
LOAD_PREPROCESSED = False # mettre True pour charger un fichier
                           existant

SAVE_MODEL = False # Pour la sauvegarde du modèle, de ses
                   hyperparamètres et de ses scores AUC
LOAD_MODEL = True

```

1.5.1 Prétraitement

Si le fichier prétraité existe et que LOAD_PREPROCESSED est True, les données sont chargées depuis le fichier parquet pour éviter de recalculer le prétraitement. Sinon, les données brutes sont prétraitées avec la fonction preproces, et si SAVE_PREPROCESSED est True, elles sont sauvegardées dans un fichier parquet. Cette étape garantit que df_clean contient toujours les colonnes tokens, target et label prêtes pour l'entraînement du modèle.

```

# Prétraitement (ou chargement)
if LOAD_PREPROCESSED and os.path.exists(PREPROCESSED_PATH):
    print(f"Chargement des données prétraitées depuis :
{PREPROCESSED_PATH}")
    df_clean = spark.read.parquet(PREPROCESSED_PATH)

```

```

else:
    print("Prétraitement...")
    train_tronq, df_preprocess_comments = preprocess(train,
limit=12500, debug=True)  # debug = True pour afficher toutes les
étapes
    df_clean = df_preprocess_comments.select("id", "tokens", "target",
"label")
    if SAVE_PREPROCESSED:
        df_clean.write.mode("overwrite").parquet(PREPROCESSED_PATH)

df_clean.show(5, truncate=False)

```

Prétraitement...

>>> AFFICHAGE DES DIFFERENTES ETAPES DU PREPROCESSING

1) regex_cleaner

```

+-----+
|clean_text|
|
+-----+
|this is so cool its like would you want your mother to read this
really great idea well done|
|thank you this would make my life a lot less anxietyinducing keep it
up and dont let anyone get in your way|
|this is such an urgent design problem kudos to you for taking it on
very impressive|
|is this something ill be able to install on my site when will you be
releasing it|
|haha you guys are a bunch of losers|
+-----+
only showing top 5 rows

```

2) tokenizer

```

+-----+
|words|
|
+-----+
|[this, is, so, cool, its, like, would, you, want, your, mother, to,
read, this, really, great, idea, well, done]|
|[thank, you, this, would, make, my, life, a, lot, less,
anxietyinducing, keep, it, up, and, dont, let, anyone, get, in, your,
way]|

```

```
|[this, is, such, an, urgent, design, problem, kudos, to, you, for,
taking, it, on, very, impressive]
|[is, this, something, ill, be, able, to, install, on, my, site, when,
will, you, be, releasing, it]
|[haha, you, guys, are, a, bunch, of, losers]
```

```
+-----+
-----+
only showing top 5 rows
```

3) stopwords remover

```
+-----+
-----+
|filtered
|
+-----+
-----+
|[cool, like, want, mother, read, really, great, idea, well, done]
|
|[thank, make, life, lot, less, anxietyinducing, keep, dont, let,
anyone, get, way]|
|[urgent, design, problem, kudos, taking, impressive]
|
|[something, ill, able, install, site, releasing]
|
|[haha, guys, bunch, losers]
```

```
+-----+
-----+
only showing top 5 rows
```

4) lemmatizer (UDF)

```
+-----+
-----+
|tokens
|
+-----+
-----+
|[cool, like, want, mother, read, really, great, idea, well, done]
|
|[thank, make, life, lot, less, anxietyinducing, keep, dont, let,
anyone, get, way]|
|[urgent, design, problem, kudos, taking, impressive]
|
|[something, ill, able, install, site, releasing]
|
|[haha, guy, bunch, loser]
```

```
|
+-----+
+-----+
only showing top 5 rows
+-----+

[Stage 20433:> (0
+ 1) / 1]

+-----+
+-----+
+-----+
|id   |tokens
|target      |label|
+-----+
+-----+
+-----+
|59848|[cool, like, want, mother, read, really, great, idea, well,
done]          |0.0          |0   |
|59849|[thank, make, life, lot, less, anxietyinducing, keep, dont,
let, anyone, get, way]|0.0          |0   |
|59852|[urgent, design, problem, kudos, taking, impressive]
|0.0          |0   |
|59855|[something, ill, able, install, site, releasing]
|0.0          |0   |
|59856|[haha, guy, bunch, loser]
|0.8936170212765957|1   |
+-----+
+-----+
+-----+
only showing top 5 rows
```

On applique la fonction `preprocess_all` sur le df prétraité des commentaires pour transformer et normaliser les autres colonnes (réactions, colonnes d'identité, features temporelles, etc.). Ensuite, on affiche un échantillon de 10 lignes pour certaines colonnes représentatives afin de vérifier les transformations

```
# Prétraitement du df complet
print("\n>>> Prétraitement du df complet :\n")

if LOAD_PREPROCESSED and os.path.exists(ALL_PREPROCESSED_PATH):
    print(f"Chargement du DataFrame complet prétraité depuis :
{ALL_PREPROCESSED_PATH}")
    df_all = spark.read.parquet(ALL_PREPROCESSED_PATH)
else:
    print("Prétraitement du DataFrame complet...")
```

```

df_all = preprocess_all(df_preprocess_comments)
if SAVE_PREPROCESSED:
    print(f"Sauvegarde du DataFrame complet -->
{ALL_PREPROCESSED_PATH}")
    df_all.write.mode("overwrite").parquet(ALL_PREPROCESSED_PATH)

print("\n>>> Prétraitement df_all OK")

cols_to_show = ["severe_toxicity", "obscene", "asian",
"female", "likes", "likes_log", "decision", "hour_sin", "month_sin"]

df_all.select(cols_to_show).show(10, truncate=20)

print(df_all.columns)

```

>>> Prétraitement du df complet :

Prétraitement du DataFrame complet...

>>> Prétraitement df_all OK

```

+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
|      severe_toxicity|      obscene|asian|female|likes|
likes_log|decision|      hour_sin|      month_sin|
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
|      0.0|      0.0| 0.0| 0.0| 0|
0.0| 0| 0.5000019150622543|-0.9999999999920783|
|      0.0|      0.0| 0.0| 0.0| 0|
0.0| 0| 0.5000019150622543|-0.9999999999920783|
|      0.0|      0.0| 0.0| 0.0| 0|
0.0| 0| 0.5000019150622543|-0.9999999999920783|
|      0.0|      0.0| 0.0| 0.0| 0|
0.0| 0| 0.5000019150622543|-0.9999999999920783|
|0.021276595744680847|      0.0| 0.0| 0.0| 1|
0.6931471805599453| 0| 0.5000019150622543|-0.9999999999920783|
|0.047619047619047616|0.6380952380952379| 0.0| 0.0| 0|
0.0| 0| 0.5000019150622543|-0.9999999999920783|
| 0.05084745762711865|0.3050847457627119| 0.0| 0.0| 0|
0.0| 0| 0.5000019150622543|-0.9999999999920783|
|      0.0|      0.0| 0.0| 0.0| 0|
0.0| 0| 0.5000019150622543|-0.9999999999920783|
|      0.0|      0.0| 0.0| 0.0| 0|
0.0| 1|-0.9999999999920783| 0.4999996169872557|
|      0.0|      0.0| 0.0| 0.0| 1|
0.6931471805599453| 1|-0.9999999999920783| 0.4999996169872557|
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
only showing top 10 rows

```

```
[ 'id', 'target', 'comment_text', 'severe_toxicity', 'obscene',
  'identity_attack', 'insult', 'threat', 'asian', 'atheist', 'bisexual',
  'black', 'buddhist', 'christian', 'female', 'heterosexual', 'hindu',
  'homosexual_gay_or_lesbian', 'intellectual_or_learning_disability',
  'jewish', 'latino', 'male', 'muslim', 'other_disability',
  'other_gender', 'other_race_or_ethnicity', 'other_religion',
  'other_sexual_orientation', 'physical_disability',
  'psychiatric_or_mental_illness', 'transgender', 'white',
  'created_date', 'funny', 'wow', 'sad', 'likes', 'disagree',
  'sexual_explicit', 'identity_annotator_count',
  'toxicity_annotator_count', 'clean_text', 'words', 'filtered',
  'tokens', 'label', 'funny_log', 'wow_log', 'sad_log', 'likes_log',
  'disagree_log', 'decision', 'hour_sin', 'hour_cos', 'month_sin',
  'month_cos', 'dow_sin', 'dow_cos', 'toxicity_annotator_count_log',
  'identity_annotator_count_log']
```

On divise ensuite le dataset en df_train et df_test :

- df_train sera utilisé pour la cross validation du modèle baseline et pour la nested CV des modèles plus complexes
- df_test sera utilisé pour tester les modèles pour obtenir différentes métriques

1.5.2 Split des données

```
from pyspark.sql import functions as F

# Pour obtenir un split précis
df_clean = df_clean.withColumn("rand", F.rand(seed=0))
train_df = df_clean.orderBy("rand").limit(int(0.8 * df_clean.count()))
test_df = df_clean.exceptAll(train_df)

print(f"Train = {train_df.count()} | Test = {test_df.count()}")

-----
-----
NameError                                Traceback (most recent call
last)
Cell In[7], line 4
      1 from pyspark.sql import functions as F
      3 # Pour obtenir un split précis
----> 4 df_clean = df_clean.withColumn("rand", F.rand(seed=0))
      5 train_df = df_clean.orderBy("rand").limit(int(0.8 *
df_clean.count()))
      6 test_df = df_clean.exceptAll(train_df)

NameError: name 'df_clean' is not defined
```

1.5.3 Modélisation Baseline avec Cross Validation

```
pipeline = build_pipeline_baseline()
print("\n>>> Pipeline OK")

# CV
evaluator = BinaryClassificationEvaluator(
    labelCol="label",
    rawPredictionCol="rawPrediction",
    metricName="areaUnderROC"
)

# Grid vide --> un seul modèle testé, baseline
param_grid = ParamGridBuilder().build()

cv = CrossValidator(
    estimator=pipeline,
    estimatorParamMaps=param_grid,
    evaluator=evaluator,
    numFolds=3, #le dataset est lourd, on laisse à 3
    parallelism=4
)

cv_model = cv.fit(train_df)

best_model = cv_model.bestModel

# Prédiction sur le test set
preds = best_model.transform(test_df)
preds.select("tokens", "target", "label", "probability",
             "prediction").show(truncate=40)

# Evaluation ROC-AUC
auc = evaluator.evaluate(preds)
print("Baseline ROC-AUC (CV + Test) = {auc:.4f}")

>>> Pipeline OK
```

```
+-----+-----+-----+
+-----+-----+-----+
|          tokens|          target|label|
probability|prediction|
+-----+-----+-----+
+-----+-----+-----+
|[applaud, civils, effort, create, new...|          0.0|    0|
|[0.9997592821439032,2.407178560968059...|    0.0|
|[bother, writing, review, going, devo...|          0.0|    0|
|[0.9999947756770127,5.224322987329444...|    0.0|
```

```

|                                     [youre, idiot]|      0.96875|      1|
[1.5056209790821627E-14,0.9999999999999999...]|      1.0|
|[think, , correct, debut, year, think...]|      0.0|      0|
[0.9999999943924422,5.607557751474701...]|      0.0|
|[wonder, excellent, colleague, kate, ...]|      0.0|      0|
[0.9999999999984717,1.528333015698990...]|      0.0|
|[tough, lose, carl, wolfsons, show, y...]|      0.0|      0|
[0.9999967669034493,3.233096550703251...]|      0.0|
|[spent, ton, time, outside, portland,...]|      0.0|      0|
[0.9999999999946474,5.352607246322805...]|      0.0|
|[best, interest, olcc, help, facilita...]|      0.0|      0|
[0.997602844836314,0.0023971551636859...]|      0.0|
|[hi, dr, know, worthycommercial, radi...]|      0.0|      0|
[0.9999999999999889,1.110223024625156...]|      0.0|
|[psu, like, university, sport, get, f...]|      0.0|      0|
[0.9995446349715168,4.553650284832011...]|      0.0|
|                                     [nigger]|0.8787878787878787|      1|
[0.9997466973500534,2.533026499466073...]|      0.0|
|[ok, , civil, site, seems, rememberin...]|      0.0|      0|
[0.7255183028621228,0.2744816971378772]|      0.0|
|[new, hampshire, us, first, communist...]|      0.0|      0|
[0.9994829718929771,5.17028107022921E-4]|      0.0|
|[klamath, county, commissioner, adorn...]|      0.0|      0|
[1.0,0.0]|      0.0|
|[say, im, little, sorry, see, christi...]|      0.0|      0|
[0.9999999999999607,3.930189507173054...]|      0.0|
|                                     [thanks, effort]|      0.0|      0|
[0.9998594880326216,1.405119673784094...]|      0.0|
|[think, best, possible, outcome, vote...]|      0.0|      0|
[0.9989758242687433,0.001024175731256...]|      0.0|
|[actually, know, raw, coal, oil, formed]|      0.0|      0|
[0.9999907800435689,9.219956431105736...]|      0.0|
|[le, able, end, without, firing, shot...]|      0.2|      0|
[0.9999997388391559,2.611608440616919...]|      0.0|
|[stay, tuned, announcement, wrongful,...]|      0.0|      0|
[0.9999999998484403,1.515596537160490...]|      0.0|
+-----+-----+
+-----+-----+
only showing top 20 rows

```

Baseline ROC-AUC (CV + Test) = {auc:.4f}

1.6 Optimisation des hyperparamètres

On ne peut pas faire une CV après avoir fait un GridSearchCV --> risque de surestimer le modèle, car on a choisi les hyperparamètres en ayant "vu" les données test.

La solution pour pouvoir utiliser GridSearchCV et évaluer de manière robuste le modèle --> faire une NESTED CV

- Nested CV = deux niveaux :
 - Boucle interne : CV + sélection d'hyperparamètres (GridSearchCV)
 - Boucle externe : CV indépendante pour évaluer le modèle choisi --> estimation non biaisée

1.6.1 Nested Cross-Validation

```
from pyspark.ml.evaluation import BinaryClassificationEvaluator
from pyspark.ml.tuning import CrossValidator

def nested_cv(model, grid, df, k_outer=3, k_inner=3, combinaison=None,
k_folds=None):
    evaluator = BinaryClassificationEvaluator(
        labelCol="label",
        rawPredictionCol="rawPrediction",
        metricName="areaUnderROC"
    )

    print(f" - Taille du jeu de données en entrée : {df.count()}")
    print(f" - Outer folds : {k_outer}")
    print(f" - Inner folds : {k_inner}")
    print(f" - Hyperparamètres testés : {len(grid)}\n")

    folds = df.randomSplit([1.0/k_outer]*k_outer, seed=42)
    outer_scores = []
    best_model = None
    best_params = None

    for i in range(k_outer):
        print("\n=====")
        print(f"OUTER FOLD {i+1}/{k_outer}")
        print("=====")

        test_outer = folds[i]
        train_outer = df.subtract(test_outer)

        print(f"Taille train_outer = {train_outer.count()}")
        print(f"Taille test_outer = {test_outer.count()}")

        # Build pipeline
        print("\nConstruction du pipeline...")
        if (combinaison == None):
            pipeline = build_pipeline(model)
        else:
            pipeline = build_pipeline_part2(model,
            combinaison=combinaison)

        print("\nLancement Inner Cross-Validation...")
```

```

print(f"    -> {len(grid)} combinaisons d'hyperparamètres")
print(f"    -> {k_inner} folds par combinaison")
print(f"    -> Total estimé : {len(grid) * k_inner}
entraînements")

# Inner CV
inner_cv = CrossValidator(
    estimator=pipeline,
    estimatorParamMaps=grid,
    evaluator=evaluator,
    numFolds=k_inner,
    parallelism=4
)

cv_model = inner_cv.fit(train_outer)

# Outer evaluation
preds = cv_model.transform(test_outer)
auc = evaluator.evaluate(preds)

print(f"AUC outer fold {i+1}/{k_outer} = {auc:.4f}")
outer_scores.append(auc)

# Update best model
current_best = max(outer_scores)
if auc == current_best:
    print("Nouveau meilleur modèle trouvé.")
    best_model = cv_model.bestModel
    best_params = cv_model.bestModel.stages[-
1].extractParamMap() # récupère les params du modèle final
else:
    print("Modèle moins performant que le meilleur actuel.")

print(f"AUC par fold : {outer_scores}")
print(f"AUC moyenne : {sum(outer_scores)/len(outer_scores):.4f}")

return best_model, best_params, outer_scores

```

1.6.2 Cross Validator

Nested CV est le mieux mais demande beaucoup de calculs, on peut utiliser CrossValidator pour plus de rapidité

```

from pyspark.ml.evaluation import BinaryClassificationEvaluator
from pyspark.ml.tuning import CrossValidator
import numpy as np

def simple_cross_validator(model, grid, df, k_folds=3,
    combinaison=None):
    """

```

```
Effectue une Grid Search simple avec Cross Validation standard.  
Retourne le meilleur modèle ré-entraîné sur l'ensemble du dataset  
'df'.  
"""
```

```
# 1. Sécuriser les données en mémoire pour éviter les crashes  
(Broken Pipe)
```

```
df.cache()  
count = df.count() # Force la matérialisation en cache
```

```
print("\nDémarrage Simple Cross-Validation...")  
print(f" - Taille du jeu de données : {count}")  
print(f" - Nombre de folds (K)      : {k_folds}")  
print(f" - Hyperparamètres testés : {len(grid)}")  
print(f" - Total entraînements      : {len(grid) * k_folds}")
```

```
# 2. Définir l'évaluateur
```

```
evaluator = BinaryClassificationEvaluator(  
    labelCol="label",  
    rawPredictionCol="rawPrediction",  
    metricName="areaUnderROC"  
)
```

```
# 3. Construire le pipeline
```

```
print("\nConstruction du pipeline...")  
if (combinaison == None):  
    pipeline = build_pipeline(model)  
else:  
    pipeline = build_pipeline_part2(model,  
    combinaison=combinaison)
```

```
# 4. Configurer le CrossValidator
```

```
cv = CrossValidator(  
    estimator=pipeline,  
    estimatorParamMaps=grid,  
    evaluator=evaluator,  
    numFolds=k_folds,  
    parallelism=2,  
    seed=0  
)
```

```
# 5. Lancer l'entraînement
```

```
print("\nLancement du CrossValidator (Training)...")  
cv_model = cv.fit(df)  
print("Recherche terminée.")
```

```
# 6. Récupérer les résultats
```

```
best_model = cv_model.bestModel  
best_score = max(cv_model.avgMetrics)
```

```

    # Récupérer les paramètres du meilleur modèle (dernier stage du
    pipeline = le modèle)
    best_params = best_model.stages[-1].extractParamMap()

    print(f"Meilleur AUC moyen (Validation) : {best_score:.4f}")

    # Nettoyage de la mémoire
    df.unpersist()

    return best_model, best_params, best_score

```

1.7 Système de sauvegarde/chargement

1.7.1 Sauvegarde des models + hyperparamètres + scores

```

def save_model(model, model_name):

    # Création si nécessaire
    os.makedirs(BASE_MODEL_PATH, exist_ok=True)

    # Chemin final du modèle
    model_path = os.path.join(BASE_MODEL_PATH, model_name)

    # Sauvegarde
    model.write().overwrite().save(model_path)
    print(f"Modèle sauvegardé dans : {model_path}")

import os
import json
from pyspark.ml.param import Param

def convert_param_map(param_map):

    clean_dict = {}

    if isinstance(param_map, dict):
        # dict avec des clés Param ou autres
        for k, v in param_map.items():
            # si la clé a un attribut 'name', c'est un Param
            key = k.name if isinstance(k, Param) else str(k)
            clean_dict[key] = v
    else:
        # Cas ParamMap Spark
        for param, value in param_map.items():
            key = param.name if isinstance(param, Param) else
str(param)
            clean_dict[key] = value

    return clean_dict

```

```

def save_best_params(params, model_name):
    file_path = os.path.join(BASE_MODEL_PATH,
f"{model_name}_params.json")
    clean_params = convert_param_map(params)

    with open(file_path, "w") as f:
        json.dump(clean_params, f, indent=4)

    print(f"Hyperparamètres sauvegardés dans : {file_path}")

def save_scores(scores, model_name):
    # Si scores est déjà un dict avec une clé "scores", on prend la
liste
    if isinstance(scores, dict) and "scores" in scores:
        while isinstance(scores, dict) and "scores" in scores:
            scores = scores["scores"]

    file_path = os.path.join(BASE_MODEL_PATH,
f"{model_name}_scores.json")

    with open(file_path, "w") as f:
        json.dump({"scores": scores}, f, indent=4)

    print(f"Scores sauvegardés dans : {file_path}")

```

1.7.2 Chargement des modèles + hyperparamètres + scores

```

from pyspark.ml import PipelineModel
from pyspark.ml.feature import Word2VecModel
import os

def load_model(model_name, model_type="pipeline"):
    path = os.path.join(BASE_MODEL_PATH, model_name)

    if not os.path.exists(path):
        raise FileNotFoundError(f"Model not found at: {path}")

    if model_type == "pipeline":
        model = PipelineModel.load(path)
    elif model_type == "word2vec":
        model = Word2VecModel.load(path)
    else:
        raise ValueError("model_type doit être 'pipeline' ou 'word2vec'")

```

```

    print(f"Modèle {model_type} chargé depuis : {path}")
    return model

def load_best_params(model_name):
    file_path = os.path.join(BASE_MODEL_PATH,
                              f"{model_name}_params.json")

    if not os.path.exists(file_path):
        raise FileNotFoundError(f"Fichier introuvable : {file_path}")

    with open(file_path, "r") as f:
        params = json.load(f)

    print(f"Hyperparamètres chargés depuis : {file_path}")
    return params

def load_scores(model_name):
    file_path = os.path.join(BASE_MODEL_PATH,
                              f"{model_name}_scores.json")

    if not os.path.exists(file_path):
        print(f"Aucun score trouvé pour {model_name}")
        return None

    with open(file_path, "r") as f:
        data = json.load(f)

    # Déplier si nécessaire
    scores = data
    while isinstance(scores, dict) and "scores" in scores:
        scores = scores["scores"]

    print(f"Scores chargés depuis : {file_path}")
    return scores

```

1.8 Vectorisation et Modélisation de modèles plus complexes

1.8.1 TF-IDF

La fonction `build_pipeline(model)` construit un pipeline de traitement de texte utilisant `CountVectorizer` + `IDF` pour produire des vecteurs TF-IDF, puis appliquer un modèle de classification ou de régression fourni en argument.

Le `CountVectorizer` parcourt tous les commentaires et construit un vocabulaire de 100 000 mots max (des mots qui reviennent dans plus de 5 commentaires) + convertit chaque commentaire en un vecteur de fréquences. = Term Frequency (TF) = un simple comptage

L'IDF (Inverse Document Frequency) sert à réduire l'importance des mots trop fréquents et à augmenter celle des mots rares mais utiles. --> intuition mot rares utiles

Les mots associés aux insultes, menaces ou discriminations obtiennent un poids plus fort, ce qui aide le modèle à détecter la toxicité.

```
def build_pipeline(model):
    cv = CountVectorizer(
        inputCol="tokens",
        outputCol="raw_features",
        vocabSize=100000,
        minDF=5
    )

    idf = IDF(
        inputCol="raw_features",
        outputCol="features"
    )

    return Pipeline(stages=[cv, idf, model])

# scaler = StandardScaler(
#     inputCol="tfidf",
#     outputCol="features",
#     withMean=False,    # nécessaire pour vecteurs clairsemés
#     withStd=True
# )

# model.setParams(featuresCol="features")

# return Pipeline(stages=[cv, idf, scaler, model])

# pour ajouter une normalisation
```

1.8.2 Définition des modèles + grid d'hyperparamètres

Nous utilisons LogisticRegression, RandomForest, LinearSVC (support vector machine), GBT (gradient booting trees)

```
lr = LogisticRegression(featuresCol="features", labelCol="label",
maxIter=50)
lr_grid = ParamGridBuilder().addGrid(lr.regParam, [0.0, 0.01,
0.1]).build()

rf = RandomForestClassifier(featuresCol="features", labelCol="label",
seed=0)

rf_grid = (ParamGridBuilder()
    .addGrid(rf.numTrees, [50, 100])
```

```

        .addGrid(rf.maxDepth, [5, 10, 15])
        .build())

svm = LinearSVC(featuresCol="features", labelCol="label", maxIter=50)

svm_grid = (ParamGridBuilder()
            .addGrid(svm.regParam, [0.1, 0.01, 0.001])
            .build())

gbt = GBTClassifier(featuresCol="features", labelCol="label", seed=0)

gbt_grid = (ParamGridBuilder()
            .addGrid(gbt.maxIter, [50, 100])
            .addGrid(gbt.maxDepth, [3, 5])
            .addGrid(gbt.stepSize, [0.05, 0.1])
            .build())

```

1.8.3 Définition de la logique train/load/save

```

def
train_or_load_nested_model(estimator, grid, train_df, n_rows, LOAD_MODEL,
SAVE_MODEL, prefixe=None, training_func=nested_cv, combinaison=None,
k_folds=3):

    func_name = training_func.__name__

    if prefixe is not None:
        model_name = f"{prefixe}_{estimator.__class__.__name__}
{n_rows}"
    else:
        model_name = f"{estimator.__class__.__name__}{n_rows}"
        print(f"model name: {model_name}")

    if LOAD_MODEL and os.path.exists(os.path.join(BASE_MODEL_PATH,
f"{model_name}_scores.json")):
        best_model = load_model(model_name)
        best_params = load_best_params(model_name)
        scores = load_scores(model_name)

    else :
        print(f"Entraînement du modèle : {model_name} avec la fonction
{func_name}")
        if (combinaison == None):
            best_model, best_params, scores = training_func(estimator,
grid, train_df)
        else:
            best_model, best_params, scores = training_func(estimator,
grid, train_df, k_folds=k_folds, combinaison=combinaison)

```

```

if SAVE_MODEL:
    save_model(best_model, model_name)
    save_best_params(best_params, model_name)
    save_scores(scores, model_name)

return model_name, best_model, best_params, scores

```

1.8.4 Entraînement des modèles + recherche des meilleurs hyperparamètres

```

model_name, best_lr, best_lr_params, lr_scores =
train_or_load_nested_model(lr, lr_grid, train_df, train_df.count(),
LOAD_MODEL, SAVE_MODEL)

```

```

model name: LogisticRegression10000
Modèle pipeline chargé depuis : models/LogisticRegression10000
Hyperparamètres chargés depuis :
models/LogisticRegression10000_params.json
Scores chargés depuis : models/LogisticRegression10000_scores.json

```

```

model_name_rf, best_rf, best_rf_params, rf_scores =
train_or_load_nested_model(rf, rf_grid, train_df, train_df.count(),
LOAD_MODEL, SAVE_MODEL)

```

```

model name: RandomForestClassifier10000
Modèle pipeline chargé depuis : models/RandomForestClassifier10000
Hyperparamètres chargés depuis :
models/RandomForestClassifier10000_params.json
Scores chargés depuis : models/RandomForestClassifier10000_scores.json

```

```

model_name_svm, best_svm, best_svm_params, svm_scores =
train_or_load_nested_model(svm, svm_grid, train_df, train_df.count(),
LOAD_MODEL, SAVE_MODEL)

```

```

model name: LinearSVC10000
Modèle pipeline chargé depuis : models/LinearSVC10000
Hyperparamètres chargés depuis : models/LinearSVC10000_params.json
Scores chargés depuis : models/LinearSVC10000_scores.json

```

```

model_name_gbt, best_gbt, best_gbt_params, gbt_scores =
train_or_load_nested_model(gbt, gbt_grid, train_df, train_df.count(),
LOAD_MODEL, SAVE_MODEL)

```

```

model name: GBTCClassifier10000
Modèle pipeline chargé depuis : models/GBTCClassifier10000

```

Hyperparamètres chargés depuis : models/GBTClassifier10000_params.json
Scores chargés depuis : models/GBTClassifier10000_scores.json

1.9 Visualisations et évaluations

1.9.1 Visualisations des résultats de la nested CV

```
import matplotlib.pyplot as plt

def visualisation_AUC_nestedCV(models_scores, df_train):
    """
    models_scores : dict
        clé = alias du modèle ("lr", "rf", "svm", "gbt")
        valeur = liste de scores OU dict contenant "scores"
    df_train : DataFrame PySpark
    """

    all_scores = []
    labels = []

    # Nombre de lignes (PySpark)
    n = df_train.count()

    for alias, scores in models_scores.items():

        # Si scores est un dict
        if isinstance(scores, dict):
            scores = scores.get("scores", [])

        # Conversion float
        scores = [float(s) for s in scores]

        all_scores.append(scores)

        label = f"{alias}{n}"
        labels.append(label)

        print(f"AUC moyenne : {sum(scores)/len(scores):.4f}")
        print(f"AUC par fold : {scores}")

    plt.boxplot(all_scores, labels=labels)
    plt.title("Distribution des AUC (Nested CV)")
    plt.ylabel("AUC")
    plt.show()

visualisation_AUC_nestedCV({
    "lr": lr_scores,
    "rf": rf_scores,
    "svm": svm_scores,
    "gbt": gbt_scores
})
```

```
}, train_df)

# A REPARER

print(lr_scores)
print(rf_scores)
print(svm_scores)
print(gbt_scores)

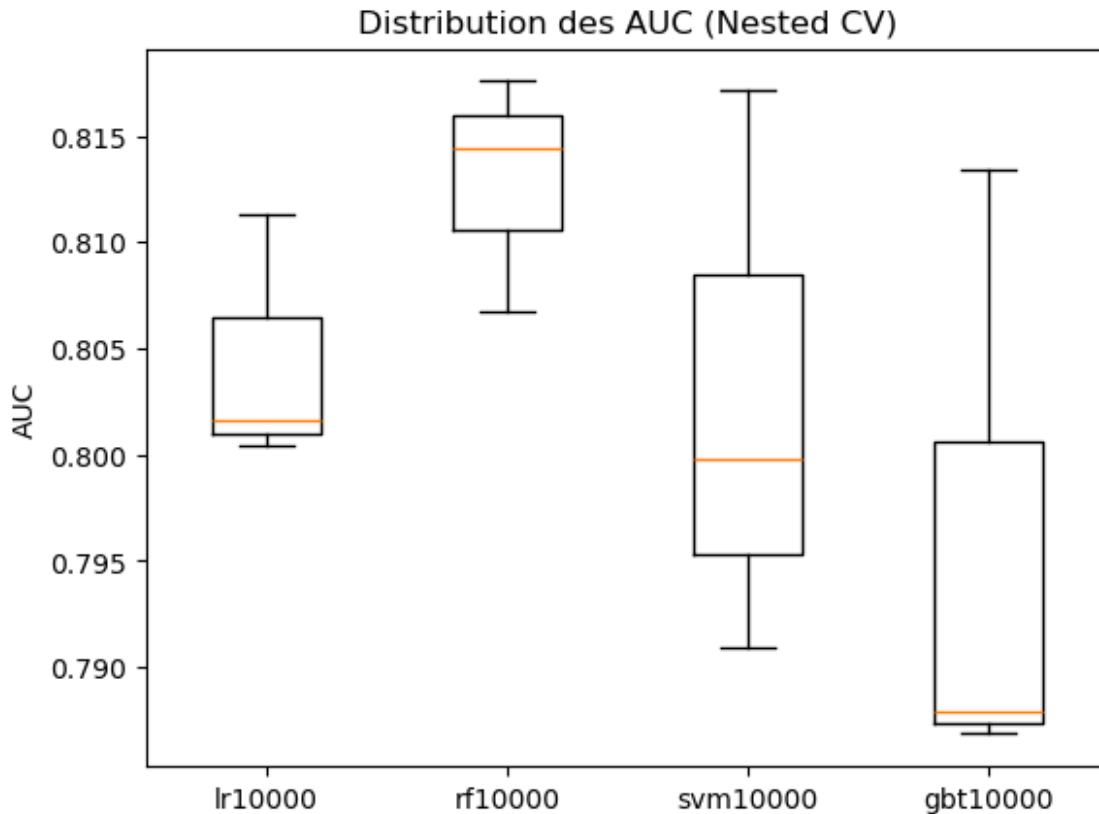
lr (lr10000)
AUC moyenne : 0.8044
AUC par fold : [0.8015740740740699, 0.8003861744741054,
0.8113205511288202]

rf (rf10000)
AUC moyenne : 0.8129
AUC par fold : [0.8143926411290335, 0.8067528100494942,
0.8175697211155383]

svm (svm10000)
AUC moyenne : 0.8026
AUC par fold : [0.7908761760752776, 0.799743607802357,
0.8171364541832745]

gbt (gbt10000)
AUC moyenne : 0.7960
AUC par fold : [0.8133988575268816, 0.7868520702433702,
0.7878627158034526]

/tmp/ipykernel_48992/1674374645.py:37: MatplotlibDeprecationWarning:
The 'labels' parameter of boxplot() has been renamed 'tick_labels'
since Matplotlib 3.9; support for the old name will be dropped in
3.11.
plt.boxplot(all_scores, labels=labels)
```



```
[0.8015740740740699, 0.8003861744741054, 0.8113205511288202]
[0.8143926411290335, 0.8067528100494942, 0.8175697211155383]
[0.7908761760752776, 0.799743607802357, 0.8171364541832745]
[0.8133988575268816, 0.7868520702433702, 0.7878627158034526]
```

1.9.2 Evaluation du modèle sur un jeu de données test

```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import (
    confusion_matrix, roc_curve, auc,
    accuracy_score, precision_score, recall_score, f1_score
)
import pandas as pd

def test_models(models, model_names=None, test_df=None):
    """
    Évalue n modèles Spark sur test_df et retourne un DataFrame de
    métriques.
    model_names est optionnel.
    """
    if model_names is not None and len(models) != len(model_names):
        raise ValueError(
            f"Nombre de modèles ({len(models)}) différent du nombre de
```

```

noms ({len(model_names)})"
    )

    # Si aucun nom fourni → utiliser le nom de classe Spark
    if model_names is None:
        model_names = [m.__class__.__name__ for m in models]

    results = []

    for model, name in zip(models, model_names):
        print(f"\n=== Évaluation du modèle : {name} ===")

        preds = model.transform(test_df)

        cols = ["label", "prediction"]
        has_prob = "probability" in preds.columns

        if has_prob:
            cols.append("probability")
        else:
            cols.append("rawPrediction")

        pdf = preds.select(*cols).toPandas()

        y_true = pdf["label"].astype(int)
        y_pred = pdf["prediction"].astype(int)

        if has_prob:
            y_score = pdf["probability"].apply(lambda x: float(x[1]))
        else:
            y_score = pdf["rawPrediction"].apply(lambda x:
float(x[1]))

        # Métriques
        acc = accuracy_score(y_true, y_pred)
        prec = precision_score(y_true, y_pred, zero_division=0)
        rec = recall_score(y_true, y_pred, zero_division=0)
        f1 = f1_score(y_true, y_pred, zero_division=0)

        fpr, tpr, _ = roc_curve(y_true, y_score)
        roc_auc = auc(fpr, tpr)

        results.append({
            "model": name,
            "accuracy": acc,
            "precision": prec,
            "recall": rec,
            "f1_score": f1,
            "roc_auc": roc_auc
        })

```

```

# Graphiques
plt.figure(figsize=(5,4))
plt.plot(fpr, tpr, label=f"AUC = {roc_auc:.3f}")
plt.plot([0, 1], [0, 1], linestyle="--")
plt.title(f"ROC - {name}")
plt.xlabel("FPR")
plt.ylabel("TPR")
plt.legend()
plt.show()

cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(4,4))
sns.heatmap(cm, annot=True, fmt='d', cmap="Blues")
plt.title(f"Matrice de confusion - {name}")
plt.xlabel("Prédiction")
plt.ylabel("Réel")
plt.show()

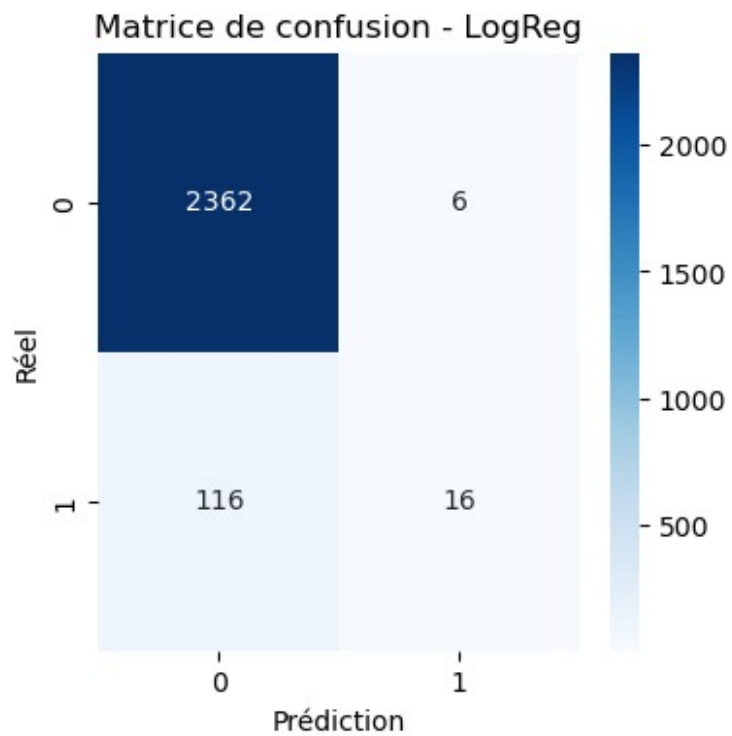
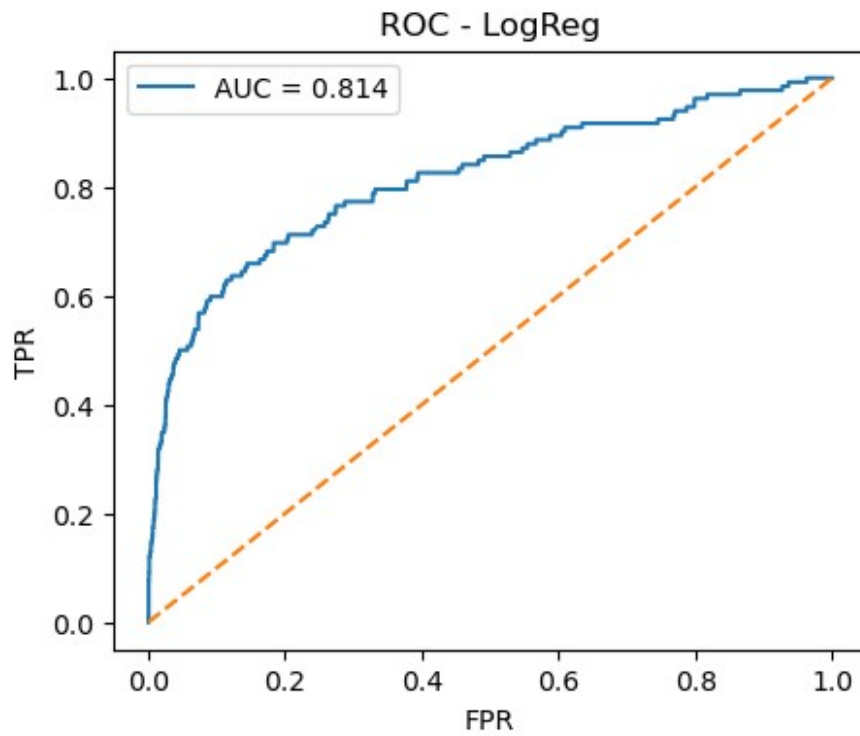
return pd.DataFrame(results)

models = [best_lr, best_rf, best_svm, best_gbt]
model_names = ["LogReg", "RandomForest", "LinearSVM", "GBT"]

metrics_df = test_models(models, model_names, test_df)

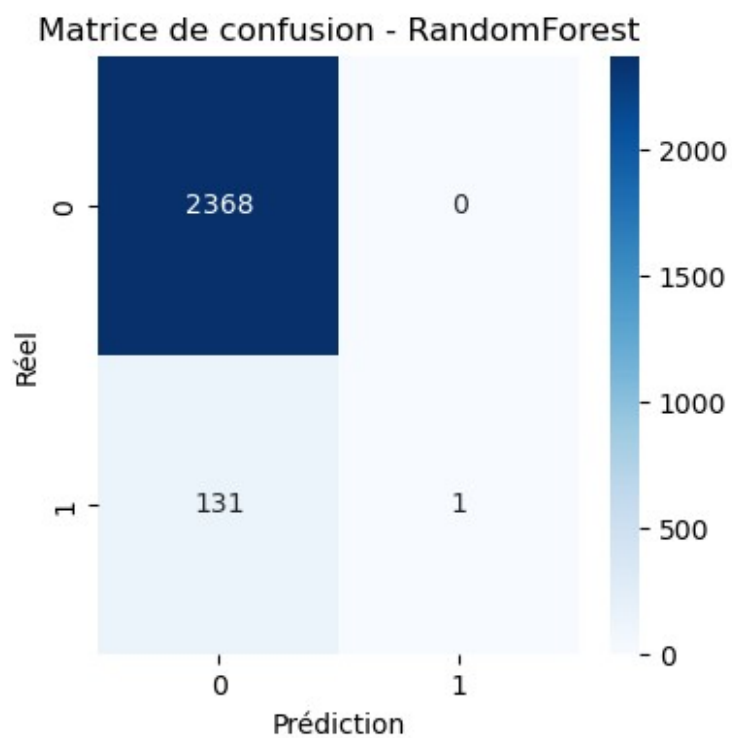
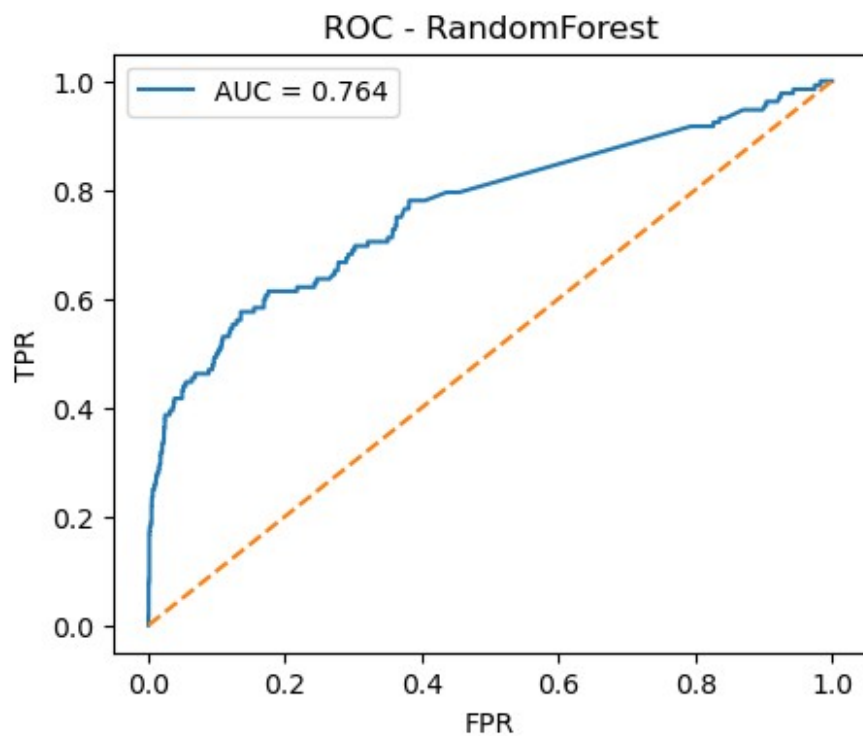
=== Évaluation du modèle : LogReg ===

```

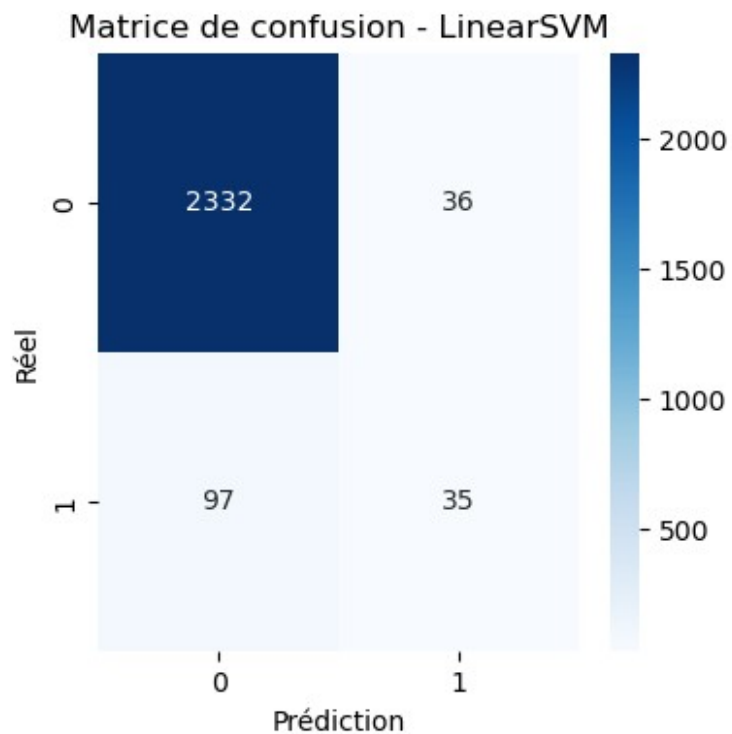
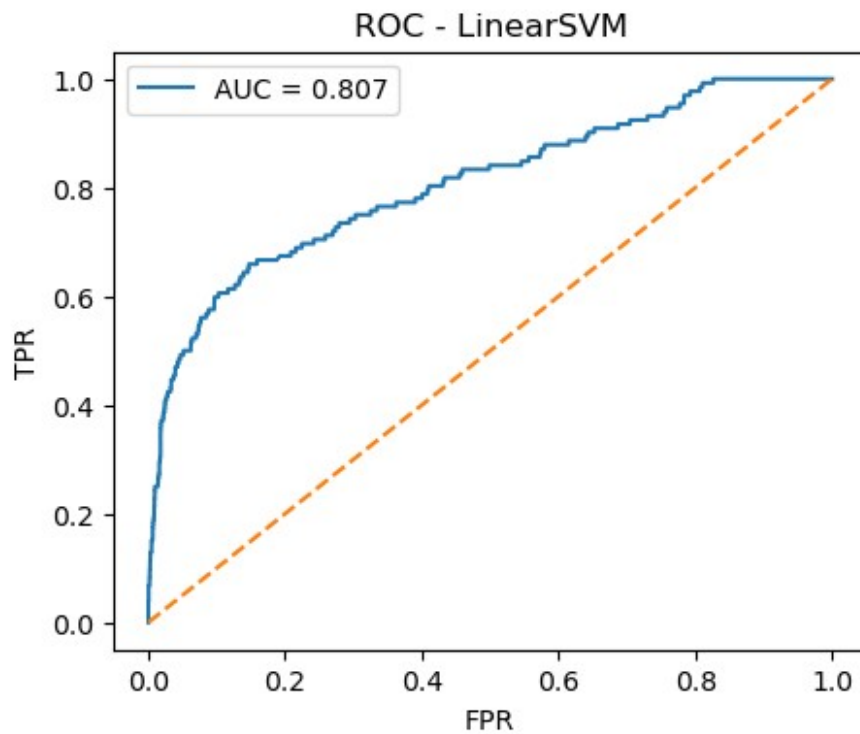


===   valuation du mod  le : RandomForest ===

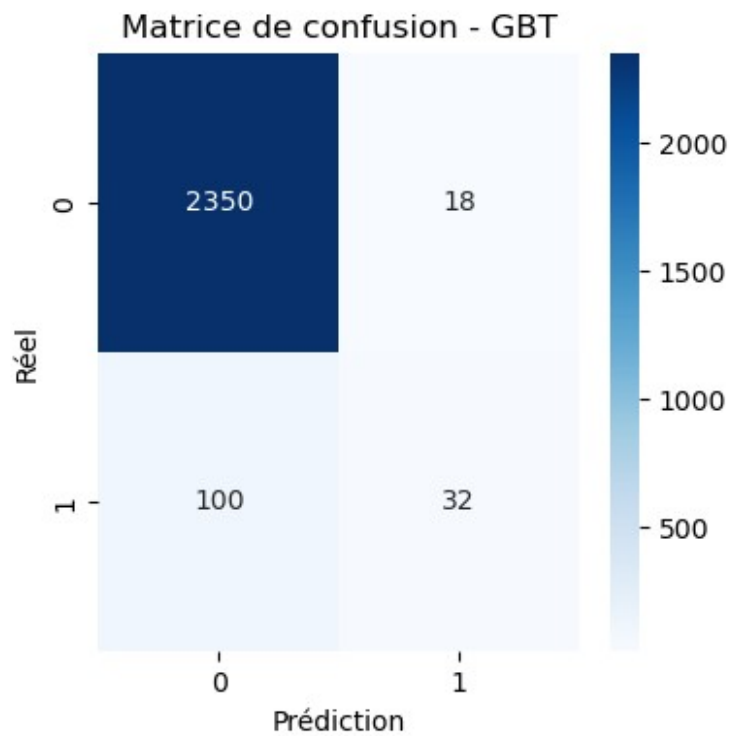
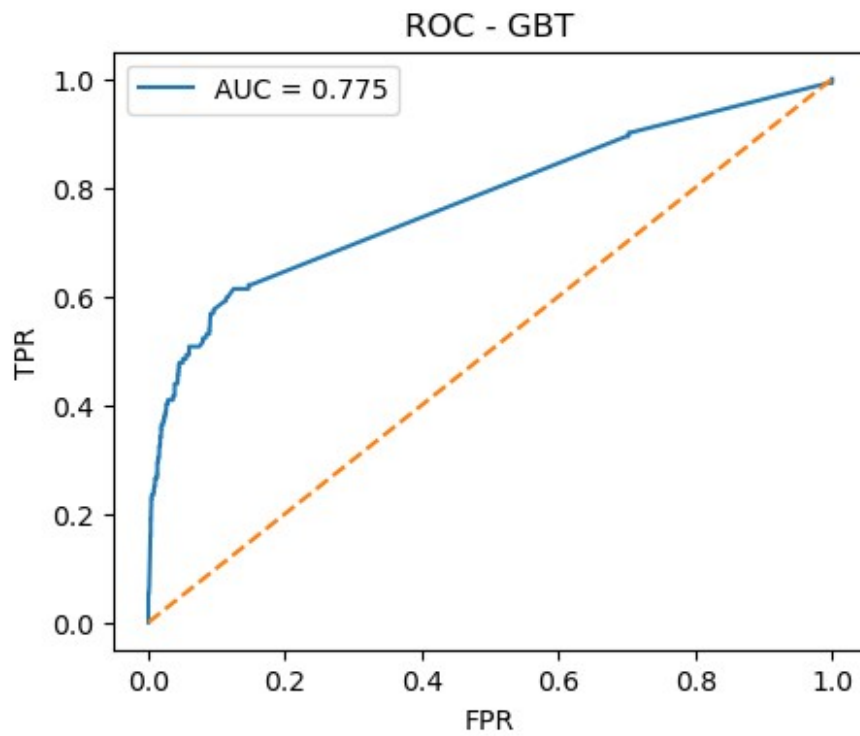
25/12/06 06:18:51 WARN DAGScheduler: Broadcasting large task binary with size 1070.9 KiB



=== Évaluation du modèle : LinearSVM ===



=== Évaluation du modèle : GBT ===



```
print(metrics_df)
```

	model	accuracy	precision	recall	f1_score	roc_auc
0	LogReg	0.9512	0.727273	0.121212	0.207792	0.814381
1	RandomForest	0.9476	1.000000	0.007576	0.015038	0.764459
2	LinearSVM	0.9468	0.492958	0.265152	0.344828	0.807157
3	GBT	0.9528	0.640000	0.242424	0.351648	0.774826

Les accuracies sont toutes très proches (entre 0.946 et 0.953). Cela indique :

- L'accuracy n'est pas un bon indicateur ici : un modèle qui prédit presque toujours la classe majoritaire peut avoir une excellente accuracy (étant donné le fort déséquilibre des classes)
- On s'appuie donc principalement sur recall, precision, f1 et AUC.

LogReg

- Très peu de positifs détectés (recall très bas).
- Quand elle prédit positif, elle a plutôt raison (precision élevée).
- Modèle conservateur, prédictions positives rares.
- AUC très bonne : le modèle sépare bien les classes, mais le seuil 0.5 n'est pas optimal.

Conclusion : bon potentiel, mais nécessite optimisation du seuil ou ajustement des poids de classes.

RandomForest

- Le modèle ne détecte quasiment aucun positif.
- Précision parfaite uniquement parce qu'il a dû prédire très peu de positifs.
- Typique d'un modèle qui s'effondre sur la classe minoritaire.
- AUC correcte, mais inutilisable en l'état.

Conclusion : modélisation trop biaisée vers la classe majoritaire. Nécessite rebalancing, tuning ou modification du seuil.

LinearSVM

- Le meilleur compromis recall/precision.
- Détecte plus de positifs que les autres.
- F1 le plus élevé → meilleur modèle opérationnel pour identifier les positifs.
- AUC très bonne, proche de LogReg.

Conclusion : modèle le plus équilibré. Performant malgré l'absence de probas natives.

GBT

- Le meilleur compromis recall/precision.
- Détecte plus de positifs que les autres.
- F1 le plus élevé → meilleur modèle opérationnel pour identifier les positifs.
- AUC très bonne, proche de LogReg.

Conclusion : modèle le plus équilibré. Performant malgré l'absence de probas natives.

Conclusion générale

- Le déséquilibre des classes est le problème principal.
- Nécessité d'ajuster le seuil --> regarder la courbe ROC
- Les modèles maximisent naturellement l'accuracy au détriment de la classe minoritaire --> changer la métrique à maximiser
- **LinearSVM et GBT sont les modèles les plus prometteurs.**
- **LogReg a un fort potentiel si l'on ajuste le seuil ou pondère les classes.**
- **RandomForest est inutilisable tant que le déséquilibre n'est pas traité.**

1.9.3 Pistes d'amélioration

Plusieurs pistes d'améliorations :

- On peut améliorer les données qu'on donne au modèle :
 - Feature engineering (longueur des textes, nb de mentions, nb de mots offensants connus...)
 - faire des n-grams
- S'attaquer au déséquilibre des classes
 - Rééchantillonnage : oversampling des positifs (SMOTE, RandomOverSampler), undersampling des négatifs
 - Weights : classWeightCol ou pondération des observations dans le loss.
 - Changer les seuils : ajuster le threshold de décision pour maximiser F1 ou PR-AUC
 - Changer la fonction coût : focal loss, weighted cross-entropy
 - Changer la métrique à maximiser
- Donner toutes les colonnes au modèle : pour apprendre des tâches auxiliaires pendant l'entraînement : - prédire si un commentaire mentionne une identité - la sortie de ce modèle peut être utilisée comme feature dérivée pour le modèle principale - = multi-task learning ou feature augmentation via tâches auxiliaires
- Autres pistes :
 - Cross-validation stratifiée
 - Augmentation de texte : synonymes, paraphrases, back-translation pour enrichir la classe minoritaire
 - **Word Embeddings (Word2Vec ou GloVe)**

2. Amélioration des performances

2.1 Feature engineering

2.1.1 Création des bigrams

--> au lieu de ne laisser qu'une liste de mot seuls lorsque qu'on tokenise, on fais des tokens de 2 mots --> car les insultes ou menaces sont souvent en expressions composées

tokens = ["you", "are", "so", "stupid"] avec un bigram : ["you are", "are so", "so stupid"]

```
from pyspark.ml.feature import NGram

def add_bigrams(df, n=2, input_col="tokens", output_col="bigrams"):
    ngram = NGram(n=n, inputCol=input_col, outputCol=output_col)
    return ngram.transform(df)

# train_df = add_bigrams(train_df)
# train_df.select("tokens", "bigrams").show(5, truncate=50)

test_df = add_bigrams(test_df)

train_df.printSchema()

root
|-- id: integer (nullable = true)
|-- tokens: array (nullable = true)
|   |-- element: string (containsNull = true)
|-- target: double (nullable = true)
|-- label: integer (nullable = false)
|-- rand: double (nullable = false)
|-- bigrams: array (nullable = true)
|   |-- element: string (containsNull = true)
```

2.1.2 Ajout de features basées sur comments

feature engineering sur la colonne comments --> on extrait des informations à partir des commentaires (longueur du commentaire, structure, ponctuation, proportion de majuscules...) Ces informations seront ensuite normalisées + vectorisées pour être utilisées par nos modèles

```
from pyspark.sql.functions import col, length, size, split,
regexp_replace, lower, udf, round
from pyspark.sql.types import IntegerType
import re

def extract_text_features(df, stopwords_list=None):
    if stopwords_list is None:
        stopwords_list = ["the", "and", "is", "in", "it", "of", "to",
"a"]

    df_features = df

    # --- Longueur et structure ---
    df_features = df_features.withColumn("char_count",
length(col("comment_text")))
    df_features = df_features.withColumn("word_count",
```

```

size(split(col("comment_text"), r"\s+"))
df_features = df_features.withColumn("sentence_count",
size(split(col("comment_text"), r"[.!?]+")))

# --- Punctuation ---
df_features = df_features.withColumn("exclamation_count",
size(split(col("comment_text"), "!")))
df_features = df_features.withColumn("question_count",
size(split(col("comment_text"), r"\?")))

# --- Majuscules consécutives ---
def count_consec_caps(text):
    return len(re.findall(r"[A-Z]{2,}", text))
count_consec_caps_udf = udf(count_consec_caps, IntegerType())
df_features = df_features.withColumn("consec_caps_count",
count_consec_caps_udf(col("comment_text")))

# --- Majuscules / minuscules ---
df_features = df_features.withColumn("uppercase_count",
size(split(regexp_replace(col("comment_text"), r"^[A-Z]", ""), "")))
df_features = df_features.withColumn("lowercase_count",
size(split(regexp_replace(col("comment_text"), r"^[a-z]", ""), "")))
df_features = df_features.withColumn("ratio_upper_lower",
col("uppercase_count") / (col("lowercase_count") + 1))

# --- Mots uniques / ratio ---
df_features = df_features.withColumn("unique_word_count",
size(split(regexp_replace(lower(col("comment_text")), r"^[^\w\s]", ""),
" ")))
df_features = df_features.withColumn("unique_ratio",
col("unique_word_count") / (col("word_count") + 1))

# --- Stopwords ---
def count_stopwords(text):
    words = re.findall(r"\b\w+\b", text.lower())
    return sum([1 for w in words if w in stopwords_list])
count_stopwords_udf = udf(count_stopwords, IntegerType())
df_features = df_features.withColumn("stopword_count",
count_stopwords_udf(col("comment_text")))

# --- Mots longs (>6 lettres) ---
def count_long_words(text):
    words = re.findall(r"\b\w+\b", text)
    return len([w for w in words if len(w) > 6])
count_long_words_udf = udf(count_long_words, IntegerType())
df_features = df_features.withColumn("long_word_count",
count_long_words_udf(col("comment_text")))
df_features = df_features.withColumn("long_word_ratio",
col("long_word_count") / (col("word_count") + 1))

```



```
['id',  
 'target',  
 'comment_text',  
 'severe_toxicity',  
 'obscene',  
 'identity_attack',  
 'insult',  
 'threat',  
 'asian',  
 'atheist',  
 'bisexual',  
 'black',  
 'buddhist',  
 'christian',  
 'female',  
 'heterosexual',  
 'hindu',  
 'homosexual_gay_or_lesbian',  
 'intellectual_or_learning_disability',  
 'jewish',  
 'latino',  
 'male',  
 'muslim',  
 'other_disability',  
 'other_gender',  
 'other_race_or_ethnicity',  
 'other_religion',  
 'other_sexual_orientation',  
 'physical_disability',  
 'psychiatric_or_mental_illness',  
 'transgender',  
 'white',  
 'created_date',  
 'publication_id',  
 'parent_id',  
 'article_id',  
 'rating',  
 'funny',  
 'wow',  
 'sad',  
 'likes',  
 'disagree',  
 'sexual_explicit',  
 'identity_annotator_count',  
 'toxicity_annotator_count',  
 'char_count',  
 'word_count',  
 'sentence_count',  
 'exclamation_count',  
 'question_count',
```

```
'consec_caps_count',
'uppercase_count',
'lowercase_count',
'ratio_upper_lower',
'unique_word_count',
'unique_ratio',
'stopword_count',
'long_word_count',
'long_word_ratio',
'special_char_count',
'tokens',
'target',
'label',
'rand',
'bigrams']
```

2.2 Word embedding

2.2.1 Entrainement du modèle Word2Vec

On enlève les stopwords pour pouvoir utiliser une fenetre plus petite et on garde la lemmatisation pour réduire le nombre de paramètres

```
from pyspark.ml.feature import Word2Vec

nothing, large_corpus_df = preprocess(train, limit=None)

corpus_size = large_corpus_df.count()
print(f"Taille du corpus : {corpus_size} lignes")

# http://localhost:4040/jobs/ pour voir les jobs

word2Vec = Word2Vec(
    vectorSize=100,      # dimension de votre vecteur de sortie, +
    grand = représente mieux les données ;
    minCount=5,          # Ignore les mots très rares (bruit)
    inputCol="tokens",   # Filtre de fréquence
    outputCol="w2v_features",
    maxIter=10,          # nombres d'epochs
    windowSize=5,        # taille de la fenetre de contexte
    stepSize=0.025       # Learning rate standard
)

model_name = f"word2Vec_{corpus_size}"

if LOAD_MODEL and os.path.exists(os.path.join(BASE_MODEL_PATH,
model_name)):
    w2v_model = load_model(model_name, model_type="word2vec")
    print(f"Modèle chargé : {model_name}")
```

```
else:
    print(f"Entraînement du modèle : {model_name}")
    w2v_model = word2Vec.fit(large_corpus_df)
```

```
if SAVE_MODEL:
    save_model(w2v_model, model_name)
```

```
25/12/07 02:56:32 WARN SparkStringUtils: Truncated the string
representation of a plan since it was too large. This behavior can be
adjusted by setting 'spark.sql.debug.maxToStringFields'.
```

```
Taille du corpus : 1804874 lignes
```

```
Modèle word2vec chargé depuis : models/word2Vec_1804874
Modèle chargé : word2Vec_1804874
```

Comme suggéré lors de notre présentation, il est judicieux d'utiliser un Word2Vec déjà entraîné sur un jeu de données plus large afin de réduire l'overfitting et d'augmenter la qualité de la vectorisation

Nous avons essayé d'intégrer un de ces modèles (Glove twitter 25), mais une prédiction=20s de chargement --> le prétraitement devient beaucoup trop long...

2.2.2 Vérification du modèle W2v entraîné

```
# Vérification sémantique : quels mots sont proches de "stupid" ou
"gay" ?
try:
    print("Mots proches de 'black' :")
    w2v_model.findSynonyms("black", 5).show()

    print("Mots proches de 'stupid' :")
    w2v_model.findSynonyms("stupid", 5).show()
except Exception as e:
    print("Le mot demandé n'est pas dans le vocabulaire (filtré par
minCount).")
```

```
Mots proches de 'black' :
```

```
+-----+-----+
|          word|      similarity|
+-----+-----+
|         white|0.7673611044883728|
|africanamericans|0.7386345267295837|
|         hispanic|0.6946250796318054|
|    peopleblack|0.6895892024040222|
|         latino|0.6759305000305176|
+-----+-----+
```

Mots proches de 'stupid' :

word	similarity
dumb	0.861616849899292
foolish	0.7260341644287109
idiotic	0.7183943390846252
ignorant	0.7167290449142456
fool	0.7095766663551331

```
import matplotlib.pyplot as plt
from sklearn.manifold import TSNE
import pandas as pd
import numpy as np

# 1. Récupération des vecteurs du modèle Spark
# getVectors() retourne un DataFrame Spark avec col "word" et col "vector"
vectors_df = w2v_model.getVectors()

# 2. On prend un échantillon pour la lisibilité (ex: les 300 premiers mots)
# Sinon le graphique sera illisible avec des milliers de mots superposés
vocab_sample = vectors_df.limit(300).toPandas()

# 3. Préparation des données pour t-SNE
# La colonne 'vector' contient des DenseVector, on doit les transformer en liste numpy
X = np.array(vocab_sample["vector"].tolist())
words = vocab_sample["word"].tolist()

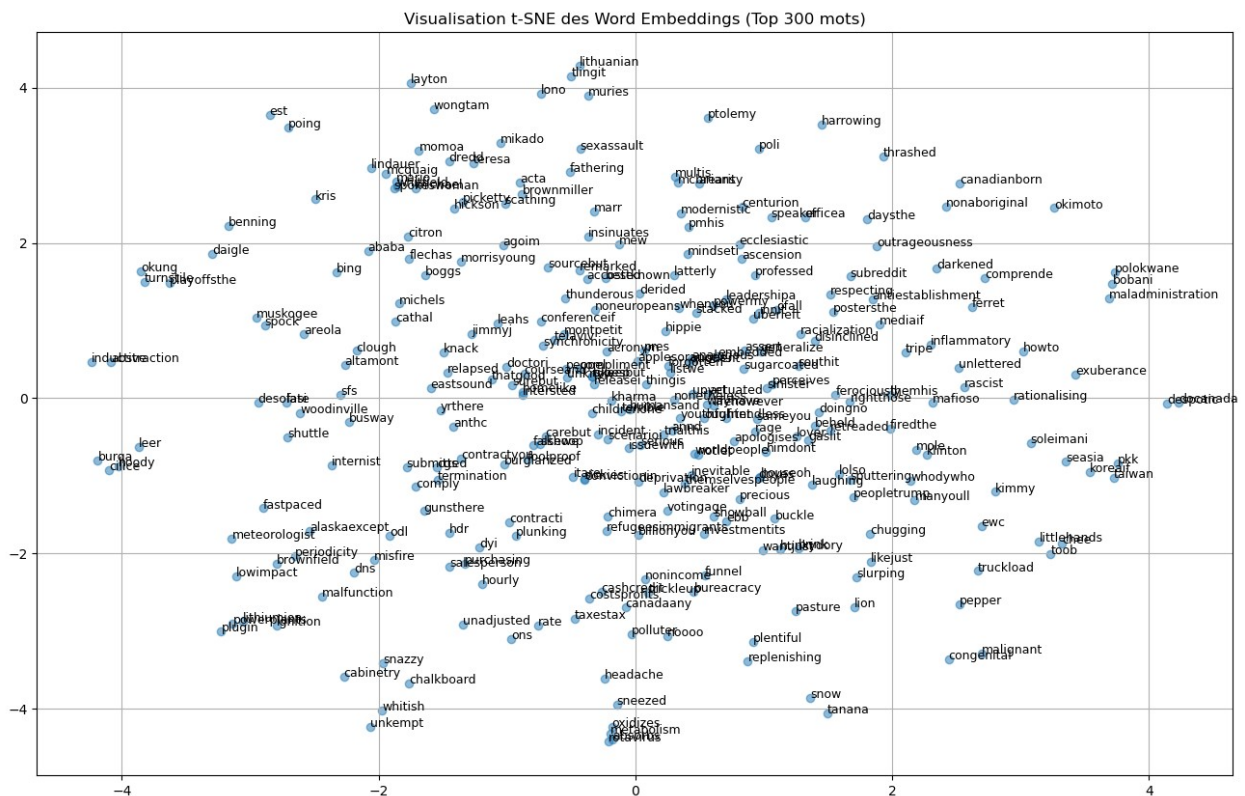
# 4. Réduction de dimension (100D -> 2D)
tsne = TSNE(n_components=2, random_state=42, perplexity=30,
init='pca', learning_rate='auto')
X_embedded = tsne.fit_transform(X)

# 5. Création du graphique
plt.figure(figsize=(16, 10))
plt.scatter(X_embedded[:, 0], X_embedded[:, 1], alpha=0.5)

# Ajouter les étiquettes (les mots) sur les points
for i, word in enumerate(words):
    plt.annotate(word, xy=(X_embedded[i, 0], X_embedded[i, 1]),
    fontsize=9)

plt.title("Visualisation t-SNE des Word Embeddings (Top 300 mots)")
```

```
plt.grid(True)
plt.show()
```



2.2.3 Transformation

--> pour appliquer w2v sur la colonne comments

```
print("--- Transformation ---")
df_final_w2v = w2v_model.transform(df_train_feat_eng)
df_final_test = w2v_model.transform(df_test_feat_eng)

print("--- Transformation terminée ---")

df_final_w2v.select("id", "tokens", "bigrams", "word_count",
                    "w2v_features").show(5)
```

```
--- Transformation ---
--- Transformation terminée ---
```

```
[Stage 21149:>
+ 1) / 1]
```

```
+-----+-----+-----+-----+
+-----+
| id | tokens | bigrams | word_count |
```

```
w2v_features|
+-----+-----+-----+-----+-----+
+-----+
|252708|[south, hill, nee...|[south hill, hill...|      19|[-
0.0387663822621...|
|247761|[murder, without,...|[murder without, ...|      24|
[0.01636873825918...|
|248786|[must, laughing, ...|[must laughing, l...|      12|
[0.09860921315848...|
|246104|      [wish, upon, st]|[wish upon, upon st]|      7|[-
0.0467201554371...|
|242675|      [buy, umbrella]|      [buy umbrella]|      6|
[0.05966314673423...|
+-----+-----+-----+-----+-----+
+-----+
only showing top 5 rows
```

2.2.4 Pipeline

--> construit dynamiquement un pipeline PySpark en fonction du type de combinaison de features

combinaisons = ["num", "w2v", "tfidf", "w2v+tfidf", "w2v+num", "tfidf+num", "all"]

num = juste les nouvelles features du feature engineering

```
from pyspark.ml import Pipeline
from pyspark.ml.feature import CountVectorizer, IDF, VectorAssembler,
StandardScaler

def build_pipeline_part2(model, combinaison, debug=False):
    """
    Construit un pipeline dynamique et affiche sa structure pour le
    débogage.

    Args:
        model: L'instance du modèle (ex: LogisticRegression)
        combinaison (str): "num", "w2v", "tfidf", "w2v+tfidf",
        "all"...
        debug (bool): Si True, affiche les étapes du pipeline dans la
        console.
    """

    combinaison = combinaison.lower()
    stages = []
    final_assembler_inputs = []
```

1. Définition des blocs de features

```
numeric_features = [
    "char_count", "word_count", "sentence_count",
    "exclamation_count",
    "question_count", "consec_caps_count", "uppercase_count",
    "lowercase_count", "ratio_upper_lower", "unique_word_count",
    "unique_ratio", "stopword_count", "long_word_count",
    "long_word_ratio", "special_char_count"
]

if "num" in combinaison or "all" in combinaison:
    assembler_num = VectorAssembler(inputCols=numeric_features,
    outputCol="vec_num_raw")
    scaler_num = StandardScaler(inputCol="vec_num_raw",
    outputCol="vec_num_scaled",
                                withStd=True, withMean=True)
    stages.extend([assembler_num, scaler_num])
    final_assembler_inputs.append("vec_num_scaled")

if "tfidf" in combinaison or "all" in combinaison:
    token_cv = CountVectorizer(inputCol="tokens",
    outputCol="tf_tokens", minDF=5)
    token_idf = IDF(inputCol="tf_tokens",
    outputCol="tfidf_tokens")

    bigram_cv = CountVectorizer(inputCol="bigrams",
    outputCol="tf_bigrams", minDF=5)
    bigram_idf = IDF(inputCol="tf_bigrams",
    outputCol="tfidf_bigrams")

    assembler_tfidf = VectorAssembler(inputCols=["tfidf_tokens",
    "tfidf_bigrams"],
    outputCol="vec_tfidf_raw")

    # withMean=False pour éviter OOM sur les vecteurs sparse
    scaler_tfidf = StandardScaler(inputCol="vec_tfidf_raw",
    outputCol="vec_tfidf_scaled",
                                withStd=True, withMean=False)

    stages.extend([token_cv, token_idf, bigram_cv, bigram_idf,
    assembler_tfidf, scaler_tfidf])
    final_assembler_inputs.append("vec_tfidf_scaled")

if "w2v" in combinaison or "all" in combinaison:
    # On suppose w2v_features déjà présent
    final_assembler_inputs.append("w2v_features")

# 2. Assemblage
```

```

    if not final_assembler_inputs:
        raise ValueError(f"Aucune feature sélectionnée pour la
combinaison : {combinaison}")

    assembler_final =
VectorAssembler(inputCols=final_assembler_inputs,
outputCol="features_final")
    stages.append(assembler_final)

# 3. Modèle

model.setParams(featuresCol="features_final", labelCol="label")
stages.append(model)

# 4. Debug printing

if debug:
    print(f"DEBUG PIPELINE - Combinaison : '{combinaison}'")

    for i, stage in enumerate(stages):
        stage_name = stage.__class__.__name__
        params = ""

        # Récupération intelligente des entrées/sorties pour
l'affichage
        if hasattr(stage, "getInputCol"):
            params += f"In: {stage.getInputCol()} "
        elif hasattr(stage, "getInputCols"):
            # On tronque si la liste est trop longue pour la
lisibilité
            cols = stage.getInputCols()
            cols_str = str(cols) if len(cols) < 5 else
f"[{cols[0]}, ..., {cols[-1]}] ({len(cols)} cols)"
            params += f"In: {cols_str} "
        elif hasattr(stage, "getFeaturesCol"): # Pour le modèle
final
            params += f"Features: {stage.getFeaturesCol()} |
Label: {stage.getLabelCol()}"

            if hasattr(stage, "getOutputCol"):
                params += f"-> Out: {stage.getOutputCol()}"

            print(f"Step {i+1:02d} : {stage_name:<20} | {params}")
            print(f"{'='*60}\n")

    return Pipeline(stages=stages)

```

2.2.5 - Apprentissage des modèles (avec les différentes combinaisons de features)

```
# Liste de toutes les combinaisons
combinaisons = ["num", "w2v", "tfidf", "w2v+tfidf", "w2v+num",
"tfidf+num", "all"]

# Dictionnaires pour stocker les résultats
models_name_lr = {}
best_lrs = {}
best_lr_params = {}
lr_scores = {}

for combinaison in combinaisons:
    # Générer des noms dynamiques pour chaque combinaison
    model_name = f"model_name_lr_{combinaison}"
    best_model_name = f"best_lr_{combinaison}"
    params_name = f"best_lr_params_{combinaison}"
    scores_name = f"lr_scores_{combinaison}"

    model, best_model, best_params, scores =
train_or_load_nested_model(
        lr,                                # modèle initial
        lr_grid,                           # grille de paramètres
        df_final_w2v,                       # dataframe
        df_final_w2v.count(),
        LOAD_MODEL,
        SAVE_MODEL,
        f"final_preprocess_{combinaison}",
        training_func=simple_cross_validator,
        combinaison=combinaison,
        k_folds=1
    )

    # Stocker les résultats dans les dictionnaires
    models_name_lr[combinaison] = model
    best_lrs[combinaison] = best_model
    best_lr_params[combinaison] = best_params
    lr_scores[combinaison] = scores
```

```
Modèle pipeline chargé depuis :
models/final_preprocess_num_LogisticRegression10000
Hyperparamètres chargés depuis :
models/final_preprocess_num_LogisticRegression10000_params.json
Scores chargés depuis :
models/final_preprocess_num_LogisticRegression10000_scores.json
Modèle pipeline chargé depuis :
models/final_preprocess_w2v_LogisticRegression10000
Hyperparamètres chargés depuis :
models/final_preprocess_w2v_LogisticRegression10000_params.json
```

```

Scores chargés depuis :
models/final_preprocess_w2v_LogisticRegression10000_scores.json
Modèle pipeline chargé depuis :
models/final_preprocess_tfidf_LogisticRegression10000
Hyperparamètres chargés depuis :
models/final_preprocess_tfidf_LogisticRegression10000_params.json
Scores chargés depuis :
models/final_preprocess_tfidf_LogisticRegression10000_scores.json
Modèle pipeline chargé depuis :
models/final_preprocess_w2v+tfidf_LogisticRegression10000
Hyperparamètres chargés depuis :
models/final_preprocess_w2v+tfidf_LogisticRegression10000_params.json
Scores chargés depuis :
models/final_preprocess_w2v+tfidf_LogisticRegression10000_scores.json
Modèle pipeline chargé depuis :
models/final_preprocess_w2v+num_LogisticRegression10000
Hyperparamètres chargés depuis :
models/final_preprocess_w2v+num_LogisticRegression10000_params.json
Scores chargés depuis :
models/final_preprocess_w2v+num_LogisticRegression10000_scores.json
Modèle pipeline chargé depuis :
models/final_preprocess_tfidf+num_LogisticRegression10000
Hyperparamètres chargés depuis :
models/final_preprocess_tfidf+num_LogisticRegression10000_params.json
Scores chargés depuis :
models/final_preprocess_tfidf+num_LogisticRegression10000_scores.json
Modèle pipeline chargé depuis :
models/final_preprocess_all_LogisticRegression10000
Hyperparamètres chargés depuis :
models/final_preprocess_all_LogisticRegression10000_params.json
Scores chargés depuis :
models/final_preprocess_all_LogisticRegression10000_scores.json

```

2.2.5 Tests sur un jeu de données TEST

```

import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix, precision_score,
recall_score, f1_score, accuracy_score
import pandas as pd
from pyspark.ml.evaluation import BinaryClassificationEvaluator

def evaluate_models_on_test_set(
    best_lrs: dict,
    test_df,
    plot_cm: bool = True
) -> dict:
    """
    Évalue les modèles en se concentrant sur la classe minoritaire (1

```

```

= Toxique).
    """
    final_test_scores = {}

    # Évaluateur Spark uniquement pour l'AUC
    binary_evaluator = BinaryClassificationEvaluator(
        labelCol="label", rawPredictionCol="rawPrediction",
        metricName="areaUnderROC"
    )

    print("Démarrage de l'évaluation CIBLÉE (Classe 1 - Toxique)...")

    # Normalisation de l'entrée : s'assurer que nous itérons
    # toujours sur une liste de tuples (nom, modèle)
    if isinstance(best_lrs, dict):
        # Cas 1 : Dictionnaire
        models_to_iterate = best_lrs.items()
    elif best_lrs is not None:
        # Cas 2 : Un seul modèle
        models_to_iterate = [("modele_unique", best_lrs)]
    else:
        # Cas 3 : L'entrée est vide/None
        models_to_iterate = []

    for combinaison, model_pipeline in models_to_iterate:
        print(f"\n{'='*40}")
        print(f"--- Modèle : {combinaison} ---")

        try:
            # 1. Prédictions Spark
            predictions = model_pipeline.transform(test_df)

            # 2. Calcul de l'AUC via Spark (toujours fiable)
            auc = binary_evaluator.evaluate(predictions)

            # 3. Conversion en Pandas pour les métriques précises de
            # la classe 1
            # On ne récupère que les colonnes nécessaires pour
            # économiser la RAM
            y_preds_pdf = predictions.select("label",
            "prediction").toPandas()

            y_true = y_preds_pdf["label"]
            y_pred = y_preds_pdf["prediction"]

            # pos_label=1 force le calcul UNIQUEMENT sur la classe
            # toxique
            prec_1 = precision_score(y_true, y_pred, pos_label=1,

```

```

zero_division=0)
    rec_1 = recall_score(y_true, y_pred, pos_label=1,
zero_division=0)
    f1_1 = f1_score(y_true, y_pred, pos_label=1,
zero_division=0)
    acc = accuracy_score(y_true, y_pred) # L'accuracy reste
globale

    # Stockage avec des noms explicites
    final_test_scores[combinaison] = {
        "AUC": auc,
        "F1_Toxic": f1_1,
        "Recall_Toxic": rec_1,
        "Precision_Toxic": prec_1,
        "Accuracy_Global": acc
    }

    print(f"TOXIC CLASS -> Recall: {rec_1:.4f} | Precision:
{prec_1:.4f} | F1: {f1_1:.4f}")
    print(f"    Global AUC: {auc:.4f}")

    # 4. Matrice de Confusion
    if plot_cm:
        cm = confusion_matrix(y_true, y_pred)
        plt.figure(figsize=(4, 3))
        sns.heatmap(cm, annot=True, fmt='d', cmap='Reds',
cbar=False,
                    xticklabels=['Sain (0)', 'Toxique (1)'],
                    yticklabels=['Sain (0)', 'Toxique (1)'])
        plt.xlabel('Prédiction')
        plt.ylabel('Réel')
        plt.title(f'Confusion Matrix - {combinaison}')
        plt.show()

    except Exception as e:
        print(f"ERREUR pour {combinaison}: {e}")
        final_test_scores[combinaison] = {"AUC": 0.0}

    print("\nÉvaluation terminée.")
    return final_test_scores

# import sys
# sys.tracebacklimit = 0

# # Appel de la fonction
# test_results = evaluate_models_on_test_set(best_lrs, df_final_test,
plot_cm=True)

# # Création du tableau récapitulatif (votre code existant)

```

```

# df_test_results = pd.DataFrame.from_dict(test_results,
orient='index')
# df_test_results.reset_index(inplace=True)
# df_test_results.rename(columns={'index': 'Combinaison'},
inplace=True)

# display(df_test_results.style.background_gradient(cmap='Greens',
subset=df_test_results.columns[1:]))

# del sys.tracebacklimit

import pandas as pd
from typing import Dict, Any

def display_comparison_tables(
    lr_scores: Dict[str, Any],
    test_results: Dict[str, Dict[str, float]]
):
    # Convertit les dictionnaires de scores (Training CV et Test
Final) en DataFrames et les affiche

    # 1. TABLEAU DE PERFORMANCE PENDANT LE TRAINING (CV)

    # Conversion et nettoyage du DataFrame de Training
    df_results = pd.DataFrame.from_dict(lr_scores, orient='index')
    df_results.reset_index(inplace=True)
    df_results.rename(columns={'index': 'Combinaison'}, inplace=True)

    # Renommer la colonne AUC qui est souvent un entier '0'
    if 0 in df_results.columns:
        df_results.rename(columns={0: 'AUC'}, inplace=True)

    # Affichage du tableau (Training)
    print("\n--- TABLEAU COMPARATIF DES PERFORMANCES PENDANT LE
TRAINING ---")

    numeric_cols_train = df_results.select_dtypes(include=['float',
'int']).columns

    styled_table_train =
df_results.style.background_gradient(cmap='Greens',
subset=numeric_cols_train)\
        .format(precision=3)\
        .set_caption("Résultats de Cross-Validation (Métrique :
AUC)")\
        .set_properties(**{'text-align': 'center'})

    display(styled_table_train)

```

```

# 2. TABLEAU DE PERFORMANCE FINALE (TEST SET)

# Conversion du DataFrame de Test
df_test_results = pd.DataFrame.from_dict(test_results,
orient='index')
df_test_results.reset_index(inplace=True)
df_test_results.rename(columns={'index': 'Combinaison'},
inplace=True)

# Affichage du tableau (Test)
print("\n--- RÉSULTATS FINAUX SUR LE JEU DE TEST ---")

numeric_cols_test =
df_test_results.select_dtypes(include=['float', 'int']).columns

styled_final_table =
df_test_results.style.background_gradient(cmap='Blues',
subset=numeric_cols_test)\
    .format(precision=4)\
    .set_caption("Performance Finale sur le Jeu de Test (LogReg) -
Métriques Ciblées sur la Toxicité (1)")\
    .set_properties(**{'text-align': 'center'})

display(styled_final_table)
display_comparison_tables(lr_scores, test_results)

>>> TABLEAU COMPARATIF DES PERFORMANCES PENDANT LE TRAINING
<pandas.io.formats.style.Styler at 0xf14b4225d590>

>>> RÉSULTATS FINAUX SUR LE JEU DE TEST (Focus Classe 1)
<pandas.io.formats.style.Styler at 0xf14b365b8e10>

```

tf+idf ET w2v marche particulièrement bien, en revanche les features obtenues avec le feature engineering ne sont pas très utiles

- w2v très prometteur étant donné que l'entraînement du modèle n'avait été fait que sur 100 000 commentaires
- pour la suite on garde w2v, tfidf, w2v + tfidf, all

Synthèse:

Avant (préprocessing standard, LogReg)

- Accuracy : 0.9512

- Precision (toxic=1) : 0.7273
- Recall (toxic=1) : 0.1212
- F1 (toxic=1) : 0.2078
- ROC AUC : 0.8144

Après (nouveau pipeline) *best model : w2v+tfidf*

- Accuracy : 0.9540
- Precision (toxic=1) : 0.8148
- Recall (toxic=1) : 0.1667
- F1 (toxic=1) : 0.2767
- ROC AUC : 0.8501

→ Le pipeline w2v+tfidf donne le meilleur rappel, même meilleur que all. Cela veut dire que tfidf capture mieux les mots rares qui signalent la toxicité, et w2v aide au contexte.

2.3 Gestion du déséquilibre des classes

2.3.0 Ajout d'une fonction pour faire du sous échantillonnage

```
def undersample_data(df, ratio_target=1.0):
    #ratio_target : Ratio désiré (Negatifs / Positifs)

    pos_df = df.filter(col("label") == 1)
    neg_df = df.filter(col("label") == 0)

    count_pos = pos_df.count()
    count_neg = neg_df.count()

    fraction = (count_pos * ratio_target) / count_neg

    print(f"Undersampling : On garde {fraction:.2%} des données négatives.")

    neg_sampled = neg_df.sample(withReplacement=False,
                                fraction=fraction, seed=42)

    return pos_df.union(neg_sampled)
```

2.3.1 Ajout d'une fonction lr avec des poids

```
from pyspark.sql.functions import when

def add_class_weights(df, ratio_multiplieur=1):
    """Ajoute une colonne de poids pour équilibrer les classes."""

    # 1. Compter les classes
    total = df.count()
    count_pos = df.filter(col("label") == 1).count()
    count_neg = df.filter(col("label") == 0).count()

    # 2. Calculer le ratio de balancement
    # Poids pour 0 = 1 (référence)
    # Poids pour 1 = (Nombre Négatifs / Nombre Positifs)
    balancing_ratio = (count_neg / count_pos)*ratio_multiplieur+1
    print(f"Ratio de déséquilibre : 1 toxique ≈ {balancing_ratio:.2f} commentaires sains.")

    # 3. Créer la colonne de poids
    df_weighted = df.withColumn("classWeight", when(col("label") == 1,
balancing_ratio).otherwise(1.0))

    return df_weighted
```

2.3.2 Définition des classifieurs weighted

```
lr_weighted = LogisticRegression(
    featuresCol="features_final",
    labelCol="label",
    weightCol="classWeight",
    maxIter=10
)

rf_weighted = RandomForestClassifier(
    featuresCol="features",
    labelCol="label",
    weightCol="final_weight",
    seed=0
)

svm_weighted = LinearSVC(
    featuresCol="features",
    labelCol="label",
    weightCol="final_weight",
    maxIter=50
)

gbt_weighted = GBTClassifier(
    featuresCol="features",
```

```

labelCol="label",
weightCol="final_weight",
seed=0
)

```

2.3.3 Entraînement

Entraînement des modèles :

- Avec UNDERSAMPLING
- Avec des poids dynamiques (pour se concentrer sur TARGET)
- Avec les 2

Application

```

train_weighted = add_class_weights(df_final_w2v)
train_balanced = undersample_data(df_final_w2v, ratio_target=0.5)
train_balanced_weighted = add_class_weights(train_balanced)

model_lr_weighted, best_model_lr_weighted, best_params_lr_weighted,
scores_lr_weighted = train_or_load_nested_model(
    lr_weighted,
    lr_grid,
    train_weighted,
    train_weighted.count(),
    LOAD_MODEL,
    SAVE_MODEL,
    f"class_weights_w2v+tfidf",
    training_func=simple_cross_validator,
    combinaison='w2v+tfidf',
    k_folds=2
)

model_lr_balanced, best_model_lr_balanced, best_params_lr_balanced,
scores_lr_balanced = train_or_load_nested_model(
    lr,
    lr_grid,
    train_balanced,
    train_balanced.count(),
    LOAD_MODEL,
    SAVE_MODEL,
    f"class_balanced_w2v+tfidf",
    training_func=simple_cross_validator,
    combinaison='w2v+tfidf',
    k_folds=2
)

model_lr_balanced_weighted, best_model_lr_balanced_weighted,
best_params_lr_balanced_weighted, scores_lr_balanced_weighted =
train_or_load_nested_model(

```

```

    lr_weighted,
    lr_grid,
    train_balanced_weighted,
    train_balanced_weighted.count(),
    LOAD_MODEL,
    SAVE_MODEL,
    f"class_weights_w2v+tfidf",
    training_func=simple_cross_validator,
    combinaison='w2v+tfidf',
    k_folds=2
)

```

Ratio de déséquilibre : 1 toxique $\approx$ 16.81 commentaires sains.

Undersampling : On garde 3.16% des données négatives.

Ratio de déséquilibre : 1 toxique $\approx$ 1.54 commentaires sains.

Entraînement du modèle :

class_weights_w2v+tfidf_LogisticRegression10000 avec la fonction simple_cross_validator

Exception ignored in: <_io.BufferedWriter name=7>48:> (0 + 1) / 1]

Traceback (most recent call last):

File "/opt/spark/python/lib/pyspark.zip/pyspark/daemon.py", line 200, in manager

BrokenPipeError: [Errno 32] Broken pipe

Démarrage Simple Cross-Validation...

- Taille du jeu de données : 10000
- Nombre de folds (K) : 2
- Hyperparamètres testés : 3
- Total entraînements : 6

Construction du pipeline...

Lancement du CrossValidator (Training)...

25/12/06 06:20:05 WARN DAGScheduler: Broadcasting large task binary with size 1050.0 KiB

25/12/06 06:20:05 WARN DAGScheduler: Broadcasting large task binary with size 1050.0 KiB

25/12/06 06:20:05 WARN DAGScheduler: Broadcasting large task binary with size 1050.6 KiB

25/12/06 06:20:05 WARN DAGScheduler: Broadcasting large task binary with size 1050.6 KiB

25/12/06 06:20:05 WARN DAGScheduler: Broadcasting large task binary with size 1050.6 KiB

25/12/06 06:20:05 WARN DAGScheduler: Broadcasting large task binary with size 1050.6 KiB

[illegible]

[illegible]

[illegible]

```
with size 1027.0 KiB
25/12/06 06:20:13 WARN DAGScheduler: Broadcasting large task binary
with size 1071.9 KiB
25/12/06 06:20:14 WARN DAGScheduler: Broadcasting large task binary
with size 1714.5 KiB
25/12/06 06:20:15 WARN DAGScheduler: Broadcasting large task binary
with size 1715.1 KiB
25/12/06 06:20:15 WARN DAGScheduler: Broadcasting large task binary
with size 1715.1 KiB
25/12/06 06:20:15 WARN DAGScheduler: Broadcasting large task binary
with size 1715.1 KiB
25/12/06 06:20:15 WARN DAGScheduler: Broadcasting large task binary
with size 1715.1 KiB
25/12/06 06:20:15 WARN DAGScheduler: Broadcasting large task binary
with size 1715.1 KiB
25/12/06 06:20:15 WARN DAGScheduler: Broadcasting large task binary
with size 1715.1 KiB
25/12/06 06:20:15 WARN DAGScheduler: Broadcasting large task binary
with size 1715.1 KiB
25/12/06 06:20:15 WARN DAGScheduler: Broadcasting large task binary
with size 1715.1 KiB
25/12/06 06:20:15 WARN DAGScheduler: Broadcasting large task binary
with size 1715.1 KiB
25/12/06 06:20:16 WARN DAGScheduler: Broadcasting large task binary
with size 1715.1 KiB
25/12/06 06:20:16 WARN DAGScheduler: Broadcasting large task binary
with size 1715.1 KiB
25/12/06 06:20:16 WARN DAGScheduler: Broadcasting large task binary
with size 1715.1 KiB
25/12/06 06:20:16 WARN DAGScheduler: Broadcasting large task binary
with size 1715.1 KiB
```

Recherche terminée.

RÉSULTATS

```
Meilleur AUC moyen (Validation) : 0.7419
Entrainement du modèle :
class_balanced_w2v+tfidf_LogisticRegression919 avec la fonction
simple_cross_validator
```

```
Exception ignored in: <_io.BufferedWriter name=7> 1][Stage 1988:> (0 +
1) / 111]
```

```
Traceback (most recent call last):
```

```
File "/opt/spark/python/lib/pyspark.zip/pyspark/daemon.py", line 200, in manager
```

```
BrokenPipeError: [Errno 32] Broken pipe
```

Démarrage Simple Cross-Validation...

```
- Taille du jeu de données : 919
- Nombre de folds (K)      : 2
- Hyperparamètres testés  : 3
- Total entraînements      : 6
```

Construction du pipeline...

Lancement du CrossValidator (Training)...
Recherche terminée.

```
=====
RÉSULTATS
```

```
=====
Meilleur AUC moyen (Validation) : 0.7736
Entraînement du modèle : class_weights_w2v+tfidf_LogisticRegression919
avec la fonction simple_cross_validator
```

Exception ignored in: <_io.BufferedWriter name=7>

```
Traceback (most recent call last):
  File "/opt/spark/python/lib/pyspark.zip/pyspark/daemon.py", line
  200, in manager
BrokenPipeError: [Errno 32] Broken pipe
```

Démarrage Simple Cross-Validation...

```
- Taille du jeu de données : 919
- Nombre de folds (K)      : 2
- Hyperparamètres testés  : 3
- Total entraînements      : 6
```

Construction du pipeline...

Lancement du CrossValidator (Training)...
Recherche terminée.

```
=====
RÉSULTATS
```

```
=====
Meilleur AUC moyen (Validation) : 0.7074
```

2.3.4 TESTS

```
test_results_weights =
evaluate_models_on_test_set(best_model_lr_weighted, df_final_test,
plot_cm=True)
test_results_balanced =
evaluate_models_on_test_set(best_model_lr_balanced, df_final_test,
plot_cm=True)
test_results_weights_balanced =
```

```
evaluate_models_on_test_set(best_model_lr_balanced_weighted,  
df_final_test, plot_cm=True)
```

Démarrage de l'évaluation CIBLÉE (Classe 1 - Toxique)...

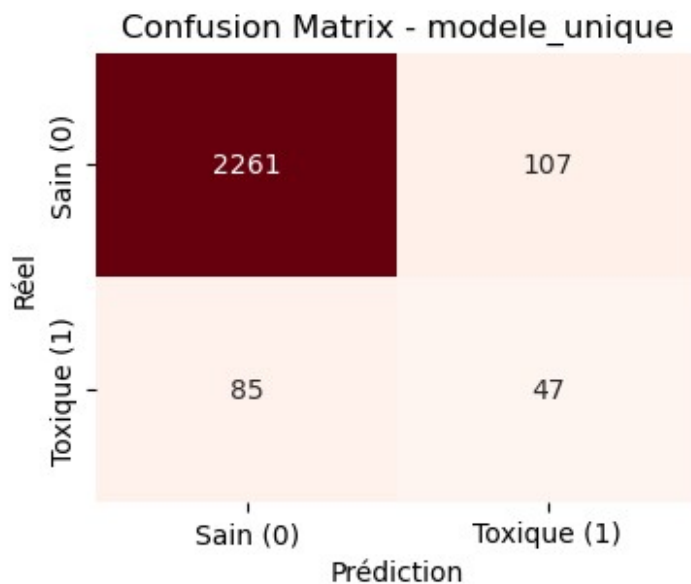
=====

--- Modèle : modele_unique ---

25/12/06 06:20:43 WARN DAGScheduler: Broadcasting large task binary
with size 1847.4 KiB

[Stage 3081:> (0
+ 2) / 2]

TOXIC CLASS -> Recall: 0.3561 | Precision: 0.3052 | F1: 0.3287
Global AUC: 0.7801



Évaluation terminée.

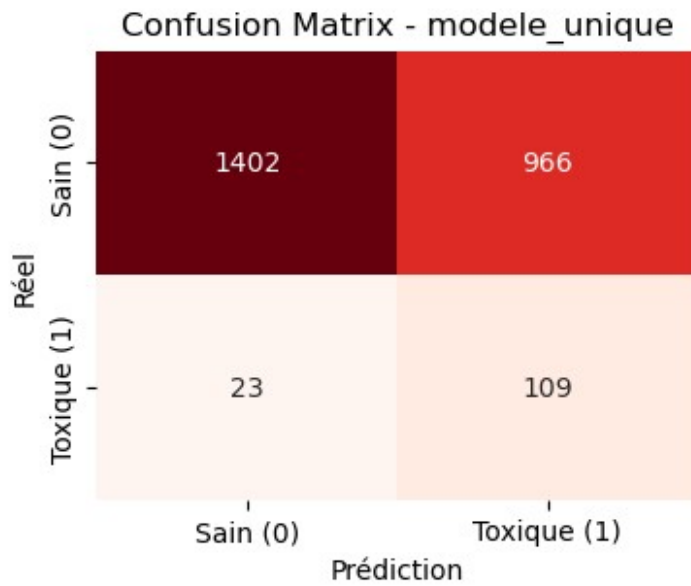
Démarrage de l'évaluation CIBLÉE (Classe 1 - Toxique)...

=====

--- Modèle : modele_unique ---

[Stage 3115:> (0
+ 2) / 2]

TOXIC CLASS -> Recall: 0.8258 | Precision: 0.1014 | F1: 0.1806
Global AUC: 0.7935



Évaluation terminée.

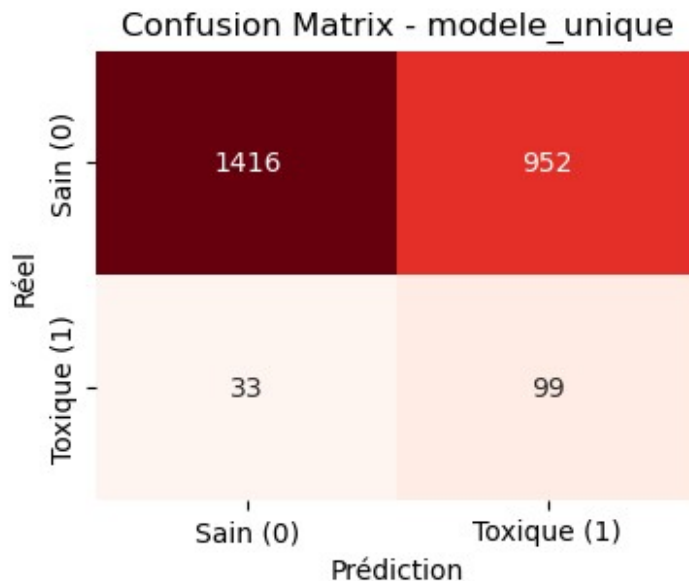
Démarrage de l'évaluation CIBLÉE (Classe 1 - Toxique)...

=====

--- Modèle : modele_unique ---

[Stage 3149:> (0
+ 2) / 2]

TOXIC CLASS -> Recall: 0.7500 | Precision: 0.0942 | F1: 0.1674
Global AUC: 0.7260



Évaluation terminée.

```
df_test_results_weights = pd.DataFrame.from_dict(test_results_weights,
orient='index')
df_test_results_weights.reset_index(inplace=True)
df_test_results_weights.rename(columns={'index': 'Combinaison'},
inplace=True)
```

```
display(
    df_test_results_weights.style.background_gradient(
        cmap='Blues',
        subset=df_test_results_weights.columns[1:] # On utilise le
        DataFrame pour les colonnes
    )
    .format(precision=4)
    .set_caption("Performance Finale sur le Jeu de Test (LogReg) -
    Avec Poids")
    .set_properties(**{'text-align': 'center'})
)
```

```
df_test_results_balanced =
pd.DataFrame.from_dict(test_results_balanced, orient='index')
df_test_results_balanced.reset_index(inplace=True)
df_test_results_balanced.rename(columns={'index': 'Combinaison'},
inplace=True)
```

```
display(
    df_test_results_balanced.style.background_gradient(
        cmap='Blues',
```

```

        subset=df_test_results_balanced.columns[1:] # On utilise le
        DataFrame pour les colonnes
    )
    .format(precision=4)
    .set_caption("Performance Finale sur le Jeu de Test (LogReg) -
    Avec Poids")
    .set_properties(**{'text-align': 'center'})
)

df_test_results_weights_balanced =
pd.DataFrame.from_dict(test_results_weights_balanced, orient='index')
df_test_results_weights_balanced.reset_index(inplace=True)
df_test_results_weights_balanced.rename(columns={'index':
'Combinaison'}, inplace=True)

display(
    df_test_results_weights_balanced.style.background_gradient(
        cmap='Blues',
        subset=df_test_results_weights_balanced.columns[1:] # On
        utilise le DataFrame pour les colonnes
    )
    .format(precision=4)
    .set_caption("Performance Finale sur le Jeu de Test (LogReg) -
    Avec Poids")
    .set_properties(**{'text-align': 'center'})
)

>>> LR : RÉSULTATS FINAUX SUR LE JEU DE TEST weights
<pandas.io.formats.style.Styler at 0xf14b2b39c050>

>>> LR : RÉSULTATS FINAUX SUR LE JEU DE TEST balanced
<pandas.io.formats.style.Styler at 0xf14b2b39c050>

>>> LR : RÉSULTATS FINAUX SUR LE JEU DE TEST weights_balanced
<pandas.io.formats.style.Styler at 0xf14b2b39c050>

import matplotlib.pyplot as plt
import pandas as pd
from pyspark.sql.functions import udf
from pyspark.sql.types import FloatType

import matplotlib.pyplot as plt
import pandas as pd
from pyspark.sql.functions import udf

```

```

from pyspark.sql.types import FloatType
from sklearn.metrics import (
    f1_score, recall_score, precision_score, roc_curve, auc,
    accuracy_score
)

def find_best_threshold(model, test_df, model_name):
    """
    Trouve le seuil optimal qui maximise le F1-Score sur la classe
    Toxique (1)
    et affiche l'AUC et l'Accuracy Générale du modèle.
    """
    # 1. On récupère les prédictions brutes (probabilités)
    predictions = model.transform(test_df)

    # On extrait la probabilité de la classe 1 (Toxique)
    extract_prob = udf(lambda x: float(x[1]), FloatType())
    preds_pd = predictions.select("label",
    extract_prob("probability").alias("score")).toPandas()

    y_true = preds_pd["label"]
    y_score = preds_pd["score"]

    # Calcul de l'AUC (Area Under ROC - Seuil-Indépendant)
    fpr, tpr, _ = roc_curve(y_true, y_score)
    roc_auc = auc(fpr, tpr)

    # 2. On teste plein de seuils entre 0.1 et 0.9
    thresholds = [i/100 for i in range(10, 90, 2)]
    scores = []

    for t in thresholds:
        # On applique le seuil manuellement
        y_pred_t = (y_score > t).astype(int)

        # On calcule les métriques sur la classe Toxique (1)
        f1 = f1_score(y_true, y_pred_t, pos_label=1, zero_division=0)
        rec = recall_score(y_true, y_pred_t, pos_label=1,
        zero_division=0)
        prec = precision_score(y_true, y_pred_t, pos_label=1,
        zero_division=0)
        acc = accuracy_score(y_true, y_pred_t)

        scores.append({"threshold": t, "f1": f1, "recall": rec,
        "precision": prec, "accuracy": acc})

    df_scores = pd.DataFrame(scores)

```

```

# 3. Trouver le meilleur F1
best_row = df_scores.loc[df_scores['f1'].idxmax()]
best_thresh = best_row['threshold']

print(f"Analyse de Seuil pour {model_name}")
print(f"AUC (Seuil-Indépendant) : {roc_auc:.4f}")
print(f"Seuil optimal (Max F1) : {best_thresh}")
print(f"    -> F1 Toxic (Opt.) : {best_row['f1']:.4f}")
print(f"    -> Recall Toxic (Opt.) : {best_row['recall']:.4f}")
print(f"    -> Precision Toxic (Opt.) :
{best_row['precision']:.4f}")
print(f"    -> Accuracy Générale : {best_row['accuracy']:.4f}") #
NOUVEAU

# 4. Graphique
plt.figure(figsize=(10, 6))
plt.plot(df_scores['threshold'], df_scores['f1'], label='F1
Score', linewidth=3)
plt.plot(df_scores['threshold'], df_scores['recall'],
label='Recall', linestyle='--')
plt.plot(df_scores['threshold'], df_scores['precision'],
label='Precision', linestyle='--')
plt.axvline(best_thresh, color='red', linestyle=':',
label=f'Optimum ({best_thresh})')
plt.xlabel('Seuil de décision')
plt.ylabel('Score')
plt.title(f'Impact du Seuil sur la détection de Toxicité
({model_name})')
plt.legend()
plt.grid(True)
plt.show()

return best_thresh

# optimal_threshold = find_best_threshold(best_lrs['w2v+tfidf'],
df_final_test, model_name="best_lrs['w2v+tfidf']")
# optimal_threshold = find_best_threshold(best_model_lr_weighted,
df_final_test, model_name='best_model_lr_weighted')
# optimal_threshold = find_best_threshold(best_model_lr_balanced,
df_final_test, model_name='best_model_lr_balanced')
# optimal_threshold =
find_best_threshold(best_model_lr_balanced_weighted, df_final_test,
model_name='best_model_lr_weighted_balanced')

```

Comme on pouvait s'y attendre on a un meilleur f1 + recall avec ces méthodes mais l'AUC baisse... à voir avec les autres modèles et à pondérer de façon plus douce --> la pondération est assez lourde + on enlève beaucoup du dataset avec balanced.

Cependant `balanced` peut s'avérer très intéressant si à la fin on se rend compte qu'on ne pourra jamais faire tourner tout le jeu de donnée car trop gros --> dans ce cas on fait d'une pierre deux coups en faisant un undersampling (on garde moins de données, mais avec une distribution de target moins inégale)

étant donné les métriques, le meilleur compromis serait un de faire un undersampling avec peu ou pas de pondération. En attendant le meilleur modèle est **RegLog + tfidf + w2c** avec simplement un seuil ajusté à 0.16

C'est mieux de faire des poids dynamique étant donné que si on fait du sous échantillonnage, le ratio positif/négatif sera modifié

2.4 Intégration de modèles plus performants

3. Gestion des biais d'identité

```
# 1. Liste des identités sensibles fournies dans le dataset Jigsaw
identity_columns = [
    'male', 'female', 'transgender', 'other_gender',
    'heterosexual', 'homosexual_gay_or_lesbian', 'bisexual',
    'other_sexual_orientation',
    'christian', 'jewish', 'muslim', 'hindu', 'buddhist', 'atheist',
    'other_religion',
    'black', 'white', 'asian', 'latino', 'other_race_or_ethnicity',
    'physical_disability', 'intellectual_or_learning_disability',
    'psychiatric_or_mental_illness', 'other_disability'
]
```

3.1 Définition d'une fonction pour regarder les biais d'identité

```
import pandas as pd
from sklearn.metrics import roc_auc_score
import numpy as np

def compute_bias_metrics(model, test_df, identity_cols,
    model_name="Model"):
    """
    Calcule les métriques de biais Jigsaw (Subgroup AUC, BPSN, BNSP).
    """
    print(f"Calcul des biais pour : {model_name} ...")

    # 1. Prédiction
    preds = model.transform(test_df)

    # On passe en Pandas pour l'évaluation détaillée (plus flexible
    pour le slicing)
```

```

# On ne récupère que les colonnes nécessaires
cols_to_select = ["label", "probability"] + identity_cols
df_eval = preds.select(*cols_to_select).toPandas()

# Extraction du score de toxicité (proba classe 1)
df_eval['score'] = df_eval['probability'].apply(lambda x:
float(x[1]))
df_eval['label'] = df_eval['label'].astype(int)

# Binarisation des identités (Jigsaw considère présent si >= 0.5)
for col_id in identity_cols:
    df_eval[col_id] = (df_eval[col_id] >= 0.5).astype(bool)

metrics = []

# AUC Global
global_auc = roc_auc_score(df_eval['label'], df_eval['score'])

for identity in identity_cols:
    # --- Masques Booléens ---
    # Sous-groupe (ex: contient 'black')
    subset_identity = df_eval[df_eval[identity] == True]

    # Background (ex: tout le reste)
    subset_background = df_eval[df_eval[identity] == False]

    # 1. Subgroup AUC : Juste sur le sous-groupe
    try:
        sub_auc = roc_auc_score(subset_identity['label'],
subset_identity['score'])
    except:
        sub_auc = np.nan # Pas assez de données

    # 2. BPSN (Background Positive, Subgroup Negative)
    # On mélange les "Toxiques SANS l'identité" (Background
Positive) et les "Sains AVEC l'identité" (Subgroup Negative)
    # C'est la métrique qui détecte les faux positifs dûs au
biais.
    bpsn_subset = pd.concat([
        subset_background[subset_background['label'] == 1], #
Vrais toxiques (hors groupe)
        subset_identity[subset_identity['label'] == 0] #
Sains (du groupe) -> Ne pas classer toxique !
    ])
    try:
        bpsn_auc = roc_auc_score(bpsn_subset['label'],
bpsn_subset['score'])
    except:
        bpsn_auc = np.nan

```

```

# 3. BNSP (Background Negative, Subgroup Positive)
bnsps_subset = pd.concat([
    subset_background[subset_background['label'] == 0],
    subset_identity[subset_identity['label'] == 1]
])
try:
    bnsps_auc = roc_auc_score(bnsps_subset['label'],
bnsps_subset['score'])
except:
    bnsps_auc = np.nan

metrics.append({
    "Identity": identity,
    "Subgroup_AUC": sub_auc,
    "BPSN_AUC": bpsn_auc,
    "BNSP_AUC": bnsps_auc
})

results_df = pd.DataFrame(metrics)
results_df = results_df.set_index("Identity")

# Score Composite Jigsaw (Moyenne Géométrique Généralisée)
final_score = np.mean([
    global_auc,
    results_df['Subgroup_AUC'].mean(),
    results_df['BPSN_AUC'].mean(),
    results_df['BNSP_AUC'].mean()
])

print(f"Score Jigsaw Final : {final_score:.4f} (Global AUC:
{global_auc:.4f})")

return results_df.sort_values(by="BPSN_AUC") # On trie par les
pires biais (BPSN bas)

# bias_results_before = compute_bias_metrics(
#     best_lrs['w2v+tfidf'],
#     df_final_w2v,
#     identity_columns,
#     "Modèle Base"
# )

# Afficher les identités les plus discriminées (BPSN faible = le
modèle confond identité et insulte)
# print("\nIdentités les plus confondues avec la toxicité (BPSN
bas) :")
#

```

```
display(bias_results_before.head(5).style.background_gradient(cmap='RdYlGn', vmin=0.5, vmax=1.0))
```

3.2 Définition d'une fonction pour faire une colonne poids dynamique sur

- les colonnes contenant des identités
- les colonnes avec un commentaire toxique

```
from pyspark.sql.functions import lit

def add_bias_mitigation_weights(df, identity_cols, ratio_multiplifier,
                                identity_weight):
    """
    Crée une colonne de poids pour mitiger les biais.
    Stratégie :
    - Poids de base = 1.0
    - Si Toxique (Label 1) : Poids += 5.0 (Pour gérer le déséquilibre global)
    - Si Identité présente : Poids += 0.5 (Booster l'importance des identités)
    - Si Identité présente ET Sain (Label 0) : Poids += 10.0 (C'est CRUCIAL : Anti-Biais)
    - Si Identité présente ET Toxique (Label 1) : Poids += 5.0
    """

    # 1. Compter les classes
    total = df.count()
    count_pos = df.filter(col("label") == 1).count()
    count_neg = df.filter(col("label") == 0).count()
    ratio = count_neg/count_pos
    print(f"Ratio de sain/toxique {ratio:.2f}")

    toxic_weight = ratio*ratio_multiplifier
    print(f"toxic_weights : {toxic_weight:.2f} + 1, identity_weight : {identity_weight}")

    from pyspark.sql.functions import expr

    condition_str = " OR ".join([f"{c} >= 0.5" for c in identity_cols])
    df = df.withColumn("has_identity", expr(condition_str))

    # 2. Calcul des poids
    # Poids de base pour tout le monde
    df = df.withColumn("final_weight", lit(1.0))
```

```

# A. Gestion du déséquilibre global (Bonus pour tous les toxiques)
df = df.withColumn("final_weight",
                    when(col("label") == 1, col("final_weight") +
toxic_weight)
                    .otherwise(col("final_weight")))

# B. Bonus si une identité est mentionnée (attirer l'attention du
modèle)
df = df.withColumn("final_weight",
                    when(col("has_identity") == True,
col("final_weight") + identity_weight)
                    .otherwise(col("final_weight")))

# C. ANTI-BIAIS (Le plus important) : Non-Toxique + Identité
df = df.withColumn("final_weight",
                    when((col("has_identity") == True) &
(col("label") == 0), col("final_weight") + identity_weight*10)
                    .otherwise(col("final_weight")))

# D. Renforcer les vrais positifs d'identité (Toxique + Identité)
df = df.withColumn("final_weight",
                    when((col("has_identity") == True) &
(col("label") == 1), col("final_weight") + identity_weight*5)
                    .otherwise(col("final_weight")))

return df

# train_debiased = add_bias_mitigation_weights(df_final_w2v,
identity_columns, 0.1, 1)
# train_debiased.select("comment_text", "label", "has_identity",
"final_weight").show(5)

# Ré-entraînement du modèle AVEC poids
lr_weighted = LogisticRegression(
    featuresCol="features_final",
    labelCol="label",
    weightCol="final_weight", # <--- C'est ici que la magie opère
    maxIter=10
)

model_lr_debiased, best_model_lr_debiased, best_params_lr_debiased,
scores_lr_debiased = train_or_load_nested_model(
    lr_weighted, # modèle initial
    lr_grid, # grille de paramètres
    train_debiased, # dataframe
    train_debiased.count(),
    LOAD_MODEL,
    SAVE_MODEL,

```

```
f"identity_biais",
training_func=simple_cross_validator,
combinaison='w2v+tfidf',
k_folds=2
)
```

□ Ré-entraînement avec Class Weights...

Entraînement du modèle : identity_biais_LogisticRegression10000 avec la fonction simple_cross_validator

Exception ignored in: <_io.BufferedWriter name=5>26:> (0 + 1) / 1]

Traceback (most recent call last):

File "/opt/spark/python/lib/pyspark.zip/pyspark/daemon.py", line 200, in manager

BrokenPipeError: [Errno 32] Broken pipe

Démarrage Simple Cross-Validation...

- Taille du jeu de données : 10000
- Nombre de folds (K) : 2
- Hyperparamètres testés : 3
- Total entraînements : 6

Construction du pipeline...

Lancement du CrossValidator (Training)...

25/12/06 06:21:11 WARN DAGScheduler: Broadcasting large task binary with size 1075.2 KiB

25/12/06 06:21:11 WARN DAGScheduler: Broadcasting large task binary with size 1075.2 KiB

25/12/06 06:21:12 WARN DAGScheduler: Broadcasting large task binary with size 1075.8 KiB

25/12/06 06:21:12 WARN DAGScheduler: Broadcasting large task binary with size 1075.8 KiB

25/12/06 06:21:12 WARN DAGScheduler: Broadcasting large task binary with size 1075.8 KiB

25/12/06 06:21:12 WARN DAGScheduler: Broadcasting large task binary with size 1075.8 KiB

25/12/06 06:21:12 WARN DAGScheduler: Broadcasting large task binary with size 1075.8 KiB

25/12/06 06:21:12 WARN DAGScheduler: Broadcasting large task binary with size 1075.8 KiB

25/12/06 06:21:12 WARN DAGScheduler: Broadcasting large task binary with size 1075.8 KiB

25/12/06 06:21:12 WARN DAGScheduler: Broadcasting large task binary with size 1075.8 KiB

25/12/06 06:21:12 WARN DAGScheduler: Broadcasting large task binary

[illegible]

[illegible]

```
with size 1052.2 KiB  
25/12/06 06:21:19 WARN DAGScheduler: Broadcasting large task binary  
with size 1052.2 KiB  
25/12/06 06:21:19 WARN DAGScheduler: Broadcasting large task binary  
with size 1052.2 KiB  
25/12/06 06:21:19 WARN DAGScheduler: Broadcasting large task binary  
with size 1052.2 KiB  
25/12/06 06:21:19 WARN DAGScheduler: Broadcasting large task binary  
with size 1052.2 KiB  
25/12/06 06:21:19 WARN DAGScheduler: Broadcasting large task binary  
with size 1052.2 KiB  
25/12/06 06:21:20 WARN DAGScheduler: Broadcasting large task binary  
with size 1097.1 KiB  
25/12/06 06:21:20 WARN DAGScheduler: Broadcasting large task binary  
with size 1097.1 KiB  
25/12/06 06:21:22 WARN DAGScheduler: Broadcasting large task binary  
with size 1051.7 KiB  
25/12/06 06:21:22 WARN DAGScheduler: Broadcasting large task binary  
with size 1052.2 KiB  
25/12/06 06:21:22 WARN DAGScheduler: Broadcasting large task binary  
with size 1052.2 KiB  
25/12/06 06:21:22 WARN DAGScheduler: Broadcasting large task binary  
with size 1052.2 KiB  
25/12/06 06:21:22 WARN DAGScheduler: Broadcasting large task binary  
with size 1052.2 KiB  
25/12/06 06:21:22 WARN DAGScheduler: Broadcasting large task binary  
with size 1052.2 KiB  
25/12/06 06:21:22 WARN DAGScheduler: Broadcasting large task binary  
with size 1052.2 KiB  
25/12/06 06:21:22 WARN DAGScheduler: Broadcasting large task binary  
with size 1052.2 KiB  
25/12/06 06:21:22 WARN DAGScheduler: Broadcasting large task binary  
with size 1052.2 KiB  
25/12/06 06:21:23 WARN DAGScheduler: Broadcasting large task binary  
with size 1052.2 KiB  
25/12/06 06:21:23 WARN DAGScheduler: Broadcasting large task binary  
with size 1052.2 KiB  
25/12/06 06:21:23 WARN DAGScheduler: Broadcasting large task binary  
with size 1052.2 KiB  
25/12/06 06:21:23 WARN DAGScheduler: Broadcasting large task binary  
with size 1097.1 KiB  
25/12/06 06:21:27 WARN DAGScheduler: Broadcasting large task binary  
with size 1739.4 KiB  
25/12/06 06:21:27 WARN DAGScheduler: Broadcasting large task binary  
with size 1740.0 KiB  
25/12/06 06:21:28 WARN DAGScheduler: Broadcasting large task binary  
with size 1740.0 KiB
```

```
25/12/06 06:21:28 WARN DAGScheduler: Broadcasting large task binary
with size 1740.0 KiB
25/12/06 06:21:28 WARN DAGScheduler: Broadcasting large task binary
with size 1740.0 KiB
25/12/06 06:21:28 WARN DAGScheduler: Broadcasting large task binary
with size 1740.0 KiB
25/12/06 06:21:28 WARN DAGScheduler: Broadcasting large task binary
with size 1740.0 KiB
25/12/06 06:21:28 WARN DAGScheduler: Broadcasting large task binary
with size 1740.0 KiB
25/12/06 06:21:28 WARN DAGScheduler: Broadcasting large task binary
with size 1740.0 KiB
25/12/06 06:21:29 WARN DAGScheduler: Broadcasting large task binary
with size 1740.0 KiB
25/12/06 06:21:29 WARN DAGScheduler: Broadcasting large task binary
with size 1740.0 KiB
25/12/06 06:21:29 WARN DAGScheduler: Broadcasting large task binary
with size 1740.0 KiB
25/12/06 06:21:29 WARN DAGScheduler: Broadcasting large task binary
with size 1740.0 KiB
```

Recherche terminée.

```
=====
RÉSULTATS
```

```
=====
Meilleur AUC moyen (Validation) : 0.7890
```

Approche finale : On prend tout le dataset en veillant à garder un ratio de 1:1 (toxique/non toxique)

- combinaison retenue "w2c + tfidf", avec une colonne bigrams, les autres colonnes du feature engineering n'augmentant quasiment pas l'AUC-ROC mais augmentant considérablement le nombre de paramètres nous ne les utiliseront pas.
- Etant donné que nous n'avons pas beaucoup de ressources avec notre VM, cela nous permettra d'alléger les entrées de nos modèles, tout en gérant le biais introduit par le déséquilibre des classes
- Correction des biais d'identité en augmentant les poids :
 - Poids de base = 1.0
 - Si Toxique (Label 1) : Poids += 5.0 (Pour gérer le déséquilibre global)
 - Si Identité présente : Poids += 0.5 (Booster l'importance des identités)
 - Si Identité présente ET Sain (Label 0) : Poids += 10.0 (C'est CRUCIAL : Anti-Biais)
 - Si Identité présente ET Toxique (Label 1) : Poids += 5.0

3.3 Définition des fonctions pour le choix du preprocessing et de l'entrainement final

3.3.1 Définition d'une fonction de preprocessing statique

Cette fonction exécute toutes les étapes de prétraitement "lourdes" et invariantes --> celles qui ne changent pas lorsqu'on ajuste les hyperparamètres d'équilibrage et de pondération. --> + mise en cache du df pour tester différents prétraitements par la suite

1. Prétraitement initial complet*
 - nettoyage (lowercase, URL, emojis, ponctuation, stopwords...)
 - tokenisation
 - normalisation
1. Split Train/Test
 - split 90% / 10%
1. Ajout des bigrams
2. Feature engineering
3. Word2Vec (W2V)
 - produit un vecteur dense 100 dimensions

À ce stade, chaque ligne contient :

- tokens
 - bigrams
 - features numériques (feature engineering)
 - vecteur Word2Vec
 - identités
 - label
1. Mise en cache

```
from pyspark.ml.classification import LogisticRegression
from pyspark.ml.evaluation import BinaryClassificationEvaluator
import itertools
import pandas as pd

def preprocessing_static_pipeline(train, limit: int=None,
                                debug=False):
    """
        Exécute les étapes de prétraitement Spark lourdes et statiques
        (nettoyage, features, W2V).
        Ces étapes ne dépendent pas des hyperparamètres de
        pondération/d'équilibrage.

        Args:
            train (DataFrame): Le DataFrame d'entrée initial (non traité).
```

limit (int): Limite facultative du nombre de lignes pour le prétraitement.

debug (bool): Active le mode débogage pour la fonction preprocess.

Returns:

tuple[DataFrame, DataFrame]: (df_train_final_all_w2v mis en cache, df_test_final_all_w2v mis en cache)
"""

```
import pyspark.sql.functions as F # Importation locale pour la fonction
```

```
print(">>> Démarrage du pré-traitement statique...")
```

```
# 1. PRÉTRAITEMENT INITIAL
```

```
df_tronq_all, df_preprocess_comments = preprocess(train, limit=limit, debug=debug)
df_clean_final = df_preprocess_comments
```

```
# 2. SPLIT
```

```
seed_split = 42
```

```
# Utilisation d'une seed différente du split de l'Outer CV pour garantir l'indépendance
```

```
train_df_all, test_df_all = df_clean_final.randomSplit([0.9, 0.1], seed=seed_split)
```

```
print("\n>>> Split OK (via randomSplit)")
```

```
print(f"Train = {train_df_all.count()} | Test = {test_df_all.count()}")
```

```
# 3. FEATURE ENGINEERING ET W2V
```

```
train_df_all_bigrams = add_bigrams(train_df_all)
```

```
test_df_all_bigrams = add_bigrams(test_df_all)
```

```
df_train_all_feat_eng = extract_text_features(train_df_all_bigrams)
```

```
df_test_all_feat_eng = extract_text_features(test_df_all_bigrams)
```

```
# Note: w2v_model doit être un modèle entraîné disponible globalement
```

```
df_train_final_all_w2v =
```

```
w2v_model.transform(df_train_all_feat_eng)
```

```
df_test_final_all_w2v = w2v_model.transform(df_test_all_feat_eng)
```

```
# Déclenchement de l'action de cache (via count)
```

```
print(f"Base d'entraînement en cache : {df_train_final_all_w2v.count()} lignes")
```

```
print(f"Base de test en cache : {df_test_final_all_w2v.count()} lignes")
```

```
# 4. MISE EN CACHE
```

```

df_train_final_all_w2v.cache()
df_test_final_all_w2v.cache()

return df_train_final_all_w2v, df_test_final_all_w2v
df_train_final_all_w2v, df_test_final_all_w2v =
preprocessing_static_pipeline(train, limit=12000, debug=False)
>>> Démarrage du pré-traitement statique...
>>> Split OK (via randomSplit)

Train = 10801 | Test = 1199

Base d'entraînement en cache : 10801 lignes

[Stage 17:> (0
+ 1) / 1]

Base de test en cache : 1199 lignes

```

3.3.2 Définition d'une nouvelle fonction pour l'undersampling

- On redéfinit la fonction d'undersampling --> le problème de la dernière était qu'elle enlevait aussi les colonnes contenant des identités
 - On appelle Groupe A = Positifs (toxiques) — toujours garder
 - Groupe B = Négatifs avec identités — toujours garder
 - Groupe C = Négatifs sans identités — à sous-échantillonner

```

from pyspark.sql.functions import col, lit, when, size, split,
array_contains, expr
from typing import List, Optional

def get_identity_condition(identity_cols: List[str]):
    """Crée une expression Spark SQL qui renvoie True si au moins une
    identité est présente."""
    condition_str = " OR ".join([f"({c} >= 0.5)" for c in
identity_cols])
    return expr(condition_str)

def undersample_data_debiased(df, identity_cols, ratio_target=1.0):
    """
    Sous-échantillonne la classe négative (0) en protégeant les

```

exemples contenant des identités.

Args:

df: DataFrame Spark d'entraînement complet.

identity_cols: Liste des colonnes d'identité.

ratio_target: Ratio désiré (Négatifs / Positifs). 1.0 = 50/50.

Returns:

DataFrame: DataFrame sous-échantillonné et dé-biaisé.

"""

1. Séparation des données en 4 groupes

A. Les Positifs (Toxiques) - TOUJOURS TOUT GARDER

pos_df = df.filter(col("label") == 1)

B. Négatifs AVEC Identité (Les "exemples anti-biais") - TOUJOURS TOUT GARDER

neg_with_id_df = df.filter((col("label") == 0) &
(get_identity_condition(identity_cols)))

C. Négatifs SANS Identité (La masse à sous-échantillonner)

neg_without_id_df = df.filter((col("label") == 0) &
~(get_identity_condition(identity_cols)))

2. Calcul du ratio d'échantillonnage pour le Groupe C

count_pos = pos_df.count()

count_neg_with_id = neg_with_id_df.count()

count_neg_without_id = neg_without_id_df.count()

guaranteed_neg_count = count_neg_with_id

required_neg_from_c = (count_pos * ratio_target) -
guaranteed_neg_count

if required_neg_from_c <= 0:

fraction = 0.0

else:

fraction = required_neg_from_c / count_neg_without_id

fraction = max(0.0, min(1.0, fraction))

print(f"\n--- Statut Undersampling Dé-Biaisé ---")

print(f"Total Toxiques (Pos) : {count_pos}")

print(f"Négatifs Protégés (Groupe B) : {count_neg_with_id}")

print(f"Négatifs à Échantillonner (Groupe C) :

{count_neg_without_id}")

print(f"Fraction à garder du Groupe C : {fraction:.4f}")

```
({fraction:.2%})")

# 3. Échantillonnage et Union

neg_sampled = neg_without_id_df.sample(withReplacement=False,
fraction=fraction, seed=42)

df_final = pos_df.union(neg_with_id_df).union(neg_sampled)

return df_final
```

3.3.3 Définition d'une fonction de preprocessing dynamique

Cette fonction constitue la dernière étape du prétraitement avant l'entraînement du modèle. Elle combine deux éléments essentiels pour lutter contre les déséquilibres de classe : Rééquilibrage des classes (undersampling dé-biaisé)

- via `undersample_data_debiased`

Pondération dynamique des exemples sensibles

- via `add_bias_mitigation_weights`

Cette fonction permet de faire la dernière étape du préprocessing avec différents ratio non toxiques/toxiques + différents poids pour les commentaires toxiques + différents poids pour les identités

```
def
preprocessing_final_dynamic(df_train_final_all_w2v=df_train_final_all_
w2v, ratio_target=1, ratio_multiplier=0.1, identity_weight=0.5):

    # Liste complète des identités (utilisée pour l'analyse)
    IDENTITY_COLUMNS = [
        'male', 'female', 'transgender', 'other_gender',
        'heterosexual', 'homosexual_gay_or_lesbian', 'bisexual',
        'other_sexual_orientation',
        'christian', 'jewish', 'muslim', 'hindu', 'buddhist',
        'atheist', 'other_religion',
        'black', 'white', 'asian', 'latino',
        'other_race_or_ethnicity',
        'physical_disability', 'intellectual_or_learning_disability',
        'psychiatric_or_mental_illness', 'other_disability'
    ]

    df_train_final_balanced =
undersample_data_debiased(df_train_final_all_w2v, IDENTITY_COLUMNS,
ratio_target=ratio_target)
    df_train_final_balanced_debiased =
add_bias_mitigation_weights(df_train_final_balanced, identity_columns,
ratio_multiplier=ratio_multiplier, identity_weight=identity_weight)
```

```

    print(f"Prétraitement terminé, df_train_final_balanced_debiased =
{df_train_final_balanced_debiased.count()} | df_test_final_all_w2v =
{df_test_final_all_w2v.count()}")

    return df_train_final_balanced_debiased

```

3.3.4 Définition d'une grid de paramètres

On va tester différentes combinaisons :

- ratio_target : contrôle le ratio d'équilibrage de classe.
- ratio_multiplier : pondération supplémentaire pour lignes avec label =1
- identity_weight : pondération supplémentaire donnée aux lignes liées aux identités.
- model : la liste des modèles à tester (LR, RF, SVM, GBT pondérés).
- combinaison : quelles combinaisons de features utiliser

```

grid_params = {
    'ratio_target': [3, 5, 7],          # Equilibre (1) ou majorité de 0
    (1.5, 2)
    'ratio_multiplier': [0.05,0.1],    # Force de la multiplication de
    biais
    'identity_weight': [0.5, 1.0],# Bonus pour les identités
    'model': [lr_weighted, rf_weighted, svm_weighted, gbt_weighted],
    # Modèles à tester
    'combinaison': ['w2v + tfidf']      # Les combinaisons de features
    définies dans build_pipeline
}

# Création de toutes les combinaisons possibles
keys, values = zip(*grid_params.items())
combinations_list = [dict(zip(keys, v)) for v in
itertools.product(*values)]

all_results = []

```

3.3.5 Tests des paramètres

```

from pyspark.ml.classification import LogisticRegression
from pyspark.ml.evaluation import BinaryClassificationEvaluator,
MulticlassClassificationEvaluator # <-- VÉRIFIE QUE CECI EST EXÉCUTÉ
import itertools
import time

print(f"\n--- Démarrage de l'itération sur {len(combinations_list)}
configurations ---")

```

```

for i, config in enumerate(combinations_list):
    start_time = time.time()
    print(f"\n[{i+1}/{len(combinations_list)}] Test config: {config}")

    try:
        # 1. Préparation des données (Dynamique)
        train_data_ready = preprocessing_final_dynamic(
            df_train_final_all_w2v,
            ratio_target=config['ratio_target'],
            ratio_multiplieur=config['ratio_multiplieur'],
            identity_weight=config['identity_weight']
        )

        # 2. Construction du Pipeline / Modèle
        pipeline = build_pipeline_part2(
            model=config['model'],
            combinaison=config['combinaison']
        )

        # 3. Entraînement
        # Le pipeline va chercher la colonne 'final_weight' créée dans
        preprocessing_final_dynamic
        model_fitted = pipeline.fit(train_data_ready)

        # 4. Prédiction sur le TEST SET (Le vrai test set, jamais
        touché par undersampling)
        preds = model_fitted.transform(df_test_final_all_w2v)

        # 5. Évaluation (Métriques Globales)
        evaluator_auc =
        BinaryClassificationEvaluator(labelCol="label",
            rawPredictionCol="rawPrediction", metricName="areaUnderROC")
        evaluator_acc =
        MulticlassClassificationEvaluator(labelCol="label",
            predictionCol="prediction", metricName="accuracy")
        evaluator_f1 =
        MulticlassClassificationEvaluator(labelCol="label",
            predictionCol="prediction", metricName="f1")

        auc_score = evaluator_auc.evaluate(preds)
        acc_score = evaluator_acc.evaluate(preds)
        f1_score = evaluator_f1.evaluate(preds)

        duration = time.time() - start_time
        print(f"    -> Result: AUC={auc_score:.4f} | F1={f1_score:.4f}
        | Time={duration:.1f}s")

        # 6. Stockage
        result_entry = config.copy()

```

```

    result_entry['AUC'] = auc_score
    result_entry['Accuracy'] = acc_score
    result_entry['F1'] = f1_score
    result_entry['Training_Size'] = train_data_ready.count() #
Pour voir l'effet de l'undersampling

```

```

    all_results.append(result_entry)

except Exception as e:
    print(f"    !!! ERREUR sur cette config : {e}")

```

>>> Démarrage de l'itération sur 48 configurations...

```

[1/48] Test config: {'ratio_target': 3, 'ratio_multiplieur': 0.05,
'identity_weight': 0.5, 'model': LogisticRegression_afae24335158,
'combinaison': 'w2v + tfidf'}

```

Undersampling : On garde 18.14% des données négatives.

Ratio de sain/toxique 3.15

toxic_weights : 0.16 + 1, identity_weight :0.5

Prétraitement terminé, df_train_final_balanced_debiased = 2556 |

df_test_final_all_w2v = 1199

-> Result: AUC=0.8103 | F1=0.8667 | Time=2.7s

```

[2/48] Test config: {'ratio_target': 3, 'ratio_multiplieur': 0.05,
'identity_weight': 0.5, 'model': RandomForestClassifier_ad394a6e557e,
'combinaison': 'w2v + tfidf'}

```

Undersampling : On garde 18.14% des données négatives.

Ratio de sain/toxique 3.15

toxic_weights : 0.16 + 1, identity_weight :0.5

Prétraitement terminé, df_train_final_balanced_debiased = 2556 |

df_test_final_all_w2v = 1199

25/12/06 06:24:17 WARN DAGScheduler: Broadcasting large task binary with size 1008.4 KiB

25/12/06 06:24:17 WARN DAGScheduler: Broadcasting large task binary with size 1031.5 KiB

25/12/06 06:24:18 WARN DAGScheduler: Broadcasting large task binary with size 1064.2 KiB

-> Result: AUC=0.7896 | F1=0.9092 | Time=3.3s

```

[3/48] Test config: {'ratio_target': 3, 'ratio_multiplieur': 0.05,
'identity_weight': 0.5, 'model': LinearSVC_8f0c0322b70e,
'combinaison': 'w2v + tfidf'}

```

Undersampling : On garde 18.14% des données négatives.

Ratio de sain/toxique 3.15

toxic_weights : 0.16 + 1, identity_weight :0.5

Prétraitement terminé, df_train_final_balanced_debiased = 2556 |

df_test_final_all_w2v = 1199

-> Result: AUC=0.8229 | F1=0.8833 | Time=3.9s

[4/48] Test config: {'ratio_target': 3, 'ratio_multiplier': 0.05, 'identity_weight': 0.5, 'model': GBTCClassifier_3b8b769d6280, 'combinaison': 'w2v + tfidf'}

Undersampling : On garde 18.14% des données négatives.

Ratio de sain/toxique 3.15

toxic_weights : 0.16 + 1, identity_weight :0.5

Prétraitement terminé, df_train_final_balanced_debiased = 2556 |
df_test_final_all_w2v = 1199

25/12/06 06:24:26 WARN DAGScheduler: Broadcasting large task binary with size 1000.5 KiB

25/12/06 06:24:26 WARN DAGScheduler: Broadcasting large task binary with size 1002.5 KiB

25/12/06 06:24:26 WARN DAGScheduler: Broadcasting large task binary with size 1004.8 KiB

25/12/06 06:24:26 WARN DAGScheduler: Broadcasting large task binary with size 1005.3 KiB

25/12/06 06:24:26 WARN DAGScheduler: Broadcasting large task binary with size 1005.9 KiB

25/12/06 06:24:27 WARN DAGScheduler: Broadcasting large task binary with size 1007.1 KiB

25/12/06 06:24:27 WARN DAGScheduler: Broadcasting large task binary with size 1009.3 KiB

25/12/06 06:24:27 WARN DAGScheduler: Broadcasting large task binary with size 1011.5 KiB

25/12/06 06:24:27 WARN DAGScheduler: Broadcasting large task binary with size 1012.0 KiB

25/12/06 06:24:27 WARN DAGScheduler: Broadcasting large task binary with size 1012.6 KiB

25/12/06 06:24:27 WARN DAGScheduler: Broadcasting large task binary with size 1013.8 KiB

25/12/06 06:24:27 WARN DAGScheduler: Broadcasting large task binary with size 1016.0 KiB

25/12/06 06:24:27 WARN DAGScheduler: Broadcasting large task binary with size 1018.2 KiB

25/12/06 06:24:27 WARN DAGScheduler: Broadcasting large task binary with size 1018.7 KiB

25/12/06 06:24:27 WARN DAGScheduler: Broadcasting large task binary with size 1019.3 KiB

25/12/06 06:24:27 WARN DAGScheduler: Broadcasting large task binary with size 1020.5 KiB

25/12/06 06:24:27 WARN DAGScheduler: Broadcasting large task binary with size 1022.7 KiB

25/12/06 06:24:27 WARN DAGScheduler: Broadcasting large task binary with size 1024.6 KiB

25/12/06 06:24:27 WARN DAGScheduler: Broadcasting large task binary with size 1025.1 KiB

25/12/06 06:24:27 WARN DAGScheduler: Broadcasting large task binary

with size 1025.6 KiB
25/12/06 06:24:28 WARN DAGScheduler: Broadcasting large task binary
with size 1026.8 KiB
25/12/06 06:24:28 WARN DAGScheduler: Broadcasting large task binary
with size 1028.6 KiB
25/12/06 06:24:28 WARN DAGScheduler: Broadcasting large task binary
with size 1030.5 KiB
25/12/06 06:24:28 WARN DAGScheduler: Broadcasting large task binary
with size 1031.0 KiB
25/12/06 06:24:28 WARN DAGScheduler: Broadcasting large task binary
with size 1031.5 KiB
25/12/06 06:24:28 WARN DAGScheduler: Broadcasting large task binary
with size 1032.7 KiB
25/12/06 06:24:28 WARN DAGScheduler: Broadcasting large task binary
with size 1035.0 KiB
25/12/06 06:24:28 WARN DAGScheduler: Broadcasting large task binary
with size 1037.7 KiB
25/12/06 06:24:28 WARN DAGScheduler: Broadcasting large task binary
with size 1038.2 KiB
25/12/06 06:24:28 WARN DAGScheduler: Broadcasting large task binary
with size 1038.7 KiB
25/12/06 06:24:28 WARN DAGScheduler: Broadcasting large task binary
with size 1039.9 KiB
25/12/06 06:24:28 WARN DAGScheduler: Broadcasting large task binary
with size 1041.7 KiB
25/12/06 06:24:28 WARN DAGScheduler: Broadcasting large task binary
with size 1044.1 KiB
25/12/06 06:24:28 WARN DAGScheduler: Broadcasting large task binary
with size 1044.6 KiB
25/12/06 06:24:28 WARN DAGScheduler: Broadcasting large task binary
with size 1045.1 KiB
25/12/06 06:24:28 WARN DAGScheduler: Broadcasting large task binary
with size 1046.3 KiB
25/12/06 06:24:29 WARN DAGScheduler: Broadcasting large task binary
with size 1048.6 KiB
25/12/06 06:24:29 WARN DAGScheduler: Broadcasting large task binary
with size 1050.8 KiB
25/12/06 06:24:29 WARN DAGScheduler: Broadcasting large task binary
with size 1051.3 KiB
25/12/06 06:24:29 WARN DAGScheduler: Broadcasting large task binary
with size 1051.8 KiB
25/12/06 06:24:29 WARN DAGScheduler: Broadcasting large task binary
with size 1053.0 KiB
25/12/06 06:24:29 WARN DAGScheduler: Broadcasting large task binary
with size 1055.0 KiB
25/12/06 06:24:29 WARN DAGScheduler: Broadcasting large task binary
with size 1056.8 KiB
25/12/06 06:24:29 WARN DAGScheduler: Broadcasting large task binary
with size 1057.3 KiB

25/12/06 06:24:29 WARN DAGScheduler: Broadcasting large task binary with size 1057.9 KiB
25/12/06 06:24:29 WARN DAGScheduler: Broadcasting large task binary with size 1059.1 KiB
25/12/06 06:24:29 WARN DAGScheduler: Broadcasting large task binary with size 1061.3 KiB
25/12/06 06:24:29 WARN DAGScheduler: Broadcasting large task binary with size 1063.4 KiB
25/12/06 06:24:29 WARN DAGScheduler: Broadcasting large task binary with size 1063.9 KiB
25/12/06 06:24:29 WARN DAGScheduler: Broadcasting large task binary with size 1064.4 KiB
25/12/06 06:24:29 WARN DAGScheduler: Broadcasting large task binary with size 1065.6 KiB
25/12/06 06:24:29 WARN DAGScheduler: Broadcasting large task binary with size 1067.6 KiB
25/12/06 06:24:30 WARN DAGScheduler: Broadcasting large task binary with size 1070.1 KiB
25/12/06 06:24:30 WARN DAGScheduler: Broadcasting large task binary with size 1070.6 KiB
25/12/06 06:24:30 WARN DAGScheduler: Broadcasting large task binary with size 1071.1 KiB
25/12/06 06:24:30 WARN DAGScheduler: Broadcasting large task binary with size 1072.3 KiB
25/12/06 06:24:30 WARN DAGScheduler: Broadcasting large task binary with size 1074.1 KiB
25/12/06 06:24:30 WARN DAGScheduler: Broadcasting large task binary with size 1076.1 KiB
25/12/06 06:24:30 WARN DAGScheduler: Broadcasting large task binary with size 1076.6 KiB
25/12/06 06:24:30 WARN DAGScheduler: Broadcasting large task binary with size 1077.2 KiB
25/12/06 06:24:30 WARN DAGScheduler: Broadcasting large task binary with size 1078.4 KiB
25/12/06 06:24:30 WARN DAGScheduler: Broadcasting large task binary with size 1080.1 KiB
25/12/06 06:24:30 WARN DAGScheduler: Broadcasting large task binary with size 1081.9 KiB
25/12/06 06:24:30 WARN DAGScheduler: Broadcasting large task binary with size 1082.4 KiB
25/12/06 06:24:30 WARN DAGScheduler: Broadcasting large task binary with size 1082.9 KiB
25/12/06 06:24:30 WARN DAGScheduler: Broadcasting large task binary with size 1084.1 KiB
25/12/06 06:24:30 WARN DAGScheduler: Broadcasting large task binary with size 1086.1 KiB
25/12/06 06:24:30 WARN DAGScheduler: Broadcasting large task binary with size 1088.2 KiB
25/12/06 06:24:30 WARN DAGScheduler: Broadcasting large task binary

with size 1088.7 KiB
25/12/06 06:24:31 WARN DAGScheduler: Broadcasting large task binary
with size 1089.3 KiB
25/12/06 06:24:31 WARN DAGScheduler: Broadcasting large task binary
with size 1090.5 KiB
25/12/06 06:24:31 WARN DAGScheduler: Broadcasting large task binary
with size 1092.8 KiB
25/12/06 06:24:31 WARN DAGScheduler: Broadcasting large task binary
with size 1095.4 KiB
25/12/06 06:24:31 WARN DAGScheduler: Broadcasting large task binary
with size 1095.9 KiB
25/12/06 06:24:31 WARN DAGScheduler: Broadcasting large task binary
with size 1096.5 KiB
25/12/06 06:24:31 WARN DAGScheduler: Broadcasting large task binary
with size 1097.7 KiB
25/12/06 06:24:31 WARN DAGScheduler: Broadcasting large task binary
with size 1099.9 KiB
25/12/06 06:24:31 WARN DAGScheduler: Broadcasting large task binary
with size 1102.1 KiB
25/12/06 06:24:31 WARN DAGScheduler: Broadcasting large task binary
with size 1102.6 KiB
25/12/06 06:24:31 WARN DAGScheduler: Broadcasting large task binary
with size 1103.2 KiB
25/12/06 06:24:32 WARN DAGScheduler: Broadcasting large task binary
with size 1104.4 KiB
25/12/06 06:24:32 WARN DAGScheduler: Broadcasting large task binary
with size 1106.4 KiB

-> Result: AUC=0.8140 | F1=0.8971 | Time=10.2s

[5/48] Test config: {'ratio_target': 3, 'ratio_multiplier': 0.05,
'identity_weight': 1.0, 'model': LogisticRegression_afae24335158,
'combinaison': 'w2v + tfidf'}

Undersampling : On garde 18.14% des données négatives.

Ratio de sain/toxique 3.15

toxic_weights : 0.16 + 1, identity_weight :1.0

Prétraitement terminé, df_train_final_balanced_debiased = 2556 |

df_test_final_all_w2v = 1199

-> Result: AUC=0.8101 | F1=0.8708 | Time=2.3s

[6/48] Test config: {'ratio_target': 3, 'ratio_multiplier': 0.05,
'identity_weight': 1.0, 'model': RandomForestClassifier_ad394a6e557e,
'combinaison': 'w2v + tfidf'}

Undersampling : On garde 18.14% des données négatives.

Ratio de sain/toxique 3.15

toxic_weights : 0.16 + 1, identity_weight :1.0

Prétraitement terminé, df_train_final_balanced_debiased = 2556 |

df_test_final_all_w2v = 1199

25/12/06 06:24:37 WARN DAGScheduler: Broadcasting large task binary with size 1008.4 KiB
25/12/06 06:24:37 WARN DAGScheduler: Broadcasting large task binary with size 1033.5 KiB
25/12/06 06:24:37 WARN DAGScheduler: Broadcasting large task binary with size 1070.5 KiB

-> Result: AUC=0.7639 | F1=0.9092 | Time=3.3s

[7/48] Test config: {'ratio_target': 3, 'ratio_multiplier': 0.05, 'identity_weight': 1.0, 'model': LinearSVC_8f0c0322b70e, 'combinaison': 'w2v + tfidf'}

Undersampling : On garde 18.14% des données négatives.

Ratio de sain/toxique 3.15

toxic_weights : 0.16 + 1, identity_weight :1.0

Prétraitement terminé, df_train_final_balanced_debiased = 2556 |
df_test_final_all_w2v = 1199

-> Result: AUC=0.8245 | F1=0.8871 | Time=3.8s

[8/48] Test config: {'ratio_target': 3, 'ratio_multiplier': 0.05, 'identity_weight': 1.0, 'model': GBTClassifier_3b8b769d6280, 'combinaison': 'w2v + tfidf'}

Undersampling : On garde 18.14% des données négatives.

Ratio de sain/toxique 3.15

toxic_weights : 0.16 + 1, identity_weight :1.0

Prétraitement terminé, df_train_final_balanced_debiased = 2556 |
df_test_final_all_w2v = 1199

25/12/06 06:24:45 WARN DAGScheduler: Broadcasting large task binary with size 1000.4 KiB

25/12/06 06:24:45 WARN DAGScheduler: Broadcasting large task binary with size 1000.9 KiB

25/12/06 06:24:45 WARN DAGScheduler: Broadcasting large task binary with size 1002.1 KiB

25/12/06 06:24:45 WARN DAGScheduler: Broadcasting large task binary with size 1003.6 KiB

25/12/06 06:24:45 WARN DAGScheduler: Broadcasting large task binary with size 1005.8 KiB

25/12/06 06:24:45 WARN DAGScheduler: Broadcasting large task binary with size 1006.3 KiB

25/12/06 06:24:45 WARN DAGScheduler: Broadcasting large task binary with size 1006.8 KiB

25/12/06 06:24:45 WARN DAGScheduler: Broadcasting large task binary with size 1008.0 KiB

25/12/06 06:24:45 WARN DAGScheduler: Broadcasting large task binary with size 1009.9 KiB

25/12/06 06:24:46 WARN DAGScheduler: Broadcasting large task binary with size 1012.0 KiB

25/12/06 06:24:46 WARN DAGScheduler: Broadcasting large task binary with size 1012.5 KiB

25/12/06 06:24:46 WARN DAGScheduler: Broadcasting large task binary with size 1013.0 KiB
25/12/06 06:24:46 WARN DAGScheduler: Broadcasting large task binary with size 1014.2 KiB
25/12/06 06:24:46 WARN DAGScheduler: Broadcasting large task binary with size 1016.5 KiB
25/12/06 06:24:46 WARN DAGScheduler: Broadcasting large task binary with size 1019.0 KiB
25/12/06 06:24:46 WARN DAGScheduler: Broadcasting large task binary with size 1019.5 KiB
25/12/06 06:24:46 WARN DAGScheduler: Broadcasting large task binary with size 1020.0 KiB
25/12/06 06:24:46 WARN DAGScheduler: Broadcasting large task binary with size 1021.2 KiB
25/12/06 06:24:46 WARN DAGScheduler: Broadcasting large task binary with size 1022.6 KiB
25/12/06 06:24:46 WARN DAGScheduler: Broadcasting large task binary with size 1024.9 KiB
25/12/06 06:24:46 WARN DAGScheduler: Broadcasting large task binary with size 1025.4 KiB
25/12/06 06:24:46 WARN DAGScheduler: Broadcasting large task binary with size 1025.9 KiB
25/12/06 06:24:46 WARN DAGScheduler: Broadcasting large task binary with size 1027.1 KiB
25/12/06 06:24:46 WARN DAGScheduler: Broadcasting large task binary with size 1029.4 KiB
25/12/06 06:24:46 WARN DAGScheduler: Broadcasting large task binary with size 1032.1 KiB
25/12/06 06:24:47 WARN DAGScheduler: Broadcasting large task binary with size 1032.6 KiB
25/12/06 06:24:47 WARN DAGScheduler: Broadcasting large task binary with size 1033.1 KiB
25/12/06 06:24:47 WARN DAGScheduler: Broadcasting large task binary with size 1034.3 KiB
25/12/06 06:24:47 WARN DAGScheduler: Broadcasting large task binary with size 1036.3 KiB
25/12/06 06:24:47 WARN DAGScheduler: Broadcasting large task binary with size 1038.6 KiB
25/12/06 06:24:47 WARN DAGScheduler: Broadcasting large task binary with size 1039.1 KiB
25/12/06 06:24:47 WARN DAGScheduler: Broadcasting large task binary with size 1039.7 KiB
25/12/06 06:24:47 WARN DAGScheduler: Broadcasting large task binary with size 1040.9 KiB
25/12/06 06:24:47 WARN DAGScheduler: Broadcasting large task binary with size 1043.1 KiB
25/12/06 06:24:47 WARN DAGScheduler: Broadcasting large task binary with size 1045.3 KiB
25/12/06 06:24:47 WARN DAGScheduler: Broadcasting large task binary

with size 1045.8 KiB
25/12/06 06:24:47 WARN DAGScheduler: Broadcasting large task binary
with size 1046.4 KiB
25/12/06 06:24:47 WARN DAGScheduler: Broadcasting large task binary
with size 1047.5 KiB
25/12/06 06:24:47 WARN DAGScheduler: Broadcasting large task binary
with size 1049.5 KiB
25/12/06 06:24:47 WARN DAGScheduler: Broadcasting large task binary
with size 1051.9 KiB
25/12/06 06:24:47 WARN DAGScheduler: Broadcasting large task binary
with size 1052.3 KiB
25/12/06 06:24:47 WARN DAGScheduler: Broadcasting large task binary
with size 1052.9 KiB
25/12/06 06:24:47 WARN DAGScheduler: Broadcasting large task binary
with size 1054.1 KiB
25/12/06 06:24:48 WARN DAGScheduler: Broadcasting large task binary
with size 1056.1 KiB
25/12/06 06:24:48 WARN DAGScheduler: Broadcasting large task binary
with size 1058.2 KiB
25/12/06 06:24:48 WARN DAGScheduler: Broadcasting large task binary
with size 1058.7 KiB
25/12/06 06:24:48 WARN DAGScheduler: Broadcasting large task binary
with size 1059.3 KiB
25/12/06 06:24:48 WARN DAGScheduler: Broadcasting large task binary
with size 1060.5 KiB
25/12/06 06:24:48 WARN DAGScheduler: Broadcasting large task binary
with size 1062.8 KiB
25/12/06 06:24:48 WARN DAGScheduler: Broadcasting large task binary
with size 1065.6 KiB
25/12/06 06:24:48 WARN DAGScheduler: Broadcasting large task binary
with size 1066.1 KiB
25/12/06 06:24:48 WARN DAGScheduler: Broadcasting large task binary
with size 1066.6 KiB
25/12/06 06:24:48 WARN DAGScheduler: Broadcasting large task binary
with size 1067.8 KiB
25/12/06 06:24:48 WARN DAGScheduler: Broadcasting large task binary
with size 1070.1 KiB
25/12/06 06:24:48 WARN DAGScheduler: Broadcasting large task binary
with size 1072.3 KiB
25/12/06 06:24:48 WARN DAGScheduler: Broadcasting large task binary
with size 1072.8 KiB
25/12/06 06:24:48 WARN DAGScheduler: Broadcasting large task binary
with size 1073.4 KiB
25/12/06 06:24:48 WARN DAGScheduler: Broadcasting large task binary
with size 1074.6 KiB
25/12/06 06:24:48 WARN DAGScheduler: Broadcasting large task binary
with size 1076.9 KiB
25/12/06 06:24:48 WARN DAGScheduler: Broadcasting large task binary
with size 1079.3 KiB

25/12/06 06:24:49 WARN DAGScheduler: Broadcasting large task binary with size 1079.8 KiB
25/12/06 06:24:49 WARN DAGScheduler: Broadcasting large task binary with size 1080.4 KiB
25/12/06 06:24:49 WARN DAGScheduler: Broadcasting large task binary with size 1081.6 KiB
25/12/06 06:24:49 WARN DAGScheduler: Broadcasting large task binary with size 1083.8 KiB
25/12/06 06:24:49 WARN DAGScheduler: Broadcasting large task binary with size 1086.1 KiB
25/12/06 06:24:49 WARN DAGScheduler: Broadcasting large task binary with size 1086.5 KiB
25/12/06 06:24:49 WARN DAGScheduler: Broadcasting large task binary with size 1087.1 KiB
25/12/06 06:24:49 WARN DAGScheduler: Broadcasting large task binary with size 1088.3 KiB
25/12/06 06:24:49 WARN DAGScheduler: Broadcasting large task binary with size 1090.5 KiB
25/12/06 06:24:49 WARN DAGScheduler: Broadcasting large task binary with size 1092.8 KiB
25/12/06 06:24:49 WARN DAGScheduler: Broadcasting large task binary with size 1093.2 KiB
25/12/06 06:24:49 WARN DAGScheduler: Broadcasting large task binary with size 1093.8 KiB
25/12/06 06:24:49 WARN DAGScheduler: Broadcasting large task binary with size 1095.0 KiB
25/12/06 06:24:49 WARN DAGScheduler: Broadcasting large task binary with size 1097.2 KiB
25/12/06 06:24:49 WARN DAGScheduler: Broadcasting large task binary with size 1099.1 KiB
25/12/06 06:24:49 WARN DAGScheduler: Broadcasting large task binary with size 1099.6 KiB
25/12/06 06:24:49 WARN DAGScheduler: Broadcasting large task binary with size 1100.2 KiB
25/12/06 06:24:50 WARN DAGScheduler: Broadcasting large task binary with size 1101.4 KiB
25/12/06 06:24:50 WARN DAGScheduler: Broadcasting large task binary with size 1103.3 KiB
25/12/06 06:24:50 WARN DAGScheduler: Broadcasting large task binary with size 1105.5 KiB
25/12/06 06:24:50 WARN DAGScheduler: Broadcasting large task binary with size 1106.0 KiB
25/12/06 06:24:50 WARN DAGScheduler: Broadcasting large task binary with size 1106.6 KiB
25/12/06 06:24:50 WARN DAGScheduler: Broadcasting large task binary with size 1107.8 KiB
25/12/06 06:24:50 WARN DAGScheduler: Broadcasting large task binary with size 1109.8 KiB

-> Result: AUC=0.7903 | F1=0.8996 | Time=8.4s

[9/48] Test config: {'ratio_target': 3, 'ratio_multiplieur': 0.1, 'identity_weight': 0.5, 'model': LogisticRegression_afae24335158, 'combinaison': 'w2v + tfidf'}

Undersampling : On garde 18.14% des données négatives.

Ratio de sain/toxique 3.15

toxic_weights : 0.31 + 1, identity_weight :0.5

Prétraitement terminé, df_train_final_balanced_debiased = 2556 |

df_test_final_all_w2v = 1199

-> Result: AUC=0.8054 | F1=0.8656 | Time=1.9s

[10/48] Test config: {'ratio_target': 3, 'ratio_multiplieur': 0.1, 'identity_weight': 0.5, 'model': RandomForestClassifier_ad394a6e557e, 'combinaison': 'w2v + tfidf'}

Undersampling : On garde 18.14% des données négatives.

Ratio de sain/toxique 3.15

toxic_weights : 0.31 + 1, identity_weight :0.5

Prétraitement terminé, df_train_final_balanced_debiased = 2556 |

df_test_final_all_w2v = 1199

25/12/06 06:24:55 WARN DAGScheduler: Broadcasting large task binary with size 1008.9 KiB

25/12/06 06:24:55 WARN DAGScheduler: Broadcasting large task binary with size 1034.0 KiB

25/12/06 06:24:55 WARN DAGScheduler: Broadcasting large task binary with size 1071.6 KiB

-> Result: AUC=0.8004 | F1=0.9112 | Time=3.3s

[11/48] Test config: {'ratio_target': 3, 'ratio_multiplieur': 0.1, 'identity_weight': 0.5, 'model': LinearSVC_8f0c0322b70e, 'combinaison': 'w2v + tfidf'}

Undersampling : On garde 18.14% des données négatives.

Ratio de sain/toxique 3.15

toxic_weights : 0.31 + 1, identity_weight :0.5

Prétraitement terminé, df_train_final_balanced_debiased = 2556 |

df_test_final_all_w2v = 1199

-> Result: AUC=0.8199 | F1=0.8868 | Time=3.7s

[12/48] Test config: {'ratio_target': 3, 'ratio_multiplieur': 0.1, 'identity_weight': 0.5, 'model': GBTClassifier_3b8b769d6280, 'combinaison': 'w2v + tfidf'}

Undersampling : On garde 18.14% des données négatives.

Ratio de sain/toxique 3.15

toxic_weights : 0.31 + 1, identity_weight :0.5

Prétraitement terminé, df_train_final_balanced_debiased = 2556 |

df_test_final_all_w2v = 1199

25/12/06 06:25:02 WARN DAGScheduler: Broadcasting large task binary with size 1000.4 KiB
25/12/06 06:25:02 WARN DAGScheduler: Broadcasting large task binary with size 1000.9 KiB
25/12/06 06:25:02 WARN DAGScheduler: Broadcasting large task binary with size 1002.1 KiB
25/12/06 06:25:02 WARN DAGScheduler: Broadcasting large task binary with size 1004.3 KiB
25/12/06 06:25:02 WARN DAGScheduler: Broadcasting large task binary with size 1006.4 KiB
25/12/06 06:25:03 WARN DAGScheduler: Broadcasting large task binary with size 1006.9 KiB
25/12/06 06:25:03 WARN DAGScheduler: Broadcasting large task binary with size 1007.5 KiB
25/12/06 06:25:03 WARN DAGScheduler: Broadcasting large task binary with size 1008.7 KiB
25/12/06 06:25:03 WARN DAGScheduler: Broadcasting large task binary with size 1010.9 KiB
25/12/06 06:25:03 WARN DAGScheduler: Broadcasting large task binary with size 1013.3 KiB
25/12/06 06:25:03 WARN DAGScheduler: Broadcasting large task binary with size 1013.8 KiB
25/12/06 06:25:03 WARN DAGScheduler: Broadcasting large task binary with size 1014.3 KiB
25/12/06 06:25:03 WARN DAGScheduler: Broadcasting large task binary with size 1015.5 KiB
25/12/06 06:25:03 WARN DAGScheduler: Broadcasting large task binary with size 1017.7 KiB
25/12/06 06:25:03 WARN DAGScheduler: Broadcasting large task binary with size 1019.8 KiB
25/12/06 06:25:03 WARN DAGScheduler: Broadcasting large task binary with size 1020.3 KiB
25/12/06 06:25:03 WARN DAGScheduler: Broadcasting large task binary with size 1020.9 KiB
25/12/06 06:25:03 WARN DAGScheduler: Broadcasting large task binary with size 1022.1 KiB
25/12/06 06:25:03 WARN DAGScheduler: Broadcasting large task binary with size 1024.1 KiB
25/12/06 06:25:03 WARN DAGScheduler: Broadcasting large task binary with size 1026.4 KiB
25/12/06 06:25:03 WARN DAGScheduler: Broadcasting large task binary with size 1026.9 KiB
25/12/06 06:25:04 WARN DAGScheduler: Broadcasting large task binary with size 1027.4 KiB
25/12/06 06:25:04 WARN DAGScheduler: Broadcasting large task binary with size 1028.6 KiB
25/12/06 06:25:04 WARN DAGScheduler: Broadcasting large task binary with size 1030.0 KiB
25/12/06 06:25:04 WARN DAGScheduler: Broadcasting large task binary with size 1032.3 KiB

25/12/06 06:25:04 WARN DAGScheduler: Broadcasting large task binary with size 1032.8 KiB
25/12/06 06:25:04 WARN DAGScheduler: Broadcasting large task binary with size 1033.3 KiB
25/12/06 06:25:04 WARN DAGScheduler: Broadcasting large task binary with size 1034.5 KiB
25/12/06 06:25:04 WARN DAGScheduler: Broadcasting large task binary with size 1036.5 KiB
25/12/06 06:25:04 WARN DAGScheduler: Broadcasting large task binary with size 1038.5 KiB
25/12/06 06:25:04 WARN DAGScheduler: Broadcasting large task binary with size 1039.0 KiB
25/12/06 06:25:04 WARN DAGScheduler: Broadcasting large task binary with size 1039.6 KiB
25/12/06 06:25:04 WARN DAGScheduler: Broadcasting large task binary with size 1040.7 KiB
25/12/06 06:25:04 WARN DAGScheduler: Broadcasting large task binary with size 1043.0 KiB
25/12/06 06:25:04 WARN DAGScheduler: Broadcasting large task binary with size 1045.2 KiB
25/12/06 06:25:04 WARN DAGScheduler: Broadcasting large task binary with size 1045.7 KiB
25/12/06 06:25:04 WARN DAGScheduler: Broadcasting large task binary with size 1046.3 KiB
25/12/06 06:25:05 WARN DAGScheduler: Broadcasting large task binary with size 1047.4 KiB
25/12/06 06:25:05 WARN DAGScheduler: Broadcasting large task binary with size 1049.2 KiB
25/12/06 06:25:05 WARN DAGScheduler: Broadcasting large task binary with size 1051.4 KiB
25/12/06 06:25:05 WARN DAGScheduler: Broadcasting large task binary with size 1051.9 KiB
25/12/06 06:25:05 WARN DAGScheduler: Broadcasting large task binary with size 1052.5 KiB
25/12/06 06:25:05 WARN DAGScheduler: Broadcasting large task binary with size 1053.7 KiB
25/12/06 06:25:05 WARN DAGScheduler: Broadcasting large task binary with size 1055.9 KiB
25/12/06 06:25:05 WARN DAGScheduler: Broadcasting large task binary with size 1058.1 KiB
25/12/06 06:25:05 WARN DAGScheduler: Broadcasting large task binary with size 1058.6 KiB
25/12/06 06:25:05 WARN DAGScheduler: Broadcasting large task binary with size 1059.2 KiB
25/12/06 06:25:05 WARN DAGScheduler: Broadcasting large task binary with size 1060.4 KiB
25/12/06 06:25:05 WARN DAGScheduler: Broadcasting large task binary with size 1062.6 KiB
25/12/06 06:25:05 WARN DAGScheduler: Broadcasting large task binary with size 1064.7 KiB

25/12/06 06:25:05 WARN DAGScheduler: Broadcasting large task binary with size 1065.2 KiB
25/12/06 06:25:05 WARN DAGScheduler: Broadcasting large task binary with size 1065.7 KiB
25/12/06 06:25:05 WARN DAGScheduler: Broadcasting large task binary with size 1066.9 KiB
25/12/06 06:25:06 WARN DAGScheduler: Broadcasting large task binary with size 1068.9 KiB
25/12/06 06:25:06 WARN DAGScheduler: Broadcasting large task binary with size 1071.1 KiB
25/12/06 06:25:06 WARN DAGScheduler: Broadcasting large task binary with size 1071.5 KiB
25/12/06 06:25:06 WARN DAGScheduler: Broadcasting large task binary with size 1072.1 KiB
25/12/06 06:25:06 WARN DAGScheduler: Broadcasting large task binary with size 1073.3 KiB
25/12/06 06:25:06 WARN DAGScheduler: Broadcasting large task binary with size 1075.3 KiB
25/12/06 06:25:06 WARN DAGScheduler: Broadcasting large task binary with size 1077.8 KiB
25/12/06 06:25:06 WARN DAGScheduler: Broadcasting large task binary with size 1078.3 KiB
25/12/06 06:25:06 WARN DAGScheduler: Broadcasting large task binary with size 1078.8 KiB
25/12/06 06:25:06 WARN DAGScheduler: Broadcasting large task binary with size 1080.0 KiB
25/12/06 06:25:06 WARN DAGScheduler: Broadcasting large task binary with size 1082.3 KiB
25/12/06 06:25:06 WARN DAGScheduler: Broadcasting large task binary with size 1084.7 KiB
25/12/06 06:25:06 WARN DAGScheduler: Broadcasting large task binary with size 1085.1 KiB
25/12/06 06:25:06 WARN DAGScheduler: Broadcasting large task binary with size 1085.7 KiB
25/12/06 06:25:06 WARN DAGScheduler: Broadcasting large task binary with size 1086.9 KiB
25/12/06 06:25:06 WARN DAGScheduler: Broadcasting large task binary with size 1088.6 KiB
25/12/06 06:25:07 WARN DAGScheduler: Broadcasting large task binary with size 1090.7 KiB
25/12/06 06:25:07 WARN DAGScheduler: Broadcasting large task binary with size 1091.2 KiB
25/12/06 06:25:07 WARN DAGScheduler: Broadcasting large task binary with size 1091.8 KiB
25/12/06 06:25:07 WARN DAGScheduler: Broadcasting large task binary with size 1093.0 KiB
25/12/06 06:25:07 WARN DAGScheduler: Broadcasting large task binary with size 1095.2 KiB
25/12/06 06:25:07 WARN DAGScheduler: Broadcasting large task binary with size 1097.9 KiB

25/12/06 06:25:07 WARN DAGScheduler: Broadcasting large task binary with size 1098.4 KiB
25/12/06 06:25:07 WARN DAGScheduler: Broadcasting large task binary with size 1098.9 KiB
25/12/06 06:25:07 WARN DAGScheduler: Broadcasting large task binary with size 1100.1 KiB
25/12/06 06:25:07 WARN DAGScheduler: Broadcasting large task binary with size 1101.9 KiB
25/12/06 06:25:07 WARN DAGScheduler: Broadcasting large task binary with size 1104.0 KiB
25/12/06 06:25:07 WARN DAGScheduler: Broadcasting large task binary with size 1104.4 KiB
25/12/06 06:25:07 WARN DAGScheduler: Broadcasting large task binary with size 1105.0 KiB
25/12/06 06:25:07 WARN DAGScheduler: Broadcasting large task binary with size 1106.2 KiB
25/12/06 06:25:07 WARN DAGScheduler: Broadcasting large task binary with size 1108.5 KiB

-> Result: AUC=0.8058 | F1=0.8970 | Time=8.4s

[13/48] Test config: {'ratio_target': 3, 'ratio_multiplier': 0.1, 'identity_weight': 1.0, 'model': LogisticRegression_afae24335158, 'combinaison': 'w2v + tfidf'}

Undersampling : On garde 18.14% des données négatives.

Ratio de sain/toxique 3.15

toxic_weights : 0.31 + 1, identity_weight :1.0

Prétraitement terminé, df_train_final_balanced_debiased = 2556 |

df_test_final_all_w2v = 1199

-> Result: AUC=0.8021 | F1=0.8689 | Time=1.9s

[14/48] Test config: {'ratio_target': 3, 'ratio_multiplier': 0.1, 'identity_weight': 1.0, 'model': RandomForestClassifier_ad394a6e557e, 'combinaison': 'w2v + tfidf'}

Undersampling : On garde 18.14% des données négatives.

Ratio de sain/toxique 3.15

toxic_weights : 0.31 + 1, identity_weight :1.0

Prétraitement terminé, df_train_final_balanced_debiased = 2556 |

df_test_final_all_w2v = 1199

25/12/06 06:25:12 WARN DAGScheduler: Broadcasting large task binary with size 1008.4 KiB

25/12/06 06:25:12 WARN DAGScheduler: Broadcasting large task binary with size 1033.7 KiB

25/12/06 06:25:12 WARN DAGScheduler: Broadcasting large task binary with size 1069.1 KiB

-> Result: AUC=0.7517 | F1=0.9112 | Time=3.3s

[15/48] Test config: {'ratio_target': 3, 'ratio_multiplier': 0.1,

```
'identity_weight': 1.0, 'model': LinearSVC_8f0c0322b70e,  
'combinaison': 'w2v + tfidf'}  
Undersampling : On garde 18.14% des données négatives.  
Ratio de sain/toxique 3.15  
toxic_weights : 0.31 + 1, identity_weight :1.0  
Prétraitement terminé, df_train_final_balanced_debiased = 2556 |  
df_test_final_all_w2v = 1199  
-> Result: AUC=0.8332 | F1=0.8849 | Time=4.0s
```

```
[16/48] Test config: {'ratio_target': 3, 'ratio_multiplieur': 0.1,  
'identity_weight': 1.0, 'model': GBTClassifier_3b8b769d6280,  
'combinaison': 'w2v + tfidf'}  
Undersampling : On garde 18.14% des données négatives.  
Ratio de sain/toxique 3.15  
toxic_weights : 0.31 + 1, identity_weight :1.0  
Prétraitement terminé, df_train_final_balanced_debiased = 2556 |  
df_test_final_all_w2v = 1199
```

```
25/12/06 06:25:20 WARN DAGScheduler: Broadcasting large task binary  
with size 1000.4 KiB  
25/12/06 06:25:20 WARN DAGScheduler: Broadcasting large task binary  
with size 1001.6 KiB  
25/12/06 06:25:20 WARN DAGScheduler: Broadcasting large task binary  
with size 1003.9 KiB  
25/12/06 06:25:20 WARN DAGScheduler: Broadcasting large task binary  
with size 1006.7 KiB  
25/12/06 06:25:20 WARN DAGScheduler: Broadcasting large task binary  
with size 1007.2 KiB  
25/12/06 06:25:21 WARN DAGScheduler: Broadcasting large task binary  
with size 1007.8 KiB  
25/12/06 06:25:21 WARN DAGScheduler: Broadcasting large task binary  
with size 1009.0 KiB  
25/12/06 06:25:21 WARN DAGScheduler: Broadcasting large task binary  
with size 1011.3 KiB  
25/12/06 06:25:21 WARN DAGScheduler: Broadcasting large task binary  
with size 1013.6 KiB  
25/12/06 06:25:21 WARN DAGScheduler: Broadcasting large task binary  
with size 1014.1 KiB  
25/12/06 06:25:21 WARN DAGScheduler: Broadcasting large task binary  
with size 1014.6 KiB  
25/12/06 06:25:21 WARN DAGScheduler: Broadcasting large task binary  
with size 1015.8 KiB  
25/12/06 06:25:21 WARN DAGScheduler: Broadcasting large task binary  
with size 1018.1 KiB  
25/12/06 06:25:21 WARN DAGScheduler: Broadcasting large task binary  
with size 1020.5 KiB  
25/12/06 06:25:21 WARN DAGScheduler: Broadcasting large task binary  
with size 1021.0 KiB  
25/12/06 06:25:21 WARN DAGScheduler: Broadcasting large task binary  
with size 1021.5 KiB
```

25/12/06 06:25:21 WARN DAGScheduler: Broadcasting large task binary with size 1022.7 KiB
25/12/06 06:25:21 WARN DAGScheduler: Broadcasting large task binary with size 1025.0 KiB
25/12/06 06:25:21 WARN DAGScheduler: Broadcasting large task binary with size 1027.3 KiB
25/12/06 06:25:22 WARN DAGScheduler: Broadcasting large task binary with size 1027.8 KiB
25/12/06 06:25:22 WARN DAGScheduler: Broadcasting large task binary with size 1028.4 KiB
25/12/06 06:25:22 WARN DAGScheduler: Broadcasting large task binary with size 1029.6 KiB
25/12/06 06:25:22 WARN DAGScheduler: Broadcasting large task binary with size 1031.9 KiB
25/12/06 06:25:22 WARN DAGScheduler: Broadcasting large task binary with size 1034.2 KiB
25/12/06 06:25:22 WARN DAGScheduler: Broadcasting large task binary with size 1034.7 KiB
25/12/06 06:25:22 WARN DAGScheduler: Broadcasting large task binary with size 1035.3 KiB
25/12/06 06:25:22 WARN DAGScheduler: Broadcasting large task binary with size 1036.4 KiB
25/12/06 06:25:22 WARN DAGScheduler: Broadcasting large task binary with size 1038.5 KiB
25/12/06 06:25:22 WARN DAGScheduler: Broadcasting large task binary with size 1041.1 KiB
25/12/06 06:25:22 WARN DAGScheduler: Broadcasting large task binary with size 1041.6 KiB
25/12/06 06:25:22 WARN DAGScheduler: Broadcasting large task binary with size 1042.1 KiB
25/12/06 06:25:22 WARN DAGScheduler: Broadcasting large task binary with size 1043.3 KiB
25/12/06 06:25:22 WARN DAGScheduler: Broadcasting large task binary with size 1045.1 KiB
25/12/06 06:25:22 WARN DAGScheduler: Broadcasting large task binary with size 1047.2 KiB
25/12/06 06:25:22 WARN DAGScheduler: Broadcasting large task binary with size 1047.6 KiB
25/12/06 06:25:23 WARN DAGScheduler: Broadcasting large task binary with size 1048.2 KiB
25/12/06 06:25:23 WARN DAGScheduler: Broadcasting large task binary with size 1049.4 KiB
25/12/06 06:25:23 WARN DAGScheduler: Broadcasting large task binary with size 1051.7 KiB
25/12/06 06:25:23 WARN DAGScheduler: Broadcasting large task binary with size 1054.3 KiB
25/12/06 06:25:23 WARN DAGScheduler: Broadcasting large task binary with size 1054.8 KiB
25/12/06 06:25:23 WARN DAGScheduler: Broadcasting large task binary

with size 1055.4 KiB
25/12/06 06:25:23 WARN DAGScheduler: Broadcasting large task binary
with size 1056.6 KiB
25/12/06 06:25:23 WARN DAGScheduler: Broadcasting large task binary
with size 1058.3 KiB
25/12/06 06:25:23 WARN DAGScheduler: Broadcasting large task binary
with size 1060.6 KiB
25/12/06 06:25:23 WARN DAGScheduler: Broadcasting large task binary
with size 1061.1 KiB
25/12/06 06:25:23 WARN DAGScheduler: Broadcasting large task binary
with size 1061.6 KiB
25/12/06 06:25:23 WARN DAGScheduler: Broadcasting large task binary
with size 1062.8 KiB
25/12/06 06:25:23 WARN DAGScheduler: Broadcasting large task binary
with size 1064.8 KiB
25/12/06 06:25:23 WARN DAGScheduler: Broadcasting large task binary
with size 1067.3 KiB
25/12/06 06:25:23 WARN DAGScheduler: Broadcasting large task binary
with size 1067.7 KiB
25/12/06 06:25:23 WARN DAGScheduler: Broadcasting large task binary
with size 1068.3 KiB
25/12/06 06:25:24 WARN DAGScheduler: Broadcasting large task binary
with size 1069.5 KiB
25/12/06 06:25:24 WARN DAGScheduler: Broadcasting large task binary
with size 1071.5 KiB
25/12/06 06:25:24 WARN DAGScheduler: Broadcasting large task binary
with size 1073.7 KiB
25/12/06 06:25:24 WARN DAGScheduler: Broadcasting large task binary
with size 1074.1 KiB
25/12/06 06:25:24 WARN DAGScheduler: Broadcasting large task binary
with size 1074.7 KiB
25/12/06 06:25:24 WARN DAGScheduler: Broadcasting large task binary
with size 1075.9 KiB
25/12/06 06:25:24 WARN DAGScheduler: Broadcasting large task binary
with size 1077.9 KiB
25/12/06 06:25:24 WARN DAGScheduler: Broadcasting large task binary
with size 1079.7 KiB
25/12/06 06:25:24 WARN DAGScheduler: Broadcasting large task binary
with size 1080.2 KiB
25/12/06 06:25:24 WARN DAGScheduler: Broadcasting large task binary
with size 1080.8 KiB
25/12/06 06:25:24 WARN DAGScheduler: Broadcasting large task binary
with size 1082.0 KiB
25/12/06 06:25:24 WARN DAGScheduler: Broadcasting large task binary
with size 1084.2 KiB
25/12/06 06:25:24 WARN DAGScheduler: Broadcasting large task binary
with size 1086.6 KiB
25/12/06 06:25:24 WARN DAGScheduler: Broadcasting large task binary
with size 1087.1 KiB

25/12/06 06:25:24 WARN DAGScheduler: Broadcasting large task binary with size 1087.6 KiB
25/12/06 06:25:24 WARN DAGScheduler: Broadcasting large task binary with size 1088.8 KiB
25/12/06 06:25:25 WARN DAGScheduler: Broadcasting large task binary with size 1091.1 KiB
25/12/06 06:25:25 WARN DAGScheduler: Broadcasting large task binary with size 1093.3 KiB
25/12/06 06:25:25 WARN DAGScheduler: Broadcasting large task binary with size 1093.8 KiB
25/12/06 06:25:25 WARN DAGScheduler: Broadcasting large task binary with size 1094.3 KiB
25/12/06 06:25:25 WARN DAGScheduler: Broadcasting large task binary with size 1095.5 KiB
25/12/06 06:25:25 WARN DAGScheduler: Broadcasting large task binary with size 1097.5 KiB
25/12/06 06:25:25 WARN DAGScheduler: Broadcasting large task binary with size 1099.7 KiB
25/12/06 06:25:25 WARN DAGScheduler: Broadcasting large task binary with size 1100.2 KiB
25/12/06 06:25:25 WARN DAGScheduler: Broadcasting large task binary with size 1100.7 KiB
25/12/06 06:25:25 WARN DAGScheduler: Broadcasting large task binary with size 1101.9 KiB
25/12/06 06:25:25 WARN DAGScheduler: Broadcasting large task binary with size 1103.7 KiB
25/12/06 06:25:25 WARN DAGScheduler: Broadcasting large task binary with size 1105.9 KiB
25/12/06 06:25:25 WARN DAGScheduler: Broadcasting large task binary with size 1106.4 KiB
25/12/06 06:25:25 WARN DAGScheduler: Broadcasting large task binary with size 1106.9 KiB
25/12/06 06:25:25 WARN DAGScheduler: Broadcasting large task binary with size 1108.1 KiB
25/12/06 06:25:25 WARN DAGScheduler: Broadcasting large task binary with size 1110.4 KiB

-> Result: AUC=0.7560 | F1=0.8794 | Time=8.7s

[17/48] Test config: {'ratio_target': 5, 'ratio_multiplier': 0.05, 'identity_weight': 0.5, 'model': LogisticRegression_afae24335158, 'combinaison': 'w2v + tfidf'}

Undersampling : On garde 30.24% des données négatives.

Ratio de sain/toxique 5.13

toxic_weights : 0.26 + 1, identity_weight :0.5

Prétraitement terminé, df_train_final_balanced_debiased = 3779 |
df_test_final_all_w2v = 1199

25/12/06 06:25:27 WARN DAGScheduler: Broadcasting large task binary with size 1041.1 KiB

```
25/12/06 06:25:27 WARN DAGScheduler: Broadcasting large task binary
with size 1041.7 KiB
25/12/06 06:25:27 WARN DAGScheduler: Broadcasting large task binary
with size 1041.7 KiB
25/12/06 06:25:28 WARN DAGScheduler: Broadcasting large task binary
with size 1041.7 KiB
25/12/06 06:25:28 WARN DAGScheduler: Broadcasting large task binary
with size 1041.7 KiB
25/12/06 06:25:28 WARN DAGScheduler: Broadcasting large task binary
with size 1041.7 KiB
25/12/06 06:25:28 WARN DAGScheduler: Broadcasting large task binary
with size 1041.7 KiB
25/12/06 06:25:28 WARN DAGScheduler: Broadcasting large task binary
with size 1041.7 KiB
25/12/06 06:25:28 WARN DAGScheduler: Broadcasting large task binary
with size 1041.7 KiB
25/12/06 06:25:28 WARN DAGScheduler: Broadcasting large task binary
with size 1041.7 KiB
25/12/06 06:25:28 WARN DAGScheduler: Broadcasting large task binary
with size 1041.7 KiB
25/12/06 06:25:28 WARN DAGScheduler: Broadcasting large task binary
with size 1041.7 KiB
25/12/06 06:25:28 WARN DAGScheduler: Broadcasting large task binary
with size 1041.7 KiB
25/12/06 06:25:28 WARN DAGScheduler: Broadcasting large task binary
with size 1041.7 KiB
25/12/06 06:25:28 WARN DAGScheduler: Broadcasting large task binary
with size 1041.7 KiB
```

```
-> Result: AUC=0.8098 | F1=0.8865 | Time=2.1s
```

```
[18/48] Test config: {'ratio_target': 5, 'ratio_multiplier': 0.05,
'identity_weight': 0.5, 'model': RandomForestClassifier_ad394a6e557e,
'combinaison': 'w2v + tfidf'}
```

```
Undersampling : On garde 30.24% des données négatives.
```

```
Ratio de sain/toxique 5.13
```

```
toxic_weights : 0.26 + 1, identity_weight :0.5
```

```
Prétraitement terminé, df_train_final_balanced_debiased = 3779 |
```

```
df_test_final_all_w2v = 1199
```

```
25/12/06 06:25:29 WARN DAGScheduler: Broadcasting large task binary
with size 1037.6 KiB
25/12/06 06:25:29 WARN DAGScheduler: Broadcasting large task binary
with size 1037.5 KiB
25/12/06 06:25:30 WARN DAGScheduler: Broadcasting large task binary
with size 1086.8 KiB
25/12/06 06:25:31 WARN DAGScheduler: Broadcasting large task binary
with size 1217.7 KiB
25/12/06 06:25:31 WARN DAGScheduler: Broadcasting large task binary
with size 1229.5 KiB
25/12/06 06:25:31 WARN DAGScheduler: Broadcasting large task binary
with size 1247.2 KiB
25/12/06 06:25:31 WARN DAGScheduler: Broadcasting large task binary
with size 1272.7 KiB
```

```
25/12/06 06:25:32 WARN DAGScheduler: Broadcasting large task binary
with size 1309.3 KiB
25/12/06 06:25:32 WARN DAGScheduler: Broadcasting large task binary
with size 1035.8 KiB
25/12/06 06:25:32 WARN DAGScheduler: Broadcasting large task binary
with size 1048.0 KiB
25/12/06 06:25:32 WARN DAGScheduler: Broadcasting large task binary
with size 1048.0 KiB
```

-> Result: AUC=0.7740 | F1=0.9072 | Time=4.2s

```
[19/48] Test config: {'ratio_target': 5, 'ratio_multiplier': 0.05,
'identity_weight': 0.5, 'model': LinearSVC_8f0c0322b70e,
'combinaison': 'w2v + tfidf'}
```

Undersampling : On garde 30.24% des données négatives.

Ratio de sain/toxique 5.13

```
toxic_weights : 0.26 + 1, identity_weight :0.5
```

```
Prétraitement terminé, df_train_final_balanced_debiased = 3779 |
```

```
df_test_final_all_w2v = 1199
```

```
25/12/06 06:25:34 WARN DAGScheduler: Broadcasting large task binary
with size 1038.6 KiB
```

```
25/12/06 06:25:34 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
```

```
25/12/06 06:25:34 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
```

```
25/12/06 06:25:34 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
```

```
25/12/06 06:25:34 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
```

```
25/12/06 06:25:34 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
```

```
25/12/06 06:25:34 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
```

```
25/12/06 06:25:34 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
```

```
25/12/06 06:25:34 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
```

```
25/12/06 06:25:34 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
```

```
25/12/06 06:25:34 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
```

```
25/12/06 06:25:34 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
```

```
25/12/06 06:25:34 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
```

```
25/12/06 06:25:34 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
```

```
25/12/06 06:25:34 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
```

[illegible]

[illegible]

[illegible]

with size 1039.1 KiB
25/12/06 06:25:36 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
25/12/06 06:25:36 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
25/12/06 06:25:36 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
25/12/06 06:25:36 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
25/12/06 06:25:37 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
25/12/06 06:25:37 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
25/12/06 06:25:37 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
25/12/06 06:25:37 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
25/12/06 06:25:37 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
25/12/06 06:25:37 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
25/12/06 06:25:37 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
25/12/06 06:25:37 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
25/12/06 06:25:37 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
25/12/06 06:25:37 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
25/12/06 06:25:37 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB

-> Result: AUC=0.8207 | F1=0.8952 | Time=4.8s

[20/48] Test config: {'ratio_target': 5, 'ratio_multiplier': 0.05,
'identity_weight': 0.5, 'model': GBTClassifier_3b8b769d6280,
'combinaison': 'w2v + tfidf'}

Undersampling : On garde 30.24% des données négatives.

Ratio de sain/toxique 5.13

toxic_weights : 0.26 + 1, identity_weight :0.5

Prétraitement terminé, df_train_final_balanced_debiased = 3779 |

df_test_final_all_w2v = 1199

25/12/06 06:25:38 WARN DAGScheduler: Broadcasting large task binary
with size 1037.3 KiB
25/12/06 06:25:38 WARN DAGScheduler: Broadcasting large task binary
with size 1037.2 KiB
25/12/06 06:25:39 WARN DAGScheduler: Broadcasting large task binary
with size 1086.5 KiB
25/12/06 06:25:40 WARN DAGScheduler: Broadcasting large task binary

with size 1211.3 KiB
25/12/06 06:25:40 WARN DAGScheduler: Broadcasting large task binary
with size 1212.0 KiB
25/12/06 06:25:40 WARN DAGScheduler: Broadcasting large task binary
with size 1212.6 KiB
25/12/06 06:25:41 WARN DAGScheduler: Broadcasting large task binary
with size 1213.8 KiB
25/12/06 06:25:41 WARN DAGScheduler: Broadcasting large task binary
with size 1215.8 KiB
25/12/06 06:25:41 WARN DAGScheduler: Broadcasting large task binary
with size 1221.9 KiB
25/12/06 06:25:41 WARN DAGScheduler: Broadcasting large task binary
with size 1222.4 KiB
25/12/06 06:25:41 WARN DAGScheduler: Broadcasting large task binary
with size 1222.9 KiB
25/12/06 06:25:41 WARN DAGScheduler: Broadcasting large task binary
with size 1224.1 KiB
25/12/06 06:25:41 WARN DAGScheduler: Broadcasting large task binary
with size 1226.4 KiB
25/12/06 06:25:41 WARN DAGScheduler: Broadcasting large task binary
with size 1229.1 KiB
25/12/06 06:25:41 WARN DAGScheduler: Broadcasting large task binary
with size 1229.6 KiB
25/12/06 06:25:41 WARN DAGScheduler: Broadcasting large task binary
with size 1230.1 KiB
25/12/06 06:25:41 WARN DAGScheduler: Broadcasting large task binary
with size 1231.3 KiB
25/12/06 06:25:42 WARN DAGScheduler: Broadcasting large task binary
with size 1233.6 KiB
25/12/06 06:25:42 WARN DAGScheduler: Broadcasting large task binary
with size 1236.0 KiB
25/12/06 06:25:42 WARN DAGScheduler: Broadcasting large task binary
with size 1236.5 KiB
25/12/06 06:25:42 WARN DAGScheduler: Broadcasting large task binary
with size 1237.0 KiB
25/12/06 06:25:42 WARN DAGScheduler: Broadcasting large task binary
with size 1238.2 KiB
25/12/06 06:25:42 WARN DAGScheduler: Broadcasting large task binary
with size 1239.9 KiB
25/12/06 06:25:42 WARN DAGScheduler: Broadcasting large task binary
with size 1241.6 KiB
25/12/06 06:25:42 WARN DAGScheduler: Broadcasting large task binary
with size 1242.0 KiB
25/12/06 06:25:42 WARN DAGScheduler: Broadcasting large task binary
with size 1242.6 KiB
25/12/06 06:25:42 WARN DAGScheduler: Broadcasting large task binary
with size 1243.8 KiB
25/12/06 06:25:42 WARN DAGScheduler: Broadcasting large task binary
with size 1245.5 KiB

25/12/06 06:25:42 WARN DAGScheduler: Broadcasting large task binary with size 1247.8 KiB
25/12/06 06:25:42 WARN DAGScheduler: Broadcasting large task binary with size 1248.3 KiB
25/12/06 06:25:43 WARN DAGScheduler: Broadcasting large task binary with size 1248.8 KiB
25/12/06 06:25:43 WARN DAGScheduler: Broadcasting large task binary with size 1250.0 KiB
25/12/06 06:25:43 WARN DAGScheduler: Broadcasting large task binary with size 1251.7 KiB
25/12/06 06:25:43 WARN DAGScheduler: Broadcasting large task binary with size 1254.2 KiB
25/12/06 06:25:43 WARN DAGScheduler: Broadcasting large task binary with size 1254.6 KiB
25/12/06 06:25:43 WARN DAGScheduler: Broadcasting large task binary with size 1255.2 KiB
25/12/06 06:25:43 WARN DAGScheduler: Broadcasting large task binary with size 1256.4 KiB
25/12/06 06:25:43 WARN DAGScheduler: Broadcasting large task binary with size 1258.7 KiB
25/12/06 06:25:43 WARN DAGScheduler: Broadcasting large task binary with size 1261.0 KiB
25/12/06 06:25:43 WARN DAGScheduler: Broadcasting large task binary with size 1261.5 KiB
25/12/06 06:25:43 WARN DAGScheduler: Broadcasting large task binary with size 1262.1 KiB
25/12/06 06:25:43 WARN DAGScheduler: Broadcasting large task binary with size 1263.3 KiB
25/12/06 06:25:44 WARN DAGScheduler: Broadcasting large task binary with size 1265.6 KiB
25/12/06 06:25:44 WARN DAGScheduler: Broadcasting large task binary with size 1267.7 KiB
25/12/06 06:25:44 WARN DAGScheduler: Broadcasting large task binary with size 1268.2 KiB
25/12/06 06:25:44 WARN DAGScheduler: Broadcasting large task binary with size 1268.8 KiB
25/12/06 06:25:44 WARN DAGScheduler: Broadcasting large task binary with size 1270.0 KiB
25/12/06 06:25:44 WARN DAGScheduler: Broadcasting large task binary with size 1272.3 KiB
25/12/06 06:25:44 WARN DAGScheduler: Broadcasting large task binary with size 1274.9 KiB
25/12/06 06:25:44 WARN DAGScheduler: Broadcasting large task binary with size 1275.4 KiB
25/12/06 06:25:44 WARN DAGScheduler: Broadcasting large task binary with size 1276.0 KiB
25/12/06 06:25:44 WARN DAGScheduler: Broadcasting large task binary with size 1277.1 KiB
25/12/06 06:25:44 WARN DAGScheduler: Broadcasting large task binary

with size 1278.9 KiB
25/12/06 06:25:44 WARN DAGScheduler: Broadcasting large task binary
with size 1281.0 KiB
25/12/06 06:25:45 WARN DAGScheduler: Broadcasting large task binary
with size 1281.5 KiB
25/12/06 06:25:45 WARN DAGScheduler: Broadcasting large task binary
with size 1282.0 KiB
25/12/06 06:25:45 WARN DAGScheduler: Broadcasting large task binary
with size 1283.2 KiB
25/12/06 06:25:45 WARN DAGScheduler: Broadcasting large task binary
with size 1285.2 KiB
25/12/06 06:25:45 WARN DAGScheduler: Broadcasting large task binary
with size 1287.4 KiB
25/12/06 06:25:45 WARN DAGScheduler: Broadcasting large task binary
with size 1287.8 KiB
25/12/06 06:25:45 WARN DAGScheduler: Broadcasting large task binary
with size 1288.4 KiB
25/12/06 06:25:45 WARN DAGScheduler: Broadcasting large task binary
with size 1289.6 KiB
25/12/06 06:25:45 WARN DAGScheduler: Broadcasting large task binary
with size 1291.9 KiB
25/12/06 06:25:45 WARN DAGScheduler: Broadcasting large task binary
with size 1294.1 KiB
25/12/06 06:25:45 WARN DAGScheduler: Broadcasting large task binary
with size 1294.5 KiB
25/12/06 06:25:45 WARN DAGScheduler: Broadcasting large task binary
with size 1295.1 KiB
25/12/06 06:25:45 WARN DAGScheduler: Broadcasting large task binary
with size 1296.3 KiB
25/12/06 06:25:46 WARN DAGScheduler: Broadcasting large task binary
with size 1298.6 KiB
25/12/06 06:25:46 WARN DAGScheduler: Broadcasting large task binary
with size 1300.9 KiB
25/12/06 06:25:46 WARN DAGScheduler: Broadcasting large task binary
with size 1301.4 KiB
25/12/06 06:25:46 WARN DAGScheduler: Broadcasting large task binary
with size 1302.0 KiB
25/12/06 06:25:46 WARN DAGScheduler: Broadcasting large task binary
with size 1303.1 KiB
25/12/06 06:25:46 WARN DAGScheduler: Broadcasting large task binary
with size 1304.9 KiB
25/12/06 06:25:46 WARN DAGScheduler: Broadcasting large task binary
with size 1307.0 KiB
25/12/06 06:25:46 WARN DAGScheduler: Broadcasting large task binary
with size 1307.5 KiB
25/12/06 06:25:46 WARN DAGScheduler: Broadcasting large task binary
with size 1308.0 KiB
25/12/06 06:25:46 WARN DAGScheduler: Broadcasting large task binary
with size 1309.2 KiB

25/12/06 06:25:46 WARN DAGScheduler: Broadcasting large task binary with size 1311.5 KiB
25/12/06 06:25:46 WARN DAGScheduler: Broadcasting large task binary with size 1314.2 KiB
25/12/06 06:25:47 WARN DAGScheduler: Broadcasting large task binary with size 1314.6 KiB
25/12/06 06:25:47 WARN DAGScheduler: Broadcasting large task binary with size 1315.2 KiB
25/12/06 06:25:47 WARN DAGScheduler: Broadcasting large task binary with size 1316.4 KiB
25/12/06 06:25:47 WARN DAGScheduler: Broadcasting large task binary with size 1318.7 KiB
25/12/06 06:25:47 WARN DAGScheduler: Broadcasting large task binary with size 1321.2 KiB
25/12/06 06:25:47 WARN DAGScheduler: Broadcasting large task binary with size 1321.7 KiB
25/12/06 06:25:47 WARN DAGScheduler: Broadcasting large task binary with size 1322.2 KiB
25/12/06 06:25:47 WARN DAGScheduler: Broadcasting large task binary with size 1323.4 KiB
25/12/06 06:25:47 WARN DAGScheduler: Broadcasting large task binary with size 1325.7 KiB
25/12/06 06:25:47 WARN DAGScheduler: Broadcasting large task binary with size 1328.1 KiB
25/12/06 06:25:47 WARN DAGScheduler: Broadcasting large task binary with size 1328.5 KiB
25/12/06 06:25:47 WARN DAGScheduler: Broadcasting large task binary with size 1329.1 KiB
25/12/06 06:25:48 WARN DAGScheduler: Broadcasting large task binary with size 1330.3 KiB
25/12/06 06:25:48 WARN DAGScheduler: Broadcasting large task binary with size 1332.2 KiB
25/12/06 06:25:48 WARN DAGScheduler: Broadcasting large task binary with size 1334.6 KiB
25/12/06 06:25:48 WARN DAGScheduler: Broadcasting large task binary with size 1335.1 KiB
25/12/06 06:25:48 WARN DAGScheduler: Broadcasting large task binary with size 1335.6 KiB
25/12/06 06:25:48 WARN DAGScheduler: Broadcasting large task binary with size 1336.8 KiB
25/12/06 06:25:48 WARN DAGScheduler: Broadcasting large task binary with size 1338.9 KiB
25/12/06 06:25:48 WARN DAGScheduler: Broadcasting large task binary with size 1341.0 KiB
25/12/06 06:25:48 WARN DAGScheduler: Broadcasting large task binary with size 1341.5 KiB
25/12/06 06:25:48 WARN DAGScheduler: Broadcasting large task binary with size 1342.0 KiB
25/12/06 06:25:48 WARN DAGScheduler: Broadcasting large task binary

with size 1343.2 KiB
25/12/06 06:25:48 WARN DAGScheduler: Broadcasting large task binary
with size 1345.4 KiB
25/12/06 06:25:49 WARN DAGScheduler: Broadcasting large task binary
with size 1066.3 KiB
25/12/06 06:25:49 WARN DAGScheduler: Broadcasting large task binary
with size 1078.7 KiB
25/12/06 06:25:49 WARN DAGScheduler: Broadcasting large task binary
with size 1078.7 KiB

-> Result: AUC=0.8004 | F1=0.9205 | Time=11.9s

[21/48] Test config: {'ratio_target': 5, 'ratio_multiplier': 0.05,
'identity_weight': 1.0, 'model': LogisticRegression_afae24335158,
'combinaison': 'w2v + tfidf'}

Undersampling : On garde 30.24% des données négatives.

Ratio de sain/toxique 5.13

toxic_weights : 0.26 + 1, identity_weight :1.0

Prétraitement terminé, df_train_final_balanced_debiased = 3779 |
df_test_final_all_w2v = 1199

25/12/06 06:25:50 WARN DAGScheduler: Broadcasting large task binary
with size 1041.1 KiB
25/12/06 06:25:50 WARN DAGScheduler: Broadcasting large task binary
with size 1041.6 KiB
25/12/06 06:25:51 WARN DAGScheduler: Broadcasting large task binary
with size 1041.6 KiB
25/12/06 06:25:51 WARN DAGScheduler: Broadcasting large task binary
with size 1041.6 KiB
25/12/06 06:25:51 WARN DAGScheduler: Broadcasting large task binary
with size 1041.6 KiB
25/12/06 06:25:51 WARN DAGScheduler: Broadcasting large task binary
with size 1041.6 KiB
25/12/06 06:25:51 WARN DAGScheduler: Broadcasting large task binary
with size 1041.6 KiB
25/12/06 06:25:51 WARN DAGScheduler: Broadcasting large task binary
with size 1041.6 KiB
25/12/06 06:25:51 WARN DAGScheduler: Broadcasting large task binary
with size 1041.6 KiB
25/12/06 06:25:51 WARN DAGScheduler: Broadcasting large task binary
with size 1041.6 KiB
25/12/06 06:25:51 WARN DAGScheduler: Broadcasting large task binary
with size 1041.6 KiB
25/12/06 06:25:51 WARN DAGScheduler: Broadcasting large task binary
with size 1041.6 KiB

-> Result: AUC=0.8033 | F1=0.8779 | Time=2.1s

[22/48] Test config: {'ratio_target': 5, 'ratio_multiplier': 0.05,
'identity_weight': 1.0, 'model': RandomForestClassifier_ad394a6e557e,

```
'combinaison': 'w2v + tfidf'}
Undersampling : On garde 30.24% des données négatives.
Ratio de sain/toxique 5.13
toxic_weights : 0.26 + 1, identity_weight :1.0
Prétraitement terminé, df_train_final_balanced_debiased = 3779 |
df_test_final_all_w2v = 1199

25/12/06 06:25:53 WARN DAGScheduler: Broadcasting large task binary
with size 1037.6 KiB
25/12/06 06:25:53 WARN DAGScheduler: Broadcasting large task binary
with size 1037.5 KiB
25/12/06 06:25:53 WARN DAGScheduler: Broadcasting large task binary
with size 1086.8 KiB
25/12/06 06:25:54 WARN DAGScheduler: Broadcasting large task binary
with size 1217.7 KiB
25/12/06 06:25:54 WARN DAGScheduler: Broadcasting large task binary
with size 1229.5 KiB
25/12/06 06:25:54 WARN DAGScheduler: Broadcasting large task binary
with size 1245.4 KiB
25/12/06 06:25:54 WARN DAGScheduler: Broadcasting large task binary
with size 1269.7 KiB
25/12/06 06:25:54 WARN DAGScheduler: Broadcasting large task binary
with size 1304.9 KiB
25/12/06 06:25:55 WARN DAGScheduler: Broadcasting large task binary
with size 1033.4 KiB
25/12/06 06:25:55 WARN DAGScheduler: Broadcasting large task binary
with size 1045.5 KiB
25/12/06 06:25:55 WARN DAGScheduler: Broadcasting large task binary
with size 1045.5 KiB

-> Result: AUC=0.8241 | F1=0.9072 | Time=3.5s

[23/48] Test config: {'ratio_target': 5, 'ratio_multiplier': 0.05,
'identity_weight': 1.0, 'model': LinearSVC_8f0c0322b70e,
'combinaison': 'w2v + tfidf'}
Undersampling : On garde 30.24% des données négatives.
Ratio de sain/toxique 5.13
toxic_weights : 0.26 + 1, identity_weight :1.0
Prétraitement terminé, df_train_final_balanced_debiased = 3779 |
df_test_final_all_w2v = 1199

25/12/06 06:25:56 WARN DAGScheduler: Broadcasting large task binary
with size 1038.6 KiB
25/12/06 06:25:56 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
25/12/06 06:25:56 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
25/12/06 06:25:56 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
25/12/06 06:25:56 WARN DAGScheduler: Broadcasting large task binary
```

[illegible]

[illegible]

[illegible]

[illegible]

25/12/06 06:25:59 WARN DAGScheduler: Broadcasting large task binary with size 1039.1 KiB

-> Result: AUC=0.8364 | F1=0.8974 | Time=4.5s

[24/48] Test config: {'ratio_target': 5, 'ratio_multiplier': 0.05, 'identity_weight': 1.0, 'model': GBTCClassifier_3b8b769d6280, 'combinaison': 'w2v + tfidf'}

Undersampling : On garde 30.24% des données négatives.

Ratio de sain/toxique 5.13

toxic_weights : 0.26 + 1, identity_weight :1.0

Prétraitement terminé, df_train_final_balanced_debiased = 3779 | df_test_final_all_w2v = 1199

25/12/06 06:26:01 WARN DAGScheduler: Broadcasting large task binary with size 1037.3 KiB

25/12/06 06:26:01 WARN DAGScheduler: Broadcasting large task binary with size 1037.2 KiB

25/12/06 06:26:01 WARN DAGScheduler: Broadcasting large task binary with size 1086.5 KiB

25/12/06 06:26:02 WARN DAGScheduler: Broadcasting large task binary with size 1211.3 KiB

25/12/06 06:26:03 WARN DAGScheduler: Broadcasting large task binary with size 1212.0 KiB

25/12/06 06:26:03 WARN DAGScheduler: Broadcasting large task binary with size 1212.6 KiB

25/12/06 06:26:03 WARN DAGScheduler: Broadcasting large task binary with size 1213.8 KiB

25/12/06 06:26:03 WARN DAGScheduler: Broadcasting large task binary with size 1215.7 KiB

25/12/06 06:26:03 WARN DAGScheduler: Broadcasting large task binary with size 1221.7 KiB

25/12/06 06:26:03 WARN DAGScheduler: Broadcasting large task binary with size 1222.2 KiB

25/12/06 06:26:03 WARN DAGScheduler: Broadcasting large task binary with size 1222.7 KiB

25/12/06 06:26:04 WARN DAGScheduler: Broadcasting large task binary with size 1223.6 KiB

25/12/06 06:26:04 WARN DAGScheduler: Broadcasting large task binary with size 1225.4 KiB

25/12/06 06:26:04 WARN DAGScheduler: Broadcasting large task binary with size 1228.0 KiB

25/12/06 06:26:04 WARN DAGScheduler: Broadcasting large task binary with size 1228.4 KiB

25/12/06 06:26:04 WARN DAGScheduler: Broadcasting large task binary with size 1229.0 KiB

25/12/06 06:26:04 WARN DAGScheduler: Broadcasting large task binary with size 1230.2 KiB

25/12/06 06:26:04 WARN DAGScheduler: Broadcasting large task binary with size 1232.5 KiB

25/12/06 06:26:04 WARN DAGScheduler: Broadcasting large task binary with size 1235.1 KiB
25/12/06 06:26:04 WARN DAGScheduler: Broadcasting large task binary with size 1235.6 KiB
25/12/06 06:26:04 WARN DAGScheduler: Broadcasting large task binary with size 1236.2 KiB
25/12/06 06:26:04 WARN DAGScheduler: Broadcasting large task binary with size 1237.3 KiB
25/12/06 06:26:04 WARN DAGScheduler: Broadcasting large task binary with size 1239.0 KiB
25/12/06 06:26:05 WARN DAGScheduler: Broadcasting large task binary with size 1240.9 KiB
25/12/06 06:26:05 WARN DAGScheduler: Broadcasting large task binary with size 1241.4 KiB
25/12/06 06:26:05 WARN DAGScheduler: Broadcasting large task binary with size 1241.9 KiB
25/12/06 06:26:05 WARN DAGScheduler: Broadcasting large task binary with size 1243.1 KiB
25/12/06 06:26:05 WARN DAGScheduler: Broadcasting large task binary with size 1245.4 KiB
25/12/06 06:26:05 WARN DAGScheduler: Broadcasting large task binary with size 1247.9 KiB
25/12/06 06:26:05 WARN DAGScheduler: Broadcasting large task binary with size 1248.4 KiB
25/12/06 06:26:05 WARN DAGScheduler: Broadcasting large task binary with size 1249.0 KiB
25/12/06 06:26:05 WARN DAGScheduler: Broadcasting large task binary with size 1250.2 KiB
25/12/06 06:26:05 WARN DAGScheduler: Broadcasting large task binary with size 1252.4 KiB
25/12/06 06:26:05 WARN DAGScheduler: Broadcasting large task binary with size 1255.1 KiB
25/12/06 06:26:05 WARN DAGScheduler: Broadcasting large task binary with size 1255.6 KiB
25/12/06 06:26:06 WARN DAGScheduler: Broadcasting large task binary with size 1256.2 KiB
25/12/06 06:26:06 WARN DAGScheduler: Broadcasting large task binary with size 1257.4 KiB
25/12/06 06:26:06 WARN DAGScheduler: Broadcasting large task binary with size 1259.6 KiB
25/12/06 06:26:06 WARN DAGScheduler: Broadcasting large task binary with size 1261.8 KiB
25/12/06 06:26:06 WARN DAGScheduler: Broadcasting large task binary with size 1262.3 KiB
25/12/06 06:26:06 WARN DAGScheduler: Broadcasting large task binary with size 1262.9 KiB
25/12/06 06:26:06 WARN DAGScheduler: Broadcasting large task binary with size 1264.1 KiB
25/12/06 06:26:06 WARN DAGScheduler: Broadcasting large task binary

with size 1266.4 KiB
25/12/06 06:26:06 WARN DAGScheduler: Broadcasting large task binary
with size 1268.5 KiB
25/12/06 06:26:06 WARN DAGScheduler: Broadcasting large task binary
with size 1269.0 KiB
25/12/06 06:26:06 WARN DAGScheduler: Broadcasting large task binary
with size 1269.6 KiB
25/12/06 06:26:06 WARN DAGScheduler: Broadcasting large task binary
with size 1270.8 KiB
25/12/06 06:26:06 WARN DAGScheduler: Broadcasting large task binary
with size 1272.8 KiB
25/12/06 06:26:06 WARN DAGScheduler: Broadcasting large task binary
with size 1275.1 KiB
25/12/06 06:26:07 WARN DAGScheduler: Broadcasting large task binary
with size 1275.6 KiB
25/12/06 06:26:07 WARN DAGScheduler: Broadcasting large task binary
with size 1276.1 KiB
25/12/06 06:26:07 WARN DAGScheduler: Broadcasting large task binary
with size 1277.3 KiB
25/12/06 06:26:07 WARN DAGScheduler: Broadcasting large task binary
with size 1279.6 KiB
25/12/06 06:26:07 WARN DAGScheduler: Broadcasting large task binary
with size 1282.3 KiB
25/12/06 06:26:07 WARN DAGScheduler: Broadcasting large task binary
with size 1282.7 KiB
25/12/06 06:26:07 WARN DAGScheduler: Broadcasting large task binary
with size 1283.3 KiB
25/12/06 06:26:07 WARN DAGScheduler: Broadcasting large task binary
with size 1284.5 KiB
25/12/06 06:26:07 WARN DAGScheduler: Broadcasting large task binary
with size 1286.8 KiB
25/12/06 06:26:07 WARN DAGScheduler: Broadcasting large task binary
with size 1288.8 KiB
25/12/06 06:26:07 WARN DAGScheduler: Broadcasting large task binary
with size 1289.3 KiB
25/12/06 06:26:07 WARN DAGScheduler: Broadcasting large task binary
with size 1289.9 KiB
25/12/06 06:26:08 WARN DAGScheduler: Broadcasting large task binary
with size 1291.1 KiB
25/12/06 06:26:08 WARN DAGScheduler: Broadcasting large task binary
with size 1293.3 KiB
25/12/06 06:26:08 WARN DAGScheduler: Broadcasting large task binary
with size 1295.5 KiB
25/12/06 06:26:08 WARN DAGScheduler: Broadcasting large task binary
with size 1296.0 KiB
25/12/06 06:26:08 WARN DAGScheduler: Broadcasting large task binary
with size 1296.6 KiB
25/12/06 06:26:08 WARN DAGScheduler: Broadcasting large task binary
with size 1297.8 KiB

25/12/06 06:26:08 WARN DAGScheduler: Broadcasting large task binary with size 1300.0 KiB
25/12/06 06:26:08 WARN DAGScheduler: Broadcasting large task binary with size 1302.4 KiB
25/12/06 06:26:08 WARN DAGScheduler: Broadcasting large task binary with size 1302.9 KiB
25/12/06 06:26:08 WARN DAGScheduler: Broadcasting large task binary with size 1303.4 KiB
25/12/06 06:26:08 WARN DAGScheduler: Broadcasting large task binary with size 1304.6 KiB
25/12/06 06:26:08 WARN DAGScheduler: Broadcasting large task binary with size 1306.9 KiB
25/12/06 06:26:08 WARN DAGScheduler: Broadcasting large task binary with size 1309.7 KiB
25/12/06 06:26:09 WARN DAGScheduler: Broadcasting large task binary with size 1310.2 KiB
25/12/06 06:26:09 WARN DAGScheduler: Broadcasting large task binary with size 1310.8 KiB
25/12/06 06:26:09 WARN DAGScheduler: Broadcasting large task binary with size 1312.0 KiB
25/12/06 06:26:09 WARN DAGScheduler: Broadcasting large task binary with size 1314.0 KiB
25/12/06 06:26:09 WARN DAGScheduler: Broadcasting large task binary with size 1316.4 KiB
25/12/06 06:26:09 WARN DAGScheduler: Broadcasting large task binary with size 1316.9 KiB
25/12/06 06:26:09 WARN DAGScheduler: Broadcasting large task binary with size 1317.5 KiB
25/12/06 06:26:09 WARN DAGScheduler: Broadcasting large task binary with size 1318.7 KiB
25/12/06 06:26:09 WARN DAGScheduler: Broadcasting large task binary with size 1320.9 KiB
25/12/06 06:26:09 WARN DAGScheduler: Broadcasting large task binary with size 1323.0 KiB
25/12/06 06:26:09 WARN DAGScheduler: Broadcasting large task binary with size 1323.5 KiB
25/12/06 06:26:09 WARN DAGScheduler: Broadcasting large task binary with size 1324.0 KiB
25/12/06 06:26:10 WARN DAGScheduler: Broadcasting large task binary with size 1325.2 KiB
25/12/06 06:26:10 WARN DAGScheduler: Broadcasting large task binary with size 1327.2 KiB
25/12/06 06:26:10 WARN DAGScheduler: Broadcasting large task binary with size 1329.5 KiB
25/12/06 06:26:10 WARN DAGScheduler: Broadcasting large task binary with size 1330.0 KiB
25/12/06 06:26:10 WARN DAGScheduler: Broadcasting large task binary with size 1330.6 KiB
25/12/06 06:26:10 WARN DAGScheduler: Broadcasting large task binary

with size 1331.8 KiB
25/12/06 06:26:10 WARN DAGScheduler: Broadcasting large task binary
with size 1333.7 KiB
25/12/06 06:26:10 WARN DAGScheduler: Broadcasting large task binary
with size 1335.6 KiB
25/12/06 06:26:10 WARN DAGScheduler: Broadcasting large task binary
with size 1336.1 KiB
25/12/06 06:26:10 WARN DAGScheduler: Broadcasting large task binary
with size 1336.6 KiB
25/12/06 06:26:10 WARN DAGScheduler: Broadcasting large task binary
with size 1337.8 KiB
25/12/06 06:26:10 WARN DAGScheduler: Broadcasting large task binary
with size 1339.5 KiB
25/12/06 06:26:11 WARN DAGScheduler: Broadcasting large task binary
with size 1341.6 KiB
25/12/06 06:26:11 WARN DAGScheduler: Broadcasting large task binary
with size 1342.1 KiB
25/12/06 06:26:11 WARN DAGScheduler: Broadcasting large task binary
with size 1342.7 KiB
25/12/06 06:26:11 WARN DAGScheduler: Broadcasting large task binary
with size 1343.6 KiB
25/12/06 06:26:11 WARN DAGScheduler: Broadcasting large task binary
with size 1345.3 KiB
25/12/06 06:26:11 WARN DAGScheduler: Broadcasting large task binary
with size 1066.0 KiB
25/12/06 06:26:11 WARN DAGScheduler: Broadcasting large task binary
with size 1078.4 KiB
25/12/06 06:26:12 WARN DAGScheduler: Broadcasting large task binary
with size 1078.4 KiB

-> Result: AUC=0.8555 | F1=0.9158 | Time=12.1s

[25/48] Test config: {'ratio_target': 5, 'ratio_multiplier': 0.1,
'identity_weight': 0.5, 'model': LogisticRegression_afae24335158,
'combinaison': 'w2v + tfidf'}

Undersampling : On garde 30.24% des données négatives.

Ratio de sain/toxique 5.13

toxic_weights : 0.51 + 1, identity_weight :0.5

Prétraitement terminé, df_train_final_balanced_debiased = 3779 |
df_test_final_all_w2v = 1199

25/12/06 06:26:13 WARN DAGScheduler: Broadcasting large task binary
with size 1041.1 KiB
25/12/06 06:26:13 WARN DAGScheduler: Broadcasting large task binary
with size 1041.6 KiB
25/12/06 06:26:13 WARN DAGScheduler: Broadcasting large task binary
with size 1041.6 KiB
25/12/06 06:26:13 WARN DAGScheduler: Broadcasting large task binary
with size 1041.6 KiB
25/12/06 06:26:13 WARN DAGScheduler: Broadcasting large task binary

with size 1041.6 KiB
25/12/06 06:26:13 WARN DAGScheduler: Broadcasting large task binary
with size 1041.6 KiB
25/12/06 06:26:13 WARN DAGScheduler: Broadcasting large task binary
with size 1041.6 KiB
25/12/06 06:26:13 WARN DAGScheduler: Broadcasting large task binary
with size 1041.6 KiB
25/12/06 06:26:13 WARN DAGScheduler: Broadcasting large task binary
with size 1041.6 KiB
25/12/06 06:26:13 WARN DAGScheduler: Broadcasting large task binary
with size 1041.6 KiB
25/12/06 06:26:13 WARN DAGScheduler: Broadcasting large task binary
with size 1041.6 KiB
25/12/06 06:26:13 WARN DAGScheduler: Broadcasting large task binary
with size 1041.6 KiB

-> Result: AUC=0.8005 | F1=0.8850 | Time=2.0s

[26/48] Test config: {'ratio_target': 5, 'ratio_multiplier': 0.1,
'identity_weight': 0.5, 'model': RandomForestClassifier_ad394a6e557e,
'combinaison': 'w2v + tfidf'}

Undersampling : On garde 30.24% des données négatives.

Ratio de sain/toxique 5.13

toxic_weights : 0.51 + 1, identity_weight :0.5

Prétraitement terminé, df_train_final_balanced_debiased = 3779 |
df_test_final_all_w2v = 1199

25/12/06 06:26:15 WARN DAGScheduler: Broadcasting large task binary
with size 1037.6 KiB
25/12/06 06:26:15 WARN DAGScheduler: Broadcasting large task binary
with size 1037.5 KiB
25/12/06 06:26:15 WARN DAGScheduler: Broadcasting large task binary
with size 1086.8 KiB
25/12/06 06:26:16 WARN DAGScheduler: Broadcasting large task binary
with size 1217.7 KiB
25/12/06 06:26:16 WARN DAGScheduler: Broadcasting large task binary
with size 1229.5 KiB
25/12/06 06:26:17 WARN DAGScheduler: Broadcasting large task binary
with size 1247.5 KiB
25/12/06 06:26:17 WARN DAGScheduler: Broadcasting large task binary
with size 1275.1 KiB
25/12/06 06:26:17 WARN DAGScheduler: Broadcasting large task binary
with size 1312.0 KiB
25/12/06 06:26:17 WARN DAGScheduler: Broadcasting large task binary
with size 1037.3 KiB
25/12/06 06:26:17 WARN DAGScheduler: Broadcasting large task binary
with size 1049.4 KiB
25/12/06 06:26:17 WARN DAGScheduler: Broadcasting large task binary
with size 1049.4 KiB

-> Result: AUC=0.8139 | F1=0.9092 | Time=3.7s

[27/48] Test config: {'ratio_target': 5, 'ratio_multiplier': 0.1, 'identity_weight': 0.5, 'model': LinearSVC_8f0c0322b70e, 'combinaison': 'w2v + tfidf'}

Undersampling : On garde 30.24% des données négatives.

Ratio de sain/toxique 5.13

toxic_weights : 0.51 + 1, identity_weight :0.5

Prétraitement terminé, df_train_final_balanced_debiased = 3779 |
df_test_final_all_w2v = 1199

25/12/06 06:26:19 WARN DAGScheduler: Broadcasting large task binary with size 1038.6 KiB

25/12/06 06:26:19 WARN DAGScheduler: Broadcasting large task binary with size 1039.1 KiB

25/12/06 06:26:19 WARN DAGScheduler: Broadcasting large task binary with size 1039.1 KiB

25/12/06 06:26:19 WARN DAGScheduler: Broadcasting large task binary with size 1039.1 KiB

25/12/06 06:26:19 WARN DAGScheduler: Broadcasting large task binary with size 1039.1 KiB

25/12/06 06:26:19 WARN DAGScheduler: Broadcasting large task binary with size 1039.1 KiB

25/12/06 06:26:19 WARN DAGScheduler: Broadcasting large task binary with size 1039.1 KiB

25/12/06 06:26:19 WARN DAGScheduler: Broadcasting large task binary with size 1039.1 KiB

25/12/06 06:26:19 WARN DAGScheduler: Broadcasting large task binary with size 1039.1 KiB

25/12/06 06:26:19 WARN DAGScheduler: Broadcasting large task binary with size 1039.1 KiB

25/12/06 06:26:19 WARN DAGScheduler: Broadcasting large task binary with size 1039.1 KiB

25/12/06 06:26:19 WARN DAGScheduler: Broadcasting large task binary with size 1039.1 KiB

25/12/06 06:26:19 WARN DAGScheduler: Broadcasting large task binary with size 1039.1 KiB

25/12/06 06:26:19 WARN DAGScheduler: Broadcasting large task binary with size 1039.1 KiB

25/12/06 06:26:19 WARN DAGScheduler: Broadcasting large task binary with size 1039.1 KiB

25/12/06 06:26:19 WARN DAGScheduler: Broadcasting large task binary with size 1039.1 KiB

25/12/06 06:26:19 WARN DAGScheduler: Broadcasting large task binary with size 1039.1 KiB

25/12/06 06:26:19 WARN DAGScheduler: Broadcasting large task binary with size 1039.1 KiB

25/12/06 06:26:19 WARN DAGScheduler: Broadcasting large task binary with size 1039.1 KiB

25/12/06 06:26:19 WARN DAGScheduler: Broadcasting large task binary

[illegible]

[illegible]

[illegible]

```
25/12/06 06:26:21 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
25/12/06 06:26:21 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
25/12/06 06:26:21 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
25/12/06 06:26:21 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
25/12/06 06:26:21 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
25/12/06 06:26:21 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
25/12/06 06:26:21 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
25/12/06 06:26:21 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
25/12/06 06:26:21 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
25/12/06 06:26:21 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
25/12/06 06:26:21 WARN DAGScheduler: Broadcasting large task binary
with size 1039.1 KiB
```

```
-> Result: AUC=0.8100 | F1=0.8963 | Time=4.3s
```

```
[28/48] Test config: {'ratio_target': 5, 'ratio_multiplier': 0.1,
'identity_weight': 0.5, 'model': GBTCClassifier_3b8b769d6280,
'combinaison': 'w2v + tfidf'}
```

```
Undersampling : On garde 30.24% des données négatives.
```

```
Ratio de sain/toxique 5.13
```

```
toxic_weights : 0.51 + 1, identity_weight :0.5
```

```
Prétraitement terminé, df_train_final_balanced_debiased = 3779 |
df_test_final_all_w2v = 1199
```

```
25/12/06 06:26:23 WARN DAGScheduler: Broadcasting large task binary
with size 1037.3 KiB
25/12/06 06:26:23 WARN DAGScheduler: Broadcasting large task binary
with size 1037.2 KiB
25/12/06 06:26:23 WARN DAGScheduler: Broadcasting large task binary
with size 1086.5 KiB
25/12/06 06:26:24 WARN DAGScheduler: Broadcasting large task binary
with size 1211.3 KiB
25/12/06 06:26:24 WARN DAGScheduler: Broadcasting large task binary
with size 1212.1 KiB
25/12/06 06:26:24 WARN DAGScheduler: Broadcasting large task binary
with size 1212.6 KiB
25/12/06 06:26:24 WARN DAGScheduler: Broadcasting large task binary
with size 1213.8 KiB
25/12/06 06:26:24 WARN DAGScheduler: Broadcasting large task binary
with size 1215.7 KiB
```

25/12/06 06:26:25 WARN DAGScheduler: Broadcasting large task binary with size 1221.9 KiB
25/12/06 06:26:25 WARN DAGScheduler: Broadcasting large task binary with size 1222.4 KiB
25/12/06 06:26:25 WARN DAGScheduler: Broadcasting large task binary with size 1222.9 KiB
25/12/06 06:26:25 WARN DAGScheduler: Broadcasting large task binary with size 1224.1 KiB
25/12/06 06:26:25 WARN DAGScheduler: Broadcasting large task binary with size 1226.4 KiB
25/12/06 06:26:25 WARN DAGScheduler: Broadcasting large task binary with size 1228.6 KiB
25/12/06 06:26:25 WARN DAGScheduler: Broadcasting large task binary with size 1229.1 KiB
25/12/06 06:26:25 WARN DAGScheduler: Broadcasting large task binary with size 1229.7 KiB
25/12/06 06:26:25 WARN DAGScheduler: Broadcasting large task binary with size 1230.8 KiB
25/12/06 06:26:25 WARN DAGScheduler: Broadcasting large task binary with size 1232.5 KiB
25/12/06 06:26:25 WARN DAGScheduler: Broadcasting large task binary with size 1234.5 KiB
25/12/06 06:26:25 WARN DAGScheduler: Broadcasting large task binary with size 1235.0 KiB
25/12/06 06:26:25 WARN DAGScheduler: Broadcasting large task binary with size 1235.6 KiB
25/12/06 06:26:26 WARN DAGScheduler: Broadcasting large task binary with size 1236.8 KiB
25/12/06 06:26:26 WARN DAGScheduler: Broadcasting large task binary with size 1239.0 KiB
25/12/06 06:26:26 WARN DAGScheduler: Broadcasting large task binary with size 1241.6 KiB
25/12/06 06:26:26 WARN DAGScheduler: Broadcasting large task binary with size 1242.0 KiB
25/12/06 06:26:26 WARN DAGScheduler: Broadcasting large task binary with size 1242.6 KiB
25/12/06 06:26:26 WARN DAGScheduler: Broadcasting large task binary with size 1243.8 KiB
25/12/06 06:26:26 WARN DAGScheduler: Broadcasting large task binary with size 1245.7 KiB
25/12/06 06:26:26 WARN DAGScheduler: Broadcasting large task binary with size 1247.8 KiB
25/12/06 06:26:26 WARN DAGScheduler: Broadcasting large task binary with size 1248.3 KiB
25/12/06 06:26:26 WARN DAGScheduler: Broadcasting large task binary with size 1248.8 KiB
25/12/06 06:26:26 WARN DAGScheduler: Broadcasting large task binary with size 1250.0 KiB
25/12/06 06:26:26 WARN DAGScheduler: Broadcasting large task binary

with size 1252.3 KiB
25/12/06 06:26:26 WARN DAGScheduler: Broadcasting large task binary
with size 1254.6 KiB
25/12/06 06:26:27 WARN DAGScheduler: Broadcasting large task binary
with size 1255.1 KiB
25/12/06 06:26:27 WARN DAGScheduler: Broadcasting large task binary
with size 1255.7 KiB
25/12/06 06:26:27 WARN DAGScheduler: Broadcasting large task binary
with size 1256.9 KiB
25/12/06 06:26:27 WARN DAGScheduler: Broadcasting large task binary
with size 1258.6 KiB
25/12/06 06:26:27 WARN DAGScheduler: Broadcasting large task binary
with size 1261.0 KiB
25/12/06 06:26:27 WARN DAGScheduler: Broadcasting large task binary
with size 1261.5 KiB
25/12/06 06:26:27 WARN DAGScheduler: Broadcasting large task binary
with size 1262.1 KiB
25/12/06 06:26:27 WARN DAGScheduler: Broadcasting large task binary
with size 1263.2 KiB
25/12/06 06:26:27 WARN DAGScheduler: Broadcasting large task binary
with size 1265.2 KiB
25/12/06 06:26:27 WARN DAGScheduler: Broadcasting large task binary
with size 1267.2 KiB
25/12/06 06:26:27 WARN DAGScheduler: Broadcasting large task binary
with size 1267.7 KiB
25/12/06 06:26:27 WARN DAGScheduler: Broadcasting large task binary
with size 1268.3 KiB
25/12/06 06:26:28 WARN DAGScheduler: Broadcasting large task binary
with size 1269.5 KiB
25/12/06 06:26:28 WARN DAGScheduler: Broadcasting large task binary
with size 1271.7 KiB
25/12/06 06:26:28 WARN DAGScheduler: Broadcasting large task binary
with size 1274.1 KiB
25/12/06 06:26:28 WARN DAGScheduler: Broadcasting large task binary
with size 1274.6 KiB
25/12/06 06:26:28 WARN DAGScheduler: Broadcasting large task binary
with size 1275.2 KiB
25/12/06 06:26:28 WARN DAGScheduler: Broadcasting large task binary
with size 1276.3 KiB
25/12/06 06:26:28 WARN DAGScheduler: Broadcasting large task binary
with size 1278.6 KiB
25/12/06 06:26:28 WARN DAGScheduler: Broadcasting large task binary
with size 1281.3 KiB
25/12/06 06:26:28 WARN DAGScheduler: Broadcasting large task binary
with size 1281.8 KiB
25/12/06 06:26:28 WARN DAGScheduler: Broadcasting large task binary
with size 1282.3 KiB
25/12/06 06:26:28 WARN DAGScheduler: Broadcasting large task binary
with size 1283.5 KiB

25/12/06 06:26:28 WARN DAGScheduler: Broadcasting large task binary with size 1285.8 KiB
25/12/06 06:26:29 WARN DAGScheduler: Broadcasting large task binary with size 1288.5 KiB
25/12/06 06:26:29 WARN DAGScheduler: Broadcasting large task binary with size 1289.0 KiB
25/12/06 06:26:29 WARN DAGScheduler: Broadcasting large task binary with size 1289.5 KiB
25/12/06 06:26:29 WARN DAGScheduler: Broadcasting large task binary with size 1290.7 KiB
25/12/06 06:26:29 WARN DAGScheduler: Broadcasting large task binary with size 1292.7 KiB
25/12/06 06:26:29 WARN DAGScheduler: Broadcasting large task binary with size 1295.0 KiB
25/12/06 06:26:29 WARN DAGScheduler: Broadcasting large task binary with size 1295.5 KiB
25/12/06 06:26:29 WARN DAGScheduler: Broadcasting large task binary with size 1296.1 KiB
25/12/06 06:26:29 WARN DAGScheduler: Broadcasting large task binary with size 1297.3 KiB
25/12/06 06:26:29 WARN DAGScheduler: Broadcasting large task binary with size 1299.6 KiB
25/12/06 06:26:29 WARN DAGScheduler: Broadcasting large task binary with size 1301.9 KiB
25/12/06 06:26:29 WARN DAGScheduler: Broadcasting large task binary with size 1302.3 KiB
25/12/06 06:26:30 WARN DAGScheduler: Broadcasting large task binary with size 1302.9 KiB
25/12/06 06:26:30 WARN DAGScheduler: Broadcasting large task binary with size 1304.1 KiB
25/12/06 06:26:30 WARN DAGScheduler: Broadcasting large task binary with size 1306.1 KiB
25/12/06 06:26:30 WARN DAGScheduler: Broadcasting large task binary with size 1308.4 KiB
25/12/06 06:26:30 WARN DAGScheduler: Broadcasting large task binary with size 1308.9 KiB
25/12/06 06:26:30 WARN DAGScheduler: Broadcasting large task binary with size 1309.5 KiB
25/12/06 06:26:30 WARN DAGScheduler: Broadcasting large task binary with size 1310.7 KiB
25/12/06 06:26:30 WARN DAGScheduler: Broadcasting large task binary with size 1312.9 KiB
25/12/06 06:26:30 WARN DAGScheduler: Broadcasting large task binary with size 1315.1 KiB
25/12/06 06:26:30 WARN DAGScheduler: Broadcasting large task binary with size 1315.6 KiB
25/12/06 06:26:30 WARN DAGScheduler: Broadcasting large task binary with size 1316.2 KiB
25/12/06 06:26:30 WARN DAGScheduler: Broadcasting large task binary

with size 1317.4 KiB
25/12/06 06:26:31 WARN DAGScheduler: Broadcasting large task binary
with size 1319.6 KiB
25/12/06 06:26:31 WARN DAGScheduler: Broadcasting large task binary
with size 1322.1 KiB
25/12/06 06:26:31 WARN DAGScheduler: Broadcasting large task binary
with size 1322.6 KiB
25/12/06 06:26:31 WARN DAGScheduler: Broadcasting large task binary
with size 1323.2 KiB
25/12/06 06:26:31 WARN DAGScheduler: Broadcasting large task binary
with size 1324.4 KiB
25/12/06 06:26:31 WARN DAGScheduler: Broadcasting large task binary
with size 1326.7 KiB
25/12/06 06:26:31 WARN DAGScheduler: Broadcasting large task binary
with size 1329.3 KiB
25/12/06 06:26:31 WARN DAGScheduler: Broadcasting large task binary
with size 1329.8 KiB
25/12/06 06:26:31 WARN DAGScheduler: Broadcasting large task binary
with size 1330.4 KiB
25/12/06 06:26:31 WARN DAGScheduler: Broadcasting large task binary
with size 1331.6 KiB
25/12/06 06:26:31 WARN DAGScheduler: Broadcasting large task binary
with size 1333.9 KiB
25/12/06 06:26:31 WARN DAGScheduler: Broadcasting large task binary
with size 1336.5 KiB
25/12/06 06:26:32 WARN DAGScheduler: Broadcasting large task binary
with size 1337.0 KiB
25/12/06 06:26:32 WARN DAGScheduler: Broadcasting large task binary
with size 1337.6 KiB
25/12/06 06:26:32 WARN DAGScheduler: Broadcasting large task binary
with size 1338.8 KiB
25/12/06 06:26:32 WARN DAGScheduler: Broadcasting large task binary
with size 1340.7 KiB
25/12/06 06:26:32 WARN DAGScheduler: Broadcasting large task binary
with size 1342.6 KiB
25/12/06 06:26:32 WARN DAGScheduler: Broadcasting large task binary
with size 1343.1 KiB
25/12/06 06:26:32 WARN DAGScheduler: Broadcasting large task binary
with size 1343.6 KiB
25/12/06 06:26:32 WARN DAGScheduler: Broadcasting large task binary
with size 1344.8 KiB
25/12/06 06:26:32 WARN DAGScheduler: Broadcasting large task binary
with size 1347.1 KiB
25/12/06 06:26:33 WARN DAGScheduler: Broadcasting large task binary
with size 1068.4 KiB
25/12/06 06:26:33 WARN DAGScheduler: Broadcasting large task binary
with size 1080.7 KiB
25/12/06 06:26:33 WARN DAGScheduler: Broadcasting large task binary
with size 1080.7 KiB

-> Result: AUC=0.8344 | F1=0.9193 | Time=11.0s

[29/48] Test config: {'ratio_target': 5, 'ratio_multiplier': 0.1, 'identity_weight': 1.0, 'model': LogisticRegression_afae24335158, 'combinaison': 'w2v + tfidf'}

Undersampling : On garde 30.24% des données négatives.

Ratio de sain/toxique 5.13

toxic_weights : 0.51 + 1, identity_weight :1.0

Prétraitement terminé, df_train_final_balanced_debiased = 3779 |
df_test_final_all_w2v = 1199

25/12/06 06:26:34 WARN DAGScheduler: Broadcasting large task binary with size 1041.1 KiB

25/12/06 06:26:34 WARN DAGScheduler: Broadcasting large task binary with size 1041.7 KiB

25/12/06 06:26:34 WARN DAGScheduler: Broadcasting large task binary with size 1041.7 KiB

25/12/06 06:26:35 WARN DAGScheduler: Broadcasting large task binary with size 1041.7 KiB

25/12/06 06:26:35 WARN DAGScheduler: Broadcasting large task binary with size 1041.7 KiB

25/12/06 06:26:35 WARN DAGScheduler: Broadcasting large task binary with size 1041.7 KiB

25/12/06 06:26:35 WARN DAGScheduler: Broadcasting large task binary with size 1041.7 KiB

25/12/06 06:26:35 WARN DAGScheduler: Broadcasting large task binary with size 1041.7 KiB

25/12/06 06:26:35 WARN DAGScheduler: Broadcasting large task binary with size 1041.7 KiB

25/12/06 06:26:35 WARN DAGScheduler: Broadcasting large task binary with size 1041.7 KiB

25/12/06 06:26:35 WARN DAGScheduler: Broadcasting large task binary with size 1041.7 KiB

25/12/06 06:26:35 WARN DAGScheduler: Broadcasting large task binary with size 1041.7 KiB

-> Result: AUC=0.7987 | F1=0.8773 | Time=2.4s

[30/48] Test config: {'ratio_target': 5, 'ratio_multiplier': 0.1, 'identity_weight': 1.0, 'model': RandomForestClassifier_ad394a6e557e, 'combinaison': 'w2v + tfidf'}

Undersampling : On garde 30.24% des données négatives.

Ratio de sain/toxique 5.13

toxic_weights : 0.51 + 1, identity_weight :1.0

Prétraitement terminé, df_train_final_balanced_debiased = 3779 |
df_test_final_all_w2v = 1199

25/12/06 06:26:36 WARN DAGScheduler: Broadcasting large task binary with size 1037.6 KiB

25/12/06 06:26:36 WARN DAGScheduler: Broadcasting large task binary

[illegible]

[illegible]

[illegible]

[illegible]

-> Result: AUC=0.8294 | F1=0.9011 | Time=4.4s

```
[32/48] Test config: {'ratio_target': 5, 'ratio_multiplier': 0.1,
'identity_weight': 1.0, 'model': GBTClassifier_3b8b769d6280,
'combinaison': 'w2v + tfidf'}
Undersampling : On garde 30.24% des données négatives.
Ratio de sain/toxique 5.13
```

```
toxic_weights : 0.51 + 1, identity_weight :1.0  
Prétraitement terminé, df_train_final_balanced_debiased = 3779 |  
df_test_final_all_w2v = 1199
```

```
25/12/06 06:26:45 WARN DAGScheduler: Broadcasting large task binary  
with size 1037.3 KiB  
25/12/06 06:26:45 WARN DAGScheduler: Broadcasting large task binary  
with size 1037.2 KiB  
25/12/06 06:26:45 WARN DAGScheduler: Broadcasting large task binary  
with size 1086.5 KiB  
25/12/06 06:26:46 WARN DAGScheduler: Broadcasting large task binary  
with size 1211.3 KiB  
25/12/06 06:26:47 WARN DAGScheduler: Broadcasting large task binary  
with size 1212.1 KiB  
25/12/06 06:26:47 WARN DAGScheduler: Broadcasting large task binary  
with size 1212.6 KiB  
25/12/06 06:26:47 WARN DAGScheduler: Broadcasting large task binary  
with size 1213.8 KiB  
25/12/06 06:26:47 WARN DAGScheduler: Broadcasting large task binary  
with size 1215.7 KiB  
25/12/06 06:26:47 WARN DAGScheduler: Broadcasting large task binary  
with size 1221.6 KiB  
25/12/06 06:26:47 WARN DAGScheduler: Broadcasting large task binary  
with size 1222.0 KiB  
25/12/06 06:26:47 WARN DAGScheduler: Broadcasting large task binary  
with size 1222.6 KiB  
25/12/06 06:26:47 WARN DAGScheduler: Broadcasting large task binary  
with size 1223.8 KiB  
25/12/06 06:26:47 WARN DAGScheduler: Broadcasting large task binary  
with size 1225.8 KiB  
25/12/06 06:26:47 WARN DAGScheduler: Broadcasting large task binary  
with size 1228.0 KiB  
25/12/06 06:26:47 WARN DAGScheduler: Broadcasting large task binary  
with size 1228.5 KiB  
25/12/06 06:26:48 WARN DAGScheduler: Broadcasting large task binary  
with size 1229.0 KiB  
25/12/06 06:26:48 WARN DAGScheduler: Broadcasting large task binary  
with size 1230.2 KiB  
25/12/06 06:26:48 WARN DAGScheduler: Broadcasting large task binary  
with size 1232.2 KiB  
25/12/06 06:26:48 WARN DAGScheduler: Broadcasting large task binary  
with size 1234.0 KiB  
25/12/06 06:26:48 WARN DAGScheduler: Broadcasting large task binary  
with size 1234.5 KiB  
25/12/06 06:26:48 WARN DAGScheduler: Broadcasting large task binary  
with size 1235.1 KiB  
25/12/06 06:26:48 WARN DAGScheduler: Broadcasting large task binary  
with size 1236.3 KiB  
25/12/06 06:26:48 WARN DAGScheduler: Broadcasting large task binary  
with size 1238.3 KiB
```

25/12/06 06:26:48 WARN DAGScheduler: Broadcasting large task binary with size 1240.4 KiB
25/12/06 06:26:48 WARN DAGScheduler: Broadcasting large task binary with size 1240.9 KiB
25/12/06 06:26:48 WARN DAGScheduler: Broadcasting large task binary with size 1241.5 KiB
25/12/06 06:26:48 WARN DAGScheduler: Broadcasting large task binary with size 1242.7 KiB
25/12/06 06:26:48 WARN DAGScheduler: Broadcasting large task binary with size 1244.7 KiB
25/12/06 06:26:49 WARN DAGScheduler: Broadcasting large task binary with size 1247.0 KiB
25/12/06 06:26:49 WARN DAGScheduler: Broadcasting large task binary with size 1247.4 KiB
25/12/06 06:26:49 WARN DAGScheduler: Broadcasting large task binary with size 1248.0 KiB
25/12/06 06:26:49 WARN DAGScheduler: Broadcasting large task binary with size 1249.2 KiB
25/12/06 06:26:49 WARN DAGScheduler: Broadcasting large task binary with size 1250.9 KiB
25/12/06 06:26:49 WARN DAGScheduler: Broadcasting large task binary with size 1253.3 KiB
25/12/06 06:26:49 WARN DAGScheduler: Broadcasting large task binary with size 1253.8 KiB
25/12/06 06:26:49 WARN DAGScheduler: Broadcasting large task binary with size 1254.4 KiB
25/12/06 06:26:49 WARN DAGScheduler: Broadcasting large task binary with size 1255.6 KiB
25/12/06 06:26:49 WARN DAGScheduler: Broadcasting large task binary with size 1257.5 KiB
25/12/06 06:26:49 WARN DAGScheduler: Broadcasting large task binary with size 1259.6 KiB
25/12/06 06:26:49 WARN DAGScheduler: Broadcasting large task binary with size 1260.0 KiB
25/12/06 06:26:49 WARN DAGScheduler: Broadcasting large task binary with size 1260.6 KiB
25/12/06 06:26:50 WARN DAGScheduler: Broadcasting large task binary with size 1261.8 KiB
25/12/06 06:26:50 WARN DAGScheduler: Broadcasting large task binary with size 1264.1 KiB
25/12/06 06:26:50 WARN DAGScheduler: Broadcasting large task binary with size 1266.4 KiB
25/12/06 06:26:50 WARN DAGScheduler: Broadcasting large task binary with size 1266.9 KiB
25/12/06 06:26:50 WARN DAGScheduler: Broadcasting large task binary with size 1267.5 KiB
25/12/06 06:26:50 WARN DAGScheduler: Broadcasting large task binary with size 1268.7 KiB
25/12/06 06:26:50 WARN DAGScheduler: Broadcasting large task binary

with size 1270.4 KiB
25/12/06 06:26:50 WARN DAGScheduler: Broadcasting large task binary
with size 1272.7 KiB
25/12/06 06:26:50 WARN DAGScheduler: Broadcasting large task binary
with size 1273.1 KiB
25/12/06 06:26:50 WARN DAGScheduler: Broadcasting large task binary
with size 1273.7 KiB
25/12/06 06:26:50 WARN DAGScheduler: Broadcasting large task binary
with size 1274.9 KiB
25/12/06 06:26:50 WARN DAGScheduler: Broadcasting large task binary
with size 1277.2 KiB
25/12/06 06:26:51 WARN DAGScheduler: Broadcasting large task binary
with size 1279.7 KiB
25/12/06 06:26:51 WARN DAGScheduler: Broadcasting large task binary
with size 1280.2 KiB
25/12/06 06:26:51 WARN DAGScheduler: Broadcasting large task binary
with size 1280.7 KiB
25/12/06 06:26:51 WARN DAGScheduler: Broadcasting large task binary
with size 1281.9 KiB
25/12/06 06:26:51 WARN DAGScheduler: Broadcasting large task binary
with size 1284.2 KiB
25/12/06 06:26:51 WARN DAGScheduler: Broadcasting large task binary
with size 1286.7 KiB
25/12/06 06:26:51 WARN DAGScheduler: Broadcasting large task binary
with size 1287.2 KiB
25/12/06 06:26:51 WARN DAGScheduler: Broadcasting large task binary
with size 1287.8 KiB
25/12/06 06:26:51 WARN DAGScheduler: Broadcasting large task binary
with size 1289.0 KiB
25/12/06 06:26:52 WARN DAGScheduler: Broadcasting large task binary
with size 1291.2 KiB
25/12/06 06:26:52 WARN DAGScheduler: Broadcasting large task binary
with size 1293.6 KiB
25/12/06 06:26:52 WARN DAGScheduler: Broadcasting large task binary
with size 1294.1 KiB
25/12/06 06:26:52 WARN DAGScheduler: Broadcasting large task binary
with size 1294.6 KiB
25/12/06 06:26:52 WARN DAGScheduler: Broadcasting large task binary
with size 1295.8 KiB
25/12/06 06:26:52 WARN DAGScheduler: Broadcasting large task binary
with size 1298.1 KiB
25/12/06 06:26:52 WARN DAGScheduler: Broadcasting large task binary
with size 1300.6 KiB
25/12/06 06:26:52 WARN DAGScheduler: Broadcasting large task binary
with size 1301.1 KiB
25/12/06 06:26:52 WARN DAGScheduler: Broadcasting large task binary
with size 1301.7 KiB
25/12/06 06:26:52 WARN DAGScheduler: Broadcasting large task binary
with size 1302.9 KiB

25/12/06 06:26:52 WARN DAGScheduler: Broadcasting large task binary with size 1305.1 KiB
25/12/06 06:26:52 WARN DAGScheduler: Broadcasting large task binary with size 1307.6 KiB
25/12/06 06:26:52 WARN DAGScheduler: Broadcasting large task binary with size 1308.1 KiB
25/12/06 06:26:53 WARN DAGScheduler: Broadcasting large task binary with size 1308.7 KiB
25/12/06 06:26:53 WARN DAGScheduler: Broadcasting large task binary with size 1309.9 KiB
25/12/06 06:26:53 WARN DAGScheduler: Broadcasting large task binary with size 1311.6 KiB
25/12/06 06:26:53 WARN DAGScheduler: Broadcasting large task binary with size 1314.0 KiB
25/12/06 06:26:53 WARN DAGScheduler: Broadcasting large task binary with size 1314.5 KiB
25/12/06 06:26:53 WARN DAGScheduler: Broadcasting large task binary with size 1315.1 KiB
25/12/06 06:26:53 WARN DAGScheduler: Broadcasting large task binary with size 1316.3 KiB
25/12/06 06:26:53 WARN DAGScheduler: Broadcasting large task binary with size 1318.3 KiB
25/12/06 06:26:53 WARN DAGScheduler: Broadcasting large task binary with size 1320.6 KiB
25/12/06 06:26:53 WARN DAGScheduler: Broadcasting large task binary with size 1321.1 KiB
25/12/06 06:26:53 WARN DAGScheduler: Broadcasting large task binary with size 1321.6 KiB
25/12/06 06:26:54 WARN DAGScheduler: Broadcasting large task binary with size 1322.8 KiB
25/12/06 06:26:54 WARN DAGScheduler: Broadcasting large task binary with size 1325.1 KiB
25/12/06 06:26:54 WARN DAGScheduler: Broadcasting large task binary with size 1327.3 KiB
25/12/06 06:26:54 WARN DAGScheduler: Broadcasting large task binary with size 1327.8 KiB
25/12/06 06:26:54 WARN DAGScheduler: Broadcasting large task binary with size 1328.3 KiB
25/12/06 06:26:54 WARN DAGScheduler: Broadcasting large task binary with size 1329.5 KiB
25/12/06 06:26:54 WARN DAGScheduler: Broadcasting large task binary with size 1331.8 KiB
25/12/06 06:26:54 WARN DAGScheduler: Broadcasting large task binary with size 1334.0 KiB
25/12/06 06:26:54 WARN DAGScheduler: Broadcasting large task binary with size 1334.5 KiB
25/12/06 06:26:54 WARN DAGScheduler: Broadcasting large task binary with size 1335.0 KiB
25/12/06 06:26:54 WARN DAGScheduler: Broadcasting large task binary

with size 1229.8 KiB
25/12/06 06:26:57 WARN DAGScheduler: Broadcasting large task binary
with size 1229.8 KiB
25/12/06 06:26:58 WARN DAGScheduler: Broadcasting large task binary
with size 1229.8 KiB
25/12/06 06:26:58 WARN DAGScheduler: Broadcasting large task binary
with size 1176.7 KiB
25/12/06 06:26:58 WARN DAGScheduler: Broadcasting large task binary
with size 1188.8 KiB
25/12/06 06:26:58 WARN DAGScheduler: Broadcasting large task binary
with size 1188.8 KiB

-> Result: AUC=0.8240 | F1=0.8921 | Time=2.5s

[34/48] Test config: {'ratio_target': 7, 'ratio_multiplier': 0.05,
'identity_weight': 0.5, 'model': RandomForestClassifier_ad394a6e557e,
'combinaison': 'w2v + tfidf'}

Undersampling : On garde 42.34% des données négatives.

Ratio de sain/toxique 7.08

toxic_weights : 0.35 + 1, identity_weight :0.5

Prétraitement terminé, df_train_final_balanced_debiased = 4978 |

df_test_final_all_w2v = 1199

25/12/06 06:26:59 WARN DAGScheduler: Broadcasting large task binary
with size 1225.8 KiB
25/12/06 06:26:59 WARN DAGScheduler: Broadcasting large task binary
with size 1225.7 KiB
25/12/06 06:26:59 WARN DAGScheduler: Broadcasting large task binary
with size 1287.5 KiB
25/12/06 06:27:01 WARN DAGScheduler: Broadcasting large task binary
with size 1445.9 KiB
25/12/06 06:27:01 WARN DAGScheduler: Broadcasting large task binary
with size 1458.0 KiB
25/12/06 06:27:01 WARN DAGScheduler: Broadcasting large task binary
with size 1475.4 KiB
25/12/06 06:27:01 WARN DAGScheduler: Broadcasting large task binary
with size 1500.2 KiB
25/12/06 06:27:01 WARN DAGScheduler: Broadcasting large task binary
with size 1530.5 KiB
25/12/06 06:27:02 WARN DAGScheduler: Broadcasting large task binary
with size 1217.8 KiB
25/12/06 06:27:02 WARN DAGScheduler: Broadcasting large task binary
with size 1230.0 KiB
25/12/06 06:27:02 WARN DAGScheduler: Broadcasting large task binary
with size 1230.0 KiB

-> Result: AUC=0.7514 | F1=0.9072 | Time=4.1s

[35/48] Test config: {'ratio_target': 7, 'ratio_multiplier': 0.05,
'identity_weight': 0.5, 'model': LinearSVC_8f0c0322b70e,

[illegible]

[illegible]

[illegible]

```
25/12/06 06:27:06 WARN DAGScheduler: Broadcasting large task binary
with size 1227.3 KiB
25/12/06 06:27:06 WARN DAGScheduler: Broadcasting large task binary
with size 1227.3 KiB
25/12/06 06:27:06 WARN DAGScheduler: Broadcasting large task binary
with size 1227.3 KiB
25/12/06 06:27:06 WARN DAGScheduler: Broadcasting large task binary
with size 1227.3 KiB
25/12/06 06:27:06 WARN DAGScheduler: Broadcasting large task binary
with size 1227.3 KiB
25/12/06 06:27:06 WARN DAGScheduler: Broadcasting large task binary
with size 1227.3 KiB
25/12/06 06:27:06 WARN DAGScheduler: Broadcasting large task binary
with size 1227.3 KiB
25/12/06 06:27:06 WARN DAGScheduler: Broadcasting large task binary
with size 1227.3 KiB
25/12/06 06:27:06 WARN DAGScheduler: Broadcasting large task binary
with size 1227.3 KiB
25/12/06 06:27:06 WARN DAGScheduler: Broadcasting large task binary
with size 1227.3 KiB
25/12/06 06:27:06 WARN DAGScheduler: Broadcasting large task binary
with size 1172.7 KiB
25/12/06 06:27:06 WARN DAGScheduler: Broadcasting large task binary
with size 1173.1 KiB
25/12/06 06:27:06 WARN DAGScheduler: Broadcasting large task binary
with size 1173.1 KiB
```

```
-> Result: AUC=0.8359 | F1=0.9050 | Time=4.3s
```

```
[36/48] Test config: {'ratio_target': 7, 'ratio_multiplier': 0.05,
'identity_weight': 0.5, 'model': GBTCClassifier_3b8b769d6280,
'combinaison': 'w2v + tfidf'}
Undersampling : On garde 42.34% des données négatives.
Ratio de sain/toxique 7.08
toxic_weights : 0.35 + 1, identity_weight :0.5
Prétraitement terminé, df_train_final_balanced_debiased = 4978 |
df_test_final_all_w2v = 1199
```

```
25/12/06 06:27:08 WARN DAGScheduler: Broadcasting large task binary
with size 1225.4 KiB
25/12/06 06:27:08 WARN DAGScheduler: Broadcasting large task binary
with size 1225.4 KiB
25/12/06 06:27:08 WARN DAGScheduler: Broadcasting large task binary
with size 1287.1 KiB
25/12/06 06:27:10 WARN DAGScheduler: Broadcasting large task binary
with size 1438.7 KiB
25/12/06 06:27:10 WARN DAGScheduler: Broadcasting large task binary
with size 1439.5 KiB
25/12/06 06:27:10 WARN DAGScheduler: Broadcasting large task binary
with size 1440.1 KiB
```

25/12/06 06:27:11 WARN DAGScheduler: Broadcasting large task binary with size 1440.9 KiB
25/12/06 06:27:11 WARN DAGScheduler: Broadcasting large task binary with size 1442.3 KiB
25/12/06 06:27:11 WARN DAGScheduler: Broadcasting large task binary with size 1448.0 KiB
25/12/06 06:27:11 WARN DAGScheduler: Broadcasting large task binary with size 1448.5 KiB
25/12/06 06:27:11 WARN DAGScheduler: Broadcasting large task binary with size 1449.1 KiB
25/12/06 06:27:11 WARN DAGScheduler: Broadcasting large task binary with size 1450.3 KiB
25/12/06 06:27:11 WARN DAGScheduler: Broadcasting large task binary with size 1452.3 KiB
25/12/06 06:27:11 WARN DAGScheduler: Broadcasting large task binary with size 1454.6 KiB
25/12/06 06:27:12 WARN DAGScheduler: Broadcasting large task binary with size 1455.1 KiB
25/12/06 06:27:12 WARN DAGScheduler: Broadcasting large task binary with size 1455.7 KiB
25/12/06 06:27:12 WARN DAGScheduler: Broadcasting large task binary with size 1456.9 KiB
25/12/06 06:27:12 WARN DAGScheduler: Broadcasting large task binary with size 1459.2 KiB
25/12/06 06:27:12 WARN DAGScheduler: Broadcasting large task binary with size 1461.8 KiB
25/12/06 06:27:12 WARN DAGScheduler: Broadcasting large task binary with size 1462.3 KiB
25/12/06 06:27:12 WARN DAGScheduler: Broadcasting large task binary with size 1462.8 KiB
25/12/06 06:27:12 WARN DAGScheduler: Broadcasting large task binary with size 1464.0 KiB
25/12/06 06:27:13 WARN DAGScheduler: Broadcasting large task binary with size 1466.3 KiB
25/12/06 06:27:13 WARN DAGScheduler: Broadcasting large task binary with size 1468.6 KiB
25/12/06 06:27:13 WARN DAGScheduler: Broadcasting large task binary with size 1469.1 KiB
25/12/06 06:27:13 WARN DAGScheduler: Broadcasting large task binary with size 1469.7 KiB
25/12/06 06:27:13 WARN DAGScheduler: Broadcasting large task binary with size 1470.9 KiB
25/12/06 06:27:13 WARN DAGScheduler: Broadcasting large task binary with size 1472.9 KiB
25/12/06 06:27:13 WARN DAGScheduler: Broadcasting large task binary with size 1475.2 KiB
25/12/06 06:27:13 WARN DAGScheduler: Broadcasting large task binary with size 1475.7 KiB
25/12/06 06:27:13 WARN DAGScheduler: Broadcasting large task binary

with size 1476.2 KiB
25/12/06 06:27:13 WARN DAGScheduler: Broadcasting large task binary
with size 1477.4 KiB
25/12/06 06:27:14 WARN DAGScheduler: Broadcasting large task binary
with size 1479.7 KiB
25/12/06 06:27:14 WARN DAGScheduler: Broadcasting large task binary
with size 1482.2 KiB
25/12/06 06:27:14 WARN DAGScheduler: Broadcasting large task binary
with size 1482.7 KiB
25/12/06 06:27:14 WARN DAGScheduler: Broadcasting large task binary
with size 1483.3 KiB
25/12/06 06:27:14 WARN DAGScheduler: Broadcasting large task binary
with size 1484.4 KiB
25/12/06 06:27:14 WARN DAGScheduler: Broadcasting large task binary
with size 1486.4 KiB
25/12/06 06:27:14 WARN DAGScheduler: Broadcasting large task binary
with size 1488.6 KiB
25/12/06 06:27:14 WARN DAGScheduler: Broadcasting large task binary
with size 1489.1 KiB
25/12/06 06:27:15 WARN DAGScheduler: Broadcasting large task binary
with size 1489.6 KiB
25/12/06 06:27:15 WARN DAGScheduler: Broadcasting large task binary
with size 1490.8 KiB
25/12/06 06:27:15 WARN DAGScheduler: Broadcasting large task binary
with size 1492.8 KiB
25/12/06 06:27:15 WARN DAGScheduler: Broadcasting large task binary
with size 1495.0 KiB
25/12/06 06:27:15 WARN DAGScheduler: Broadcasting large task binary
with size 1495.5 KiB
25/12/06 06:27:15 WARN DAGScheduler: Broadcasting large task binary
with size 1496.0 KiB
25/12/06 06:27:15 WARN DAGScheduler: Broadcasting large task binary
with size 1497.2 KiB
25/12/06 06:27:15 WARN DAGScheduler: Broadcasting large task binary
with size 1499.5 KiB
25/12/06 06:27:15 WARN DAGScheduler: Broadcasting large task binary
with size 1501.7 KiB
25/12/06 06:27:16 WARN DAGScheduler: Broadcasting large task binary
with size 1502.2 KiB
25/12/06 06:27:16 WARN DAGScheduler: Broadcasting large task binary
with size 1502.7 KiB
25/12/06 06:27:16 WARN DAGScheduler: Broadcasting large task binary
with size 1503.9 KiB
25/12/06 06:27:16 WARN DAGScheduler: Broadcasting large task binary
with size 1505.9 KiB
25/12/06 06:27:16 WARN DAGScheduler: Broadcasting large task binary
with size 1508.4 KiB
25/12/06 06:27:16 WARN DAGScheduler: Broadcasting large task binary
with size 1508.9 KiB

25/12/06 06:27:16 WARN DAGScheduler: Broadcasting large task binary with size 1509.4 KiB
25/12/06 06:27:16 WARN DAGScheduler: Broadcasting large task binary with size 1510.6 KiB
25/12/06 06:27:16 WARN DAGScheduler: Broadcasting large task binary with size 1512.9 KiB
25/12/06 06:27:16 WARN DAGScheduler: Broadcasting large task binary with size 1515.1 KiB
25/12/06 06:27:16 WARN DAGScheduler: Broadcasting large task binary with size 1515.6 KiB
25/12/06 06:27:17 WARN DAGScheduler: Broadcasting large task binary with size 1516.2 KiB
25/12/06 06:27:17 WARN DAGScheduler: Broadcasting large task binary with size 1517.4 KiB
25/12/06 06:27:17 WARN DAGScheduler: Broadcasting large task binary with size 1519.6 KiB
25/12/06 06:27:17 WARN DAGScheduler: Broadcasting large task binary with size 1521.8 KiB
25/12/06 06:27:17 WARN DAGScheduler: Broadcasting large task binary with size 1522.3 KiB
25/12/06 06:27:17 WARN DAGScheduler: Broadcasting large task binary with size 1522.9 KiB
25/12/06 06:27:17 WARN DAGScheduler: Broadcasting large task binary with size 1524.1 KiB
25/12/06 06:27:17 WARN DAGScheduler: Broadcasting large task binary with size 1526.3 KiB
25/12/06 06:27:17 WARN DAGScheduler: Broadcasting large task binary with size 1528.5 KiB
25/12/06 06:27:18 WARN DAGScheduler: Broadcasting large task binary with size 1529.0 KiB
25/12/06 06:27:18 WARN DAGScheduler: Broadcasting large task binary with size 1529.6 KiB
25/12/06 06:27:18 WARN DAGScheduler: Broadcasting large task binary with size 1530.8 KiB
25/12/06 06:27:18 WARN DAGScheduler: Broadcasting large task binary with size 1533.0 KiB
25/12/06 06:27:18 WARN DAGScheduler: Broadcasting large task binary with size 1534.9 KiB
25/12/06 06:27:18 WARN DAGScheduler: Broadcasting large task binary with size 1535.4 KiB
25/12/06 06:27:18 WARN DAGScheduler: Broadcasting large task binary with size 1536.0 KiB
25/12/06 06:27:18 WARN DAGScheduler: Broadcasting large task binary with size 1537.2 KiB
25/12/06 06:27:18 WARN DAGScheduler: Broadcasting large task binary with size 1539.4 KiB
25/12/06 06:27:19 WARN DAGScheduler: Broadcasting large task binary with size 1541.8 KiB
25/12/06 06:27:19 WARN DAGScheduler: Broadcasting large task binary

with size 1542.3 KiB
25/12/06 06:27:19 WARN DAGScheduler: Broadcasting large task binary
with size 1542.8 KiB
25/12/06 06:27:19 WARN DAGScheduler: Broadcasting large task binary
with size 1544.0 KiB
25/12/06 06:27:19 WARN DAGScheduler: Broadcasting large task binary
with size 1546.3 KiB
25/12/06 06:27:19 WARN DAGScheduler: Broadcasting large task binary
with size 1548.6 KiB
25/12/06 06:27:19 WARN DAGScheduler: Broadcasting large task binary
with size 1549.1 KiB
25/12/06 06:27:19 WARN DAGScheduler: Broadcasting large task binary
with size 1549.7 KiB
25/12/06 06:27:19 WARN DAGScheduler: Broadcasting large task binary
with size 1550.9 KiB
25/12/06 06:27:19 WARN DAGScheduler: Broadcasting large task binary
with size 1553.1 KiB
25/12/06 06:27:20 WARN DAGScheduler: Broadcasting large task binary
with size 1555.7 KiB
25/12/06 06:27:20 WARN DAGScheduler: Broadcasting large task binary
with size 1556.2 KiB
25/12/06 06:27:20 WARN DAGScheduler: Broadcasting large task binary
with size 1556.7 KiB
25/12/06 06:27:20 WARN DAGScheduler: Broadcasting large task binary
with size 1557.9 KiB
25/12/06 06:27:20 WARN DAGScheduler: Broadcasting large task binary
with size 1560.2 KiB
25/12/06 06:27:20 WARN DAGScheduler: Broadcasting large task binary
with size 1562.9 KiB
25/12/06 06:27:20 WARN DAGScheduler: Broadcasting large task binary
with size 1563.4 KiB
25/12/06 06:27:20 WARN DAGScheduler: Broadcasting large task binary
with size 1563.9 KiB
25/12/06 06:27:20 WARN DAGScheduler: Broadcasting large task binary
with size 1565.1 KiB
25/12/06 06:27:20 WARN DAGScheduler: Broadcasting large task binary
with size 1567.4 KiB
25/12/06 06:27:21 WARN DAGScheduler: Broadcasting large task binary
with size 1569.6 KiB
25/12/06 06:27:21 WARN DAGScheduler: Broadcasting large task binary
with size 1570.1 KiB
25/12/06 06:27:21 WARN DAGScheduler: Broadcasting large task binary
with size 1570.6 KiB
25/12/06 06:27:21 WARN DAGScheduler: Broadcasting large task binary
with size 1571.6 KiB
25/12/06 06:27:21 WARN DAGScheduler: Broadcasting large task binary
with size 1573.2 KiB
25/12/06 06:27:21 WARN DAGScheduler: Broadcasting large task binary
with size 1255.0 KiB
25/12/06 06:27:21 WARN DAGScheduler: Broadcasting large task binary

with size 1267.3 KiB
25/12/06 06:27:22 WARN DAGScheduler: Broadcasting large task binary
with size 1267.3 KiB

-> Result: AUC=0.8292 | F1=0.9322 | Time=15.1s

[37/48] Test config: {'ratio_target': 7, 'ratio_multiplier': 0.05,
'identity_weight': 1.0, 'model': LogisticRegression_afae24335158,
'combinaison': 'w2v + tfidf'}

Undersampling : On garde 42.34% des données négatives.

Ratio de sain/toxique 7.08

toxic_weights : 0.35 + 1, identity_weight :1.0

Prétraitement terminé, df_train_final_balanced_debiased = 4978 |
df_test_final_all_w2v = 1199

25/12/06 06:27:23 WARN DAGScheduler: Broadcasting large task binary
with size 1229.2 KiB

25/12/06 06:27:23 WARN DAGScheduler: Broadcasting large task binary
with size 1229.8 KiB

25/12/06 06:27:23 WARN DAGScheduler: Broadcasting large task binary
with size 1229.8 KiB

25/12/06 06:27:23 WARN DAGScheduler: Broadcasting large task binary
with size 1229.8 KiB

25/12/06 06:27:23 WARN DAGScheduler: Broadcasting large task binary
with size 1229.8 KiB

25/12/06 06:27:23 WARN DAGScheduler: Broadcasting large task binary
with size 1229.8 KiB

25/12/06 06:27:23 WARN DAGScheduler: Broadcasting large task binary
with size 1229.8 KiB

25/12/06 06:27:24 WARN DAGScheduler: Broadcasting large task binary
with size 1229.8 KiB

25/12/06 06:27:24 WARN DAGScheduler: Broadcasting large task binary
with size 1229.8 KiB

25/12/06 06:27:24 WARN DAGScheduler: Broadcasting large task binary
with size 1229.8 KiB

25/12/06 06:27:24 WARN DAGScheduler: Broadcasting large task binary
with size 1229.8 KiB

25/12/06 06:27:24 WARN DAGScheduler: Broadcasting large task binary
with size 1229.8 KiB

25/12/06 06:27:24 WARN DAGScheduler: Broadcasting large task binary
with size 1176.7 KiB

25/12/06 06:27:24 WARN DAGScheduler: Broadcasting large task binary
with size 1188.8 KiB

25/12/06 06:27:24 WARN DAGScheduler: Broadcasting large task binary
with size 1188.8 KiB

-> Result: AUC=0.8184 | F1=0.8958 | Time=2.4s

[38/48] Test config: {'ratio_target': 7, 'ratio_multiplier': 0.05,
'identity_weight': 1.0, 'model': RandomForestClassifier_ad394a6e557e,

```
'combinaison': 'w2v + tfidf'}
Undersampling : On garde 42.34% des données négatives.
Ratio de sain/toxique 7.08
toxic_weights : 0.35 + 1, identity_weight :1.0
Prétraitement terminé, df_train_final_balanced_debiased = 4978 |
df_test_final_all_w2v = 1199

25/12/06 06:27:25 WARN DAGScheduler: Broadcasting large task binary
with size 1225.8 KiB
25/12/06 06:27:26 WARN DAGScheduler: Broadcasting large task binary
with size 1225.7 KiB
25/12/06 06:27:26 WARN DAGScheduler: Broadcasting large task binary
with size 1287.5 KiB
25/12/06 06:27:27 WARN DAGScheduler: Broadcasting large task binary
with size 1445.9 KiB
25/12/06 06:27:27 WARN DAGScheduler: Broadcasting large task binary
with size 1458.0 KiB
25/12/06 06:27:27 WARN DAGScheduler: Broadcasting large task binary
with size 1476.4 KiB
25/12/06 06:27:27 WARN DAGScheduler: Broadcasting large task binary
with size 1502.5 KiB
25/12/06 06:27:28 WARN DAGScheduler: Broadcasting large task binary
with size 1536.6 KiB
25/12/06 06:27:28 WARN DAGScheduler: Broadcasting large task binary
with size 1221.5 KiB
25/12/06 06:27:28 WARN DAGScheduler: Broadcasting large task binary
with size 1233.6 KiB
25/12/06 06:27:28 WARN DAGScheduler: Broadcasting large task binary
with size 1233.6 KiB

-> Result: AUC=0.8080 | F1=0.9072 | Time=4.2s

[39/48] Test config: {'ratio_target': 7, 'ratio_multiplier': 0.05,
'identity_weight': 1.0, 'model': LinearSVC_8f0c0322b70e,
'combinaison': 'w2v + tfidf'}
Undersampling : On garde 42.34% des données négatives.
Ratio de sain/toxique 7.08
toxic_weights : 0.35 + 1, identity_weight :1.0
Prétraitement terminé, df_train_final_balanced_debiased = 4978 |
df_test_final_all_w2v = 1199

25/12/06 06:27:30 WARN DAGScheduler: Broadcasting large task binary
with size 1226.7 KiB
25/12/06 06:27:30 WARN DAGScheduler: Broadcasting large task binary
with size 1227.3 KiB
25/12/06 06:27:30 WARN DAGScheduler: Broadcasting large task binary
with size 1227.3 KiB
25/12/06 06:27:30 WARN DAGScheduler: Broadcasting large task binary
with size 1227.3 KiB
25/12/06 06:27:30 WARN DAGScheduler: Broadcasting large task binary
```

[illegible]

[illegible]

[illegible]

[illegible]

```
with size 1227.3 KiB
25/12/06 06:27:32 WARN DAGScheduler: Broadcasting large task binary
with size 1227.3 KiB
25/12/06 06:27:32 WARN DAGScheduler: Broadcasting large task binary
with size 1172.7 KiB
25/12/06 06:27:33 WARN DAGScheduler: Broadcasting large task binary
with size 1173.1 KiB
25/12/06 06:27:33 WARN DAGScheduler: Broadcasting large task binary
with size 1173.1 KiB
```

```
-> Result: AUC=0.8391 | F1=0.9061 | Time=4.3s
```

```
[40/48] Test config: {'ratio_target': 7, 'ratio_multiplier': 0.05,
'identity_weight': 1.0, 'model': GBTCClassifier_3b8b769d6280,
'combinaison': 'w2v + tfidf'}
```

```
Undersampling : On garde 42.34% des données négatives.
```

```
Ratio de sain/toxique 7.08
```

```
toxic_weights : 0.35 + 1, identity_weight :1.0
```

```
Prétraitement terminé, df_train_final_balanced_debiased = 4978 |
df_test_final_all_w2v = 1199
```

```
25/12/06 06:27:34 WARN DAGScheduler: Broadcasting large task binary
with size 1225.4 KiB
25/12/06 06:27:34 WARN DAGScheduler: Broadcasting large task binary
with size 1225.4 KiB
25/12/06 06:27:34 WARN DAGScheduler: Broadcasting large task binary
with size 1287.2 KiB
25/12/06 06:27:36 WARN DAGScheduler: Broadcasting large task binary
with size 1438.8 KiB
25/12/06 06:27:37 WARN DAGScheduler: Broadcasting large task binary
with size 1439.5 KiB
25/12/06 06:27:37 WARN DAGScheduler: Broadcasting large task binary
with size 1440.1 KiB
25/12/06 06:27:37 WARN DAGScheduler: Broadcasting large task binary
with size 1441.3 KiB
25/12/06 06:27:37 WARN DAGScheduler: Broadcasting large task binary
with size 1443.3 KiB
25/12/06 06:27:37 WARN DAGScheduler: Broadcasting large task binary
with size 1449.5 KiB
25/12/06 06:27:37 WARN DAGScheduler: Broadcasting large task binary
with size 1450.0 KiB
25/12/06 06:27:37 WARN DAGScheduler: Broadcasting large task binary
with size 1450.5 KiB
25/12/06 06:27:37 WARN DAGScheduler: Broadcasting large task binary
with size 1451.4 KiB
25/12/06 06:27:37 WARN DAGScheduler: Broadcasting large task binary
with size 1453.2 KiB
25/12/06 06:27:38 WARN DAGScheduler: Broadcasting large task binary
with size 1455.9 KiB
25/12/06 06:27:38 WARN DAGScheduler: Broadcasting large task binary
```

with size 1456.4 KiB
25/12/06 06:27:38 WARN DAGScheduler: Broadcasting large task binary
with size 1457.0 KiB
25/12/06 06:27:38 WARN DAGScheduler: Broadcasting large task binary
with size 1457.9 KiB
25/12/06 06:27:38 WARN DAGScheduler: Broadcasting large task binary
with size 1459.6 KiB
25/12/06 06:27:38 WARN DAGScheduler: Broadcasting large task binary
with size 1462.2 KiB
25/12/06 06:27:38 WARN DAGScheduler: Broadcasting large task binary
with size 1462.6 KiB
25/12/06 06:27:38 WARN DAGScheduler: Broadcasting large task binary
with size 1463.2 KiB
25/12/06 06:27:38 WARN DAGScheduler: Broadcasting large task binary
with size 1464.1 KiB
25/12/06 06:27:38 WARN DAGScheduler: Broadcasting large task binary
with size 1465.8 KiB
25/12/06 06:27:39 WARN DAGScheduler: Broadcasting large task binary
with size 1468.4 KiB
25/12/06 06:27:39 WARN DAGScheduler: Broadcasting large task binary
with size 1468.9 KiB
25/12/06 06:27:39 WARN DAGScheduler: Broadcasting large task binary
with size 1469.4 KiB
25/12/06 06:27:39 WARN DAGScheduler: Broadcasting large task binary
with size 1470.6 KiB
25/12/06 06:27:39 WARN DAGScheduler: Broadcasting large task binary
with size 1472.8 KiB
25/12/06 06:27:39 WARN DAGScheduler: Broadcasting large task binary
with size 1474.8 KiB
25/12/06 06:27:39 WARN DAGScheduler: Broadcasting large task binary
with size 1475.2 KiB
25/12/06 06:27:39 WARN DAGScheduler: Broadcasting large task binary
with size 1475.8 KiB
25/12/06 06:27:39 WARN DAGScheduler: Broadcasting large task binary
with size 1477.0 KiB
25/12/06 06:27:39 WARN DAGScheduler: Broadcasting large task binary
with size 1478.7 KiB
25/12/06 06:27:39 WARN DAGScheduler: Broadcasting large task binary
with size 1480.7 KiB
25/12/06 06:27:40 WARN DAGScheduler: Broadcasting large task binary
with size 1481.1 KiB
25/12/06 06:27:40 WARN DAGScheduler: Broadcasting large task binary
with size 1481.7 KiB
25/12/06 06:27:40 WARN DAGScheduler: Broadcasting large task binary
with size 1482.9 KiB
25/12/06 06:27:40 WARN DAGScheduler: Broadcasting large task binary
with size 1485.0 KiB
25/12/06 06:27:40 WARN DAGScheduler: Broadcasting large task binary
with size 1486.7 KiB

25/12/06 06:27:40 WARN DAGScheduler: Broadcasting large task binary with size 1487.2 KiB
25/12/06 06:27:40 WARN DAGScheduler: Broadcasting large task binary with size 1487.8 KiB
25/12/06 06:27:40 WARN DAGScheduler: Broadcasting large task binary with size 1489.0 KiB
25/12/06 06:27:40 WARN DAGScheduler: Broadcasting large task binary with size 1491.3 KiB
25/12/06 06:27:40 WARN DAGScheduler: Broadcasting large task binary with size 1493.8 KiB
25/12/06 06:27:41 WARN DAGScheduler: Broadcasting large task binary with size 1494.2 KiB
25/12/06 06:27:41 WARN DAGScheduler: Broadcasting large task binary with size 1494.8 KiB
25/12/06 06:27:41 WARN DAGScheduler: Broadcasting large task binary with size 1496.0 KiB
25/12/06 06:27:41 WARN DAGScheduler: Broadcasting large task binary with size 1498.0 KiB
25/12/06 06:27:41 WARN DAGScheduler: Broadcasting large task binary with size 1500.0 KiB
25/12/06 06:27:41 WARN DAGScheduler: Broadcasting large task binary with size 1500.4 KiB
25/12/06 06:27:41 WARN DAGScheduler: Broadcasting large task binary with size 1501.0 KiB
25/12/06 06:27:41 WARN DAGScheduler: Broadcasting large task binary with size 1502.2 KiB
25/12/06 06:27:41 WARN DAGScheduler: Broadcasting large task binary with size 1503.6 KiB
25/12/06 06:27:41 WARN DAGScheduler: Broadcasting large task binary with size 1505.2 KiB
25/12/06 06:27:41 WARN DAGScheduler: Broadcasting large task binary with size 1505.7 KiB
25/12/06 06:27:42 WARN DAGScheduler: Broadcasting large task binary with size 1506.3 KiB
25/12/06 06:27:42 WARN DAGScheduler: Broadcasting large task binary with size 1507.5 KiB
25/12/06 06:27:42 WARN DAGScheduler: Broadcasting large task binary with size 1509.5 KiB
25/12/06 06:27:42 WARN DAGScheduler: Broadcasting large task binary with size 1511.3 KiB
25/12/06 06:27:42 WARN DAGScheduler: Broadcasting large task binary with size 1511.8 KiB
25/12/06 06:27:42 WARN DAGScheduler: Broadcasting large task binary with size 1512.3 KiB
25/12/06 06:27:42 WARN DAGScheduler: Broadcasting large task binary with size 1513.5 KiB
25/12/06 06:27:42 WARN DAGScheduler: Broadcasting large task binary with size 1515.7 KiB
25/12/06 06:27:42 WARN DAGScheduler: Broadcasting large task binary

with size 1517.3 KiB
25/12/06 06:27:42 WARN DAGScheduler: Broadcasting large task binary
with size 1517.8 KiB
25/12/06 06:27:43 WARN DAGScheduler: Broadcasting large task binary
with size 1518.4 KiB
25/12/06 06:27:43 WARN DAGScheduler: Broadcasting large task binary
with size 1519.3 KiB
25/12/06 06:27:43 WARN DAGScheduler: Broadcasting large task binary
with size 1521.0 KiB
25/12/06 06:27:43 WARN DAGScheduler: Broadcasting large task binary
with size 1523.4 KiB
25/12/06 06:27:43 WARN DAGScheduler: Broadcasting large task binary
with size 1523.9 KiB
25/12/06 06:27:43 WARN DAGScheduler: Broadcasting large task binary
with size 1524.4 KiB
25/12/06 06:27:43 WARN DAGScheduler: Broadcasting large task binary
with size 1525.6 KiB
25/12/06 06:27:43 WARN DAGScheduler: Broadcasting large task binary
with size 1527.5 KiB
25/12/06 06:27:43 WARN DAGScheduler: Broadcasting large task binary
with size 1529.4 KiB
25/12/06 06:27:43 WARN DAGScheduler: Broadcasting large task binary
with size 1529.9 KiB
25/12/06 06:27:44 WARN DAGScheduler: Broadcasting large task binary
with size 1530.5 KiB
25/12/06 06:27:44 WARN DAGScheduler: Broadcasting large task binary
with size 1531.7 KiB
25/12/06 06:27:44 WARN DAGScheduler: Broadcasting large task binary
with size 1533.4 KiB
25/12/06 06:27:44 WARN DAGScheduler: Broadcasting large task binary
with size 1535.3 KiB
25/12/06 06:27:44 WARN DAGScheduler: Broadcasting large task binary
with size 1535.8 KiB
25/12/06 06:27:44 WARN DAGScheduler: Broadcasting large task binary
with size 1536.4 KiB
25/12/06 06:27:44 WARN DAGScheduler: Broadcasting large task binary
with size 1537.6 KiB
25/12/06 06:27:44 WARN DAGScheduler: Broadcasting large task binary
with size 1539.8 KiB
25/12/06 06:27:44 WARN DAGScheduler: Broadcasting large task binary
with size 1541.9 KiB
25/12/06 06:27:44 WARN DAGScheduler: Broadcasting large task binary
with size 1542.4 KiB
25/12/06 06:27:45 WARN DAGScheduler: Broadcasting large task binary
with size 1542.9 KiB
25/12/06 06:27:45 WARN DAGScheduler: Broadcasting large task binary
with size 1544.1 KiB
25/12/06 06:27:45 WARN DAGScheduler: Broadcasting large task binary
with size 1546.3 KiB

25/12/06 06:27:45 WARN DAGScheduler: Broadcasting large task binary with size 1548.1 KiB
25/12/06 06:27:45 WARN DAGScheduler: Broadcasting large task binary with size 1548.6 KiB
25/12/06 06:27:45 WARN DAGScheduler: Broadcasting large task binary with size 1549.2 KiB
25/12/06 06:27:45 WARN DAGScheduler: Broadcasting large task binary with size 1550.4 KiB
25/12/06 06:27:45 WARN DAGScheduler: Broadcasting large task binary with size 1552.6 KiB
25/12/06 06:27:45 WARN DAGScheduler: Broadcasting large task binary with size 1554.7 KiB
25/12/06 06:27:45 WARN DAGScheduler: Broadcasting large task binary with size 1555.1 KiB
25/12/06 06:27:46 WARN DAGScheduler: Broadcasting large task binary with size 1555.7 KiB
25/12/06 06:27:46 WARN DAGScheduler: Broadcasting large task binary with size 1556.6 KiB
25/12/06 06:27:46 WARN DAGScheduler: Broadcasting large task binary with size 1558.3 KiB
25/12/06 06:27:46 WARN DAGScheduler: Broadcasting large task binary with size 1560.7 KiB
25/12/06 06:27:46 WARN DAGScheduler: Broadcasting large task binary with size 1561.2 KiB
25/12/06 06:27:46 WARN DAGScheduler: Broadcasting large task binary with size 1561.8 KiB
25/12/06 06:27:46 WARN DAGScheduler: Broadcasting large task binary with size 1562.9 KiB
25/12/06 06:27:46 WARN DAGScheduler: Broadcasting large task binary with size 1564.4 KiB
25/12/06 06:27:47 WARN DAGScheduler: Broadcasting large task binary with size 1246.3 KiB
25/12/06 06:27:47 WARN DAGScheduler: Broadcasting large task binary with size 1258.6 KiB
25/12/06 06:27:47 WARN DAGScheduler: Broadcasting large task binary with size 1258.6 KiB

-> Result: AUC=0.8322 | F1=0.9287 | Time=14.1s

[41/48] Test config: {'ratio_target': 7, 'ratio_multiplier': 0.1, 'identity_weight': 0.5, 'model': LogisticRegression_afae24335158, 'combinaison': 'w2v + tfidf'}

Undersampling : On garde 42.34% des données négatives.

Ratio de sain/toxique 7.08

toxic_weights : 0.71 + 1, identity_weight :0.5

Prétraitement terminé, df_train_final_balanced_debiased = 4978 |
df_test_final_all_w2v = 1199

25/12/06 06:27:48 WARN DAGScheduler: Broadcasting large task binary with size 1229.2 KiB

25/12/06 06:27:48 WARN DAGScheduler: Broadcasting large task binary with size 1229.8 KiB
25/12/06 06:27:49 WARN DAGScheduler: Broadcasting large task binary with size 1229.8 KiB
25/12/06 06:27:49 WARN DAGScheduler: Broadcasting large task binary with size 1229.8 KiB
25/12/06 06:27:49 WARN DAGScheduler: Broadcasting large task binary with size 1229.8 KiB
25/12/06 06:27:49 WARN DAGScheduler: Broadcasting large task binary with size 1229.8 KiB
25/12/06 06:27:49 WARN DAGScheduler: Broadcasting large task binary with size 1229.8 KiB
25/12/06 06:27:49 WARN DAGScheduler: Broadcasting large task binary with size 1229.8 KiB
25/12/06 06:27:49 WARN DAGScheduler: Broadcasting large task binary with size 1229.8 KiB
25/12/06 06:27:49 WARN DAGScheduler: Broadcasting large task binary with size 1229.8 KiB
25/12/06 06:27:49 WARN DAGScheduler: Broadcasting large task binary with size 1229.8 KiB
25/12/06 06:27:49 WARN DAGScheduler: Broadcasting large task binary with size 1229.8 KiB
25/12/06 06:27:49 WARN DAGScheduler: Broadcasting large task binary with size 1229.8 KiB
25/12/06 06:27:49 WARN DAGScheduler: Broadcasting large task binary with size 1229.8 KiB
25/12/06 06:27:49 WARN DAGScheduler: Broadcasting large task binary with size 1176.7 KiB
25/12/06 06:27:49 WARN DAGScheduler: Broadcasting large task binary with size 1188.8 KiB
25/12/06 06:27:50 WARN DAGScheduler: Broadcasting large task binary with size 1188.8 KiB

-> Result: AUC=0.8207 | F1=0.8891 | Time=2.8s

[42/48] Test config: {'ratio_target': 7, 'ratio_multiplier': 0.1, 'identity_weight': 0.5, 'model': RandomForestClassifier_ad394a6e557e, 'combinaison': 'w2v + tfidf'}

Undersampling : On garde 42.34% des données négatives.

Ratio de sain/toxique 7.08

toxic_weights : 0.71 + 1, identity_weight : 0.5

Prétraitement terminé, df_train_final_balanced_debiased = 4978 |

df_test_final_all_w2v = 1199

25/12/06 06:27:51 WARN DAGScheduler: Broadcasting large task binary with size 1225.8 KiB
25/12/06 06:27:51 WARN DAGScheduler: Broadcasting large task binary with size 1225.7 KiB
25/12/06 06:27:52 WARN DAGScheduler: Broadcasting large task binary with size 1287.5 KiB
25/12/06 06:27:54 WARN DAGScheduler: Broadcasting large task binary with size 1445.9 KiB

[illegible]

[illegible]

[illegible]


```
Ratio de sain/toxique 7.08
toxic_weights : 0.71 + 1, identity_weight :0.5
Prétraitement terminé, df_train_final_balanced_debiased = 4978 |
df_test_final_all_w2v = 1199

25/12/06 06:28:07 WARN DAGScheduler: Broadcasting large task binary
with size 1225.4 KiB
25/12/06 06:28:07 WARN DAGScheduler: Broadcasting large task binary
with size 1225.4 KiB
25/12/06 06:28:07 WARN DAGScheduler: Broadcasting large task binary
with size 1287.1 KiB
25/12/06 06:28:09 WARN DAGScheduler: Broadcasting large task binary
with size 1438.7 KiB
25/12/06 06:28:10 WARN DAGScheduler: Broadcasting large task binary
with size 1439.5 KiB
25/12/06 06:28:10 WARN DAGScheduler: Broadcasting large task binary
with size 1440.1 KiB
25/12/06 06:28:10 WARN DAGScheduler: Broadcasting large task binary
with size 1440.9 KiB
25/12/06 06:28:10 WARN DAGScheduler: Broadcasting large task binary
with size 1442.3 KiB
25/12/06 06:28:11 WARN DAGScheduler: Broadcasting large task binary
with size 1448.0 KiB
25/12/06 06:28:11 WARN DAGScheduler: Broadcasting large task binary
with size 1448.5 KiB
25/12/06 06:28:11 WARN DAGScheduler: Broadcasting large task binary
with size 1449.1 KiB
25/12/06 06:28:11 WARN DAGScheduler: Broadcasting large task binary
with size 1450.3 KiB
25/12/06 06:28:11 WARN DAGScheduler: Broadcasting large task binary
with size 1452.5 KiB
25/12/06 06:28:12 WARN DAGScheduler: Broadcasting large task binary
with size 1455.1 KiB
25/12/06 06:28:12 WARN DAGScheduler: Broadcasting large task binary
with size 1455.6 KiB
25/12/06 06:28:12 WARN DAGScheduler: Broadcasting large task binary
with size 1456.2 KiB
25/12/06 06:28:12 WARN DAGScheduler: Broadcasting large task binary
with size 1457.3 KiB
25/12/06 06:28:12 WARN DAGScheduler: Broadcasting large task binary
with size 1459.3 KiB
25/12/06 06:28:13 WARN DAGScheduler: Broadcasting large task binary
with size 1461.7 KiB
25/12/06 06:28:13 WARN DAGScheduler: Broadcasting large task binary
with size 1462.1 KiB
25/12/06 06:28:13 WARN DAGScheduler: Broadcasting large task binary
with size 1462.7 KiB
25/12/06 06:28:13 WARN DAGScheduler: Broadcasting large task binary
with size 1463.9 KiB
25/12/06 06:28:13 WARN DAGScheduler: Broadcasting large task binary
```

with size 1465.6 KiB
25/12/06 06:28:14 WARN DAGScheduler: Broadcasting large task binary
with size 1468.0 KiB
25/12/06 06:28:14 WARN DAGScheduler: Broadcasting large task binary
with size 1468.5 KiB
25/12/06 06:28:14 WARN DAGScheduler: Broadcasting large task binary
with size 1469.1 KiB
25/12/06 06:28:14 WARN DAGScheduler: Broadcasting large task binary
with size 1470.3 KiB
25/12/06 06:28:15 WARN DAGScheduler: Broadcasting large task binary
with size 1472.5 KiB
25/12/06 06:28:15 WARN DAGScheduler: Broadcasting large task binary
with size 1475.1 KiB
25/12/06 06:28:15 WARN DAGScheduler: Broadcasting large task binary
with size 1475.6 KiB
25/12/06 06:28:15 WARN DAGScheduler: Broadcasting large task binary
with size 1476.1 KiB
25/12/06 06:28:15 WARN DAGScheduler: Broadcasting large task binary
with size 1477.3 KiB
25/12/06 06:28:15 WARN DAGScheduler: Broadcasting large task binary
with size 1479.3 KiB
25/12/06 06:28:16 WARN DAGScheduler: Broadcasting large task binary
with size 1481.6 KiB
25/12/06 06:28:16 WARN DAGScheduler: Broadcasting large task binary
with size 1482.1 KiB
25/12/06 06:28:16 WARN DAGScheduler: Broadcasting large task binary
with size 1482.7 KiB
25/12/06 06:28:16 WARN DAGScheduler: Broadcasting large task binary
with size 1483.9 KiB
25/12/06 06:28:16 WARN DAGScheduler: Broadcasting large task binary
with size 1486.2 KiB
25/12/06 06:28:16 WARN DAGScheduler: Broadcasting large task binary
with size 1489.0 KiB
25/12/06 06:28:16 WARN DAGScheduler: Broadcasting large task binary
with size 1489.5 KiB
25/12/06 06:28:17 WARN DAGScheduler: Broadcasting large task binary
with size 1490.0 KiB
25/12/06 06:28:17 WARN DAGScheduler: Broadcasting large task binary
with size 1491.2 KiB
25/12/06 06:28:17 WARN DAGScheduler: Broadcasting large task binary
with size 1492.9 KiB
25/12/06 06:28:17 WARN DAGScheduler: Broadcasting large task binary
with size 1494.9 KiB
25/12/06 06:28:17 WARN DAGScheduler: Broadcasting large task binary
with size 1495.4 KiB
25/12/06 06:28:18 WARN DAGScheduler: Broadcasting large task binary
with size 1495.9 KiB
25/12/06 06:28:18 WARN DAGScheduler: Broadcasting large task binary
with size 1497.1 KiB

25/12/06 06:28:18 WARN DAGScheduler: Broadcasting large task binary with size 1499.4 KiB
25/12/06 06:28:18 WARN DAGScheduler: Broadcasting large task binary with size 1501.8 KiB
25/12/06 06:28:18 WARN DAGScheduler: Broadcasting large task binary with size 1502.2 KiB
25/12/06 06:28:18 WARN DAGScheduler: Broadcasting large task binary with size 1502.8 KiB
25/12/06 06:28:18 WARN DAGScheduler: Broadcasting large task binary with size 1504.0 KiB
25/12/06 06:28:19 WARN DAGScheduler: Broadcasting large task binary with size 1506.2 KiB
25/12/06 06:28:19 WARN DAGScheduler: Broadcasting large task binary with size 1508.3 KiB
25/12/06 06:28:19 WARN DAGScheduler: Broadcasting large task binary with size 1508.8 KiB
25/12/06 06:28:19 WARN DAGScheduler: Broadcasting large task binary with size 1509.3 KiB
25/12/06 06:28:19 WARN DAGScheduler: Broadcasting large task binary with size 1510.5 KiB
25/12/06 06:28:19 WARN DAGScheduler: Broadcasting large task binary with size 1512.3 KiB
25/12/06 06:28:20 WARN DAGScheduler: Broadcasting large task binary with size 1514.4 KiB
25/12/06 06:28:20 WARN DAGScheduler: Broadcasting large task binary with size 1514.8 KiB
25/12/06 06:28:20 WARN DAGScheduler: Broadcasting large task binary with size 1515.4 KiB
25/12/06 06:28:20 WARN DAGScheduler: Broadcasting large task binary with size 1516.6 KiB
25/12/06 06:28:20 WARN DAGScheduler: Broadcasting large task binary with size 1518.9 KiB
25/12/06 06:28:20 WARN DAGScheduler: Broadcasting large task binary with size 1521.4 KiB
25/12/06 06:28:20 WARN DAGScheduler: Broadcasting large task binary with size 1521.9 KiB
25/12/06 06:28:20 WARN DAGScheduler: Broadcasting large task binary with size 1522.4 KiB
25/12/06 06:28:21 WARN DAGScheduler: Broadcasting large task binary with size 1523.6 KiB
25/12/06 06:28:21 WARN DAGScheduler: Broadcasting large task binary with size 1525.9 KiB
25/12/06 06:28:21 WARN DAGScheduler: Broadcasting large task binary with size 1528.2 KiB
25/12/06 06:28:21 WARN DAGScheduler: Broadcasting large task binary with size 1528.7 KiB
25/12/06 06:28:21 WARN DAGScheduler: Broadcasting large task binary with size 1529.3 KiB
25/12/06 06:28:21 WARN DAGScheduler: Broadcasting large task binary

with size 1530.5 KiB
25/12/06 06:28:21 WARN DAGScheduler: Broadcasting large task binary
with size 1532.8 KiB
25/12/06 06:28:21 WARN DAGScheduler: Broadcasting large task binary
with size 1535.4 KiB
25/12/06 06:28:21 WARN DAGScheduler: Broadcasting large task binary
with size 1535.9 KiB
25/12/06 06:28:22 WARN DAGScheduler: Broadcasting large task binary
with size 1536.5 KiB
25/12/06 06:28:22 WARN DAGScheduler: Broadcasting large task binary
with size 1537.7 KiB
25/12/06 06:28:22 WARN DAGScheduler: Broadcasting large task binary
with size 1539.6 KiB
25/12/06 06:28:22 WARN DAGScheduler: Broadcasting large task binary
with size 1541.5 KiB
25/12/06 06:28:22 WARN DAGScheduler: Broadcasting large task binary
with size 1542.0 KiB
25/12/06 06:28:22 WARN DAGScheduler: Broadcasting large task binary
with size 1542.5 KiB
25/12/06 06:28:23 WARN DAGScheduler: Broadcasting large task binary
with size 1543.7 KiB
25/12/06 06:28:23 WARN DAGScheduler: Broadcasting large task binary
with size 1545.8 KiB
25/12/06 06:28:23 WARN DAGScheduler: Broadcasting large task binary
with size 1548.2 KiB
25/12/06 06:28:23 WARN DAGScheduler: Broadcasting large task binary
with size 1548.7 KiB
25/12/06 06:28:23 WARN DAGScheduler: Broadcasting large task binary
with size 1549.2 KiB
25/12/06 06:28:23 WARN DAGScheduler: Broadcasting large task binary
with size 1550.4 KiB
25/12/06 06:28:23 WARN DAGScheduler: Broadcasting large task binary
with size 1552.7 KiB
25/12/06 06:28:23 WARN DAGScheduler: Broadcasting large task binary
with size 1555.1 KiB
25/12/06 06:28:23 WARN DAGScheduler: Broadcasting large task binary
with size 1555.5 KiB
25/12/06 06:28:24 WARN DAGScheduler: Broadcasting large task binary
with size 1556.1 KiB
25/12/06 06:28:24 WARN DAGScheduler: Broadcasting large task binary
with size 1557.3 KiB
25/12/06 06:28:24 WARN DAGScheduler: Broadcasting large task binary
with size 1559.3 KiB
25/12/06 06:28:24 WARN DAGScheduler: Broadcasting large task binary
with size 1561.4 KiB
25/12/06 06:28:24 WARN DAGScheduler: Broadcasting large task binary
with size 1561.9 KiB
25/12/06 06:28:24 WARN DAGScheduler: Broadcasting large task binary
with size 1562.5 KiB

```
25/12/06 06:28:24 WARN DAGScheduler: Broadcasting large task binary
with size 1563.7 KiB
25/12/06 06:28:24 WARN DAGScheduler: Broadcasting large task binary
with size 1565.9 KiB
25/12/06 06:28:24 WARN DAGScheduler: Broadcasting large task binary
with size 1567.7 KiB
25/12/06 06:28:25 WARN DAGScheduler: Broadcasting large task binary
with size 1568.1 KiB
25/12/06 06:28:25 WARN DAGScheduler: Broadcasting large task binary
with size 1568.7 KiB
25/12/06 06:28:25 WARN DAGScheduler: Broadcasting large task binary
with size 1569.9 KiB
25/12/06 06:28:25 WARN DAGScheduler: Broadcasting large task binary
with size 1572.1 KiB
25/12/06 06:28:25 WARN DAGScheduler: Broadcasting large task binary
with size 1253.9 KiB
25/12/06 06:28:25 WARN DAGScheduler: Broadcasting large task binary
with size 1266.2 KiB
25/12/06 06:28:26 WARN DAGScheduler: Broadcasting large task binary
with size 1266.2 KiB
```

-> Result: AUC=0.8540 | F1=0.9308 | Time=20.0s

```
[45/48] Test config: {'ratio_target': 7, 'ratio_multiplier': 0.1,
'identity_weight': 1.0, 'model': LogisticRegression_afae24335158,
'combinaison': 'w2v + tfidf'}
Undersampling : On garde 42.34% des données négatives.
Ratio de sain/toxique 7.08
toxic_weights : 0.71 + 1, identity_weight :1.0
Prétraitement terminé, df_train_final_balanced_debiased = 4978 |
df test final all w2v = 1199
```

[illegible]

```
25/12/06 06:28:28 WARN DAGScheduler: Broadcasting large task binary
with size 1229.8 KiB
25/12/06 06:28:28 WARN DAGScheduler: Broadcasting large task binary
with size 1229.8 KiB
25/12/06 06:28:28 WARN DAGScheduler: Broadcasting large task binary
with size 1229.8 KiB
25/12/06 06:28:28 WARN DAGScheduler: Broadcasting large task binary
with size 1176.7 KiB
25/12/06 06:28:28 WARN DAGScheduler: Broadcasting large task binary
with size 1188.8 KiB
25/12/06 06:28:29 WARN DAGScheduler: Broadcasting large task binary
with size 1188.8 KiB
```

```
-> Result: AUC=0.8131 | F1=0.8899 | Time=3.0s
```

```
[46/48] Test config: {'ratio_target': 7, 'ratio_multiplier': 0.1,
'identity_weight': 1.0, 'model': RandomForestClassifier_ad394a6e557e,
'combinaison': 'w2v + tfidf'}
Undersampling : On garde 42.34% des données négatives.
Ratio de sain/toxique 7.08
toxic_weights : 0.71 + 1, identity_weight :1.0
Prétraitement terminé, df_train_final_balanced_debiased = 4978 |
df_test_final_all_w2v = 1199
```

```
25/12/06 06:28:30 WARN DAGScheduler: Broadcasting large task binary
with size 1225.8 KiB
25/12/06 06:28:30 WARN DAGScheduler: Broadcasting large task binary
with size 1225.7 KiB
25/12/06 06:28:30 WARN DAGScheduler: Broadcasting large task binary
with size 1287.5 KiB
25/12/06 06:28:32 WARN DAGScheduler: Broadcasting large task binary
with size 1445.9 KiB
25/12/06 06:28:33 WARN DAGScheduler: Broadcasting large task binary
with size 1458.0 KiB
25/12/06 06:28:33 WARN DAGScheduler: Broadcasting large task binary
with size 1476.4 KiB
25/12/06 06:28:33 WARN DAGScheduler: Broadcasting large task binary
with size 1502.2 KiB
25/12/06 06:28:33 WARN DAGScheduler: Broadcasting large task binary
with size 1534.0 KiB
25/12/06 06:28:34 WARN DAGScheduler: Broadcasting large task binary
with size 1218.6 KiB
25/12/06 06:28:34 WARN DAGScheduler: Broadcasting large task binary
with size 1230.7 KiB
25/12/06 06:28:34 WARN DAGScheduler: Broadcasting large task binary
with size 1230.7 KiB
```

```
-> Result: AUC=0.7716 | F1=0.9072 | Time=5.4s
```

```
[47/48] Test config: {'ratio_target': 7, 'ratio_multiplier': 0.1,
```

```
'identity_weight': 1.0, 'model': LinearSVC_8f0c0322b70e,  
'combinaison': 'w2v + tfidf'}  
Undersampling : On garde 42.34% des données négatives.  
Ratio de sain/toxique 7.08  
toxic_weights : 0.71 + 1, identity_weight :1.0  
Prétraitement terminé, df_train_final_balanced_debiased = 4978 |  
df_test_final_all_w2v = 1199
```

[illegible]

[illegible]

[illegible]

[illegible]

with size 1227.3 KiB
25/12/06 06:28:40 WARN DAGScheduler: Broadcasting large task binary
with size 1227.3 KiB
25/12/06 06:28:40 WARN DAGScheduler: Broadcasting large task binary
with size 1227.3 KiB
25/12/06 06:28:40 WARN DAGScheduler: Broadcasting large task binary
with size 1227.3 KiB
25/12/06 06:28:41 WARN DAGScheduler: Broadcasting large task binary
with size 1227.3 KiB
25/12/06 06:28:41 WARN DAGScheduler: Broadcasting large task binary
with size 1227.3 KiB
25/12/06 06:28:41 WARN DAGScheduler: Broadcasting large task binary
with size 1227.3 KiB
25/12/06 06:28:41 WARN DAGScheduler: Broadcasting large task binary
with size 1227.3 KiB
25/12/06 06:28:41 WARN DAGScheduler: Broadcasting large task binary
with size 1227.3 KiB
25/12/06 06:28:41 WARN DAGScheduler: Broadcasting large task binary
with size 1172.7 KiB
25/12/06 06:28:41 WARN DAGScheduler: Broadcasting large task binary
with size 1173.1 KiB
25/12/06 06:28:41 WARN DAGScheduler: Broadcasting large task binary
with size 1173.1 KiB

-> Result: AUC=0.8358 | F1=0.9059 | Time=7.2s

[48/48] Test config: {'ratio_target': 7, 'ratio_multiplier': 0.1,
'identity_weight': 1.0, 'model': GBTCClassifier_3b8b769d6280,
'combinaison': 'w2v + tfidf'}

Undersampling : On garde 42.34% des données négatives.

Ratio de sain/toxique 7.08

toxic_weights : 0.71 + 1, identity_weight :1.0

Prétraitement terminé, df_train_final_balanced_debiased = 4978 |
df_test_final_all_w2v = 1199

25/12/06 06:28:43 WARN DAGScheduler: Broadcasting large task binary
with size 1225.4 KiB
25/12/06 06:28:43 WARN DAGScheduler: Broadcasting large task binary
with size 1225.4 KiB
25/12/06 06:28:44 WARN DAGScheduler: Broadcasting large task binary
with size 1287.2 KiB
25/12/06 06:28:45 WARN DAGScheduler: Broadcasting large task binary
with size 1438.8 KiB
25/12/06 06:28:45 WARN DAGScheduler: Broadcasting large task binary
with size 1439.5 KiB
25/12/06 06:28:46 WARN DAGScheduler: Broadcasting large task binary
with size 1440.1 KiB
25/12/06 06:28:46 WARN DAGScheduler: Broadcasting large task binary

with size 1441.3 KiB
25/12/06 06:28:46 WARN DAGScheduler: Broadcasting large task binary
with size 1443.2 KiB
25/12/06 06:28:46 WARN DAGScheduler: Broadcasting large task binary
with size 1449.4 KiB
25/12/06 06:28:46 WARN DAGScheduler: Broadcasting large task binary
with size 1449.8 KiB
25/12/06 06:28:46 WARN DAGScheduler: Broadcasting large task binary
with size 1450.4 KiB
25/12/06 06:28:46 WARN DAGScheduler: Broadcasting large task binary
with size 1451.3 KiB
25/12/06 06:28:46 WARN DAGScheduler: Broadcasting large task binary
with size 1453.0 KiB
25/12/06 06:28:47 WARN DAGScheduler: Broadcasting large task binary
with size 1455.4 KiB
25/12/06 06:28:47 WARN DAGScheduler: Broadcasting large task binary
with size 1455.9 KiB
25/12/06 06:28:47 WARN DAGScheduler: Broadcasting large task binary
with size 1456.5 KiB
25/12/06 06:28:47 WARN DAGScheduler: Broadcasting large task binary
with size 1457.4 KiB
25/12/06 06:28:47 WARN DAGScheduler: Broadcasting large task binary
with size 1459.1 KiB
25/12/06 06:28:47 WARN DAGScheduler: Broadcasting large task binary
with size 1461.3 KiB
25/12/06 06:28:47 WARN DAGScheduler: Broadcasting large task binary
with size 1461.8 KiB
25/12/06 06:28:47 WARN DAGScheduler: Broadcasting large task binary
with size 1462.4 KiB
25/12/06 06:28:48 WARN DAGScheduler: Broadcasting large task binary
with size 1463.6 KiB
25/12/06 06:28:48 WARN DAGScheduler: Broadcasting large task binary
with size 1465.3 KiB
25/12/06 06:28:48 WARN DAGScheduler: Broadcasting large task binary
with size 1467.6 KiB
25/12/06 06:28:48 WARN DAGScheduler: Broadcasting large task binary
with size 1468.0 KiB
25/12/06 06:28:48 WARN DAGScheduler: Broadcasting large task binary
with size 1468.6 KiB
25/12/06 06:28:48 WARN DAGScheduler: Broadcasting large task binary
with size 1469.8 KiB
25/12/06 06:28:49 WARN DAGScheduler: Broadcasting large task binary
with size 1471.8 KiB
25/12/06 06:28:49 WARN DAGScheduler: Broadcasting large task binary
with size 1474.1 KiB
25/12/06 06:28:49 WARN DAGScheduler: Broadcasting large task binary
with size 1474.6 KiB
25/12/06 06:28:49 WARN DAGScheduler: Broadcasting large task binary
with size 1475.2 KiB

25/12/06 06:28:49 WARN DAGScheduler: Broadcasting large task binary with size 1476.3 KiB
25/12/06 06:28:49 WARN DAGScheduler: Broadcasting large task binary with size 1478.2 KiB
25/12/06 06:28:49 WARN DAGScheduler: Broadcasting large task binary with size 1480.0 KiB
25/12/06 06:28:49 WARN DAGScheduler: Broadcasting large task binary with size 1480.5 KiB
25/12/06 06:28:49 WARN DAGScheduler: Broadcasting large task binary with size 1481.1 KiB
25/12/06 06:28:50 WARN DAGScheduler: Broadcasting large task binary with size 1482.3 KiB
25/12/06 06:28:50 WARN DAGScheduler: Broadcasting large task binary with size 1484.5 KiB
25/12/06 06:28:50 WARN DAGScheduler: Broadcasting large task binary with size 1486.7 KiB
25/12/06 06:28:50 WARN DAGScheduler: Broadcasting large task binary with size 1487.2 KiB
25/12/06 06:28:50 WARN DAGScheduler: Broadcasting large task binary with size 1487.8 KiB
25/12/06 06:28:50 WARN DAGScheduler: Broadcasting large task binary with size 1489.0 KiB
25/12/06 06:28:50 WARN DAGScheduler: Broadcasting large task binary with size 1491.3 KiB
25/12/06 06:28:50 WARN DAGScheduler: Broadcasting large task binary with size 1493.9 KiB
25/12/06 06:28:50 WARN DAGScheduler: Broadcasting large task binary with size 1494.4 KiB
25/12/06 06:28:51 WARN DAGScheduler: Broadcasting large task binary with size 1495.0 KiB
25/12/06 06:28:51 WARN DAGScheduler: Broadcasting large task binary with size 1496.1 KiB
25/12/06 06:28:51 WARN DAGScheduler: Broadcasting large task binary with size 1498.1 KiB
25/12/06 06:28:51 WARN DAGScheduler: Broadcasting large task binary with size 1500.3 KiB
25/12/06 06:28:51 WARN DAGScheduler: Broadcasting large task binary with size 1500.8 KiB
25/12/06 06:28:51 WARN DAGScheduler: Broadcasting large task binary with size 1501.4 KiB
25/12/06 06:28:51 WARN DAGScheduler: Broadcasting large task binary with size 1502.5 KiB
25/12/06 06:28:51 WARN DAGScheduler: Broadcasting large task binary with size 1504.6 KiB
25/12/06 06:28:51 WARN DAGScheduler: Broadcasting large task binary with size 1506.4 KiB
25/12/06 06:28:52 WARN DAGScheduler: Broadcasting large task binary with size 1506.8 KiB
25/12/06 06:28:52 WARN DAGScheduler: Broadcasting large task binary

with size 1507.4 KiB
25/12/06 06:28:52 WARN DAGScheduler: Broadcasting large task binary
with size 1508.6 KiB
25/12/06 06:28:52 WARN DAGScheduler: Broadcasting large task binary
with size 1510.6 KiB
25/12/06 06:28:52 WARN DAGScheduler: Broadcasting large task binary
with size 1513.1 KiB
25/12/06 06:28:52 WARN DAGScheduler: Broadcasting large task binary
with size 1513.6 KiB
25/12/06 06:28:53 WARN DAGScheduler: Broadcasting large task binary
with size 1514.1 KiB
25/12/06 06:28:53 WARN DAGScheduler: Broadcasting large task binary
with size 1515.3 KiB
25/12/06 06:28:53 WARN DAGScheduler: Broadcasting large task binary
with size 1517.6 KiB
25/12/06 06:28:53 WARN DAGScheduler: Broadcasting large task binary
with size 1519.8 KiB
25/12/06 06:28:53 WARN DAGScheduler: Broadcasting large task binary
with size 1520.3 KiB
25/12/06 06:28:53 WARN DAGScheduler: Broadcasting large task binary
with size 1520.8 KiB
25/12/06 06:28:54 WARN DAGScheduler: Broadcasting large task binary
with size 1522.0 KiB
25/12/06 06:28:54 WARN DAGScheduler: Broadcasting large task binary
with size 1523.7 KiB
25/12/06 06:28:54 WARN DAGScheduler: Broadcasting large task binary
with size 1525.7 KiB
25/12/06 06:28:54 WARN DAGScheduler: Broadcasting large task binary
with size 1526.2 KiB
25/12/06 06:28:54 WARN DAGScheduler: Broadcasting large task binary
with size 1526.7 KiB
25/12/06 06:28:54 WARN DAGScheduler: Broadcasting large task binary
with size 1527.6 KiB
25/12/06 06:28:54 WARN DAGScheduler: Broadcasting large task binary
with size 1529.4 KiB
25/12/06 06:28:54 WARN DAGScheduler: Broadcasting large task binary
with size 1531.7 KiB
25/12/06 06:28:54 WARN DAGScheduler: Broadcasting large task binary
with size 1532.2 KiB
25/12/06 06:28:55 WARN DAGScheduler: Broadcasting large task binary
with size 1532.8 KiB
25/12/06 06:28:55 WARN DAGScheduler: Broadcasting large task binary
with size 1534.0 KiB
25/12/06 06:28:55 WARN DAGScheduler: Broadcasting large task binary
with size 1535.9 KiB
25/12/06 06:28:55 WARN DAGScheduler: Broadcasting large task binary
with size 1538.1 KiB
25/12/06 06:28:55 WARN DAGScheduler: Broadcasting large task binary
with size 1538.6 KiB

25/12/06 06:28:55 WARN DAGScheduler: Broadcasting large task binary with size 1539.2 KiB
25/12/06 06:28:56 WARN DAGScheduler: Broadcasting large task binary with size 1540.4 KiB
25/12/06 06:28:56 WARN DAGScheduler: Broadcasting large task binary with size 1542.6 KiB
25/12/06 06:28:56 WARN DAGScheduler: Broadcasting large task binary with size 1544.5 KiB
25/12/06 06:28:56 WARN DAGScheduler: Broadcasting large task binary with size 1545.0 KiB
25/12/06 06:28:56 WARN DAGScheduler: Broadcasting large task binary with size 1545.5 KiB
25/12/06 06:28:56 WARN DAGScheduler: Broadcasting large task binary with size 1546.7 KiB
25/12/06 06:28:56 WARN DAGScheduler: Broadcasting large task binary with size 1548.9 KiB
25/12/06 06:28:56 WARN DAGScheduler: Broadcasting large task binary with size 1551.2 KiB
25/12/06 06:28:56 WARN DAGScheduler: Broadcasting large task binary with size 1551.7 KiB
25/12/06 06:28:57 WARN DAGScheduler: Broadcasting large task binary with size 1552.3 KiB
25/12/06 06:28:57 WARN DAGScheduler: Broadcasting large task binary with size 1553.4 KiB
25/12/06 06:28:57 WARN DAGScheduler: Broadcasting large task binary with size 1554.8 KiB
25/12/06 06:28:57 WARN DAGScheduler: Broadcasting large task binary with size 1556.9 KiB
25/12/06 06:28:57 WARN DAGScheduler: Broadcasting large task binary with size 1557.4 KiB
25/12/06 06:28:57 WARN DAGScheduler: Broadcasting large task binary with size 1558.0 KiB
25/12/06 06:28:57 WARN DAGScheduler: Broadcasting large task binary with size 1559.2 KiB
25/12/06 06:28:57 WARN DAGScheduler: Broadcasting large task binary with size 1561.1 KiB
25/12/06 06:28:57 WARN DAGScheduler: Broadcasting large task binary with size 1563.5 KiB
25/12/06 06:28:58 WARN DAGScheduler: Broadcasting large task binary with size 1564.0 KiB
25/12/06 06:28:58 WARN DAGScheduler: Broadcasting large task binary with size 1564.5 KiB
25/12/06 06:28:58 WARN DAGScheduler: Broadcasting large task binary with size 1565.7 KiB
25/12/06 06:28:58 WARN DAGScheduler: Broadcasting large task binary with size 1568.0 KiB
25/12/06 06:28:59 WARN DAGScheduler: Broadcasting large task binary with size 1249.7 KiB

25/12/06 06:28:59 WARN DAGScheduler: Broadcasting large task binary with size 1262.0 KiB

-> Result: AUC=0.8416 | F1=0.9215 | Time=17.6s

25/12/06 06:28:59 WARN DAGScheduler: Broadcasting large task binary with size 1262.0 KiB

Conversion en Pandas pour analyse facile

```
df_results_pd = pd.DataFrame(all_results)
```

Trier par meilleur AUC

```
print(df_results_pd.sort_values(by="AUC", ascending=False).head(5))
```

Sauvegarde CSV

```
df_results_pd.to_csv("grid_search_results.csv", index=False)
```

```
meilleurs_params_globaux = df_results_pd[[
    'ratio_target',
    'ratio_multiplieur',
    'identity_weight',
    'model',
    'combinaison'
]].iloc[0].to_dict()
```

```
print(meilleurs_params_globaux)
```

	ratio_target	ratio_multiplieur	identity_weight	\
23	5	0.05	1.0	
43	7	0.10	0.5	
47	7	0.10	1.0	
38	7	0.05	1.0	
22	5	0.05	1.0	

	model	combinaison	AUC	Accuracy
F1 \				
23	GBTCClassifier_3b8b769d6280	w2v + tfidf	0.855480	0.913261
0.915778				
43	GBTCClassifier_3b8b769d6280	w2v + tfidf	0.853950	0.935780
0.930850				
47	GBTCClassifier_3b8b769d6280	w2v + tfidf	0.841566	0.920767
0.921491				
38	LinearSVC_8f0c0322b70e	w2v + tfidf	0.839146	0.893244
0.906060				
22	LinearSVC_8f0c0322b70e	w2v + tfidf	0.836429	0.876564
0.897393				

	Training_Size
23	3779
43	4978
47	4978

```

38         4978
22         3779
{'ratio_target': 3, 'ratio_multiplifier': 0.05, 'identity_weight': 0.5,
'model': LogisticRegression_afae24335158, 'combinaison': 'w2v +
tfidf'}

```

3.3.5 Trouver les hyperparamètres pour le modèle + prétraitement retenus

```

def trouver_grid_finale(model, grid, combinaison, name, df):

    model_lr_prefinal, best_model_lr_prefinal,
    best_params_lr_prefinal, scores_lr_prefinal =
    train_or_load_nested_model(
        model,
        grid,
        df,
        df.count(),
        LOAD_MODEL,
        SAVE_MODEL,
        name,
        #training_func=simple_cross_validator, # C EST MIEUX D
UTILISER NESTED CV SI POSSIBLE, SI CA PREND TROP DE TEMPS PASSER A
simple_cross_validator
        training_func=nested_cv,
        combinaison=combinaison,
        k_folds=3
    )

    return model_lr_prefinal, best_model_lr_prefinal,
    best_params_lr_prefinal, scores_lr_prefinal

```

Limite que l'on peut changer - on aurait voulu 100k lignes, mais Spark et la VM ne fonctionnent pas (erreurs out of memory + connection refused)... Pour la suite, nous avons gardé la grid d'hyperparamètres que nous avons trouvé pour 10k lignes

```

df_train_final_all_w2v, df_test_final_all_w2v =
preprocessing_static_pipeline(
    train,
    limit=100000,
    debug=False
)

# Application du pré-traitement dynamique avec les meilleurs
paramètres
df_train_final_balanced_debiased = preprocessing_final_dynamic(
    df_train_final_all_w2v,
    ratio_target=meilleurs_params_globaux['ratio_target'],
    ratio_multiplifier=meilleurs_params_globaux['ratio_multiplifier'],

```

```

        identity_weight=meilleurs_params_globaux['identity_weight']
    )

model_lr_prefinal, best_model_lr_prefinal, best_params_lr_prefinal,
scores_lr_prefinal = trouver_grid_finale(
    gbt_weighted, # à changer si besoin
    gbt_grid, # pareil
    meilleurs_params_globaux['combinaison'],
    name="modele_grid_finale",
    df=df_train_final_balanced_debiased
)

print("\nRecherche d'hyperparamètres internes terminée.")

best_params_raw = best_params_lr_prefinal

# Création d'un nouveau dictionnaire lisible
best_params_clean = {}

for param_obj, value in best_params_raw.items():
    # Extrait la propriété 'name' de l'objet Param PySpark
    param_name = param_obj.name
    best_params_clean[param_name] = value

print("\n--- Meilleurs hyperparamètres ---")
print(best_params_clean)

```

3.3.6 Entraînement final

Entraînement final avec le meilleur modèle, la meilleure combinaison de features et les meilleurs paramètres et hyperparamètres

```

df_train_final_all_w2v, df_test_final_all_w2v =
preprocessing_static_pipeline(
    train,
    limit=1000,
    debug=False
)

# Application du pré-traitement dynamique avec les meilleurs
paramètres
df_train_final_balanced_debiased = preprocessing_final_dynamic(
    df_train_final_all_w2v,
    ratio_target=3,
    ratio_multiplieur=0.05,
    identity_weight=0.5
)

gbt_weighted_final = GBTClassifier(
    featuresCol="features_final",

```

```

labelCol="label",
weightCol="final_weight",
seed=0,

# Hyperparamètres optimaux
cacheNodeIds=False,
checkpointInterval=10,
featureSubsetStrategy='all',
impurity='variance', # Impureté utilisée pour mesurer la qualité
de la division
leafCol='',
lossType='logistic', # Le type de fonction de perte pour la
classification binaire
maxBins=32, # Nombre maximal de "bins" (compartiments)
pour discrétiser les features continues
maxDepth=5, # Profondeur maximale de l'arbre
maxIter=20, # Nombre maximal d'itérations (nombre
d'arbres)
maxMemoryInMB=256,
minInfoGain=0.0,
minInstancesPerNode=1,
minWeightFractionPerNode=0.0,
stepSize=0.1, # Taux d'apprentissage (Learning Rate)
subsamplingRate=1.0, # Fraction des données à utiliser pour
entraîner chaque arbre
validationTol=0.01
)

pipeline = build_pipeline_part2(model=gbt_weighted_final,
combinaison='w2v + tfidf')

# 3. Entraînement
# Le pipeline va chercher la colonne 'final_weight' créée dans
preprocessing_final_dynamic
model_final = pipeline.fit(df_train_final_balanced_debiased)

>>> Démarrage du pré-traitement statique...

>>> Split OK (via randomSplit)

Train = 924 | Test = 76

Base d'entraînement en cache : 924 lignes

Base de test en cache : 76 lignes

```

```
25/12/07 03:08:26 WARN InstanceBuilder: Failed to load implementation
from:dev.ludovic.netlib.blas.JNIBLAS
```

```
--- Statut Undersampling Dé-Biaisé ---
Total Toxiques (Pos) : 56
Négatifs Protégés (Groupe B) : 32
Négatifs à Échantillonner (Groupe C) : 104
Fraction à garder du Groupe C : 1.0000 (100.00%)
Ratio de sain/toxique 2.43
toxic_weights : 0.12 + 1, identity_weight :0.5
```

```
Prétraitement terminé, df_train_final_balanced_debiased = 192 |
df_test_final_all_w2v = 76
```

```
save_model(model_final, './model_final')
```

```
Modèle sauvegardé dans : models/./model_final
```

3.3.7 Résultats finaux

Voir le pdf pour nos meilleurs résultats

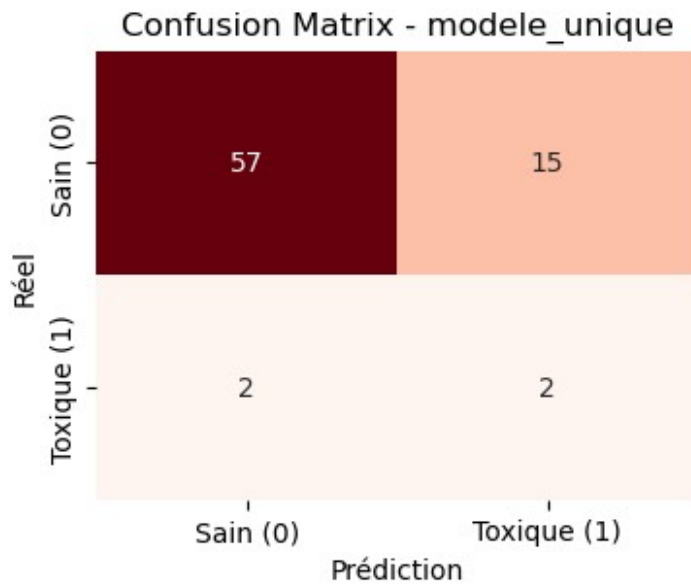
```
# Appel de la fonction
test_results_final = evaluate_models_on_test_set(model_final,
df_test_final_all_w2v, plot_cm=True)

# Création du tableau récapitulatif (votre code existant)
df_test_results_final = pd.DataFrame.from_dict(test_results_final,
orient='index')
df_test_results_final.reset_index(inplace=True)
df_test_results_final.rename(columns={'index': 'Combinaison'},
inplace=True)

# Affichage du tableau final coloré
display(df_test_results_final.style.background_gradient(cmap='Greens',
subset=df_test_results_final.columns[1:]))

Démarrage de l'évaluation CIBLÉE (Classe 1 - Toxique)...

=====
--- Modèle : modele_unique ---
TOXIC CLASS -> Recall: 0.5000 | Precision: 0.1176 | F1: 0.1905
Global AUC: 0.6007
```



Évaluation terminée.

<pandas.io.formats.style.Styler at 0x7c37292270e0>

```
optimal_threshold_final = find_best_threshold(model_final,  
df_test_final_all_w2v, model_name="GBT_Final")
```

Analyse de Seuil pour GBT_Final

AUC (Seuil-Indépendant) : 0.6007

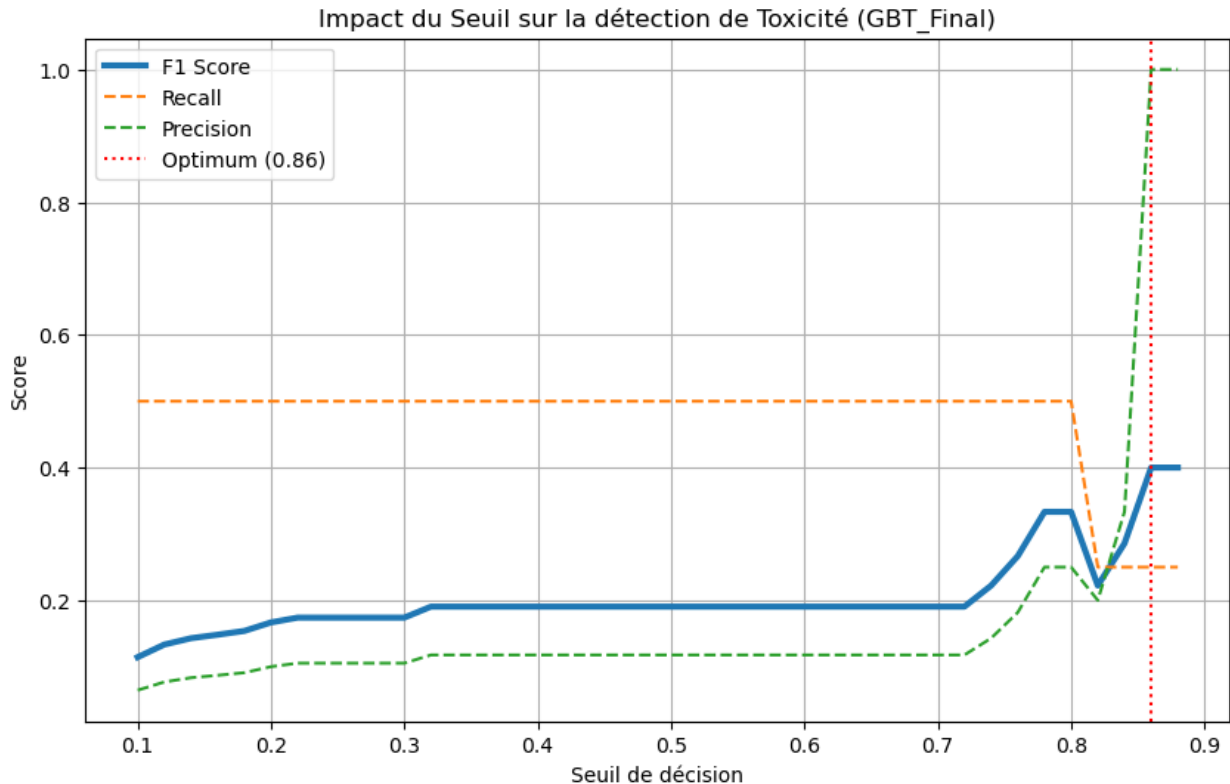
Seuil optimal (Max F1) : 0.86

-> F1 Toxic (Opt.) : 0.4000

-> Recall Toxic (Opt.) : 0.2500

-> Precision Toxic (Opt.) : 1.0000

-> Accuracy Générale : 0.9605



```
bias_results_before = compute_bias_metrics(
    model_final,
    df_test_final_all_w2v,
    identity_columns,
    "Modèle Final"
)
```

```
#Afficher les identités les plus discriminées (BPSN faible = le modèle confond identité et insulte)
print("\nIdentités les plus confondues avec la toxicité (BPSN bas) :")
display(bias_results_before.head(5).style.background_gradient(cmap='RdYlGn', vmin=0.5, vmax=1.0))
```

```
Calcul des biais pour : Modèle Final ...
Score Jigsaw Final : 0.7438 (Global AUC: 0.6007)
```

```
Identités les plus confondues avec la toxicité (BPSN bas) :
```

```
/home/vboxuser/ENTER/lib/python3.13/site-packages/sklearn/metrics/_ranking.py:379: UndefinedMetricWarning: Only one class is present in y_true. ROC AUC score is not defined in that case.
  warnings.warn(
/home/vboxuser/ENTER/lib/python3.13/site-packages/sklearn/metrics/_ranking.py:379: UndefinedMetricWarning: Only one class is present in y_true. ROC AUC score is not defined in that case.
  warnings.warn(
```

[illegible]

[illegible]

```
y_true. ROC AUC score is not defined in that case.  
warnings.warn(  
/home/vboxuser/ENTER/lib/python3.13/site-packages/sklearn/metrics/  
_ranking.py:379: UndefinedMetricWarning: Only one class is present in  
y_true. ROC AUC score is not defined in that case.  
warnings.warn(  
/home/vboxuser/ENTER/lib/python3.13/site-packages/sklearn/metrics/  
_ranking.py:379: UndefinedMetricWarning: Only one class is present in  
y_true. ROC AUC score is not defined in that case.  
warnings.warn(  
/home/vboxuser/ENTER/lib/python3.13/site-packages/sklearn/metrics/  
_ranking.py:379: UndefinedMetricWarning: Only one class is present in  
y_true. ROC AUC score is not defined in that case.  
warnings.warn(  
/home/vboxuser/ENTER/lib/python3.13/site-packages/sklearn/metrics/  
_ranking.py:379: UndefinedMetricWarning: Only one class is present in  
y_true. ROC AUC score is not defined in that case.  
warnings.warn(  
/home/vboxuser/ENTER/lib/python3.13/site-packages/sklearn/metrics/  
_ranking.py:379: UndefinedMetricWarning: Only one class is present in  
y_true. ROC AUC score is not defined in that case.  
warnings.warn(  
/home/vboxuser/ENTER/lib/python3.13/site-packages/sklearn/metrics/  
_ranking.py:379: UndefinedMetricWarning: Only one class is present in  
y_true. ROC AUC score is not defined in that case.  
warnings.warn(  
/home/vboxuser/ENTER/lib/python3.13/site-packages/sklearn/metrics/  
_ranking.py:379: UndefinedMetricWarning: Only one class is present in  
y_true. ROC AUC score is not defined in that case.  
warnings.warn(  
/home/vboxuser/ENTER/lib/python3.13/site-packages/sklearn/metrics/  
_ranking.py:379: UndefinedMetricWarning: Only one class is present in  
y_true. ROC AUC score is not defined in that case.  
warnings.warn(  
/home/vboxuser/ENTER/lib/python3.13/site-packages/sklearn/metrics/  
_ranking.py:379: UndefinedMetricWarning: Only one class is present in  
y_true. ROC AUC score is not defined in that case.  
warnings.warn(  
/home/vboxuser/ENTER/lib/python3.13/site-packages/sklearn/metrics/  
_ranking.py:379: UndefinedMetricWarning: Only one class is present in  
y True. ROC AUC score is not defined in that case.
```

```
warnings.warn(
/home/vboxuser/ENTER/lib/python3.13/site-packages/sklearn/metrics/_ranking.py:379: UndefinedMetricWarning: Only one class is present in y_true. ROC AUC score is not defined in that case.
warnings.warn(
/home/vboxuser/ENTER/lib/python3.13/site-packages/sklearn/metrics/_ranking.py:379: UndefinedMetricWarning: Only one class is present in y_true. ROC AUC score is not defined in that case.
warnings.warn(
/home/vboxuser/ENTER/lib/python3.13/site-packages/sklearn/metrics/_ranking.py:379: UndefinedMetricWarning: Only one class is present in y_true. ROC AUC score is not defined in that case.
warnings.warn(
/home/vboxuser/ENTER/lib/python3.13/site-packages/sklearn/metrics/_ranking.py:379: UndefinedMetricWarning: Only one class is present in y_true. ROC AUC score is not defined in that case.
warnings.warn(
/home/vboxuser/ENTER/lib/python3.13/site-packages/sklearn/metrics/_ranking.py:379: UndefinedMetricWarning: Only one class is present in y_true. ROC AUC score is not defined in that case.
warnings.warn(
/home/vboxuser/ENTER/lib/python3.13/site-packages/sklearn/metrics/_ranking.py:379: UndefinedMetricWarning: Only one class is present in y_true. ROC AUC score is not defined in that case.
warnings.warn(
/home/vboxuser/ENTER/lib/python3.13/site-packages/sklearn/metrics/_ranking.py:379: UndefinedMetricWarning: Only one class is present in y_true. ROC AUC score is not defined in that case.
```

<pandas.io.formats.style.Styler at 0x7c37288d6490>

4. Approche moderne : Transformers (BERT)

Pour tenter un d'obtenir de meilleures performances de compréhension sémantique, nous nous tournons vers l'état de l'art du Traitement du Langage Naturel (NLP) : les Transformers.

Nous utiliserons ici DistilBERT, version allégée et plus rapide de BERT (Bidirectional Encoder Representations from Transformers), capable de comprendre le contexte d'un mot en fonction de toute la phrase.

L'idée était ici d'utiliser les modèles BERT pour le traitement des données, via la surcouche Sparknlp. Malgré un downgrade vers Spark 3.5 pour des soucis de compatibilité (Spark 4 et Scala 2.12 ne sont pas compatibles), cela n'a pas suffi, notamment à cause du fait d'utiliser une VM linux sur une architecture mac ARM. Le choix a donc été fait de passer outre Sparknlp et d'utiliser Hugging Face.

<https://www.swisstransfer.com/d/f3dcb660-ad11-4435-aa12-9357a523e31d>

Lien de téléchargement des données tokenisées (70k uniquement), du modèle entraîné avec ces données (260Mo, d'où ce lien) et des résultats pré-inférés pour ce même modèle et ces données. Fichier à unzip et placer dans le répertoire courant du notebook. Lien valable jusqu'au 5/01/2026 12h.

4.1 Préparation des données pour BERT

4.1.0 Installation des librairies nécessaires

```
# Installation des librairies Python nécessaires pour BERT
#!pip install datasets transformers torch onnx onnxruntime tf2onnx
#!pip install accelerate -U
```

4.1.1 SparkSession adaptée pour le pré-traitement pour BERT

Configuration spécifique pour l'usage de BERT. Le but est de préparer la SparkSession pour optimiser au mieux la durée d'exécution des cellules suivantes" Allocation de 12g au driver afin de supporter le passage toPandas. Activation de Apache Arrow pour accélérer la conversion toPandas().

```
import os
import sys
import findspark

# 1. Initialisation via findspark (pour retrouver le Spark 4 dans
/opt/spark)
findspark.init()

import pyspark
from pyspark.sql import SparkSession

# 2. Configuration Optimisée pour BERT sur Spark 4
spark = SparkSession.builder \
    .appName("Jigsaw BERT Turbo (Spark 4)") \
    .master("local[*]") \
    .config("spark.driver.memory", "12g") \
    .config("spark.driver.maxResultSize", "2g") \
    .config("spark.sql.execution.arrow.pyspark.enabled", "true") \
    .getOrCreate()

print(f"Session Spark active.")
print(f"Spark Version : {spark.version}")
# Doit afficher une version 4.x.x
print(f"CPUs disponibles : {os.cpu_count()}")
```

4.1.2 Chargement et nettoyage des données

Chargement des données brutes pour le pré-traitement pour BERT.

Gestion des retours à la ligne. Configuration du chemin d'accès aux données. Mise en place d'une limite pour la taille de l'échantillon pour éviter les saturations mémoire et pour accélérer la vérification de l'aspect fonctionnel de la procédure de prétraitement des données. On force la conversion de 'target' en float pour éviter les erreurs avec le texte corrompu qui pourrait s'y trouver.

Le but de l'exercice est de classer un commentaire selon s'il est, ou non, toxique. BERT est conçu pour la classification discrète, ainsi nous transformons 'target' en des labels, valant 0 ou 1. Le découpage entre ces deux classes se fait selon le score 'target' en coupant l'espace en deux (0,5 appartient à la classe 1). Ensuite il faut envoyer les données sur Pandas. Cette étape sort nos données du traitement via Spark mais est indispensable étant donné que l'entraînement n'est pas possible ici via Spark. Nous faisons donc le maximum sur Spark mais il faut en sortir pour l'entraînement.

Un des soucis de nos données est la sur représentation de la classe 'sain' par rapport à la classe 'toxique' ; l'autre soucis est que nous avons un grand nombre de données, souvent trop grand pour pouvoir être utilisé au complet. Une solution à ses deux problèmes est le downsampling avec un ratio. Le downsampling permet de limiter considérablement notre quantité finale de données. Le fait d'instaurer un ratio 'sain/toxique' permet d'avoir une quantité suffisante d'éléments toxiques pour assurer l'apprentissage.

```
from pyspark.sql.functions import col, when, rand
import pandas as pd

# --- CONFIGURATION ---
DATA_PATH =
    './jigsaw-unintended-bias-in-toxicity-classification/train.csv'

# CONFIGURATION DU RATIO
TARGET_TOXIC_COUNT = 10000 # max toxiques dispo ~144k
RATIO_SAIN_TOXIQUE = 6     # resultat du calcul sain/toxique

print(f"--- Démarrage de l'extraction ---")
print(f"Cible : {TARGET_TOXIC_COUNT} toxiques + {TARGET_TOXIC_COUNT *
    RATIO_SAIN_TOXIQUE} sains")

# 1. Chargement des données brutes
df_raw = spark.read.csv(DATA_PATH, header=True, inferSchema=True,
    multiLine=True, escape='')

# 2. Nettoyage et Labelisation
df_clean = df_raw.select("comment_text", "target") \
    .filter(col("comment_text").isNotNull()) \
    .filter(col("comment_text") != "") \
    .withColumn("target", col("target").cast("float")) \
    .filter(col("target").isNotNull()) \
    .withColumn("label", when(col("target") >= 0.5, 1).otherwise(0))

# 3. Échantillonnage dans Spark
print("Séparation des classes dans Spark...")
```

```

df_toxic = df_clean.filter(col("label") == 1)
df_safe = df_clean.filter(col("label") == 0)

# Récupérer les toxiques
df_toxic_sample = df_toxic.limit(TARGET_TOXIC_COUNT)

# Récupérer les sains
count_safe_needed = TARGET_TOXIC_COUNT * RATIO_SAIN_TOXIQUE
df_safe_sample = df_safe.limit(count_safe_needed)

# Fusion
df_balanced_spark = df_toxic_sample.union(df_safe_sample)

# 4. Conversion Spark -> Pandas
print("Conversion Spark -> Pandas en cours...")
pdf_sample = df_balanced_spark.toPandas()

# 5. Mélange pour ne pas avoir tous les toxiques au début
pdf_sample = pdf_sample.sample(frac=1,
random_state=42).reset_index(drop=True)

# VÉRIFICATIONS
print("-" * 30)
print(f"Succès ! Dimensions du DataFrame Pandas : {pdf_sample.shape}")

n_toxic = len(pdf_sample[pdf_sample['label'] == 1])
n_safe = len(pdf_sample[pdf_sample['label'] == 0])

print(f"Nombre de Toxiques (Class 1) : {n_toxic}")
print(f"Nombre de Sains (Class 0) : {n_safe}")

if n_toxic > 0:
    real_ratio = n_safe / n_toxic
    print(f"Ratio réel obtenu : 1 Toxique pour {real_ratio:.2f} Sains")
else:
    print("Erreur : Aucun toxique trouvé !")
print("-" * 30)

print(pdf_sample.head(5))

--- Démarrage de l'extraction ---
Cible : 10000 toxiques + 60000 sains

Séparation des classes dans Spark...
Conversion Spark -> Pandas en cours...

```

```
-----
Succès ! Dimensions du DataFrame Pandas : (70000, 3)
Nombre de Toxiques (Class 1) : 10000
Nombre de Sains (Class 0) : 60000
Ratio réel obtenu : 1 Toxique pour 6.00 Sains
-----
```

	comment_text	target	label
0	I keep trying to have a civil conversation wit...	0.0	0
1	Daniel wants Alaskans to lose their dividends ...	0.0	0
2	My bad for misinterpreting the article. Thank ...	0.0	0
3	Fair point -- and this week I've noticed troop...	0.0	0
4	How utterly thoughtful and diplomatic of Chena...	0.0	0

4.1.3 Tokenisation

La tokenisation est indispensable pour les modèles BERT, c'est ce qui leur permet de 'comprendre' la donnée textuelle. La tokenisation permet au modèle de comprendre le langage, elle permet aussi de faire en sorte que toutes les entrées aient des longueurs fixes pour que le réseau de neurones puisse les traiter correctement. Enfin elle permet une meilleure gestion du vocabulaire utilisé.

C'est en particulier cette étape de tokenisation qui justifie un prétraitement séparé entre l'approche classique et l'approche BERT. C'est également l'étape la plus longue du pré-traitement des données.

Cette cellule s'exécute hors Spark, dans le driver. Le modèle 'distilbert' est chargé (plus léger qu'un BERT classique), nous définissons aussi une taille maximale pour l'enregistrement des données, afin d'éviter les trop gros fichiers. Nous avons choisi d'échanger de la précision pour de la rapidité en tronquant à 64 tokens au lieu de 512 par défaut. Cela réduit la taille maximale des entrées mais diminue drastiquement la complexité, quadratique en le nombre de tokens.

Une fois la tokenisation faite, nous pouvons supprimer les colonnes contenant le texte brut, ainsi que la colonne 'target' devenue obsolète depuis la création de la colonne 'label'. Ensuite nous effectuons un split des données entre le train et le test (pas de set de validation car nous savons déjà quel modèle utiliser et il n'y a pas beaucoup d'hyperparamètres à choisir).

Finalement nous sauvegardons les données pré-traitées sur le disque afin de pouvoir les utiliser postérieurement lors de la phase d'entraînement du modèle. Ces données sont fragmentées en petits fichiers pour éviter tout risque de saturation mémoire, en écriture comme en lecture.

```
import shutil
import os
from transformers import DistilBertTokenizer
from datasets import Dataset

# --- CONFIGURATION ---
MODEL_NAME = "distilbert-base-uncased"
SAVE_PATH =
"./jigsaw-unintended-bias-in-toxicity-classification/data/Bert/process
ed_data_bert_10k_ratio6_"
```

```

MAX_SHARD_SIZE = "50MB"

print("--- Démarrage de la Tokenisation et Sauvegarde ---")

# 1. Initialisation
tokenizer = DistilBertTokenizer.from_pretrained(MODEL_NAME)

def tokenize_function(examples):
    # 64 tokens only
    return tokenizer(examples["comment_text"], padding="max_length",
truncation=True, max_length=64)

# 2. Pandas DataFrame to Dataset Hugging Face
hf_dataset = Dataset.from_pandas(pdf_sample)

# 3. Tokenisation
print("Tokenisation en cours...")
tokenized_datasets = hf_dataset.map(tokenize_function, batched=True)

# 4. Nettoyage
cols_to_remove = ["comment_text", "target"]
if "__index_level_0__" in tokenized_datasets.column_names:
    cols_to_remove.append("__index_level_0__")

tokenized_datasets = tokenized_datasets.remove_columns(cols_to_remove)
tokenized_datasets.set_format("torch")

# Changement de colonne pour le stratify_by_column
print("Conversion de la colonne 'label' en ClassLabel...")
tokenized_datasets = tokenized_datasets.class_encode_column("label")

tokenized_datasets.set_format("torch")

# 5. Split Train/Test (80/20)
print("Division Train / Test...")
dataset_dict = tokenized_datasets.train_test_split(test_size=0.2,
stratify_by_column="label")

# 6. Sauvegarde
if os.path.exists(SAVE_PATH):
    print(f"Nettoyage de l'ancien dossier : {SAVE_PATH}")
    shutil.rmtree(SAVE_PATH)

print(f"Sauvegarde dans : {SAVE_PATH}")
dataset_dict.save_to_disk(SAVE_PATH, max_shard_size=MAX_SHARD_SIZE)

print("--- Sauvegarde terminée ! ---")

--- Démarrage de la Tokenisation et Sauvegarde ---
Tokenisation en cours...

```

```
{"model_id": "7baddf9647fe43938cbc7c4cd09b58e2", "version_major": 2, "version_minor": 0}
```

Conversion de la colonne 'label' en ClassLabel...

```
{"model_id": "0d578519d4344b3b827acd2fb2fbd5df", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "e0dfdc06c4d64e44aa795753ad13bf3c", "version_major": 2, "version_minor": 0}
```

Division Train / Test...

Sauvegarde dans : ./jigsaw-unintended-bias-in-toxicity-classification/data/Bert/processed_data_bert_10k_ratio6_

```
{"model_id": "fcff34104bb648f1830f36a99feb6c1a", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "36bfdbe361aa4ac390af3265d08108c4", "version_major": 2, "version_minor": 0}
```

--- Sauvegarde terminée ! ---

4.1.4 Chargement des données tokenisées

```
import os
from datasets import load_from_disk, DatasetDict

# On va chercher les données sauvegardées
encoded_data_path = "./jigsaw-unintended-bias-in-toxicity-classification/data/Bert/processed_data_bert_10k_ratio6_"

if os.path.exists(encoded_data_path):
    print(f"Chargement du dataset tokenisé depuis {encoded_data_path}...")

    dataset_split = load_from_disk(encoded_data_path)

    print("Chargement terminé.")
    print(f"Structure du dataset : {dataset_split}")

else:
    # Si n'y a pas de données tokenisées
    print(f"ERREUR : Le dossier {encoded_data_path} est introuvable.")
    print("Veuillez lancer la tokenisation et la sauvegarde (4.1).")

# Vérification
if 'train' not in dataset_split or 'test' not in dataset_split:
    raise ValueError("Le dataset chargé ne contient pas 'train' et 'test'.")
```

```
print("--- Dataset prêt pour l'entraînement. ---")

Chargement du dataset tokenisé depuis ./jigsaw-unintended-bias-in-
toxicity-classification/data/Bert/processed_data_bert_10k_ratio6...
Chargement terminé.
Structure du dataset : DatasetDict({
  train: Dataset({
    features: ['label', 'input_ids', 'attention_mask'],
    num_rows: 56000
  })
  test: Dataset({
    features: ['label', 'input_ids', 'attention_mask'],
    num_rows: 14000
  })
})
--- Dataset prêt pour l'entraînement. ---
```

4.2 Entraînement du modèle BERT

4.2.1 Entraînement

L'entraînement est très long. La méthode du transfer learning est utilisée pour diminuer au maximum la durée d'entraînement. En effet, plutôt que de fine-tuner tout le modèle nous n'entraînons que la surcouche de classification de distilBERT. Dans le détail il y a le pre-classifier et le classifieur à entraîner. Nous gelons les couches internes en désactivant le calcul de gradient pour n'entraîner que la tête de classification.

Cette méthode n'en reste pas moins plutôt lente. A titre d'exemple il faut compter 1 heure par époque avec 100k lignes de données (CPU uniquement étant donné que la configuration GPU ne passe pas au travers de Virtual Box). Par contraintes temporelles seulement une faible fraction du dataset a été utilisée. L'entraînement le plus complet que nous avons pu réaliser comporte 70k données pour 10k 'toxiques'.

```
import time
from transformers import DistilBertForSequenceClassification, Trainer,
TrainingArguments

MODEL_NAME = "distilbert-base-uncased"

print("--- Chargement du modèle pré-entraîné ---")
# On fixe à 2 le nombre de labels pour la classification binaire
model =
DistilBertForSequenceClassification.from_pretrained(MODEL_NAME,
num_labels=2)

# Freeze des couches internes
print("--- Figeage des couches internes de DistilBERT ---")
for param in model.distilbert.parameters():
```

```

    param.requires_grad = False

# Libération de la couche de classification (par sécurité)
for param in model.classifier.parameters():
    param.requires_grad = True

# Vérification des paramètres dont l'entraînement est actif
print("Paramètres actifs pour l'entraînement :")
for name, param in model.named_parameters():
    if param.requires_grad:
        print(f" - {name}")

# Configuration de l'entraînement
training_args = TrainingArguments(
    output_dir='./results',
    num_train_epochs=3,
    per_device_train_batch_size=32,
    per_device_eval_batch_size=32,
    learning_rate=1e-3,
    weight_decay=0.01,
    eval_strategy="epoch",
    save_strategy="epoch",
    logging_dir='./logs',
)

# Initialisation du Trainer
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=dataset_split["train"],
    eval_dataset=dataset_split["test"],
)

print(f"--- Démarrage de l'entraînement ---")
start_time = time.time()

trainer.train()

print(f"--- Entraînement terminé en {(time.time() -
start_time)/60:.2f} minutes. ---")

```

4.2.2 Sauvegarde du modèle

Une fois l'entraînement terminé nous sauvegardons le modèle afin de pouvoir le réutiliser ultérieurement. Nous faisons la sauvegarde via pytorch, le plus simple à la fois pour la sauvegarde et pour les inférences par la suite.

```

import os
import torch

```

```

from transformers import DistilBertTokenizer

# 1. SAUVEGARDE STANDARD (PyTorch)
save_directory = "./models/distilbert_jigsaw_final"
print(f"Sauvegarde du modèle PyTorch standard dans : {save_directory}")

trainer.save_model(save_directory)
# Il ne faut pas oublier de sauvegarder le tokenizer pour pouvoir
inférer avec le modèle à l'avenir
tokenizer.save_pretrained(save_directory)

```

4.3 Utilisation du modèle

4.3.1 Initialisation pour l'inférence

```

from transformers import DistilBertForSequenceClassification,
DistilBertTokenizer
import torch
import torch.nn.functional as F

# 1. Chargement du modèle
model_path = "./models/distilbert_jigsaw_final"
print("--- Chargement du modèle pour inférence ---")

loaded_model =
DistilBertForSequenceClassification.from_pretrained(model_path)
loaded_tokenizer = DistilBertTokenizer.from_pretrained(model_path)

# Configuration pour l'évaluation
loaded_model.to("cpu")
loaded_model.eval()

# 2. Fonction de prédiction
def predict_toxicity(text):
    # Max length 64 pour coller à l'entraînement)
    inputs = loaded_tokenizer(text, return_tensors="pt",
truncation=True, padding="max_length", max_length=64)

    with torch.no_grad():
        outputs = loaded_model(**inputs)

    logits = outputs.logits
    probs = F.softmax(logits, dim=1)

    # Récupération de la probabilité de toxicité
    toxic_score = probs[0][1].item()

```

```
prediction = "TOXIQUE" if toxic_score >= 0.5 else "NON TOXIQUE"  
return prediction, toxic_score
```

```
--- Chargement du modèle pour inférence ---
```

4.3.2 Inférences

Dans le dataset nous avons drop le contenu textuel pour ne garder que le contenu tokeniser, et ce même pour le test. Cette opération permet de réduire la taille des datasets et de faciliter leur manipulation. En revanche, nous ne pouvons plus voir le text en clair lors de nos test (ce n'est pas forcément utile il y a trop de données dans le test pour toutes les analyser nous même). C'est pourquoi il y a ci-dessous un exemple de phrase afin de voir comment le modèle réagit, notamment avec des variantes orthographiques ou de ponctuation.

```
sentences = [  
    "I really appreciate your help, thank you!",  
    "You are incredibly stupid and annoying.",  
    "Why are you acting like such an idiot?",  
    "The weather is nice in Montreal today."  
    "You are gay",  
    "I'm a tall black guy",  
    "I like boobs",  
    "Japanese fool",  
    "I shit in my pants",  
    "You pooped yourself",  
    "I hate vegetable !!",  
    "That's freaking awful",  
    "I'm terrified !!",  
    "Go to hell ?",  
    "go to hell !",  
    "What the fuck ?!",  
    "fuck",  
    "f**k",  
    "****",  
    "HELLLL",  
    "u r dumb",  
    "u suck",  
    "you suck",  
    "you sucks",  
    "you su*k",  
    "roten apple",  
    "Grandma died last weekend",  
    "You scare me",  
    "I miss you",  
    "He is gonna kill you, he's a wolf and you're an ewe !"  
]  
  
print("\n--- RÉSULTATS ---")  
for s in sentences:
```

```
label, score = predict_toxicity(s)
print(f"Phrase : \"{s}\"")
# Affichage : rouge = toxique, vert = sain
color = "\033[91m" if label == "TOXIQUE" else "\033[92m"
reset = "\033[0m"
print(f" --- {color}{label}{reset} (Confiance : {score:.4f})\n")
```

--- RÉSULTATS ---

Phrase : "I really appreciate your help, thank you!"

--- NON TOXIQUE (Confiance : 0.0006)

Phrase : "You are incredibly stupid and annoying."

--- TOXIQUE (Confiance : 0.9089)

Phrase : "Why are you acting like such an idiot?"

--- TOXIQUE (Confiance : 0.8486)

Phrase : "The weather is nice in Montreal today.You are gay"

--- NON TOXIQUE (Confiance : 0.0212)

Phrase : "I'm a tall black guy"

--- NON TOXIQUE (Confiance : 0.2673)

Phrase : "I like boobs"

--- NON TOXIQUE (Confiance : 0.3219)

Phrase : "Japanese fool"

--- TOXIQUE (Confiance : 0.5917)

Phrase : "I shit in my pants"

--- TOXIQUE (Confiance : 0.8608)

Phrase : "You pooped yourself"

--- TOXIQUE (Confiance : 0.6549)

Phrase : "I hate vegetable!!"

--- NON TOXIQUE (Confiance : 0.3931)

Phrase : "That's freaking awful"

--- TOXIQUE (Confiance : 0.6627)

Phrase : "I'm terrified !!"

--- NON TOXIQUE (Confiance : 0.0872)

Phrase : "Go to hell ?"

--- NON TOXIQUE (Confiance : 0.4002)

Phrase : "go to hell !"

--- TOXIQUE (Confiance : 0.7274)

```
Phrase : "What the fuck ?!"  
--- TOXIQUE (Confiance : 0.8039)  
  
Phrase : "fuck"  
--- TOXIQUE (Confiance : 0.8483)  
  
Phrase : "f**k"  
--- TOXIQUE (Confiance : 0.6453)  
  
Phrase : "*****"  
--- NON TOXIQUE (Confiance : 0.0085)  
  
Phrase : "HELLLL"  
--- NON TOXIQUE (Confiance : 0.1157)  
  
Phrase : "u r dumb"  
--- NON TOXIQUE (Confiance : 0.0263)  
  
Phrase : "u suck"  
--- NON TOXIQUE (Confiance : 0.0669)  
  
Phrase : "you suck"  
--- TOXIQUE (Confiance : 0.7030)  
  
Phrase : "you sucks"  
--- NON TOXIQUE (Confiance : 0.3236)  
  
Phrase : "you su*k"  
--- NON TOXIQUE (Confiance : 0.4452)  
  
Phrase : "roten apple"  
--- NON TOXIQUE (Confiance : 0.0205)  
  
Phrase : "Grandma died last weekend"  
--- NON TOXIQUE (Confiance : 0.0254)  
  
Phrase : "You scare me"  
--- NON TOXIQUE (Confiance : 0.4242)  
  
Phrase : "I miss you"  
--- NON TOXIQUE (Confiance : 0.0027)  
  
Phrase : "He is gonna kill you, he's a wolf and you're an ewe !"  
--- NON TOXIQUE (Confiance : 0.1715)
```

4.3.3 Utilisation du jeu de test

```
import torch  
import numpy as np
```

```

import pandas as pd
from datasets import load_from_disk
from transformers import DistilBertForSequenceClassification, Trainer,
TrainingArguments
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# --- CONFIGURATION ---
DATASET_PATH =
"./jigsaw-unintended-bias-in-toxicity-classification/data/Bert/process
ed_data_bert_10k_ratio6"
MODEL_PATH = "./models/distilbert_jigsaw_final"

print("--- Chargement des données et du modèle ---")

# Chargement du dataset
dataset = load_from_disk(DATASET_PATH)
test_dataset = dataset["test"] # partie Test

# Chargement du modèle
model =
DistilBertForSequenceClassification.from_pretrained(MODEL_PATH)

# Création d'un Trainer pour la prédiction
training_args = TrainingArguments(
    output_dir="./results_inference",
    per_device_eval_batch_size=64,
    do_train=False,
    do_predict=True,
    use_cpu=True # Configuration pour la VM via virtual Box pour
éviter les erreurs avec le GPU
)

trainer = Trainer(
    model=model,
    args=training_args
)

print(f"Nombre d'échantillons de test : {len(test_dataset)}")

--- Chargement des données et du modèle ---
Nombre d'échantillons de test : 14000

print("--- Lancement des prédictions sur le jeu de test ---")
# Étape longue
predictions_output = trainer.predict(test_dataset)

# Récupération des logits

```

```

logits = predictions_output.predictions
y_preds = np.argmax(logits, axis=1)
# Récupération des labels
y_true = predictions_output.label_ids
print("--- Prédictions terminées ---")
--- Lancement des prédictions sur le jeu de test ---
<IPython.core.display.HTML object>
--- Prédictions terminées ---

```

4.3.3bis

Enregistrement des résultats de l'inférence afin de ne pas attendre 10minutes à chaque fois pour les afficher

```

import os
import numpy as np

OUTPUT_DIR = "./models/distilbert_jigsaw_final/results"

os.makedirs(OUTPUT_DIR, exist_ok=True)
file_path = os.path.join(OUTPUT_DIR, "resultats_bert_test.npz")

print(f"Sauvegarde des prédictions dans : {file_path}")

np.savez_compressed(
    file_path,
    logits=logits,
    y_true=y_true,
    y_preds=y_preds
)

print("--- Sauvegarde terminée avec succès ---")

Sauvegarde des prédictions dans :
./models/distilbert_jigsaw_final/results/resultats_bert_test.npz
--- Sauvegarde terminée avec succès ---

```

Récupération des résultats de l'inférence sauvegardée - À run de votre côté pour voir les résultats

```

import os
import numpy as np

INPUT_DIR = "./models/distilbert_jigsaw_final/results"

```

```

file_path = os.path.join(INPUT_DIR, "resultats_bert_test.npz")
if os.path.exists(file_path):
    print(f"Chargement des données depuis : {file_path}")

    data = np.load(file_path)

    logits = data['logits']
    y_true = data['y_true']
    y_preds = data['y_preds']

    print(" Variables chargées : 'logits', 'y_true', 'y_preds'")
    print(f"Dimensions des logits : {logits.shape}")

else:
    print(f"--- Erreur : Le fichier {file_path} est introuvable. ---")

Chargement des données depuis :
./models/distilbert_jigsaw_final/results/resultats_bert_test.npz
Variables chargées : 'logits', 'y_true', 'y_preds'
Dimensions des logits : (14000, 2)

```

4.3.4 Affichage des résultats

```

print("\n--- RAPPORT DE PERFORMANCES ---")

acc = accuracy_score(y_true, y_preds)
print(f"Précision globale (Accuracy) : {acc:.2%}")

target_names = ["Non Toxique (0)", "Toxique (1)"]
print("\nClassification Report :")
print(classification_report(y_true, y_preds,
    target_names=target_names))

# --- Calcul de l'AUC ROC ---
probs = torch.nn.functional.softmax(torch.tensor(logits), dim=-1)[: ,
1].numpy()
auc_score = roc_auc_score(y_true, probs)
print(f"Score ROC AUC : {auc_score:.4f}")

# --- Visualisation ---
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# Matrice de Confusion
cm = confusion_matrix(y_true, y_preds)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
    xticklabels=target_names, yticklabels=target_names,
    ax=axes[0])
axes[0].set_xlabel('Prédit')
axes[0].set_ylabel('Réel')

```

```
axes[0].set_title('Matrice de Confusion')
```

```
# Courbe ROC
```

```
fpr, tpr, thresholds = roc_curve(y_true, probs)
axes[1].plot(fpr, tpr, color='darkorange', lw=2, label=f'Courbe ROC (AUC = {auc_score:.2f})')
axes[1].plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
axes[1].set_xlim([0.0, 1.0])
axes[1].set_ylim([0.0, 1.05])
axes[1].set_xlabel('Taux de Faux Positifs (False Positive Rate)')
axes[1].set_ylabel('Taux de Vrais Positifs (True Positive Rate)')
axes[1].set_title('Courbe ROC (Receiver Operating Characteristic)')
axes[1].legend(loc="lower right")
```

```
plt.tight_layout()
plt.show()
```

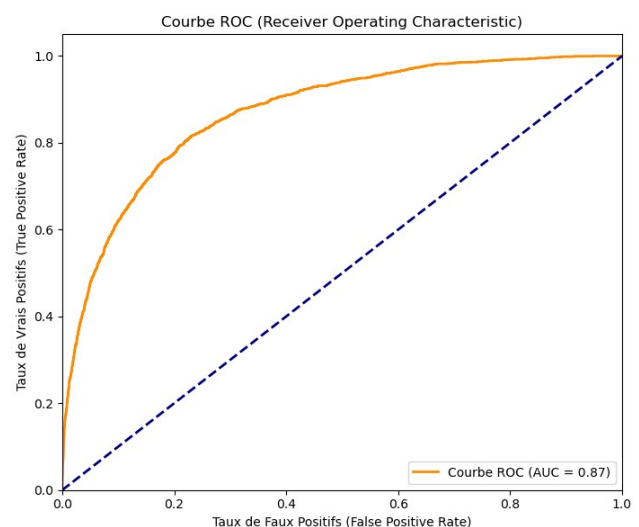
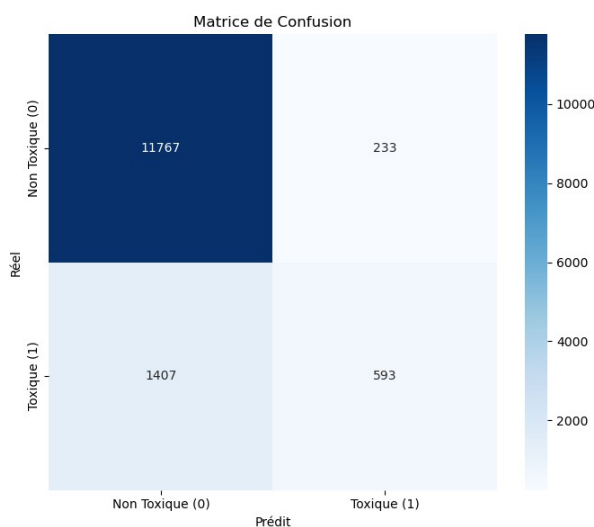
--- RAPPORT DE PERFORMANCES ---

Précision globale (Accuracy) : 88.29%

Classification Report :

	precision	recall	f1-score	support
Non Toxique (0)	0.89	0.98	0.93	12000
Toxique (1)	0.72	0.30	0.42	2000
accuracy			0.88	14000
macro avg	0.81	0.64	0.68	14000
weighted avg	0.87	0.88	0.86	14000

Score ROC AUC : 0.8695



5. Comparaison et évaluation générale

6. Amélioration et hybridation

7. Soumission Kaggle et documentation